Karlsruhe Institute of Technology

Test, Validation, Analysis

Uniform Random Sampling for Cardinality-Based Feature Models

Bachelor's Thesis
of

Marco Niklas Sircar

at the Department of Informatics
Institute of Information Security and Dependability
Test, Validation and Analysis (TVA)

Reviewer:	Prof. Dr.-Ing. Ina Schaefer
Second Reviewer:	Prof. Dr. Ralf Reussner
Advisor:	M.Sc. Lukas Güthing

Completion period: 30. November 2025 – 30. March 2026

Ich versichere wahrheitsgemäß, die Arbeit selbstständig verfasst, alle benutzten Hilfsmittel vollständig und genau angegeben und alles kenntlich gemacht zu haben, was aus Arbeiten anderer unverändert oder mit Abänderungen entnommen wurde sowie die Satzung des KIT zur Sicherung guter wissenschaftlicher Praxis in der jeweils gültigen Fassung beachtet zu haben.

Karlsruhe, den 30. März 2026 _____

Zusammenfassung

Feature-Modelle (FMs) und ihre Verallgemeinerung zu kardinalitätsbasierten Feature-Modellen (CFMs) sind zentrale Formalismen für die Modellierung von Variabilität in Produktlinien, insbesondere in Software-Produktlinien (SPLs). CFMs besitzen eine hohe Ausdrucksmächtigkeit, da sie Multiplizitäten ähnlich der Unified Modeling Language (UML) erlauben. Diese Ausdrucksmächtigkeit erschwert jedoch grundlegende Analyseaufgaben erheblich. Insbesondere ist die gleichverteilte Erzeugung zufälliger Konfigurationen für CFMs bislang kaum untersucht, obwohl sie einen wichtigen Baustein für die Analyse und das Testen von SPLs darstellt. Diese Arbeit adressiert dieses Problem, indem sie neue theoretische Grundlagen und Algorithmen zur Erzeugung gleichverteilter Konfigurationen von CFMs entwickelt. Ein zentraler Ausgangspunkt ist die Erkenntnis, dass die häufig verwendete instanzbasierte Konfigurationssemantik für diesen Zweck ungeeignet ist. Beliebig gewählte Indizes von Feature-Instanzen führen zu sogenannten Index-Anomalien. Diese verhindern, dass rekursiv über CFMs und deren Konfigurationen argumentiert werden kann. Insbesondere ist es nicht möglich, Konfigurationen systematisch aus kleineren Teilmodellen zu konstruieren. Als ersten Beitrag führt diese Arbeit daher mehrere alternative Konfigurationsbegriffe ein, die diese problematischen Artefakte beheben. Darauf aufbauend wird ein auf dynamischer Programmierung (DP) basierendes Framework entwickelt, das das Zählen und Unranken kanonischer Konfigurationen von CFMs ohne Cross-Tree-Constraints erlaubt. Auf Basis dieses Frameworks werden zwei Algorithmen vorgestellt, die nachweislich gleichverteilte Konfigurationen erzeugen. Neben der theoretischen Analyse, die die Gleichverteilung beweist und die Laufzeit- sowie Speicherkomplexität untersucht, erfolgt eine empirische Evaluation. Dabei wird das DP-basierte Framework mit einer State-of-the-Art-Baseline auf Basis eines Solvers für Constraint-Satisfaction-Probleme (CSPs) verglichen. Die Ergebnisse zeigen, dass der DP-basierte Ansatz hocheffizient ist. Er übertrifft die CP-SAT-Baseline um ein bis zwei Größenordnungen und in einigen Fällen sogar noch deutlicher. Insgesamt stellt diese Arbeit die gleichverteilte Stichprobenziehung für CFMs auf eine solide formale Grundlage. Sie führt neue Konfigurationssemantiken sowie ein solverunabhängiges, auf DP basierendes Framework für die Analyse von CFMs ein. Beide Beiträge gehen über die gleichverteilte Erzeugung von Konfigurationen hinaus. Sie deuten auf einen grundlegenden Wandel hin: weg von allgemeiner Solver-Technologie und hin zu direkten, strukturausnutzenden Methoden für CFMs und ihre möglicherweise vielfältigen Konfigurationssemantiken.

Abstract

Feature models (FMs) and their extension to cardinality-based feature models (CFMs) are key formalisms for modelling variability in product lines, especially software product lines (SPLs). While CFMs provide the expressive power needed to model complex systems, as they allow UML-like multiplicities, this expressiveness also makes fundamental analysis tasks substantially harder. In particular, uniform random sampling of configurations, an important building block for the analysis and testing of SPLs, has so far remained largely unexplored for CFMs. This thesis addresses this problem by developing new theoretical foundations and algorithms for uniform random sampling of CFMs. A central starting point is the observation that the commonly used instance-based configuration semantics of CFMs is not appropriate for this purpose. Arbitrary feature-instance indices introduce index-related anomalies that hinder recursive reasoning about CFMs and their configurations. In particular, they prevent the systematic construction of configurations from smaller sub-models. As a first contribution, this thesis therefore introduces several alternative configuration notions that eliminate these problematic artifacts. Building on this foundation, this thesis develops a dynamic programming (DP)-based framework for counting and unranking canonical configurations of CFMs without cross-tree constraints. This framework enables two provably uniform random sampling algorithms. In addition to the theoretical analysis, which proves uniformity and analyses space and time complexity, the DP-based counting and unranking framework is empirically evaluated against a CP-SAT baseline. The results indicate that the DP-based approach is highly efficient, outperforming the CP-SAT baseline by one to two orders of magnitude, and in some cases by even more. Taken together, this thesis provides a rigorous formal foundation for uniform random sampling for CFMs. It introduces several new configuration semantics and a solver-independent, DP-based framework for the analysis of CFMs. Both contributions extend beyond uniform random sampling itself and point towards a broader shift from general-purpose solver technology to direct, structure-exploiting methods for reasoning about CFMs and their possible diverse configuration semantics.

Contents

Zusammenfassung **i**

Abstract **iii**

List of Figures **x**

List of Tables **xi**

List of Algorithms **xiii**

List of Listings **xv**

Acronyms **xviii**

Notation **xix**

1 Introduction **1**

2 Theoretical Background **5**
2.1 Feature Models 5
2.1.1 A Bathroom Product Line 5
2.1.2 Formal Definition 8
2.2 Cardinality-Based Feature Models 11
2.2.1 A House Product Line 11
2.2.2 Formal Definition 14
2.2.3 Normalisation and Bounding 21
2.3 Uniform Random Sampling Techniques 24
2.3.1 Feature Models 24
2.3.2 Cardinality-Based Feature Models 25
2.4 Combinatorics 29
2.4.1 Binomial Coefficients 29
2.4.2 Multisets 31
2.4.3 Ranking and Unranking of Combinatorial Objects 35

3 Semi-Structural and Structural Configurations **43**
3.1 Problems of Instance-Based Configurations 43
3.1.1 Index Anomalies 44
3.1.2 Sub-Models 45
3.1.3 Sub-Configurations 46

3.1.4 Merging Instance-Based Configurations . . . 50
3.1.5 Conclusion . . . 57
3.2 Structural Configurations . . . 57
3.2.1 Equivalence Class Representation . . . 57
3.2.2 Index-Free Tree Representation . . . 61
3.2.3 Canonical Representation . . . 64
3.2.4 Merging Canonical Configurations . . . 83
3.3 Semi-Structural Configurations . . . 87
3.3.1 Equivalence Class Representation . . . 87
3.3.2 Index-Free Tree Representation . . . 90
3.3.3 Semi-Canonical Representation . . . 93
3.3.4 Merging Semi-Canonical Configurations . . . 95
3.4 Summary and Comparison of Configuration Notions . . . 97
3.4.1 Relationships and Properties . . . 97
3.4.2 Interpretation of Configuration Semantics . . . 100
3.4.3 Configuration Semantics in the Literature . . . 106
3.4.4 Implications for Uniform Random Sampling . . . 108
3.4.5 Outlook: Tailoring Configuration Semantics . . . 110
3.4.6 Overview of Definitions, Theorems, and Lemmas . . . 110

4 Uniform Random Sampling 113
4.1 Uniform Random Sampling without Cross-Tree Constraints . . . 114
4.1.1 Counting Canonical Configurations via Dynamic Programming 114
4.1.2 The UniSample-BT Algorithm . . . 123
4.1.3 The UniSample-UR Algorithm . . . 127
4.2 Uniform Random Sampling with Cross-Tree Constraints . . . 143
4.3 Summary and Outlook . . . 143

5 Theoretical Evaluation 147
5.1 Correctness . . . 147
5.1.1 UniSample-BT . . . 147
5.1.2 UniSample-UR . . . 156
5.1.3 UniSample-CTC . . . 156
5.2 Time and Space Complexity . . . 158
5.2.1 ComputeDP . . . 158
5.2.2 UniSample-BT . . . 163
5.2.3 UniSample-UR . . . 175
5.2.4 UniSample-CTC . . . 181
5.2.5 Comparison of Unisample-BT and Unisample-UR . . . 182
5.2.6 Eliminating Sorting of Child Configurations . . . 188
5.3 Summary and Outlook . . . 193

6 Implementation 195
6.1 General Architecture and Framework . . . 195
6.1.1 Core Model Component . . . 196
6.1.2 CLI Pipeline Component . . . 198
6.1.3 Native Backend . . . 201
6.2 Algorithm Implementations in Rust . . . 204
6.2.1 Representation of Structural Configurations . . . 204

6.2.2 ComputeDP . . . 209
6.2.3 UniSample-BT . . . 211
6.2.4 UniSample-UR and UnrankConfig . . . 214
6.3 SAT-Based Reference Implementation . . . 217

7 Empirical Evaluation **225**
7.1 Evaluation Goals and Research Questions . . . 225
7.1.1 Research Questions . . . 226
7.1.2 Rationale and Scope of the Evaluation Goals . . . 226
7.2 Experimental Methodology . . . 227
7.2.1 Methodology for Performance Assessment (G1) . . . 227
7.2.2 Methodology for Uniformity Validation (G2) . . . 237
7.2.3 Dataset . . . 241
7.2.4 Hardware and Software Environment . . . 248
7.3 Results . . . 248
7.3.1 RQ1: Time to First Configuration . . . 248
7.3.2 RQ2: Enumeration Throughput . . . 252
7.3.3 RQ3: Influence of Model Type . . . 253
7.3.4 RQ4: Empirical Uniformity . . . 261
7.4 Discussion of Findings and Failure Cases . . . 263
7.4.1 Answers to Research Questions . . . 263
7.4.2 Interpretation of Failure Cases . . . 264
7.5 Threats to Validity . . . 265
7.5.1 Internal Validity . . . 265
7.5.2 Construct Validity . . . 267
7.5.3 Conclusion Validity . . . 268
7.5.4 External Validity . . . 269
7.6 Summary and Outlook . . . 270

8 Related Work **271**

9 Conclusion and Outlook **275**
9.1 Summary of Contributions . . . 275
9.2 Future Work . . . 277

Bibliography **281**

A Use of AI Tools **293**

List of Figures

2.1 FM modelling a bathroom product line 8
2.2 FM modelling a bathroom product line (compact form) 8
2.3 CFM modelling a house product line 14
2.4 Example instance-based configurations of the house CFM 19
2.5 Big-M bounding of a CFM 24

3.1 CFM used in the index anomaly examples 44
3.2 Index anomalies of instance-based configurations 45
3.3 Sub-model of the house CFM, rooted at the feature `Unit` 46
3.4 Sub-configuration of the house CFM 49
3.5 Relationship between a CFM, its sub-model, and their configurations 50
3.6 Limitation of the MERGE procedure 56
3.7 Tree-isomorphic and non-tree-isomorphic configurations 59
3.8 Illustration of structural configurations 63
3.9 Index anomaly: Subtree index inversion 65
3.10 Index anomaly: Sibling index inversion 65
3.11 Instance order computation 69
3.12 Illustration of semi-structural configurations 92
3.13 Overview and relationships between configuration semantics 99
3.14 Parking garage CFM and configurations of all types 104
3.15 CFM together with its structural configuration space 109

4.1 Hierarchical decomposition of the rank set into blocks, grids, and fibers 136

7.1 Chain CFM with parameters h and k 242
7.2 Evaluation dataset: Distribution of feature tree properties across CFMs, FMs, and the complete dataset 245
7.3 Evaluation dataset: Distribution of cross-tree constraint characteristics across CFMs, FMs, and the complete dataset 246
7.4 Evaluation dataset: Distribution of total instance bounds and estimated DP runtime across CFMs, FMs, and the complete dataset . . . 247
7.5 RQ1: Cactus plots for TTFC 250
7.6 RQ1: Performance profiles for TTFC 251
7.7 RQ2: Cactus plots for enumeration throughput 252
7.8 RQ2: Performance profiles for enumeration throughput 253
7.9 RQ3: Cactus plots for TTFC partitioned by model type 257
7.10 RQ3: Performance profiles for TTFC partitioned by model type . . . 258
7.11 RQ3: Cactus plots for enumeration throughput partitioned by model type 259

7.12 RQ3: Performance profiles for enumeration throughput partitioned by model type 260
7.13 CFMs where DP has a construction timeout and CP-SAT an enumeration timeout 265

List of Tables

3.1 Comparison of configuration notions and their properties 101

5.1 Comparison of the main runtime components of UniSample-BT and UniSample-UR 183
5.2 Description of the algorithmic steps referenced in Table 5.1 184
5.3 Comparison of the main runtime components of the improved versions of UniSample-BT and UniSample-UR 192

7.1 Summary statistics of the total instance bound $B_{\text{tot}} = \sum_{f \in F} B_f$ for the 999 models in the folder `Generator_examples` 230
7.2 The ten models with the smallest total instance bounds B_{tot} among the 999 models in the folder `Generator_examples` 230
7.3 Models used for the uniformity validation 237
7.4 Supported UVL cross-tree constraints and their encoding in CFM semantics 243
7.5 Evaluation dataset: Models excluded due to empty configuration spaces 244
7.6 Evaluation dataset: Minimum, median, mean, and maximum values of feature tree properties, cross-tree constraints, and runtime estimation metrics 247
7.7 RQ1: Overview of solver outcomes and success rates 249
7.8 RQ1: Contingency comparison of solver outcomes 249
7.9 RQ1: PAR-1 of TTFC 250
7.10 RQ3: Overview of solver outcomes and success rates partitioned by model type 254
7.11 RQ3: Contingency comparison of solver outcomes partitioned by model type 255
7.12 RQ3: PAR-1 of TTFC partitioned by model type 256
7.13 RQ4: Per-model uniformity validation 262
7.14 RQ4: Global uniformity validation 263

List of Algorithms

2.1 Algorithm for naive random sampling in a CFM based on recursive construction and rejection sampling 27
2.2 Algorithm for naive random generation of child feature instances in a CFM 28
2.3 Algorithm for ranking a combination 37
2.4 Algorithm for unranking a combination 38
2.5 Algorithm for ranking a multiset 42
2.6 Algorithm for unranking a multiset 42

3.1 Algorithm for merging child configurations of a CFM 54

4.1 Algorithm for computing configuration-count DP tables in a CFM without cross-tree constraints 123
4.2 Algorithm for uniform random sampling in a CFM without cross-tree constraints via DP-guided recursive construction 126
4.3 Algorithm for uniform random sampling in a CFM without cross-tree constraints via unranking 128
4.4 Algorithm for unranking canonical configurations in a CFM without cross-tree constraints 141
4.5 Algorithm for uniform random sampling in a CFM with cross-tree constraints via rejection sampling 143

List of Listings

6.1 CFMTOOLS Python core: Feature identifiers and feature-indexed containers . . . 197
6.2 CFMTOOLS Python core: Simple and compound cardinality intervals . 197
6.3 CFMTOOLS Python core: CFMs and cross-tree constraints . . . 199
6.4 CFMTOOLS Python core: CFM builder for constructing a CFM . . . 200
6.5 CFMTOOLS CLI pipeline: Loader plugin for importing CFMs from JSON files . . . 202
6.6 CFMTOOLS CLI pipeline: Sampler plugin implementing UniSample-BT . . . 203
6.7 CFMTOOLS Rust core: Structural configuration representation . . . 205
6.8 CFMTOOLS Rust core: Structural configuration builder . . . 206
6.9 CFMTOOLS Rust core: Key used for node interning . . . 207
6.10 CFMTOOLS Rust core: Interning of nodes . . . 208
6.11 CFMTOOLS Rust core: Implementation of ComputeDP . . . 210
6.12 CFMTOOLS Rust core: Top-level implementation of UniSample-BT . 212
6.13 CFMTOOLS Rust core: Recursive implementation of UniSample-BT 213
6.14 CFMTOOLS Rust core: Log-space computation of the multiset rejection probability . . . 214
6.15 CFMTOOLS Rust core: Implementation of UniSample-UR . . . 215
6.16 CFMTOOLS Rust core: Implementation of UnrankConfig . . . 216
6.17 CFMTOOLS Python CP-SAT plugin: Guarded encoding of cardinality intervals . . . 222
6.18 CFMTOOLS Python CP-SAT plugin: Reification of cardinality interval predicates for cross-tree constraints . . . 223

7.1 Rust release profile for compiling the native backend . . . 248
7.2 Build command for compiling the native backend and installing the Python package in development mode . . . 248

Acronyms

AHU Aho, Hopcroft, and Ullman

CDCL conflict-driven clause learning

CDF cumulative distribution function

CFM cardinality-based feature model

CLI command-line interface

CoLex colexicographic

CP constraint programming

CSP constraint satisfaction problem

CTC cross-tree constraint

DAG directed acyclic graph

DFS depth-first search

DP dynamic programming

FM feature model

GMP GNU Multiple Precision Arithmetic Library

i.i.d. independent and identically distributed

ILP integer linear programming

JSON JavaScript Object Notation

KIT Karlsruhe Institute of Technology

LTO link-time optimization

MCMC Markov Chain Monte Carlo

PAR penalized average runtime

RAM random access machine

S.P.L.O.T. Software Product Lines Online Tools

SAT Boolean satisfiability problem

SMT satisfiability modulo theories

SPL software product line

TTFC time to first configuration

TVA Institute for Test Validation and Analysis of Software-Intensive Systems

UML Unified Modeling Language

UVL Universal Variability Language

WSL Windows Subsystem for Linux

Notation

Mathematical Symbols

Symbol	Description
$\mathbb{N}$	Set of natural numbers
$\mathbb{N}_0$	Set of natural numbers including zero
$\mathbb{Z}$	Set of integers
$\mathbb{Q}$	Set of rational numbers
$\mathbb{R}$	Set of real numbers
$\mathbb{E}$	Expectation operator
$\mathbb{P}$	Probability measure
$\mathbf{X}$	Random variable. In general, bold uppercase letters denote random variables.
$\mathbb{B}$	Set of Boolean truth values
false	Boolean truth value false
true	Boolean truth value true
$alternative_n$	Boolean function: exclusive disjunction of n arguments (exactly one argument is true)
and_n	Boolean function: conjunction of n arguments
opt_n	Boolean function: constant true
or_n	Boolean function: disjunction of n arguments
$\binom{n}{k}$	Binomial coefficient counting the number of k-subsets of an n-element set
$n!$	Factorial of n
$\left(\!\binom{n}{k}\!\right)$	Number of k-multisets over an n-element set
$\{\!\{\cdot\}\!\}$	Multiset
$\prec^*$	Reflexive–transitive closure of a binary relation $\prec$
$\lfloor x \rfloor$	Floor function returning the greatest integer less than or equal to x
id_A	Identity map on the set A

$\mathbb{1}_{\{A\}}$	Indicator function that evaluates to 1 if statement A holds and 0 otherwise
$a \bmod b$	Remainder of the division of a by b

CFM-Specific Symbols

Model syntax

Symbol	Description	Page
$\mathcal{M}$	Model object: a FM or CFM	16
F	Finite non-empty set of features	9
$\prec_F$	Feature decomposition relation	9
f_r	Root feature of the feature tree	9
$(F, \prec_F)$	Feature tree consisting of the feature set F and the decomposition relation $\prec_F$	9
$\leq_F$	Total order on the feature set F	66
λ	Relation-type function of a FM	9
λ_I^F	Feature-instance cardinality function $\lambda_I^F : F \to \mathcal{I}$	16
λ_I^G	Group-instance cardinality function $\lambda_I^G : F \to \mathcal{I}$	16
λ_T^G	Group-type cardinality function $\lambda_T^G : F \to \mathcal{I}$	16
ϕ_R	Require-edge relation; for FMs, $\phi_R \subseteq F \times F$, and for CFMs, $\phi_R \subseteq F \times \mathcal{I} \times \mathcal{I} \times F$	9
ϕ_X	Exclude-edge relation; for FMs, $\phi_X \subseteq F \times F$, and for CFMs, $\phi_X \subseteq F \times \mathcal{I} \times \mathcal{I} \times F$	9

Cardinality intervals

Symbol	Description	Page
$\mathcal{I}$	Set of all (compound) cardinality intervals	14
$\langle l, u \rangle$	Simple cardinality interval with lower bound l and upper bound u	14
I	Compound cardinality interval $I \in \mathcal{I}$, i.e. a finite set of simple intervals	14
$k \in I$	Cardinality k admitted by interval I	14
$\lvert I \rvert$	Number of distinct cardinalities admitted by interval I	14
$\max(I)$	Maximum cardinality admitted by interval I	14
$\text{convex}(I)$	Non-convex bound of a cardinality interval I (used for Big-M bounding)	21

Instance-based and Multiset-based Configurations

Symbol	Description	Page
M	Multiplicity function $M : F \to \mathbb{N}_0$ describing the number of instances of each feature	17
$K = (M, \prec)$	Instance-based configuration consisting of a multiplicity function M and a feature-instance decomposition relation $\prec$	17
$\text{Inst}(M)$	Set of feature instances generated by a multiplicity function M	17
$(\text{Inst}(M), \prec)$	Feature instance tree of a configuration $K = (M, \prec)$	17
π	Projection from instance-based configurations to multiset-based configurations	20

Sub-Models and Sub-Configurations

Symbol	Description	Page
F_f	Set of features in the subtree rooted at feature f	45
$\mathcal{M}_f$	Sub-model of $\mathcal{M}$ rooted at feature f	45
$\mathrm{Inst}(M)\|_{f^i}$	Feature instance set reachable from the feature instance f^i	47
$M\|_{f^i}$	Multiplicity function restricted to the subtree rooted at f^i	47
$\prec\|_{f^i}$	Restricted feature-instance decomposition relation for a sub-configuration	47
K_{f^i}	Sub-configuration of $K = (M, \prec)$ rooted at the feature instance f^i	47
ρ	Renumbering bijection mapping sparse instance indices to consecutive indices in a sub-configuration	47
$C_{\mathcal{M}}(f)$	Set of child features of feature f in the CFM $\mathcal{M}$	51
$C_K(f^i)$	Set of child feature instances of f^i in configuration K	51
$C_K(f^i, g)$	Set of child instances of f^i in configuration K whose feature type is g	51
$(a, b)^\Delta$	Shifted decomposition edge obtained from $(a, b) \in \prec$ by increasing instance indices according to Δ	52
$\prec^\Delta$	Shifted feature-instance decomposition relation $\prec^\Delta = \{(a, b)^\Delta \mid (a, b) \in \prec\}$	52
M^Δ	Shifted multiplicity function defined by $M^\Delta(g) = M(g) + \Delta(g)$	52
$K \oplus \Delta$	Shifted configuration obtained from $K = (M, \prec)$ by applying the offset function Δ	52

Structural and Semi-Structural Configurations

Symbol	Description	Page
$\cong_{\text{tree}}$	Tree-isomorphism relation between instance-based configurations	57
$\cong_{\text{order}}$	Order-isomorphism relation between instance-based configurations	87
$[K]_{\text{Struct}}$	Structural configuration (tree-isomorphism equivalence class)	61
$[K]_{\text{SemiStruct}}$	Semi-structural configuration (order-isomorphism equivalence class)	90
$\Phi_K^{\text{tree}}(x)$	Index-free tree representation of a feature instance x in a instance-based configuration K	61
$\Phi^{\text{tree}}(K)$	Index-free tree representation of an instance-based configuration K	61
$\Phi^{\text{tree}}([K]_{\text{Struct}})$	Index-free tree representation of a structural configuration	61
$\Phi_K^{\text{order}}(x)$	Index-free ordered-tree representation of a feature instance x in a instance-based configuration K	90
$\Phi^{\text{order}}(K)$	Index-free ordered-tree representation of an instance-based configuration K	90
$\Phi^{\text{order}}([K]_{\text{SemiStruct}})$	Index-free ordered-tree representation of a semi-structural configuration	90
$\trianglelefteq$	Instance order: total preorder on feature instances induced by a configuration	66
$\trianglelefteq_{\text{lex}}$	Lexicographic preorder induced by the instance order	66

Sampling and Dynamic Programming

Symbol	Description	Page
$C_f^i(c,k)$	Number of canonical configurations of the filled sub-model $\mathcal{M}_f^{\text{filled}}$ with realised root-group cardinalities (c,k) when only the first i children of f may contribute instances	117
$C_f(c,k)$	Number of canonical configurations of the sub-model $\mathcal{M}_f$ whose realised root-group cardinalities are (c,k)	117
T_f	Total number of canonical configurations of the sub-model $\mathcal{M}_f$	117
$\gamma_T(K)$	Number of distinct child feature types of the root feature realised in configuration K	115
$\gamma_I(K)$	Total number of child feature instances of the root feature realised in configuration K	115
$\mathcal{M}_f^{\text{filled}}$	Filled sub-model of $\mathcal{M}_f$ obtained by removing gaps in the group cardinalities of f	116
$\psi_T^G(f)$	Filled group-type cardinality interval of feature $f \in F$; $\left\langle 0, \max \lambda_T^G(f) \right\rangle$; used in the filled sub-model	116
$\psi_I^G(f)$	Filled group-instance cardinality interval of feature $f \in F$; $\left\langle 0, \max \lambda_I^G(f) \right\rangle$; used in the filled sub-model	116

Theoretical Analysis

Symbol	Description	Page
Λ_T^G	Maximum value occurring in any group-type cardinality interval of the model; $\Lambda_T^G = \max_{f \in F} \max(\lambda_T^G(f))$.	158
Λ_I^G	Maximum value occurring in any group-instance cardinality interval of the model; $\Lambda_I^G = \max_{f \in F} \max(\lambda_I^G(f))$.	158
Λ_I^F	Maximum value occurring in any feature-instance cardinality interval of the model; $\Lambda_I^F = \max_{f \in F} \max(\lambda_I^F(f))$.	158
Ξ_T^G	Maximum size of any group-type cardinality interval of the model; $\Xi_T^G = \max_{f \in F} \lvert\lambda_T^G(f)\rvert$.	158

Symbol	Description	Page
Ξ_I^G	Maximum size of any group-instance cardinality interval of the model; $\Xi_I^G = \max_{f \in F} \lvert\lambda_I^G(f)\rvert$.	158
Ξ_I^F	Maximum size of any feature-instance cardinality interval of the model; $\Xi_I^F = \max_{f \in F} \lvert\lambda_I^F(f)\rvert$.	158
$\mathbf{C}$	Random realised group-type cardinality	147
$\mathbf{K}$	Random realised group-instance cardinality	147
$\mathbf{R}$	Random vector $(\mathbf{R}_1, \ldots, \mathbf{R}_n)$ of child multiplicities where $\mathbf{R}_i$ denotes the number of instances of child feature c_i	147
$\mathbf{L}_i$	Random list of r_i sampled child configurations for child feature c_i	147
$\mathbf{RT}(f)$	Random runtime of the algorithm UniSample-BT$(f, \mathcal{M})$	163
Size(K)	Number of feature instances in configuration K	163
$\overline{\text{Size}}(\mathcal{M}_f)$	Average size of canonical configurations of the sub-model $\mathcal{M}_f$	163
$\mathbf{K}_f$	Random configuration produced by UniSample-BT$(f, \mathcal{M})$	173
RT$(\mathcal{R}, f)$	Deterministic runtime of the unranking algorithm UnrankConfig$(f, \mathcal{R}, \mathcal{M})$	175
$\mathcal{R}$	Rank	175
$\boldsymbol{\mathcal{R}}_f$	Random rank drawn uniformly from $\{0, \ldots, T_f - 1\}$	175
$\mathbf{RT}_{\text{Base}}$	Random runtime of the base sampling algorithm used in CTC-UniSample	181
$\mathbf{RT}_{\text{CTC}}$	Random runtime of the rejection sampling algorithm CTC-UniSample	181

Configuration Spaces

Symbol	Description	Page
$[\![\mathcal{M}]\!]$	Configuration space of a model (FM or CFM)	9
$\mathcal{C}_{\text{Inst}}(\mathcal{M})$	Domain of instance-based configurations	17
$\mathcal{C}_{\text{Multiset}}(\mathcal{M})$	Domain of multiset-based configurations	20
$\mathcal{C}_{\text{Struct}}(\mathcal{M})$	Domain of structural configurations	61
$\mathcal{C}_{\text{SemiStruct}}(\mathcal{M})$	Domain of semi-structural configurations	90
$[\![\mathcal{M}]\!]_{\text{Inst}}$	Instance-based configuration space	17
$[\![\mathcal{M}]\!]_{\text{Multiset}}$	Multiset-based configuration space	20
$[\![\mathcal{M}]\!]_{\text{Struct}}$	Structural configuration space	61
$[\![\mathcal{M}]\!]_{\text{Canon}}$	Canonical configuration space	76
$[\![\mathcal{M}]\!]_{\text{SemiStruct}}$	Semi-structural configuration space	90
$[\![\mathcal{M}]\!]_{\text{SemiCanon}}$	Semi-canonical configuration space	93

Utilities

Symbol	Description	Page
$\text{feat}(x)$	Underlying feature type of a feature instance $x \in \text{Inst}(M)$	17
$\text{index}(x)$	Index of a feature instance $x \in \text{Inst}(M)$	17
$\text{ht}(\cdot)$	Tree height	72

1. Introduction

Feature models (FMs) provide a formal framework for describing and reasoning about variability in families of related systems and product lines [1, 2, 3]. Such families may range from physical systems to, more commonly, software-intensive systems known as *software product lines* (SPLs). A SPL consists of a set of components, called features, that can be composed according to specific constraints, thereby yielding highly variable and often complex families of software systems [1, 4, 5, 6, 7].

The original notion of FMs, introduced by Kang et al. [8] in 1990, considered only *Boolean* features, that is, features that can either be present or absent in a product. These features are arranged in a tree-shaped structure together with additional cross-tree constraints, thereby modelling the overall space of possible products [3, 5, 9]. Many industrial domains, such as cloud computing or cyber-physical systems, require the ability to express multiplicities (e.g., "up to five", "at least two", or "exactly three") [10, 11, 12]. Cardinality-based feature models (CFMs) address this limitation by extending FMs with feature-instance and group cardinalities, similar in spirit to UML multiplicities, thus enabling the modelling of even more complex and variable systems [9, 11, 13, 14, 15]. This additional expressiveness comes at a cost: the configuration spaces of CFMs grow dramatically and may even become infinite unless cardinalities are bounded [10, 11].

Sampling configurations is a key enabler for the analysis and testing of SPLs, since exhaustive exploration of the configuration space is usually infeasible [11, 16]. Instead of testing every possible configuration, one generates a representative subset according to a chosen sampling criterion and performs the subsequent analysis only on this subset. Two widely used sampling objectives are *combinatorial sampling* (e.g., t-wise sampling) and *uniform random sampling*. Combinatorial approaches aim to ensure that all valid combinations of a fixed size are covered, whereas uniform random sampling aims to draw configurations uniformly at random from the valid configuration space [11, 16, 17].

Although numerous algorithms exist for both strategies in Boolean feature models, extending them to CFMs is not straightforward. The main challenges arise from the

fact that the configuration spaces of CFMs may be infinite or non-convex, and that standard sampling criteria often lead to excessively large search spaces [11].

Recent work has begun to address these challenges. In particular, Güthing et al. [11] and Zhang [18] study how combinatorial t-wise sampling can be adapted to CFMs. Zhang also proposes a naive algorithm for uniform random sampling, but this approach does not, in general, produce samples uniformly.

In this thesis, we build on the prior work of Zhang and develop two novel uniform random sampling algorithms for CFMs, based on dynamic programming (DP), which we prove to be correct and uniform. The fundamental idea is to recursively reduce the difficult problem of uniform random sampling for an arbitrary CFM to smaller subproblems until they are sufficiently small for uniform sampling to become immediate. The resulting samples are then assembled bottom-up to obtain a sample of the full model.

To achieve this goal, however, several new concepts and techniques for CFMs first have to be developed. In particular, the existing configuration semantics of CFMs must be clarified and extended, since this notion exhibits undesirable properties that prohibit systematic reasoning about sub-models. This leads to the development of four novel configuration semantics, which then provide the foundation for applying DP to CFMs and, subsequently, for deriving the two uniform random sampling algorithms.

The tools and techniques developed in this thesis are not only useful for uniform random sampling. In the future, we aim to use them, in particular the DP-based counting of valid configurations, to replace the use of Boolean satisfiability problem (SAT) and #SAT solvers for analysis tasks on Boolean FMs and, more generally, general-purpose satisfiability modulo theories (SMT), integer linear programming (ILP), and constraint satisfaction problem (CSP) solvers for corresponding analysis tasks on CFMs. Such tasks include, for example, the detection of dead cardinalities, dead features, and void (empty) models [10, 19, 20, 21]. This opens the prospect of achieving performance improvements of several orders of magnitude for a broad range of variability-analysis problems.

Moreover, the techniques used to introduce the new configuration semantics can themselves be generalized, potentially yielding a systematic tool for customizing configuration semantics to the specific properties and requirements of the system to be modelled. Exploring this direction is also a goal for future work.

Thus, although the primary objective of this thesis is the development of uniform random sampling algorithms for CFMs, which serves as both a guiding motivation and a unifying theme, the resulting concepts and techniques reach beyond uniform sampling itself.

Structure of the Thesis

The remainder of this thesis is organized as follows.

- Chapter 2 presents the theoretical background necessary for this thesis.
- Chapter 3 introduces the new configuration semantics. Concretely, it first identifies the flaws of instance-based configurations and then introduces structural and semi-structural configurations, as well as their canonical representations.

- Chapter 4 develops the DP-based counting framework and presents the uniform sampling algorithms for CFMs without cross-tree constraints, together with their extension to cross-tree constraints via rejection sampling.
- Chapter 5 establishes the correctness (uniformity and termination) of the proposed algorithms and analyses their time and space complexity, as well as discussing possible optimizations.
- Chapter 6 describes the open-source tool CFMTOOLS and explains how the theoretical algorithms are realized in practice.
- Chapter 7 empirically evaluates the performance of the DP-based counting and enumeration algorithm proposed in Chapter 4 against a CP-SAT baseline and validates the uniformity of the sampling algorithms.

2. Theoretical Background

This chapter introduces the theoretical foundations required for this thesis. We begin by presenting feature models (FMs) and cardinality-based feature models (CFMs), including both their formal syntax and their configuration semantics. We then review selected existing algorithms and techniques for uniform random sampling in FMs and CFMs, with particular emphasis on the approaches that form the basis for later developments in this thesis. The final part of this chapter introduces the relevant combinatorial background, including the definitions, notation, and theorems used throughout the thesis.

2.1 Feature Models

The primary focus of this thesis lies in CFMs and the problem of uniform random sampling for such models. Nevertheless, since CFMs generalize classical FMs, and many fundamental concepts directly carry over, we first introduce FMs.

The formal definition presented in this section is largely based on the framework proposed by Schobbens et al. [5] and further refined using elements described by Benavides et al. [9]. We slightly adapt the notation to align with the later definition of CFMs.

To motivate and illustrate the formal definition, we first introduce a running example that guides the introduction of the necessary concepts.

2.1.1 A Bathroom Product Line

Assume we want to model different variants of a `Bathroom`. Although every bathroom serves the same general purpose, concrete realizations may differ.

In feature modelling [8], such variability is described using *features*, where a feature is generally understood as a distinguishable system property or component [6, 8, 22, 23, 24, 25]. For each feature, a *configuration* (product) determines whether it is *selected* (included in the product) or *not selected*. A *valid* product is therefore obtained by selecting a set of features that satisfies all imposed constraints.

In our example product line, a bathroom must satisfy the following requirements:

- Every bathroom consists of a `Toilet`, a `Sink`, a `Bathing Fixture`, and a `Storage Unit`.
- A `Mirror` may optionally be present.
- The `Bathing Fixture` must be *exactly one* of the following: a `Bathtub` or a `Shower`.
- The `Storage Unit` must contain *at least one* of the following: a `Shelf` or a `Closet`.

Thus, a concrete bathroom configuration could, for example, be the set

{`Bathroom`, `Toilet`, `Sink`, `Bathing Fixture`, `Shower`, `Storage Unit`, `Shelf`}.

Not every subset of features yields a valid configuration; for instance, selecting both `Bathtub` and `Shower` would violate these requirements. The purpose of an FM is to formally capture precisely those feature combinations that are valid.

Hierarchical Feature Structure

A first observation is that features are naturally organized hierarchically. For example, `Bathtub` and `Shower` refine the feature `Bathing Fixture`, whereas `Shelf` and `Closet` refine `Storage Unit`.

This motivates the first core component of a FM: a finite rooted directed tree whose nodes are features, called the *feature tree*. Formally, we assume a finite set F of features together with a binary relation $\prec_F \subseteq F \times F$ such that $(F, \prec_F)$ is a directed tree with distinguished root $f_r \in F$. In particular, $(p, f) \in \prec_F$ holds if and only if f is a child feature of p.

Since, in most cases, the root feature f_r represents the overall system, it is also called the *concept* [1, 5].

In our example, the feature set is

$$F = \Big\{ \texttt{Bathroom}, \texttt{Bathing Fixture}, \texttt{Bathtub}, \texttt{Shower}, \texttt{Toilet}, \texttt{Sink}, \texttt{Storage Unit}, \texttt{Closet}, \texttt{Shelf}, \texttt{Mirror} \Big\},$$

and the root feature is `Bathroom`.

Relations Between Children

The hierarchical structure alone is not sufficient to capture the intended semantics of the bathroom example. In particular, it remains to specify how the children of a feature may be selected whenever their parent is selected. To capture this variability, we specify how the children of a feature relate to each other.

Formally, we introduce the *relation-type function*

$$\lambda : F \to \mathcal{R},$$

which assigns to each feature the type of relation it has with its children. Intuitively, for a feature $f \in F$, the value $\lambda(f)$ determines which subsets of its children may be selected in a valid configuration when f itself is selected. More concretely, we consider the following relation types:

- **AND (Mandatory).** All children of f must be selected whenever f is selected. In the bathroom example, `Toilet`, `Sink`, `Bathing Fixture`, and `Storage Unit` are mandatory children of `Bathroom`.
- **OR.** At least one child must be selected whenever f is selected, but several children may be selected simultaneously. For example, a `Storage Unit` may contain a `Shelf`, a `Closet`, or both.
- **XOR (Alternative).** Exactly one child must be selected whenever f is selected. In our example, the `Bathing Fixture` must be either a `Bathtub` or a `Shower`, but not both.
- **Optional.** Each child may be selected or not selected independently (provided the parent is selected). The feature `Mirror` is optional with respect to `Bathroom`.

Cross-Tree Constraints

While the hierarchical structure and relation types are sufficient for the bathroom example, more complex systems may require additional constraints that do not follow the tree hierarchy.

For example, one could require that selecting a `Bathtub` implies the selection of a `Mirror`. Furthermore, one might forbid selecting `Mirror` and `Shelf` simultaneously.

To model such dependencies, FMs allow additional binary constraints between features:

- A *require* constraint: if feature f is selected, then feature g must also be selected.
- An *exclude* constraint: features f and g must not be selected simultaneously.

Since these constraints are independent of the hierarchical decomposition, they are referred to as *cross-tree constraints* [1, 5, 13].

Graphical Representation

Figure 2.1 shows the visual representation of the FM modelling the bathroom product line and illustrates the different relation types.

Note that, under our current definition of FMs, the feature `Mirror` cannot be placed directly as a child of the root feature `Bathroom`. The reason is that the relation-type function λ assigns exactly one relation type to each feature. Hence, all children of a feature must be governed by the same relation.

In particular, we cannot simultaneously express that `Toilet`, `Sink`, `Bathing Fixture`, and `Storage Unit` are mandatory children of `Bathroom`, while `Mirror` is optional, if all of them are direct children of the root feature. To resolve this, we introduce a dummy feature `Dummy`, whose sole purpose is to separate the optional feature `Mirror` from the mandatory children.

More generally, this dummy construction can be used to simulate mixed relation types under a single parent: children can be partitioned into groups by introducing intermediate dummy features, and each group can then be assigned its own relation type via λ. Thus, although the formalism enforces a single relation type per feature, arbitrary combinations of group relations can still be expressed structurally [26].

Since this construction becomes cumbersome when many optional features are involved, we use the syntactic sugar shown in Figure 2.2, where mandatory and optional features are represented explicitly.

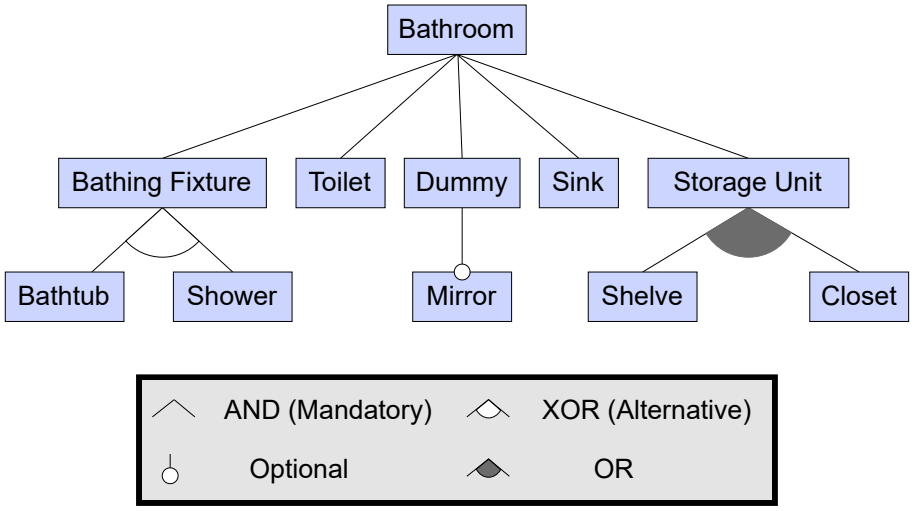


Figure 2.1: FM modelling a bathroom product line

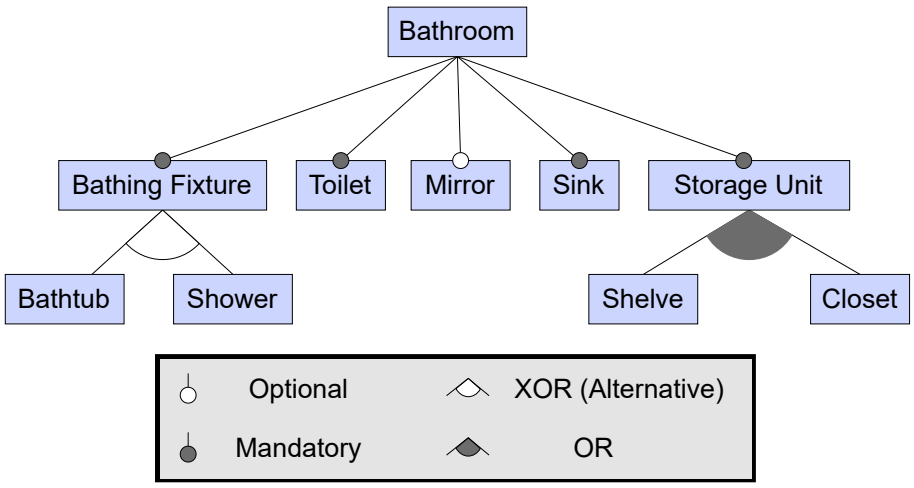


Figure 2.2: FM modelling a bathroom product line (compact form)

2.1.2 Formal Definition

We now collect the previously introduced elements into a compact mathematical definition. The following definitions fix the formal syntax of a FM as well as its configuration semantics.

Definition 2.1 (Boolean Functions)**.** Let $\mathbb{B} \coloneqq \{\text{true}, \text{false}\}$ be the set of Boolean truth values. For $n \in \mathbb{N}_0$, an n-ary Boolean function is a function $f : \mathbb{B}^n \to \mathbb{B}$.

For $n \in \mathbb{N}$, we define the following Boolean functions:

- $and_n(x_1, \ldots, x_n) \coloneqq x_1 \wedge \cdots \wedge x_n$
- $or_n(x_1, \ldots, x_n) \coloneqq x_1 \vee \cdots \vee x_n$
- $alternative_n(x_1, \ldots, x_n) \coloneqq \Big(x_1 \vee \cdots \vee x_n\Big) \wedge \bigwedge_{1 \leq i < j \leq n} \neg(x_i \wedge x_j)$
- $opt_n(x_1, \ldots, x_n) \coloneqq \text{true}$

For $n = 0$, each of these functions is defined to be the constant-true function:

$$or_0() \coloneqq alternative_0() \coloneqq and_0() \coloneqq opt_0() = \text{true}.$$

Definition 2.2 (Feature Model)**.** A *feature model* over a non-empty finite set F of features is a tuple

$$\mathcal{M} = (\prec_F, \lambda, \phi_R, \phi_X),$$

where:

- $\prec_F \subseteq F \times F$ is the *feature decomposition relation* such that $(F, \prec_F)$ is a directed tree with root $f_r \in F$.
- λ is a *relation-type function* that assigns to every feature $p \in F$ a Boolean function governing the selection of the children of p, i.e.,

  $$\lambda : F \to \bigcup_{k \in \mathbb{N}_0} \{and_k, or_k, alternative_k\} \cup \{opt_1\}.$$

  Moreover, the arity of $\lambda(p)$ has to match the number of children of p. In particular, $\lambda(p) = opt_1$ is only possible if p has exactly one child.
- $\phi_R \subseteq F \times F$ is the *require-edge relation*.
- $\phi_X \subseteq F \times F$ is the *exclude-edge relation*.

Together, $\prec_F$ and λ describe the *feature tree*, while ϕ_R and ϕ_X capture the *cross-tree constraints*.

As introduced in Section 2.1.1, a configuration of a feature model consists of the set of selected features. Formally, we define a configuration as follows.

Definition 2.3 (Configuration of a Feature Model)**.** Let $\mathcal{M} = (\prec_F, \lambda, \phi_R, \phi_X)$ be a feature model over the feature set F.

1. **Configuration.** A *configuration* of $\mathcal{M}$ is a mapping

   $$K : F \to \mathbb{B},$$

   where $K(f) = \text{true}$ means that feature f is selected.
2. **Valid configuration.** A configuration K is *valid* if all of the following conditions hold:

- Root constraint: $K(f_r) = \text{true}$.
- Hierarchy constraint: If $K(f) = \text{true}$ and $f \neq f_r$, then its parent feature p must also satisfy $K(p) = \text{true}$.
- Group constraint: For every feature p with children $f_1, \ldots, f_n$,
$$\lambda(p)\Big(K(f_1), \ldots, K(f_n)\Big) = \text{true}$$
whenever $K(p) = \text{true}$.
- Cross-tree constraints:
  - For each $(f, g) \in \phi_R$,
  $$K(f) \implies K(g).$$
  - For each $(f, g) \in \phi_X$,
  $$\neg\Big(K(f) \wedge K(g)\Big).$$

3. **Configuration space.** The *configuration space* of $\mathcal{M}$, denoted by $[\![\mathcal{M}]\!]$, is the set of all valid configurations of $\mathcal{M}$.

Technical Remarks

We conclude with several remarks clarifying modelling choices and technical subtleties underlying the above definitions.

Nullary operators and leaf features. For a leaf feature f (i.e. a feature without children), the concrete value of $\lambda(f)$ is irrelevant. By Definition 2.1, every nullary Boolean function evaluates to true and therefore imposes no additional constraint. Consequently, the particular relation type assigned to leaf features has no semantic impact on the configuration space.

Root selection. Requiring the root feature f_r to be selected in every valid configuration reflects the intended interpretation of a feature model: the root represents the overall system or concept. A configuration corresponds to a concrete product of this system and therefore necessarily includes the root feature [5, 9].

Cross-tree constraints. In our definition, cross-tree constraints are restricted to binary *require* and *exclude* relations. This restriction is primarily motivated by alignment with the existing literature and, in particular, by our later extension to cardinality-based feature model [5, 9, 26].

This restriction is not imposed by any inherent limitation of the formalism. In general, cross-tree constraints may be expressed as arbitrary propositional formulas over the set of features [8, 9, 27]. In that case, the validity condition for configurations requires that a configuration K satisfies all such formulas under the standard Boolean interpretation.

At first glance, allowing arbitrary propositional constraints appears strictly more expressive. However, Knüppel et al. [26] show that feature models with arbitrary propositional cross-tree constraints can be transformed into equivalent feature models that use only require and exclude constraints by introducing additional abstract (dummy) features. This transformation preserves the set of products but may increase the size of the model substantially.

2.2 Cardinality-Based Feature Models

Cardinality-based feature models (CFMs) generalize classical (Boolean) FMs by allowing features to occur multiple times. Whereas a configuration (i.e. product) of a FM decides only whether a feature is selected, a configuration of a CFM determines *how many instances* of each feature are present and *how* these instances are organized hierarchically.

In the following, we formally introduce CFMs. The formalization presented here follows the framework and notation of Weckesser et al. [10] and is consistent with the presentation in Güthing et al. [11]. We introduce minor notational adaptations where convenient for later developments.

As before, we begin with a running example that motivates the additional concepts required for CFMs.

2.2.1 A House Product Line

Assume we want to model different variants of a `House`. In contrast to the bathroom example from Section 2.1.1, a house exhibits an inherently repetitive structure. In particular, it is no longer sufficient to decide whether a feature is present or absent; instead, we must also express how many times it occurs.

For example, a house may contain multiple units (living units), and each unit may contain multiple rooms. Concretely, in our example product line, a house must satisfy the following requirements:

- Every `House` consists of exactly one `Heating System`, at least one `Unit`, and an arbitrary number of `Solar Panels`.
- Each `Unit` consists of between 1 and 3 `Living Rooms`, exactly one `Kitchen`, and between 1 and 3 `Bathrooms`, with the additional restriction that the total number of rooms is exactly 5.
- Each `Kitchen` may contain between 0 and 2 `Gas Stoves` and between 0 and 2 `Electric Stoves`. However, exactly one stove type must be selected, and for the selected type, between 1 and 2 instances must be present.
- The `Heating System` consists of either one `Heat Pump` or one `Gas Boiler`.
- If the total number of `Unit` instances is at least 3, then at least 5 `Solar Panel` instances must be present.
- If there is at least one `Gas Stove` in the entire house, then at least one `Gas Boiler` must be present.

Hierarchical Feature Structure

As in FMs, features are organized hierarchically in a finite rooted directed tree, called the *feature tree*. The crucial difference to FMs lies not in the tree itself, but in its interpretation; that is, in its configuration semantics.

In a FM, a configuration is simply a Boolean assignment $K\colon F \to \mathbb{B}$, deciding for each feature whether it is selected. In contrast, CFMs allow features to occur multiple times. Therefore, a configuration must not only specify *whether* a feature is present, but also *how many instances* of each feature occur and *how these instances are arranged hierarchically*. Accordingly, a configuration of a CFM consists of a finite *feature instance tree*. Its nodes are *feature instances*, and each feature instance is labelled by the feature it instantiates. The resulting feature instance tree thus makes explicit both the multiplicities of features and the hierarchical structure in which their instances are organized, that is, the concrete tree shape formed by the feature instances [2, 5, 14, 28].

To illustrate why this richer representation is necessary, consider the following example. Suppose a configuration contains two `Unit` instances. Each unit has its own local collection of rooms. If we only knew that there are four `Bathroom` instances and four `Living Room` instances in total, this information would be insufficient to reconstruct the structure of the configuration. We could not determine whether one unit contains three bathrooms and one living room while the other contains one bathroom and three living rooms, or whether both units contain two bathrooms and two living rooms each.

In other words, global multiplicities alone do not capture how feature instances are distributed among their parents. The feature instance tree makes this distribution explicit by representing the concrete parent-child relationships between feature instances.

This explicit structural information is essential for CFMs. However, moving from Boolean assignments to feature instance trees also makes the configuration semantics more involved. In particular, once configurations are modelled as trees of feature instances, one must carefully define what exactly constitutes a configuration and what it means for such a structure to be valid. As we will see, different reasonable interpretations are possible, which we will formalize and discuss in detail in Section 2.2.2 and Chapter 3.

Feature Groups and Feature Instance Groups

Since we now distinguish between the feature tree and the feature instance tree, it becomes necessary to also distinguish between *feature groups* and *feature instance groups*.

- A *feature group* consists of a feature p together with all its child features in the feature tree.
- A *feature instance group* consists of a concrete instance together with all of its child instances within a configuration. It is the counterpart of a feature group inside the feature instance tree.

These terms are introduced purely for convenience when referring to a parent feature (or feature instance) together with its respective children. In contrast to earlier definitions of CFMs, which distinguish between explicitly grouped features (i.e. features that participate in a group such as an *or*-group or a cardinality-constrained group) and isolated features [2, 28, 29], we always use the term *feature (instance) group* to denote the parent feature (or feature instance) together with all of its child features (or child instances).

Feature-Instance Cardinalities

The key extension from FMs to CFMs is that child features may occur multiple times. To express admissible multiplicities, CFMs use *cardinality intervals*. For example, "at least one `Unit`" corresponds to the interval $\langle 1, \infty \rangle$, whereas "exactly one `Heating System`" corresponds to $\langle 1, 1 \rangle$.

Formally, we introduce a function

$$\lambda_I^F \colon F \to \mathcal{I},$$

which assigns to each feature its corresponding cardinality interval, called its *feature-instance cardinality* (where $\mathcal{I}$ denotes the set of all cardinality intervals; see Definition 2.4).

A crucial property of feature-instance cardinalities is that they are interpreted *locally*. That is, the assigned interval must be satisfied within each *feature instance group*, i.e., for every concrete feature instance together with its child instances. This local interpretation is essential to express conditions such as "each `Kitchen` may contain between 0 and 2 `Gas Stoves`". If cardinalities were interpreted globally, we would need to know the total number of `Kitchen` instances in order to constrain the total number of `Gas Stoves`. Under a local interpretation, however, it suffices to assign `Gas Stove` the interval $\langle 0, 2 \rangle$.

Relations between Children

Similarly to FMs, we must also specify how the children of a feature relate to one another. However, in contrast to the Boolean setting of FMs, where such relations are expressed via Boolean functions such as *and* or *or*, CFMs use cardinality intervals.

For every *feature group*, that is, for every parent feature together with its child features, we introduce two types of cardinality constraints: the *group-type cardinality* and the *group-instance cardinality*. The group-type cardinality restricts how many distinct child feature types may be present, whereas the group-instance cardinality restricts the total number of child feature instances.

Formally, we define two functions

$$\lambda_T^G \colon F \to \mathcal{I} \qquad \text{and} \qquad \lambda_I^G \colon F \to \mathcal{I},$$

which assign to each parent feature its group-type and group-instance cardinality with respect to its children.

As with feature-instance cardinalities, these constraints are interpreted *locally*, that is, within each feature instance group. In the house example, the requirement that each `Unit` contains exactly five rooms in total is modelled by assigning the group-instance cardinality $\langle 5, 5 \rangle$. In contrast, the requirement that each `Kitchen` allows exactly one stove type is captured by setting its group-type cardinality to $\langle 1, 1 \rangle$.

Cross-Tree Constraints

Using the mechanisms introduced so far, we can express all hierarchical conditions of the house example. However, not all constraints are hierarchical. For instance,

the choice of the heating system may depend on whether any `Gas Stove` occurs in the house. Such dependencies are expressed by means of *cross-tree constraints*.

As in FMs, cross-tree constraints may relate arbitrary features in the feature tree, independently of their position in the hierarchy. They are again formulated using cardinality intervals. In contrast to feature-instance, group-type, and group-instance cardinalities, however, cross-tree constraints are interpreted *globally*. That is, they restrict the total number of instances of a feature across the entire configuration, regardless of where these instances occur in the feature instance tree.

In the house example, the condition "if there is at least one `Gas Stove` in the entire house, then there must be at least one `Gas Boiler`" is modelled as a *require* cross-tree constraint from `Gas Stove` to `Gas Boiler`, where both features are associated with the interval $\langle 1, \infty \rangle$.

Graphical Representation

Figure 2.3 shows the visual representation of the CFM modelling the house product line. In particular, it illustrates the feature-instance, group-type, and group-instance cardinalities, as well as the cross-tree constraints.

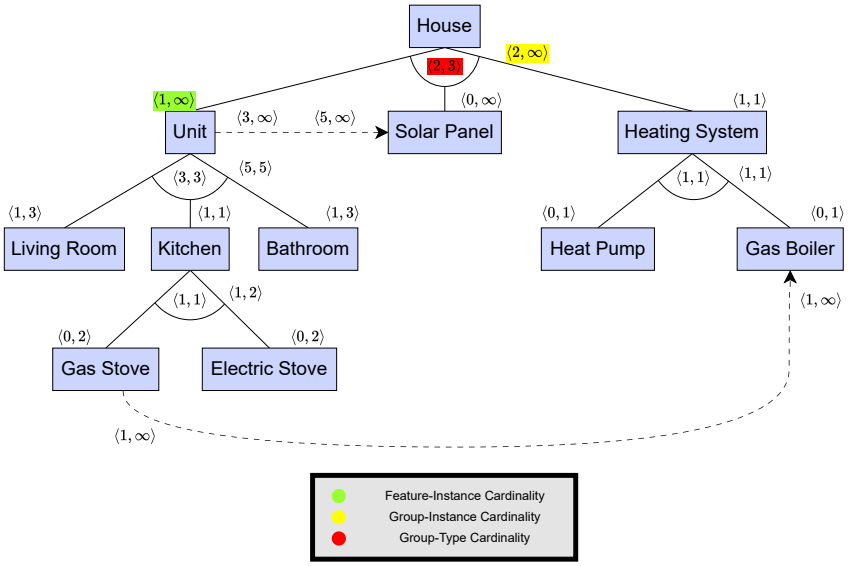


Figure 2.3: CFM modelling a house product line

2.2.2 Formal Definition

With the introduced concepts in Section 2.2.1, we can now formally define CFMs.

2.2.2.1 Cardinality Intervals

Before defining the syntax and configuration semantics, we first formalize cardinality intervals.

Definition 2.4 (Cardinality Interval)**.** A *simple cardinality interval* is a pair

$$\langle l, u \rangle \in \mathbb{N}_0 \times (\mathbb{N}_0 \cup \{\infty\}) \quad \text{with } l \leq u.$$

It represents the set

$$\{\, k \in \mathbb{N}_0 \mid l \leq k \leq u \,\}.$$

Intervals of the form $\langle l, \infty \rangle$ are called *unbounded*, since they admit an infinite number of cardinalities.

A *(compound) cardinality interval* is a non-empty finite set of simple cardinality intervals. We denote by $\mathcal{I}$ the set of all (compound) cardinality intervals.

In the following, we often identify a simple cardinality interval $\langle l, u \rangle$ with the singleton compound cardinality interval

$$\{\, \langle l, u \rangle \,\}.$$

Accordingly, whenever no ambiguity arises, we write $\langle l, u \rangle$ instead of $\{\, \langle l, u \rangle \,\}$. Since compound cardinality intervals are the more general notion and will be used throughout the remainder of this thesis, we usually adopt this simplified notation.

For a cardinality interval $I \in \mathcal{I}$, we additionally use the following notation:

$k \in I$	if there exists a simple interval $\langle l, u \rangle \in I$ such that $k \in \langle l, u \rangle$;
$\lvert I \rvert$	the number of distinct cardinalities admitted by I;
$\max(I)$	the maximum cardinality admitted by I.

Formally,

$$|I| \coloneqq \left|\{\, k \in \mathbb{N}_0 \mid k \in I \,\}\right| \in \mathbb{N}_0 \cup \{\infty\},$$

and

$$\max(I) \coloneqq \sup\{\, k \in \mathbb{N}_0 \mid k \in I \,\} \in \mathbb{N}_0 \cup \{\infty\}.$$

Expressiveness and Limitations

A cardinality interval $I \in \mathcal{I}$ represents a subset of $\mathbb{N}_0$ that can be written as a *finite union* of (simple) integer intervals. Equivalently, the set

$$\{\, k \in \mathbb{N}_0 \mid k \in I \,\}$$

has only finitely many "gaps", i.e., only finitely many non-convex regions. As a consequence, not every subset of $\mathbb{N}_0$ can be expressed as a compound cardinality interval. For example, the set of even numbers

$$\{\, 2n \mid n \in \mathbb{N}_0 \,\}$$

cannot be represented as a finite union of intervals, since between any two even numbers, there is an odd number, yielding infinitely many gaps.

However, this limitation has an important advantage. Since there are only finitely many gaps, there exists a number $k \in \mathbb{N}_0$ such that, if we restrict I to all cardinalities at most k, all non-convex regions of I are preserved.

Concretely, let

$$I = \{\, \langle l_1, u_1 \rangle, \ldots, \langle l_n, u_n \rangle \,\},$$

be a compound cardinality interval. For each simple interval $\langle l_i, u_i \rangle$, define

$$b_i \coloneqq \begin{cases} u_i & \text{if } u_i \in \mathbb{N}_0, \\ l_i & \text{if } u_i = \infty. \end{cases}$$

That is, for bounded intervals, we take their finite upper bound, and for unbounded intervals, we take their lower bound. Since there are only finitely many simple intervals, the set $\{b_1, \ldots, b_n\}$ is finite. We can therefore define

$$k \coloneqq \max\{\, b_1, \ldots, b_n \,\}.$$

By construction, k is at least as large as every relevant boundary value of I. In particular, replacing each unbounded interval $\langle l_i, \infty \rangle$ by the bounded interval $\langle l_i, k \rangle$ preserves all non-convex regions of I, since all changes between membership and non-membership occur at or below k.

This observation is important, since we will later use precisely this technique to handle CFMs whose configuration space is infinite. By bounding unbounded cardinality intervals in this manner, we obtain a model with finite configuration space while preserving all non-convex, and therefore interesting regions of the configuration space [10, 11].

2.2.2.2 Syntax of CFMs

With cardinality intervals formally defined, a CFM is defined as follows.

Definition 2.5 (Cardinality-Based Feature Model)**.** A *cardinality-based feature model* (CFM) over a non-empty finite set F of features is a tuple

$$\left(\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \phi_R, \phi_X\right),$$

where

- $\prec_F$ is the *feature decomposition relation* such that $(F, \prec_F)$ is a directed tree with root $f_r \in F$ (the feature tree);
- $\lambda_I^F : F \to \mathcal{I}$ is the feature-instance cardinality function;
- $\lambda_I^G : F \to \mathcal{I}$ is the group-instance cardinality function;
- $\lambda_T^G : F \to \mathcal{I}$ is the group-type cardinality function;
- $\phi_R \subseteq F \times \mathcal{I} \times \mathcal{I} \times F$ is the require-edge relation;
- $\phi_X \subseteq F \times \mathcal{I} \times \mathcal{I} \times F$ is the exclude-edge relation.

We furthermore require:

- $\lambda_I^F(f_r) = \langle 1, 1 \rangle$;
- if f is a leaf feature, then

$$\lambda_I^G(f) = \langle 0, 0 \rangle \quad \text{and} \quad \lambda_T^G(f) = \langle 0, 0 \rangle .$$

Together, $\prec_F$, λ_I^F, λ_I^G, and λ_T^G capture the hierarchical constraints, while ϕ_R and ϕ_X represent the cross-tree constraints.

2.2.2.3 Instance-Based Configurations

Configurations of a CFM must capture both the *multiplicity* of each feature and the *hierarchical distribution* of its instances in the resulting product. Accordingly, they consist of two components: a multiplicity function $M\colon F \to \mathbb{N}_0$, which assigns to each feature its number of feature instances, and a decomposition relation that describes the corresponding feature instance tree.

Definition 2.6 (Instance-Based Configuration)**.** Let $\mathcal{M} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \phi_R, \phi_X)$ be a CFM over a feature set F.

An *instance-based configuration* is a pair $(M, \prec)$ where:

- $M : F \to \mathbb{N}_0$ is a multiplicity function over F;
- $\mathrm{Inst}(M) \coloneqq \{\, f^1, \ldots, f^{M(f)} \mid f \in F,\ M(f) > 0 \,\}$ is the set of all feature instances;
- $\prec \subseteq \mathrm{Inst}(M) \times \mathrm{Inst}(M)$ is a binary relation on feature instances, called the *feature-instance decomposition relation*.

We denote by

$$\mathcal{C}_{\mathrm{Inst}}(\mathcal{M}) \coloneqq \Big\{\, (M, \prec) \mid M : F \to \mathbb{N}_0,\ \prec \subseteq \mathrm{Inst}(M) \times \mathrm{Inst}(M) \,\Big\}$$

the set of all instance-based configurations of $\mathcal{M}$. Furthermore, for a feature instance $x \in \mathrm{Inst}(M)$, we write $\mathrm{feat}(x)$ for the feature of which x is an instance, and $\mathrm{index}(x)$ for its associated index (i.e. $\mathrm{feat}(f^i) = f$ and $\mathrm{index}(f^i) = i$).

At this point, $\prec$ is an arbitrary relation on the instance set $\mathrm{Inst}(M)$. Structural well-formedness, in particular tree shape and compatibility with the feature tree $(F, \prec_F)$, and the satisfaction of all cardinality constraints are enforced only for *valid* configurations. Here, *compatibility with the feature tree* means that $\prec$ may only relate instances whose underlying features are in a parent-child relation in the feature tree: whenever $p^i \prec f^j$, we must have $p \prec_F f$.

As discussed in Section 2.2.1, hierarchical cardinality constraints are interpreted *locally* per feature instance group, whereas cross-tree constraints are interpreted *globally* over the multiplicity function M. In total, this leads to the following definition.

Definition 2.7 (Valid Instance-Based Configuration)**.** Let $\mathcal{M} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \phi_R, \phi_X)$ be a CFM over a feature set F. An instance-based configuration $(M, \prec)$ of $\mathcal{M}$ is valid if all of the following conditions hold:

- Root constraint: $M(f_r) = 1$.
- Hierarchy constraint: $(\mathrm{Inst}(M), \prec)$ forms a directed tree with root f_r^1, and if $p^i \prec f^j$, then $p \prec_F f$.
- Group constraints:

  - For each $p^i \in \mathrm{Inst}(M)$ and each $f \in F$ with $p \prec_F f$,

$$\Big|\{\, x \in \mathrm{Inst}(M) \mid p^i \prec x \wedge \mathrm{feat}(x) = f \,\}\Big| \in \lambda_I^F(f).$$

  - For each $p^i \in \mathrm{Inst}(M)$,

$$\Big|\{\, x \in \mathrm{Inst}(M) \mid p^i \prec x \,\}\Big| \in \lambda_I^G(p).$$

  - For each $p^i \in \mathrm{Inst}(M)$,

$$\Big|\{\, f \in F \mid \exists j : p^i \prec f^j \,\}\Big| \in \lambda_T^G(p).$$

- Cross-tree constraints:

  - For each $(f, I_f, I_g, g) \in \phi_R$,

$$M(f) \in I_f \Rightarrow M(g) \in I_g.$$

  - For each $(f, I_f, I_g, g) \in \phi_X$,

$$\neg\Big(M(f) \in I_f \wedge M(g) \in I_g\Big).$$

We denote by

$$[\![\mathcal{M}]\!]_{\mathrm{Inst}}$$

the set of all valid instance-based configurations of $\mathcal{M}$.

Figure 2.4 shows two valid and one invalid instance-based configurations of the house CFM introduced in Section 2.2.1. The configuration in Figure 2.4c is invalid because the second `Unit` instance violates the group-instance cardinality: its feature instance group contains only four room instances, whereas exactly five are required.

Remark on the formalisation. Our formalisation slightly refines the presentation of instance-based configurations by Weckesser et al. [10]. In particular, they describe an instance-based configuration as a pair $(M, \prec)$ with a relation

$$\prec \subseteq M \times M,$$

where M is a multiset of features.

Formally, however, a multiset M over F is a function

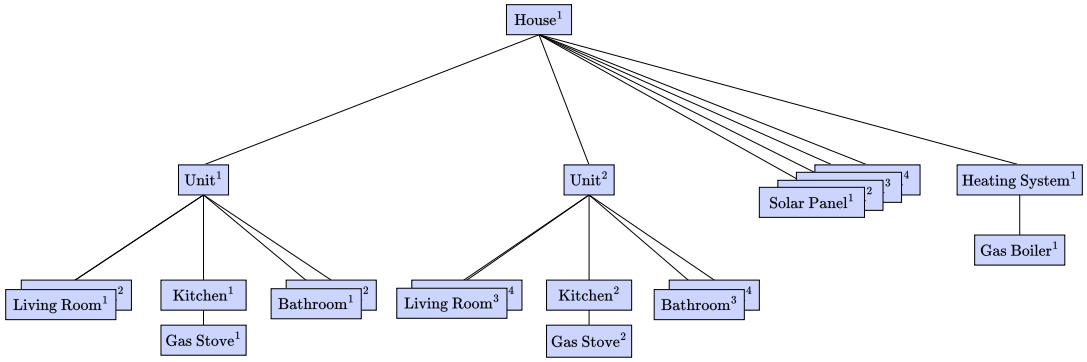
$$M \colon F \to \mathbb{N}_0$$

(see Section 2.4.2), which coincides exactly with our notion of a multiplicity function. Hence, from a set-theoretic perspective, M is the set

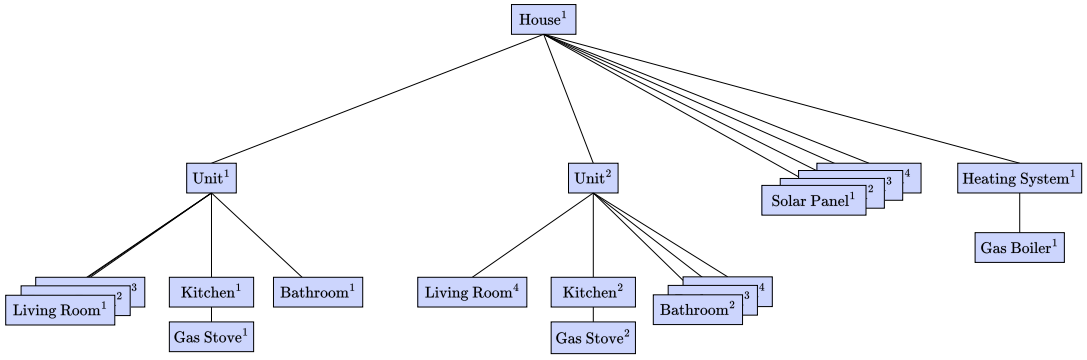
$$\{(f, M(f)) \mid f \in F\}.$$

The Cartesian product $M \times M$ would therefore consist of pairs of such tuples, i.e.,

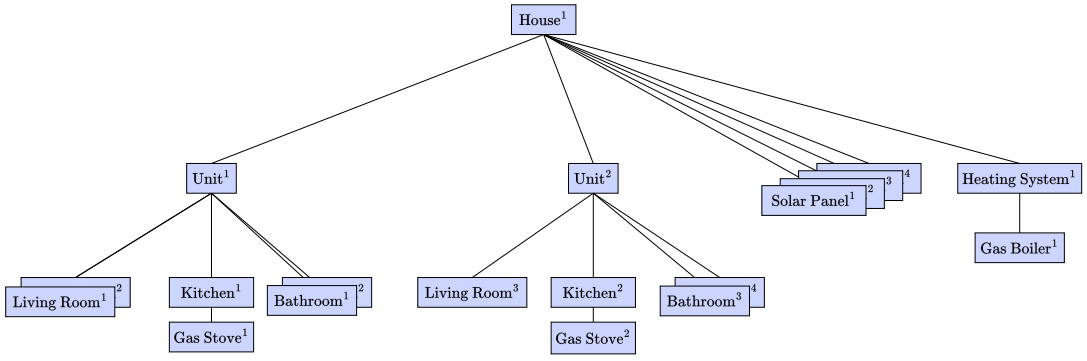
$$\Big((f, M(f)), (g, M(g))\Big),$$



(a) Valid instance-based configuration.



(b) Another valid instance-based configuration.



(c) Invalid instance-based configuration that violates the group-instance cardinality of `Unit` (only four rooms).

Figure 2.4: Example instance-based configurations of the house CFM: two valid configurations and one invalid configuration.

which does not correspond to the intended configuration semantics of Weckesser et al. In particular, the feature-instance decomposition relation is meant to relate individual feature instances rather than feature-multiplicity pairs.

To make this precise, we explicitly introduce the instance set

$$\mathrm{Inst}(M) \coloneqq \{\, f^1, \ldots, f^{M(f)} \mid f \in F,\ M(f) > 0 \,\},$$

which represents the actual feature instances generated by the multiplicity function M. The decomposition relation is then defined as

$$\prec \subseteq \mathrm{Inst}(M) \times \mathrm{Inst}(M).$$

2.2.2.4 Multiset-based Configurations

Instance-based configurations distinguish products that have the same total multiplicities but differ in the parent-child assignment of feature instances. Formally, they consist of both a multiplicity function M and a feature-instance decomposition relation $\prec$. In some settings, however, one may instead consider *multiset-based* (or *flat*) configurations, which abstract from the decomposition relation and record only the global feature multiplicities [11, 14, 28].

This abstraction trades semantic precision for improved scalability, e.g., to mitigate the combinatorial explosion of configuration spaces in sample-based testing [11]. Moreover, multiset-based configurations are closer to the Boolean notion of a feature-model configuration, which can simplify the adaptation of classical Boolean sampling criteria and algorithms to cardinality-based models [18].

Definition 2.8 (Multiset-Based Configuration)**.** Let $\mathcal{M} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \phi_R, \phi_X)$ be a CFM over a feature set F. A *multiset-based configuration* of $\mathcal{M}$ is a multiplicity function

$$M : F \to \mathbb{N}_0,$$

which we interpret as a multiset over F.

We denote by

$$\mathcal{C}_{\text{Multiset}}(\mathcal{M}) \coloneqq \left\{\, M \mid M : F \to \mathbb{N}_0 \,\right\}$$

the set of all multiset-based configurations of $\mathcal{M}$.

Definition 2.9 (Valid Multiset-Based Configuration)**.** Let $\mathcal{M} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \phi_R, \phi_X)$ be a CFM over a feature set F. A multiset-based configuration M of $\mathcal{M}$ is *valid* if there exists a relation

$$\prec \subseteq \mathrm{Inst}(M) \times \mathrm{Inst}(M),$$

such that $(M, \prec)$ is a valid instance-based configuration of $\mathcal{M}$.

We denote by

$$[\![\mathcal{M}]\!]_{\text{Multiset}}$$

the set of all valid multiset-based configurations of $\mathcal{M}$.

Although multiset-based configurations abstract from the explicit hierarchical structure, their validity remains inherently tied to instance-based configurations: a multiset-based configuration M is valid only if it can be extended to at least one valid instance-based configuration $(M, \prec)$.

For a given multiplicity function M, three situations may arise:

1. every instance-based configuration $(M, \prec)$ extending M is valid;
2. some instance-based configurations extending M are valid, while others are not;
3. no instance-based configuration extending M is valid.

Determining which of these situations holds for a given multiplicity function is, however, non-trivial. In particular, checking validity requires deciding the existence of a suitable feature-instance decomposition relation $\prec$ that satisfies all local hierarchical constraints. In contrast, the cross-tree constraints are evaluated directly over the multiplicity function M and can therefore be checked independently of any specific feature instance structure [11, 18, 28, 30].

Figures 2.4a and 2.4b show two valid instance-based configurations of the house CFM. In particular, both correspond to the same multiset-based configuration, since they have the same global multiplicities for every feature.

There is also a natural projection from instance-based configurations to multiset-based configurations that forgets the hierarchical decomposition and retains only global multiplicities. Formally, we define

$$\pi \colon \mathcal{C}_{\text{Inst}}(\mathcal{M}) \to \mathcal{C}_{\text{Multiset}}(\mathcal{M}), \quad (M, \prec) \mapsto M.$$

The map π is surjective but, in general, not injective, since different instance-based configurations may have the same multiplicity function.

2.2.3 Normalisation and Bounding

In contrast to FMs, the configuration semantics of CFMs is much richer and more expressive, which leads to multiple characteristics and behaviours that one has to deal with. Two of those characteristics, which are relevant for this thesis, are the potentially infinite configuration space of CFMs (regardless of whether one uses multiset-based or instance-based configurations), and the presence of dead cardinalities, which are a type of anomaly (i.e. unwanted behaviour) in CFMs.

Trivial Dead Cardinalities

Dead cardinalities are values that are syntactically permitted by some cardinality interval of a CFM, in particular, within a feature-instance, group-instance, or group-type cardinality, but are never realised in any valid configuration [10, 31]. Since cardinality intervals are interpreted *locally* for each group of feature instances, a dead cardinality is a value that cannot occur as the multiplicity of such a group in any valid configuration.

In general, detecting all dead cardinalities is non-trivial and typically requires the use of SMT, ILP, or CSP solvers [10, 31]. However, certain *trivial* dead cardinalities can be eliminated by imposing safe local upper bounds.

For a feature $f \in F$, let $C_{\mathcal{M}}(f)$ denote the set of child features of f in the feature tree (see Definition 3.4). Then the following bounds can be enforced without changing the set of valid configurations:

1. $\max\Big(\lambda_I^G(f)\Big) \;\leq\; \sum_{c \in C_{\mathcal{M}}(f)} \max\Big(\lambda_I^F(c)\Big)$,
2. $\max\Big(\lambda_I^F(c)\Big) \;\leq\; \max\Big(\lambda_I^G(f)\Big)$ for every child $c \in C_{\mathcal{M}}(f)$,
3. $\max\Big(\lambda_T^G(f)\Big) \;\leq\; |C_{\mathcal{M}}(f)|$.

These bounds are sound because, in any valid configuration:

1. the number of instances in a group cannot exceed the sum of the maximum instances of its children,
2. the number of instances of a child feature cannot exceed the maximum group-instance cardinality of its parent, and
3. the number of distinct feature types selected in a group cannot exceed the number of available children.

Since the first two bounds constrain each other, tightening one may enable further tightening of the other. Therefore, these bounds are applied iteratively until a fixed point is reached, i.e., until no upper bound can be reduced further.

Big-M Bounding

Even after eliminating trivial dead cardinalities, a CFM may still have an infinite configuration space due to unbounded intervals of the form $\langle l, \infty \rangle$ [11]. To obtain a finite model suitable for sampling, we apply a *Big-M* transformation [10].

The idea is to replace every occurrence of ∞ in a feature-instance cardinality by a sufficiently large finite bound M so that all *non-convex* regions of the configuration space, which are typically the most interesting ones, are preserved [11].

Computation of the bound. For each feature $f \in F$, we compute a feature-specific upper bound M_f.

Let

$$N_f \coloneqq \Big|\{\, g \in F \mid g \text{ is a descendant of } f \text{ or } g = f \text{ in } (F, \prec_F)\,\}\Big|$$

denote the number of features in the subtree rooted at f (including f itself).

We define

$$M_f \coloneqq \max\Big(1,\ \operatorname{convex}\big(\lambda_I^F(f)\big),\ \operatorname{convex}\big(\lambda_I^G(f)\big),\ \operatorname{convex}\big(\lambda_T^G(f)\big),\ \max_{(g,I_g,I_h,h)\in\phi_R\cup\phi_X} \max\big(\operatorname{convex}(I_g),\ \operatorname{convex}(I_h)\big)\cdot N_f\Big),$$

where $\operatorname{convex}(I)$ denotes the non-convex bound of the cardinality interval I, i.e., the largest boundary required to preserve all non-convex parts of I (see Section 2.2.2.1).

Every simple interval of the form $\langle l, \infty\rangle$ in the feature-instance cardinality of f is then replaced by $\langle l, M_f\rangle$. After bounding, trivial dead cardinalities may again be eliminated, propagating the finite feature-instance bounds to group-instance and group-type cardinalities. This step is optional, however, since finite feature-instance cardinalities already ensure a finite configuration space.

Rationale. The first three components in the definition of M_f ensure that every non-convex boundary occurring in feature-instance, group-instance, or group-type cardinalities of f is preserved. Hence, all interval gaps and boundary effects local to f remain observable in the bounded model.

Cross-tree constraints, however, are interpreted globally. To preserve their non-convex behaviour, we must ensure that sufficiently many instances of f can be generated so that all relevant boundary combinations can be realised independently within its subtree.

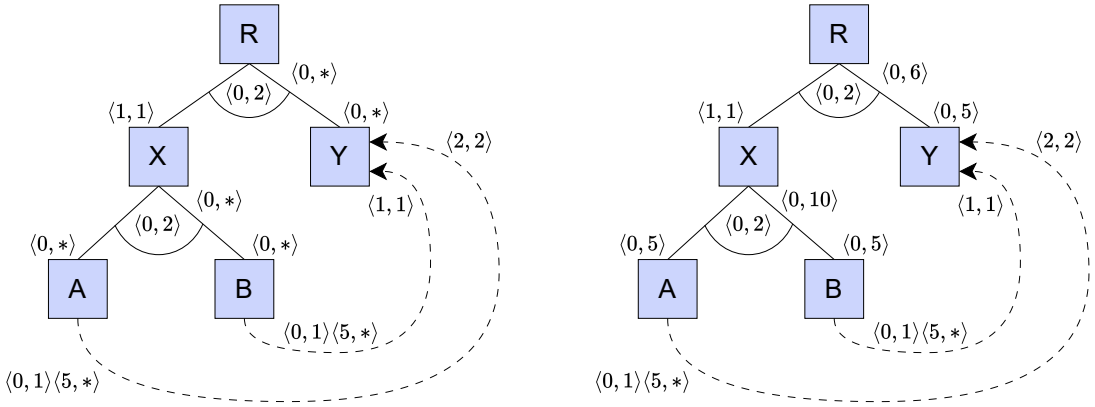
Let B denote the maximum non-convex bound occurring in any cross-tree constraint. In the worst case, each feature in the subtree of f may contribute independently to satisfying such constraints. Therefore, allowing at least $B \cdot N_f$ instances of f guarantees that every non-convex boundary induced by cross-tree constraints remains reachable in the bounded model.

The constant 1 ensures that at least one instance of every feature can be selected, even in pathological cases where all feature-instance, group-instance, and group-type cardinalities are of the form $\langle 0, \infty\rangle$ and thus $\operatorname{convex}(I) = 0$.

Relation to existing Big-M approaches. Güthing et al. [11] approximate a safe global upper bound M by multiplying the upper bounds of all feature-instance cardinalities along branches that contain no ∞, and taking the maximum over all such branches. This bound is then used to replace unbounded intervals.

This construction presupposes the existence of at least one fully bounded root-to-leaf branch. In cases where every branch contains an unbounded cardinality interval, the branch-based multiplication does not directly yield a finite bound. Moreover, the approach does not explicitly incorporate global cross-tree constraints when determining M.

Figure 2.5 illustrates the effect of our proposed Big-M transformation. In this example, no fully bounded branch exists. Furthermore, considering only the local



(a) Unbounded CFM. (b) Big-M bounded CFM.

Figure 2.5: Big-M bounding of a CFM. The unbounded model (a) has an infinite configuration space. After bounding (b), the configuration space becomes finite. The bound must account not only for feature-instance and group cardinalities, but also for cross-tree constraints.

feature-instance and group cardinalities would render certain non-convex regions induced by cross-tree constraints unreachable.

2.3 Uniform Random Sampling Techniques

Uniform random sampling aims to generate configurations from the configuration space of a (cardinality-based) feature model such that each valid configuration is selected with equal probability [18, 32, 33]. Formally, given a (cardinality-based) feature model $\mathcal{M}$ with configuration space $[\![\mathcal{M}]\!]$, a sampler is *uniform* if it produces each configuration $K \in [\![\mathcal{M}]\!]$ with probability

$$\mathbb{P}(K) = \frac{1}{|[\![\mathcal{M}]\!]|}.$$

In the following, we first survey the most common techniques for (Boolean) FMs. We then turn to CFMs.

2.3.1 Feature Models

For classical (Boolean) FMs, a configuration corresponds to a set of selected features satisfying all hierarchical and cross-tree constraints. In particular, uniform random sampling can be reduced directly to uniform sampling of satisfying assignments of Boolean formulas. This observation implies that feature-model configuration sampling is not restricted to FM-specific techniques. Instead, it can leverage the substantial body of research on uniform random sampling for SAT problems [20, 33, 34, 35, 36].

Two of the most common approaches are the following:

Counting-based samplers. Counting-based samplers exploit the close relationship between uniform random sampling and model counting, i.e., computing the number of satisfying assignments of a problem [17, 36, 37, 38, 39]. Prominent examples include `SPUR` [37] and `SMARCH` [36], both of which rely on #SAT solvers for SAT-based problems. The #SAT problem is the counting version of SAT: instead of deciding whether a Boolean formula is satisfiable, it asks how many satisfying assignments exist [38, 40].

The key idea is to first count the number of valid solutions and then use this information to map each solution to a unique integer. Sampling can then be performed by selecting an integer uniformly at random and reconstructing the corresponding solution. In mathematics and combinatorics, this technique is commonly known as *ranking and unranking* (see Section 2.4.3).

Hashing-based samplers. Hashing-based samplers partition the solution space into random buckets (or cells) by adding randomly generated XOR constraints. Once a bucket of sufficiently small size is identified, its solutions are enumerated, and a sample is drawn uniformly from within that bucket [17, 33, 41, 42].

The most prominent hashing-based sampler is the `UniGen` family, which includes the original `UniGen` solver [41], `UniGen2` [43], and `UniGen3` [42].

This classification is not strict. For instance, solvers such as `UniGen` internally rely on approximate model counters like `ApproxMC` [42, 43, 44].

Moreover, several other approaches do not neatly fall into either category. For example, `QuickSampler` [45] generates solutions by heuristically mutating existing ones. Other methods rely on Markov Chain Monte Carlo (MCMC) techniques to explore the solution space probabilistically [45, 46].

These techniques can also be classified according to the strength of the uniformity guarantees they provide. Counting-based samplers such as `SPUR` and `SMARCH` achieve perfect uniformity. Hashing-based samplers like the `UniGen` family provide strong theoretical guarantees of almost-uniformity, although they do not achieve perfect uniformity. In contrast, heuristic-based samplers such as `QuickSampler` offer no formal uniformity guarantees, but can generate samples significantly faster [33, 41, 45, 47].

2.3.2 Cardinality-Based Feature Models

For cardinality-based feature models, uniform random sampling can be pursued in two principal ways. One approach relies on general-purpose solver technology, encoding the model into a formalism such as SMT, ILP, or CSP and applying existing sampling or counting techniques. Alternatively, one may design CFM-specific algorithms that operate directly on the hierarchical structure and cardinality constraints of the model.

2.3.2.1 General-Purpose Solver-Based Approaches

While classical FMs and their configuration semantics can be encoded directly as propositional formulas, CFMs require substantially more involved encodings. In particular, such encodings must faithfully capture cardinality interval constraints,

feature instances, and tree structures. Consequently, CFMs are typically translated into formalisms supported by SMT, ILP, or CSP solvers [11, 18, 31, 48, 49].

Although model counting and uniform random sampling techniques have been proposed for SMT, ILP, and even general CSP problems, this line of research remains significantly less mature than the SAT setting [50, 51, 52, 53, 54]. Moreover, we are not aware of any existing work that combines these techniques with CFMs.

Because of this, we only employ a naive CSP reference implementation in our implementation and evaluation (see Chapters 6 and 7). This baseline does not rely on any sophisticated solving or sampling techniques; instead, it merely supports the straightforward computation and enumeration of all solutions. The concrete encoding is described in Chapter 6 and is based on the formulations presented by Zhang [18] and Eger [21].

2.3.2.2 CFM-Specific Approach

In addition to solver-based encodings, Zhang [18] proposes a CFM-specific naive sampling algorithm that does not rely on an SMT or CSP solver. Instead, it generates instance-based configurations directly by recursively constructing a feature instance tree from the root feature downwards. Conceptually, the algorithm performs a randomized top-down traversal of the feature tree, making random decisions for each feature instance group encountered during the construction. Both local hierarchical constraints and global cross-tree constraints are enforced via rejection sampling [55]: whenever a violation is detected during the construction process, the current partial or complete configuration is discarded, and the corresponding step is repeated. The full algorithm is shown in Algorithms 2.1 and 2.2.

Starting at the root feature f_r, the procedure GenerateChildren first creates one instance of the current feature and then determines a random number of distinct child feature types according to the group-type cardinality $\lambda_T^G(f)$ that should occur in the configuration. This number k_T^G is then used to select a random set of k_T^G children of f, where mandatory children (i.e. children c for which $0 \notin \lambda_I^F(c)$) are always included.

For each selected child feature f', a random feature instance multiplicity is drawn from the feature-instance cardinality $\lambda_I^F(f')$. The sum of these multiplicities is validated against the group-instance cardinality $\lambda_I^G(f)$. If the chosen combination violates the group-instance constraint, the process is repeated until a valid combination is obtained. For every selected child instance, the procedure is then applied recursively.

This recursive construction guarantees that all *local* hierarchical constraints, namely feature-instance, group-type, and group-instance cardinalities, are satisfied by construction. Cross-tree constraints, however, are global and may depend on multiplicities accumulated across the entire configuration. Therefore, once a complete configuration has been generated, it is checked against the cross-tree constraints. If it violates a require or exclude constraint, it is discarded, and the generation process is repeated.

Limitations and Possible Improvements

Although simple and solver-independent, this approach does not guarantee uniform sampling. In particular, cardinalities are chosen uniformly at random from the ad-

missible intervals, even though different values may induce vastly different numbers of valid configurations. As a consequence, configurations associated with certain multiplicities can be over- or under-represented.

To mitigate this effect, Zhang [18] proposes a simple heuristic that prefers larger cardinalities to smaller ones, based on the intuition that larger values often admit more configurations.

However, as we show in Chapter 3, the algorithm of Zhang is not suitable for generating instance-based configurations. Instead, it produces what we call *semi-structural configurations*, a new configuration type introduced in this thesis. Nevertheless, the central idea of weighting choices according to the number of compatible configurations can be retained and refined. In particular, we adopt this idea and develop it into a provably uniform random sampling algorithm in Chapter 4.

Algorithm 2.1 NaiveSample($\mathcal{M}, f_r, n$): Algorithm for naive random sampling in a CFM based on recursive construction and rejection sampling

Input:
- CFM $\mathcal{M}$ with root feature f_r
- Number n of configurations to sample

Output: Sample $\mathcal{S}$ of n configurations

```
1: S ← ∅
2: for i = 1 to n do
3:     while true do
4:         M(f) ← 0 for all f ∈ F
5:         ≺ ← ∅
6:         K ← GenerateChildren(M, f_r, (M, ≺))
7:         if validate(K) then
8:             S ← S ∪ {K}
9:             break
10:        end if
11:    end while
12: end for
13: return S
```

Algorithm 2.2 GenerateChildren($\mathcal{M}, f, K$): Algorithm for naive random generation of child feature instances in a CFM

Input:
- CFM $\mathcal{M} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \phi_R, \phi_X)$
- Feature f
- (Partial) configuration $K = (M, \prec)$

Output: Extended (partial) configuration K

1: $M(f) \leftarrow M(f) + 1$ ▷ add one instance of f to K
2: **if** f has no children **then**
3: **return** K
4: **end if**

5: **while** true **do**
6: $k_T^G \leftarrow$ RandomCardinality($\lambda_T^G(f)$)
7: $R \leftarrow \emptyset$
8: $k_I^G \leftarrow 0$
9: $F' \leftarrow$ RandomSubsetWithMandatory(k_T^G, $\{f' \mid f \prec_F f'\}$)

10: **for all** $f' \in F'$ **do**
11: $r \leftarrow$ RandomCardinality($\lambda_I^F(f')$)
12: $k_I^G \leftarrow k_I^G + r$
13: $R \leftarrow R \cup \{(f', r)\}$
14: **end for**

15: **if** $k_I^G \in \lambda_I^G(f)$ **then**
16: **break**
17: **end if**
18: **end while**

19: $p \leftarrow f^{M(f)}$ ▷ feature instance to which we append children
20: **for all** $(f', r) \in R$ **do**
21: **for** $j = 1$ to r **do**
22: $(M, \prec) \leftarrow$ GenerateChildren($\mathcal{M}, f', K$)
23: $k \leftarrow M(f')$
24: $\prec \leftarrow \prec \cup \{(p, f'^k)\}$
25: $K \leftarrow (M, \prec)$
26: **end for**
27: **end for**
28: **return** K

2.4 Combinatorics

This section collects fundamental combinatorial facts and definitions concerning binomial coefficients and multisets that are used throughout this thesis. Furthermore, we introduce the concepts of ranking and unranking of combinatorial objects. Unless stated otherwise, the definitions and results are adopted from Mazur [56], Stanley [57], Mladenović [58], and Henze [59].

2.4.1 Binomial Coefficients

We begin with binomial coefficients, which play a central role in counting combinations and multisets. A *k-combination* of an n-element set is a subset of cardinality k, and the number of such subsets is given by the binomial coefficient $\binom{n}{k}$.

Definition 2.10 (Binomial Coefficient)**.** For $n, k \in \mathbb{N}_0$, the *binomial coefficient* $\binom{n}{k}$ is defined as

$$\binom{n}{k} := \begin{cases} \dfrac{n!}{k!\,(n-k)!}, & \text{if } k \leq n, \\ 0, & \text{if } k > n. \end{cases}$$

In particular,

$$\binom{n}{0} = 1 \quad \text{for all } n \in \mathbb{N}_0, \qquad \text{and} \qquad \binom{0}{0} = 1.$$

Lemma 2.11 (Basic Identities of Binomial Coefficients)**.** For all $n, k \in \mathbb{N}_0$ with $k \leq n$, the following properties hold:

1. **Symmetry:**
$$\binom{n}{k} = \binom{n}{n-k}.$$

2. **Pascal's identity:** For $1 \leq k \leq n$,
$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}.$$

3. **Monotonicity in n:** For fixed $k \in \mathbb{N}_0$, the function
$$n \mapsto \binom{n}{k}$$
is non-decreasing for all $n \geq k$. Moreover, it is strictly increasing for all $n \geq k$ when $k \geq 1$.

4. **Multiplicative formula:** For $n \geq k$,
$$\binom{n+1}{k} = \binom{n}{k} \cdot \frac{n+1}{n+1-k}.$$

Proof. **(1)** Using the factorial definition,

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} = \frac{n!}{(n-k)!k!} = \binom{n}{n-k}.$$

(2) We distinguish cases.

If $k = n$, then

$$\binom{n-1}{k} + \binom{n-1}{k-1} = \binom{n-1}{n} + \binom{n-1}{n-1} = 0 + 1 = 1 = \binom{n}{n} = \binom{n}{k}.$$

Now assume $1 \leq k < n$. Then

$$\begin{aligned}\binom{n-1}{k} + \binom{n-1}{k-1} &= \frac{(n-1)!}{k!\,(n-1-k)!} + \frac{(n-1)!}{(k-1)!\,(n-k)!} \\ &= \frac{(n-1)!}{k!\,(n-k)!}\Big((n-k)+k\Big) \\ &= \frac{n!}{k!\,(n-k)!} = \binom{n}{k}.\end{aligned}$$

(3) If $k = 0$, then

$$\binom{n}{0} = 1$$

for all $n \in \mathbb{N}_0$, and hence the function is constant and therefore non-decreasing.

Now let $k \geq 1$ and $n \geq k$. Then

$$\frac{\binom{n+1}{k}}{\binom{n}{k}} = \frac{n+1}{n+1-k}.$$

Since $k \geq 1$, we have $n+1 > n+1-k$, and therefore

$$\frac{\binom{n+1}{k}}{\binom{n}{k}} > 1.$$

Thus $\binom{n+1}{k} > \binom{n}{k}$ for all $n \geq k$, which shows that the function is strictly increasing in n for $k \geq 1$.

(4) For $n \geq k$, we compute

$$\begin{aligned}\binom{n+1}{k} &= \frac{(n+1)!}{k!\,(n+1-k)!} \\ &= \frac{(n+1)n!}{k!\,(n+1-k)(n-k)!} \\ &= \frac{n+1}{n+1-k} \cdot \frac{n!}{k!\,(n-k)!} = \binom{n}{k} \cdot \frac{n+1}{n+1-k}.\end{aligned}$$

□

2.4.2 Multisets

Multisets generalize ordinary sets by allowing elements to occur multiple times. Formally, a multiset over a finite set assigns to each element a non-negative integer representing its multiplicity.

Definition 2.12 (Multiset)**.** Let A be a finite set with $n \in \mathbb{N}_0$ elements. A *multiset* over A is a function

$$M \colon A \to \mathbb{N}_0,$$

where $M(a)$ denotes the *multiplicity* of $a \in A$. The *size* (or *cardinality*) of M is defined as

$$|M| \coloneqq \sum_{a \in A} M(a).$$

A multiset M with $|M| = k \in \mathbb{N}_0$ is also called a *k-multiset of A*.

For $k \in \mathbb{N}_0$, we denote by

$$\left(\!\binom{n}{k}\!\right),$$

the number of k-multisets over A. It is well known that

$$\left(\!\binom{n}{k}\!\right) = \begin{cases} 1, & \text{if } n = 0 \text{ and } k = 0, \\ 0, & \text{if } n = 0 \text{ and } k > 0, \\ \binom{n+k-1}{k}, & \text{if } n > 0. \end{cases}$$

When explicitly listing the elements of a multiset, we use double curly braces $\{\!\{ \}\!\}$ to distinguish multisets from ordinary sets. For example,

$$\{\!\{a, a, b\}\!\}$$

denotes the multiset in which a has multiplicity 2 and b has multiplicity 1. In general, an element $a \in A$ appears exactly $M(a)$ times inside the double braces.

Besides defining a multiset as a function $M : A \to \mathbb{N}_0$, it admits several equivalent representations that are useful in different contexts. In particular, for the definition of instance-based configurations (Definition 2.6), we already introduced the *instance set representation*, which contains exactly $M(a)$ distinguishable copies of every element $a \in A$.

Definition 2.13 (Multiset Representations)**.** Let A be a finite set and let $M : A \to \mathbb{N}_0$ be a multiset over A. We define the following equivalent representations of M:

1. **Instance set representation.** The *instance set* of M is defined as

$$\mathrm{Inst}(M) \;\coloneqq\; \left\{ a^1, \ldots, a^{M(a)} \;\middle|\; a \in A,\ M(a) > 0 \right\},$$

   where a^t denotes the t-th *instance* of the element $a \in A$.

For the remaining representations, we assume that the underlying set A is equipped with a total order $\leq_A$:

$$A = \{a_1, \ldots, a_n\} \quad \text{with} \quad a_1 \leq_A \cdots \leq_A a_n.$$

2. **Multiplicity-vector representation.** The *multiplicity vector* of M is the tuple
$$(M(a_1), \ldots, M(a_n)) \in \mathbb{N}_0^n,$$
which records the multiplicity of each element of A according to the fixed order.

3. **Ordered-sequence representation.** The *ordered-sequence representation* of M is the unique non-decreasing sequence
$$(x_1, \ldots, x_{|M|}) \in A^{|M|}$$
in which each element $a \in A$ occurs exactly $M(a)$ times. In contrast to the multiplicity-vector representation, this sequence omits zero-multiplicity elements.

4. **Compressed ordered-sequence representation.** The ordered-sequence representation can be compressed by grouping consecutive identical elements. Since the sequence
$$(x_1, \ldots, x_{|M|})$$
is non-decreasing, identical elements appear in consecutive blocks. In particular, it can be written in the form
$$(\underbrace{a_{i_1}, \ldots, a_{i_1}}_{m_1}, \underbrace{a_{i_2}, \ldots, a_{i_2}}_{m_2}, \ldots, \underbrace{a_{i_t}, \ldots, a_{i_t}}_{m_t}),$$
where
$$a_{i_1} <_A a_{i_2} <_A \cdots <_A a_{i_t} \quad \text{and} \quad m_j \geq 1.$$
Recording only the distinct elements together with their multiplicities yields the compressed ordered-sequence representation:
$$(a_{i_1}, m_1), \ldots, (a_{i_t}, m_t),$$
where each a_{i_j} is distinct and m_j denotes its multiplicity.

Every representation in Definition 2.13 has different strengths and weaknesses in certain situations. In particular, we use the multiplicity-vector representation to calculate the expected number of times a particular element appears in a uniformly generated k-multiset, which we later use for the analysis of the sampling algorithms (see Section 5.2).

Lemma 2.14 (Expected Multiplicity in a Uniform Random Multiset)**.** For $n \in \mathbb{N}$, let $A = \{a_1, \ldots, a_n\}$ be a finite set and let $k \in \mathbb{N}_0$. Let X be a random k-multiset on A drawn uniformly from the set of all k-multisets over A. Thus,
$$\mathbb{P}(X = M) = \frac{1}{\binom{n+k-1}{k}},$$
for every k-multiset M on A. Then, for every $j \in \{1, \ldots, n\}$,
$$\mathbb{E}[X(a_j)] = \frac{k}{n},$$
where $X(a_j)$ denotes the multiplicity of a_j in the random multiset X.

Proof. A k-multiset M on $A = \{a_1, \dots, a_n\}$ is uniquely represented by the vector

$$(M(a_1), \dots, M(a_n)) \in \mathbb{N}_0^n \quad \text{with} \quad \sum_{j=1}^{n} M(a_j) = k.$$

Conversely, every vector $(v_1, \dots, v_n) \in \mathbb{N}_0^n$ with $\sum_{j=1}^{n} v_j = k$ corresponds to a k-multiset over A. Hence, a uniformly distributed random k-multiset X induces a uniformly distributed random vector

$$\Big(X(a_1), \dots, X(a_n)\Big)$$

on the set

$$\Big\{(v_1, \dots, v_n) \in \mathbb{N}_0^n \;\Big|\; v_1 + \cdots + v_n = k\Big\}.$$

The distribution of the random vector

$$\Big(X(a_1), \dots, X(a_n)\Big)$$

is invariant under permutations of the coordinates, since every vector $(v_1, \dots, v_n) \in \mathbb{N}_0^n$ with $v_1 + \cdots + v_n = k$ occurs with the same probability. In particular, for any permutation π of $\{1, \dots, n\}$,

$$\Big(X(a_1), \dots, X(a_n)\Big) \overset{d}{=} \Big(X(a_{\pi(1)}), \dots, X(a_{\pi(n)})\Big).$$

Taking expectations yields

$$\Big(\mathbb{E}[X(a_1)], \dots, \mathbb{E}[X(a_n)]\Big) = \Big(\mathbb{E}[X(a_{\pi(1)})], \dots, \mathbb{E}[X(a_{\pi(n)})]\Big),$$

and hence

$$\mathbb{E}[X(a_1)] = \mathbb{E}[X(a_2)] = \cdots = \mathbb{E}[X(a_n)].$$

Using linearity of expectation, we obtain

$$k = \mathbb{E}\left[\sum_{j=1}^{n} X(a_j)\right] = \sum_{j=1}^{n} \mathbb{E}[X(a_j)] = n \cdot \mathbb{E}[X(a_j)],$$

for any $j \in \{1, \dots, n\}$, and therefore

$$\mathbb{E}[X(a_j)] = \frac{k}{n}.$$

□

In addition to the expected number of times a certain element appears in a k-multiset, we also need the probability that a certain element appears at all.

Lemma 2.15 (Presence Probability in a Uniform Random Multiset)**.** For $n \in \mathbb{N}$, let $A = \{a_1, \dots, a_n\}$ be a finite set and let $k \in \mathbb{N}_0$. Let X be a random k-multiset on A drawn uniformly from the set of all k-multisets over A. Thus,

$$\mathbb{P}(X = M) = \frac{1}{\binom{n+k-1}{k}},$$

for every k-multiset M on A. Then, for every $j \in \{1, \ldots, n\}$,

$$\mathbb{P}\Big(X(a_j) > 0\Big) = \begin{cases} 0, & \text{if } k = 0, \\ \dfrac{k}{n+k-1}, & \text{if } k \geq 1. \end{cases},$$

where $X(a_j)$ denotes the multiplicity of a_j in the random multiset X.

Proof. If $k = 0$, then X is the empty multiset with probability 1, hence

$$\mathbb{P}\Big(X(a_j) > 0\Big) = 0.$$

Now assume $k \geq 1$. The total number of k-multisets over A is

$$\binom{n+k-1}{k}.$$

A k-multiset M satisfies $M(a_j) = 0$ if and only if all k elements are chosen from the remaining $n-1$ elements of $A \setminus \{a_j\}$. The number of such multisets is therefore

$$\binom{(n-1)+k-1}{k} = \binom{n+k-2}{k}.$$

Since X is uniformly distributed over all k-multisets on A, we obtain

$$\mathbb{P}\Big(X(a_j) = 0\Big) = \frac{\binom{n+k-2}{k}}{\binom{n+k-1}{k}}.$$

If $n = 1$, then

$$\binom{n+k-2}{k} = \binom{k-1}{k} = 0,$$

so

$$\mathbb{P}\Big(X(a_j) > 0\Big) = 1 = \frac{k}{1+k-1} = \frac{k}{k}.$$

Now assume $n \geq 2$. Then

$$\begin{aligned} \mathbb{P}\Big(X(a_j) > 0\Big) = 1 - \frac{\binom{n+k-2}{k}}{\binom{n+k-1}{k}} &= 1 - \frac{(n+k-2)!}{(n-2)!\,k!} \cdot \frac{(n-1)!\,k!}{(n+k-1)!} \\ &= 1 - \frac{1}{n+k-1} \cdot (n-1) \\ &= \frac{n+k-1}{n+k-1} - \frac{n-1}{n+k-1} = \frac{k}{n+k-1}. \end{aligned}$$

□

2.4.3 Ranking and Unranking of Combinatorial Objects

An important tool that we will later employ in the development of algorithms for uniform random sampling in CFMs is the concept of *ranking* and *unranking*. In combinatorics and computer science, *ranking* refers to the task of computing the *rank* (i.e. an index) of an object drawn from a finite set of combinatorial objects S, where we assume that a fixed order on S has been chosen. For example, S may denote the set of all permutations of a given length, the set of all combinations of fixed cardinality drawn from a finite set, or the set of all trees with a fixed number of nodes. Formally, ranking establishes a bijection between S and the rank set $\{0, \ldots, |S| - 1\}$ by assigning each object its position in the ordering. The inverse task, called *unranking*, consists of reconstructing the unique object corresponding to a given rank and thus allows one to recover combinatorial objects from their ranks [60, 61, 62, 63].

While ranking and unranking are in principle possible for any finite set of objects and any fixed order by means of a global look-up table or a sorted list containing all objects, the central algorithmic challenge lies in the development of *efficient* ranking and unranking algorithms that exploit the structural properties of the underlying combinatorial objects [60, 64, 65].

As discussed by Myrvold and Ruskey [66], the existence of efficient ranking and unranking algorithms, as well as their computational and implementation complexity, depends crucially on the a priori order imposed on the combinatorial objects. For example, ranking and unranking permutations in lexicographic order is closely tied to the Lehmer code of a permutation, whose i-th entry counts the number of entries to the right of position i that are smaller than the entry in position i [66, 67]. This leads to a time complexity of $O(n^2)$ for naive implementations and $O(n \log n)$ when more sophisticated data structures are used [66].

If, however, the specific order is not important and one merely requires an efficient bijection between the combinatorial objects and a rank set, it can be advantageous to construct efficient ranking and unranking procedures first and let them induce the order. Using this approach, Myrvold and Ruskey obtained ranking and unranking algorithms for permutations that run in $O(n)$ arithmetic operations. In this setting, following Kreher and Stinson [60], the order is defined by the ranking via

$$s < t \quad :\Longleftrightarrow \quad \operatorname{rank}(s) < \operatorname{rank}(t).$$

One important application of efficient ranking and unranking procedures is the task of uniform random sampling from the set S. Indeed, if unranking can be performed efficiently, uniform sampling can be achieved by drawing a rank uniformly at random from the rank set $\{0, \ldots, |S| - 1\}$ and then applying the unranking procedure to construct the corresponding object [60]. This general and conceptually simple approach is precisely how we will later use ranking and unranking in the context of CFMs. Namely, we develop an efficient ranking scheme for configurations of a CFM, which allows us to uniformly sample from the configuration space.

To this end, we introduce a ranking scheme for combinations, which can be extended to work for multisets.

2.4.3.1 Ranking and Unranking of Combinations

Although there exist numerous ranking and unranking schemes for combinations, differing in their underlying orders and associated properties, we will focus on a method that is based on the combinatorial number system [61]. This choice is motivated by the efficiency of this method and by the fact that it is simpler and more natural than lexicographic approaches [60, 68, 69]. Moreover, for our purposes, the concrete underlying order is irrelevant, since we do not use it to generate all combinations but rather for uniform random sampling. The following construction is adapted from Kreher and Stinson [60] and Kruchinin et al. [69].

Let $S \coloneqq \{0, \dots, n-1\}$ be a finite ground set. Our goal is to assign to each k-combination of S a unique rank in the set

$$\left\{0, \dots, \binom{n}{k} - 1\right\},$$

since there are exactly $\binom{n}{k}$ k-combinations of S.

Every k-combination of S can be represented uniquely as a strictly increasing sequence $(c_1, \dots, c_k)$, where $c_i \in S$ and

$$c_1 < c_2 < \cdots < c_k.$$

Colexicographic order. Using the sequence representation, we define the *colexicographic (CoLex) order* on the set of all k-combinations. The CoLex order works similarly to the lexicographic order; however, in contrast, sequences are compared from right to left.

Formally, the CoLex order is defined as follows. For two k-combinations $(a_1, \dots, a_k)$ and $(b_1, \dots, b_k)$,

$$(a_1, \dots, a_k) \leq_{\text{colex}} (b_1, \dots, b_k)$$

if and only if either

- $(a_1, \dots, a_k) = (b_1, \dots, b_k)$, or
- there exists an index $s \in \{1, \dots, k\}$ such that

$$a_s < b_s \quad \text{and} \quad a_i = b_i \text{ for all } i > s.$$

Ranking. By definition, the rank of a combination is its position in the CoLex order. A straightforward ranking algorithm therefore consists of counting how many combinations precede the given one.

Let $C = (c_1, \dots, c_k)$ be a combination. The combinations that precede C in CoLex order can be grouped as follows:

- Combinations $C' = (c'_1, \dots, c'_k)$ with $c'_k < c_k$;
- Combinations $C' = (c'_1, \dots, c'_k)$ with $c'_k = c_k$ and $c'_{k-1} < c_{k-1}$;

$\vdots$

- Combinations $C' = (c'_1, \ldots, c'_k)$ with $c'_2 = c_2, \ldots, c'_k = c_k$ and $c'_1 < c_1$.

More generally, for each $i \in \{1, \ldots, k\}$, consider the combinations C' such that

$$c'_{i+1} = c_{i+1}, \ldots, c'_k = c_k \quad \text{and} \quad c'_i < c_i.$$

Since the sequence representation of a combination is strictly increasing, any such combination necessarily satisfies

$$c'_1 < \cdots < c'_{i-1} < c'_i < c_i.$$

Therefore, the prefix $(c'_1, \ldots, c'_i)$ is an i-element subset of $\{0, \ldots, c_i - 1\}$. Conversely, every such subset uniquely determines a valid combination C' with the prescribed suffix. Hence, the number of combinations satisfying the above conditions is exactly $\binom{c_i}{i}$.

With respect to the CoLex order, the rank of a k-combination $(c_1, \ldots, c_k)$ is thus given by

$$\operatorname{rank}(c_1, \ldots, c_k) \;=\; \sum_{i=1}^{k} \binom{c_i}{i}.$$

A key advantage of this formula is that it is independent of the size n of the underlying universe, which makes ranking efficient even for very large ground sets S.

The full ranking algorithm is presented in Algorithm 2.3.

Algorithm 2.3 RankCombination$(N, K, (c_1, \ldots, c_K))$: Algorithm for ranking a combination in CoLex order

Input:
- $N \in \mathbb{N}$: size of the ground set $\{0, \ldots, N-1\}$
- $K \in \mathbb{N}_0$: size of the combination
- $0 \le c_1 < \cdots < c_K \le N - 1$: combination representation

Output: The rank $R \in \left\{0, \ldots, \binom{N}{K} - 1\right\}$ of the combination in CoLex order

```
if K = 0 then
    return 0
end if

Compute rank in colex order
R ← 0
for i = 1 to K do
    R ← R + C(c_i, i)
end for
return R
```

Unranking. Conversely, given a rank

$$R \in \left\{0, \ldots, \binom{n}{k} - 1\right\},$$

the corresponding k-combination $(c_1, \ldots, c_k)$ can be reconstructed by the greedy procedure shown in Algorithm 2.4. Starting from the last position $i = k$ and proceeding backwards to $i = 1$, we choose c_i as the largest integer such that $\binom{c_i}{i}$ does not exceed the remaining rank. We then subtract this value from the rank and continue with the next smaller index. This successively fixes the entries of the combination from right to left and terminates once all k elements have been determined. The full unranking algorithm is given in Algorithm 2.4.

Algorithm 2.4 UnrankCombination(N, K, R): Algorithm for unranking a combination in CoLex order

Input:
- $N \in \mathbb{N}$: size of the ground set $\{0, \ldots, N-1\}$
- $K \in \mathbb{N}_0$: size of the combination
- $R \in \{0, \ldots, \binom{N}{K} - 1\}$: rank in CoLex order

Output: The K-combination $(c_1, \ldots, c_K)$ of rank R in CoLex order

1: **if** $K = 0$ **then**
2: **return** ()
3: **end if**

4: **Greedy reconstruction of the combination**
5: **for** $i = K$ down to 1 **do**
6: $c_i \leftarrow \max\left\{x \in \mathbb{N}_0 \;\middle|\; \binom{x}{i} \leq R\right\}$
7: $R \leftarrow R - \binom{c_i}{i}$
8: **end for**
9: **return** $(c_1, \ldots, c_K)$

To justify the correctness of the greedy choice, let

$$C = (c_1, \ldots, c_k)$$

denote the k-combination with rank R in CoLex order. By definition of the CoLex order, all k-combinations whose largest element equals c_k occupy a contiguous block of ranks: combinations with a smaller largest element appear strictly before this block, while those with a larger largest element appear strictly after it.

Among all k-combinations whose largest element is c_k, the smallest one in CoLex order is

$$(0, 1, \ldots, k-2, c_k),$$

whose rank is $\binom{c_k}{k}$, since all terms in the rank formula except the last one vanish. Furthermore, there are exactly $\binom{c_k}{k-1}$ k-combinations whose largest element is c_k. Hence, the ranks in this block are

$$\left\{\binom{c_k}{k}, \ldots, \binom{c_k}{k} + \binom{c_k}{k-1} - 1\right\} = \left\{\binom{c_k}{k}, \ldots, \binom{c_k+1}{k} - 1\right\},$$

where the equality follows from Pascal's identity (Lemma 2.11). Since the rank R lies inside this block, c_k is uniquely determined as the largest integer satisfying

$$\binom{c_k}{k} \leq R.$$

Once c_k has been determined, we know that C is the

$$R - \binom{c_k}{k}\text{-th}$$

combination among all k-combinations whose largest element is c_k. The remaining entries $(c_1, \ldots, c_{k-1})$ can therefore be recovered inductively by applying the same argument to this reduced problem.

Moreover, we can show that the greedy reconstruction automatically yields a strictly increasing sequence

$$c_1 < c_2 < \cdots < c_k.$$

Let $R^{(i)}$ denote the value of the rank variable at the beginning of iteration i of the algorithm. At this step, the algorithm chooses

$$c_i = \max\left\{ x \in \mathbb{N}_0 \;\middle|\; \binom{x}{i} \leq R^{(i)} \right\},$$

and then updates the rank according to

$$R^{(i-1)} = R^{(i)} - \binom{c_i}{i}.$$

Now let $i \in \{2, \ldots, k\}$. By maximality of c_i we have $\binom{c_i+1}{i} > R^{(i)}$, and therefore after the update

$$R^{(i-1)} < \binom{c_i + 1}{i} - \binom{c_i}{i} = \binom{c_i}{i-1},$$

where the equality follows from Pascal's identity (Lemma 2.11). In the next iteration (with index $i-1$), the algorithm selects the largest x satisfying

$$\binom{x}{i-1} \leq R^{(i-1)} < \binom{c_i}{i-1}.$$

Since $i - 1 \geq 1$, the function

$$x \mapsto \binom{x}{i-1}$$

is strictly increasing for all $x \geq i - 1$ (Lemma 2.11). Hence, the strict upper bound implies $x \leq c_i - 1$. Consequently,

$$c_{i-1} \leq c_i - 1,$$

and thus

$$c_{i-1} < c_i.$$

Since this holds for every $i \in \{2, \ldots, k\}$, it follows that

$$c_1 < c_2 < \cdots < c_k.$$

Naive complexity. In a straightforward implementation, each value c_i is determined by linearly increasing x until the inequality

$$\binom{x}{i} > R^{(i)}$$

is satisfied. This search is performed independently for each $i = k, k-1, \ldots, 1$.

For a fixed i, we have $i \leq c_i \leq n-1$. In the worst case, the linear search therefore tests up to $n-i$ candidate values. If each binomial coefficient $\binom{x}{i}$ is computed from scratch, it can be evaluated in $O(i)$ arithmetic operations. Hence, the worst-case cost of determining a single value c_i is

$$O\Big((n-i)\,i\Big).$$

Summing over all $i = 1, \ldots, k$ yields

$$\sum_{i=1}^{k} O\Big((n-i)\,i\Big) = O\left(\sum_{i=1}^{k}(n-i)\,i\right) = O\left(n\sum_{i=1}^{k} i - \sum_{i=1}^{k} i^2\right) \subseteq O(nk^2).$$

Incremental binomial evaluation. The above bound assumes that each binomial coefficient is recomputed independently. In practice, this is unnecessary. Using the identity

$$\binom{x+1}{i} = \binom{x}{i} \cdot \frac{x+1}{x+1-i},$$

from Lemma 2.11, binomial coefficients can be updated incrementally in constant time (up to the cost of the underlying arithmetic operations).

With this incremental strategy, the inner linear search over x requires only $O(1)$ arithmetic updates per step. Consequently, determining a single value c_i costs at most $O(n-i)$ arithmetic operations, and the overall worst-case complexity becomes

$$O\left(\sum_{i=1}^{k}(n-i)\right) = O(nk).$$

Improved unranking for large n and small k. The naive approach is inefficient when n is large and k is small. To address this issue, Kruchinin et al. [69] propose an improved unranking algorithm whose running time depends only on k.

The key idea is to replace the linear search for c_i by deriving an analytic lower bound for the solution of $\binom{x}{i} > R^{(i)}$ via Stirling's approximation for factorials. This yields an initial estimate for c_i that does not exceed the true solution and is subsequently corrected by a small number of exact binomial comparisons.

As a result, the number of tested values of x in each iteration is bounded independently of n, leading to a total time complexity of

$$O(k^2).$$

The method is particularly effective when n is very large and k is small, or by symmetry (Lemma 2.11) when $n-k$ is small, using the identity

$$\binom{n}{k} = \binom{n}{n-k}.$$

Although this $O(k^2)$ approach provides superior asymptotic guarantees, it requires analytic approximations and additional numerical care. Since our primary focus lies on conceptual clarity, we employ the incremental binomial strategy described above in our implementation (see Chapter 6).

2.4.3.2 Ranking and Unranking of Multisets

Building on the ranking and unranking scheme for combinations, we now derive a corresponding scheme for multisets. The key idea is to establish a simple bijection between k-multisets and k-combinations. Once such a bijection is available, ranking and unranking multisets reduces immediately to the already solved problem for combinations.

Let $S \coloneqq \{0, \dots, n-1\}$ be a finite ground set. By the *ordered-sequence representation* from Definition 2.13, every k-multiset over S can be represented uniquely as a non-decreasing sequence $(m_1, \dots, m_k)$, where $m_i \in S$ and

$$0 \leq m_1 \leq m_2 \leq \cdots \leq m_k \leq n-1.$$

In contrast to the sequence representation of combinations, duplicate values are allowed.

Shift bijection. With this sequence representation, a bijection between multisets and combinations can be obtained by a simple shift transformation [57].

Given a k-multiset $(m_1, \dots, m_k)$ over S, define

$$c_i \coloneqq m_i + (i-1), \qquad i = 1, \dots, k.$$

Then the resulting sequence $(c_1, \dots, c_k)$ is strictly increasing and satisfies

$$0 \leq c_1 < c_2 < \cdots < c_k \leq n+k-2.$$

Hence, $(c_1, \dots, c_k)$ represents a k-combination of the set

$$\{0, \dots, n+k-2\}.$$

Conversely, any strictly increasing sequence $(c_1, \dots, c_k)$ with

$$0 \leq c_1 < \cdots < c_k \leq n+k-2$$

uniquely determines a k-multiset over S by the inverse transformation

$$m_i \coloneqq c_i - (i-1), \qquad i = 1, \dots, k.$$

These two constructions are inverse to each other and thus define a bijection between k-multisets over S and k-combinations of the set $\{0, \dots, n+k-2\}$, which has size $n+k-1$.

Ranking and unranking. Using the shift bijection, ranking and unranking multisets is reduced to ranking and unranking combinations. More precisely, to rank a multiset, we first convert it into a combination via the shift bijection and then apply the combination ranking algorithm from Section 2.4.3.1. Unranking proceeds analogously in the reverse direction.

The full ranking procedure is given in Algorithm 2.5, while the corresponding unranking procedure is shown in Algorithm 2.6.

Algorithm 2.5 RankMultiset$(N, K, (m_1, \ldots, m_K))$: Algorithm for ranking a multiset

Input:
- $N \in \mathbb{N}$: size of the ground set $\{0, \ldots, N-1\}$
- $K \in \mathbb{N}_0$: size of the multiset
- $(m_1, \ldots, m_K)$: K-multiset of $\{0, \ldots, N-1\}$, represented as a non-decreasing sequence

Output: The rank $R \in \left\{0, \ldots, \binom{N+K-1}{K} - 1\right\}$ of the multiset

1: **if** $K = 0$ **then**
2: **return** 0
3: **end if**

4: **Step 1: Convert multiset to combination**
5: $L \leftarrow N + K - 1$
6: **for** $j = 1$ to K **do**
7: $c_j \leftarrow m_j + (j - 1)$
8: **end for**

9: **Step 2: Rank combination**
10: $R \leftarrow$ RankCombination$(L, K, (c_1, \ldots, c_K))$
11: **return** R

Algorithm 2.6 UnrankMultiset(N, K, R): Algorithm for unranking a multiset

Input:
- $N \in \mathbb{N}$: size of the ground set $\{0, \ldots, N-1\}$
- $K \in \mathbb{N}_0$: size of the multiset
- $R \in \left\{0, \ldots, \binom{N+K-1}{K} - 1\right\}$: rank of the multiset

Output: The R-th multiset, represented as a non-decreasing sequence $(m_1, \ldots, m_K)$

1: **if** $K = 0$ **then**
2: **return** ()
3: **end if**

4: **Step 1: Unrank combination**
5: $L \leftarrow N + K - 1$
6: $(c_1, \ldots, c_K) \leftarrow$ UnrankCombination(L, K, R)

7: **Step 2: Convert combination to multiset**
8: **for** $j = 1$ to K **do**
9: $m_j \leftarrow c_j - (j - 1)$
10: **end for**
11: **return** $(m_1, \ldots, m_K)$

3. Semi-Structural and Structural Configurations

This chapter introduces two new configuration semantics for cardinality-based feature models, namely *structural* and *semi-structural* configurations, together with their corresponding instance-based representatives, called *canonical* and *semi-canonical* configurations. In later chapters, structural configurations, represented via canonical configurations, will serve as the primary configuration type for which we develop uniform random sampling algorithms.

To motivate their introduction, we first develop the notions of sub-models and sub-configurations, which allow us to isolate smaller parts of a CFM. These concepts are not only useful from a conceptual perspective, but also form the basis for the uniform random sampling techniques introduced later in this thesis. After introducing the formal definitions, we investigate to what extent valid configurations of sub-models can be merged into configurations of the full model. We will see that this is not possible within the framework of instance-based configurations, which leads to the development of new configuration semantics.

3.1 Problems of Instance-Based Configurations

The instance-based configuration semantics introduced in Definition 2.6 distinguishes configurations by explicitly enumerating feature instances and by specifying a feature-instance decomposition relation $\prec \subseteq \mathrm{Inst}(M) \times \mathrm{Inst}(M)$ that determines how instances are arranged. At first glance, this appears to be a natural and precise representation: every feature instance is uniquely identified, and configurations can be compared via their concrete feature instance trees.

However, this level of detail is excessively fine-grained. In particular, two instance-based configurations are considered different even if they have the same tree structure and differ only in how instance indices are assigned. More generally, instance-based configurations exhibit two anomalies that arise solely from index assignments.

3.1.1 Index Anomalies

To illustrate these index anomalies, we use the CFM depicted in Figure 3.1. The model enforces exactly two instances of feature `A`. Each instance of `A` must have exactly one child, which is either an instance of `B` or an instance of `C`.

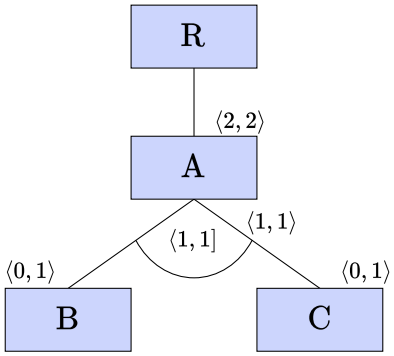


Figure 3.1: CFM used to illustrate index anomalies of the instance-based configuration semantics. The model enforces exactly two instances of `A`, each independently refined by either `B` or `C`.

Subtree Index Inversion

The first anomaly, which we call *subtree index inversion*, is illustrated in Figures 3.2a and 3.2b. The configuration shown in Figure 3.2a follows an intuitive indexing discipline: instances with smaller indices are consistently placed below parent instances with smaller indices. In contrast, the configuration in Figure 3.2b violates this discipline. There, the feature instance $\texttt{B}^2$ is a child of $\texttt{A}^1$, while $\texttt{B}^1$ is a child of $\texttt{A}^2$.

Importantly, *both* configurations are valid according to the instance-based semantics, and they are considered distinct because their feature instance trees (and thus their decomposition relations $\prec$) differ. The difference, however, is solely due to indexing choices.

Sibling Index Inversion

The second anomaly, called *sibling index inversion*, is illustrated in Figures 3.2c and 3.2d. Both figures show valid configurations, and the two configurations are distinct because their feature instance trees differ. The anomaly is that this distinction disappears once concrete instance indices are ignored.

More precisely, from a purely structural perspective that abstracts from indices, the two configurations are identical: both contain exactly two instances of `A`, one having a `B` child and the other having a `C` child. However, at the instance-based level, the two configurations are distinguished because the `B`- and `C`-subtrees are assigned to different sibling instances of `A`. Thus, the instance-based semantics yields two distinct representations of what is, structurally, the same configuration.

These anomalies show that the instance-based semantics distinguishes configurations that are structurally identical and differ only in their index assignments. This is not just a cosmetic issue. It affects how configurations can be constructed from smaller parts.

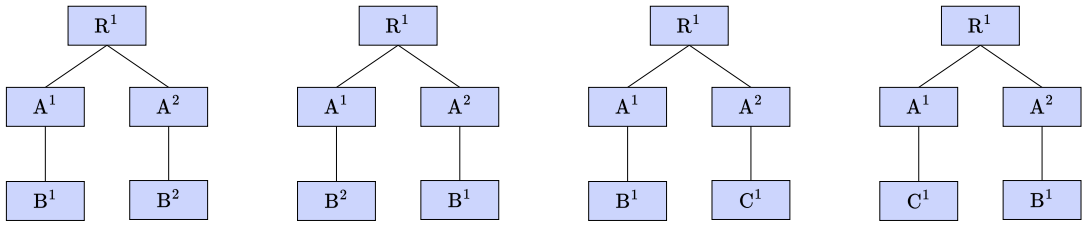


Figure 3.2: Illustration of index anomalies. From left to right: (a) normal configuration, (b) subtree index inversion, (c–d) sibling index inversion, where $\texttt{A}^1$ and $\texttt{A}^2$ swap child assignments ($\texttt{B}^1$ and $\texttt{C}^1$).

In particular, when building a configuration of the full model from configurations of its subtrees, the dependence on indices prevents a clean and canonical combination. To make this precise, we introduce sub-models and sub-configurations, which allow us to isolate and reason about individual subtrees of a CFM.

3.1.2 Sub-Models

CFMs are inherently tree-structured. This suggests a natural decomposition principle: we may isolate the subtree rooted at a given feature and consider it as a stand-alone CFM. Such sub-models allow us to reason locally about parts of a model and to study whether valid configurations of sub-models can be combined into valid configurations of the full model.

Beyond this conceptual role in analysing compositionality, sub-models will later also serve as the basis for a uniform random sampling procedure based on DP.

We therefore introduce the notion of a *sub-model*, obtained by extracting the subtree of the original CFM rooted at a chosen feature f and treating this subtree as a stand-alone CFM.

Definition 3.1 (Sub-Model)**.** Let $\mathcal{M} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \phi_R, \phi_X)$ be a CFM over a feature set F. For a feature $f \in F$, let $F_f \subseteq F$ be the set of features in the subtree rooted at f; that is,

$$F_f := \Big\{ g \in F \;\Big|\; \exists n \in \mathbb{N}_0 \, \exists x_0, \ldots, x_n \in F \text{ such that } x_0 = f, x_n = g, \\ \text{and } x_i \prec_F x_{i+1} \text{ for all } i < n \Big\}.$$

The *sub-model* of $\mathcal{M}$ at f, denoted $\mathcal{M}_f$, is the CFM induced by the subtree rooted at f. That is,

$$\mathcal{M}_f := \big(\prec_F |_f,\ \lambda_I^F|_f,\ \lambda_I^G|_f,\ \lambda_T^G|_f,\ \phi_R|_f,\ \phi_X|_f \big),$$

where

- $\prec_F |_f := \{(a,b) \in \prec_F |\ a, b \in F_f\}$;
- $\lambda_I^F|_f \colon F_f \to \mathcal{I}, \quad \lambda_I^F|_f(g) = \begin{cases} \langle 1, 1 \rangle, & \text{if } g = f, \\ \lambda_I^F(g), & \text{otherwise;} \end{cases}$

- $\lambda_I^G|_f \colon F_f \to \mathcal{I}, \quad \lambda_I^G|_f(g) = \lambda_I^G(g)$ for all $g \in F_f$;
- $\lambda_T^G|_f \colon F_f \to \mathcal{I}, \quad \lambda_T^G|_f(g) = \lambda_T^G(g)$ for all $g \in F_f$;
- $\phi_R|_f \coloneqq \{(a, I_a, I_b, b) \in \phi_R \mid a, b \in F_f\}$;
- $\phi_X|_f \coloneqq \{(a, I_a, I_b, b) \in \phi_X \mid a, b \in F_f\}$.

Remark. For any CFM $\mathcal{M}$ with root feature f_r, we have $\mathcal{M}_{f_r} = \mathcal{M}$. Furthermore, cross-tree constraints whose endpoints are not both contained in F_f do not appear in $\mathcal{M}_f$. This loss of constraints is harmless for our purposes, since our later sampling procedure operates on CFMs without cross-tree constraints. The general definition is given here for completeness.

Figure 3.3 shows the sub-model of the house CFM (see Section 2.2.1), rooted at the feature `Unit`. The sub-model is obtained by restricting the original model to the subtree below the chosen feature, removing cross-tree constraints that are not entirely contained in this subtree, and setting the feature-instance cardinality of the new root to $\langle 1, 1 \rangle$ so that the result is again a valid CFM. All other cardinalities remain unchanged.

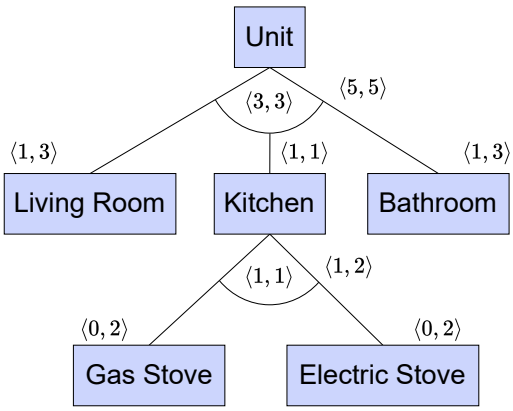


Figure 3.3: Sub-model of the house CFM, rooted at the feature `Unit`

3.1.3 Sub-Configurations

Building on the notion of a sub-model, we now define the corresponding concept of a sub-configuration.

Intuitively, given an instance-based configuration $K = (M, \prec)$ and a feature instance $f^i \in \mathrm{Inst}(M)$, we would like to "cut out" the subtree rooted at f^i and treat it as a stand-alone configuration. Conceptually, this mirrors the construction of a sub-model.

However, in contrast to the model-level case, two technical considerations are necessary:

1. Configurations are defined via multiplicity functions. Hence, after extracting a subtree, we must reconstruct a multiplicity function that counts exactly the feature instances occurring in that subtree.

2. Feature instances of each feature g are required to be enumerated consecutively from g^1 to $g^{M(g)}$. Since subtree extraction may destroy this enumeration, a systematic renumbering is required.

With these considerations in mind, we define sub-configurations as follows.

Definition 3.2 (Sub-Configuration)**.** Let $K = (M, \prec)$ be an instance-based configuration of a CFM $\mathcal{M}$, and let $f^i \in \mathrm{Inst}(M)$.

Let $\prec^*$ denote the reflexive-transitive closure of $\prec$ and define

$$\mathrm{Inst}(M)|_{f^i} \coloneqq \Big\{\, g^j \in \mathrm{Inst}(M) \mid f^i \prec^* g^j \,\Big\}.$$

Thus, $\mathrm{Inst}(M)|_{f^i}$ is the set of feature instances reachable from f^i.

We define the *restricted multiplicity function* $M|_{f^i} : F \to \mathbb{N}_0$ by

$$M|_{f^i}(g) \coloneqq \Big|\{\, g^j \in \mathrm{Inst}(M)|_{f^i} \,\}\Big|.$$

That is, $M|_{f^i}(g)$ counts the number of instances of feature g occurring in the subtree rooted at f^i.

Since the feature instances in $\mathrm{Inst}(M)|_{f^i}$ are generally not consecutively enumerated, we renumber them. Concretely, we define a bijection

$$\rho\colon\ \mathrm{Inst}(M)|_{f^i} \to \mathrm{Inst}\Big(M|_{f^i}\Big),$$

which maps the sparse instance set (possibly with index gaps) to the dense instance set induced by the multiplicity function $M|_{f^i}$.

For each feature $g \in F$, let

$$g^{j_1}, g^{j_2}, \ldots, g^{j_{M|_{f^i}(g)}}$$

be the instances of g in $\mathrm{Inst}(M)|_{f^i}$, listed in increasing order (i.e. $j_k < j_{k+1}$). We then define

$$\rho(g^{j_k}) \coloneqq g^k \qquad \text{for } k = 1, \ldots, M|_{f^i}(g).$$

The *restricted feature-instance decomposition relation* is defined by

$$\prec|_{f^i} \coloneqq \Big\{\, (\rho(a), \rho(b)) \mid (a, b) \in \prec,\ a, b \in \mathrm{Inst}(M)|_{f^i} \,\Big\}.$$

The *sub-configuration of K at f^i* is then defined as

$$K_{f^i} \coloneqq \Big(M|_{f^i},\ \prec|_{f^i}\Big).$$

The notion of a sub-configuration should not be confused with that of a *child configuration*. While sub-configurations arise from restricting an existing configuration to the subtree rooted at a specific feature instance, child configurations are configurations of a specific sub-model.

Definition 3.3 (Child Model and Child Configuration)**.** Let $\mathcal{M}$ be a CFM over the feature set F, and let $p \in F$ be a non-leaf feature with child feature $c \in F$.

The *child model* of c is the sub-model $\mathcal{M}_c$, that is, the CFM obtained by restricting $\mathcal{M}$ to the subtree rooted at c.

An (instance-based) *child configuration* of c is a configuration of the child model $\mathcal{M}_c$.

Example Sub-Configuration of the House CFM

We illustrate Definition 3.2 using the house CFM. Figure 3.4a shows an excerpt of an instance-based configuration $K = (M, \prec)$. We focus on the subtree rooted at the feature instance $\texttt{Unit}^1$.

This subtree contains two `LivingRoom` instances, one instance of `Kitchen` with a `GasStove`, and two `Bathroom` instances. Notably, the configuration exhibits a *subtree index inversion*: the visible subtree contains $\texttt{Bathroom}^1$ and $\texttt{Bathroom}^4$, while $\texttt{Bathroom}^2$ and $\texttt{Bathroom}^3$ occur below another (higher-indexed) unit instance not shown in the figure. Consequently, the indices inside the subtree are not consecutive.

We now construct the sub-configuration $K_{\texttt{Unit}^1}$.

Step 1: Extract the reachable instance set. The restricted instance set is

$$\begin{aligned}\mathrm{Inst}(M)|_{\texttt{Unit}^1} = \{\, &\texttt{Unit}^1,\, \texttt{LivingRoom}^1, \texttt{LivingRoom}^2,\, \texttt{Kitchen}^1,\\ &\texttt{GasStove}^1,\, \texttt{Bathroom}^1,\, \texttt{Bathroom}^4 \,\}.\end{aligned}$$

The gap in the bathroom indices shows that renumbering will be necessary.

Step 2: Construct the restricted multiplicity function. Counting occurrences in the restricted instance set yields

$$\begin{aligned} M|_{\texttt{Unit}^1}(\texttt{Unit}) &= 1,\\ M|_{\texttt{Unit}^1}(\texttt{LivingRoom}) &= 2,\\ M|_{\texttt{Unit}^1}(\texttt{Kitchen}) &= 1,\\ M|_{\texttt{Unit}^1}(\texttt{GasStove}) &= 1,\\ M|_{\texttt{Unit}^1}(\texttt{Bathroom}) &= 2, \end{aligned}$$

and $M|_{\texttt{Unit}^1}(g) = 0$ for all other features g.

Step 3: Define the renumbering bijection. For `Bathroom`, the instances in the subtree are

$$\texttt{Bathroom}^1, \texttt{Bathroom}^4.$$

Since these indices are not consecutive, we define

$$\rho(\texttt{Bathroom}^1) = \texttt{Bathroom}^1, \qquad \rho(\texttt{Bathroom}^4) = \texttt{Bathroom}^2.$$

All other feature instances already form consecutive sequences and are therefore mapped to themselves.

Step 4: Restrict the decomposition relation. The restricted decomposition relation is defined as

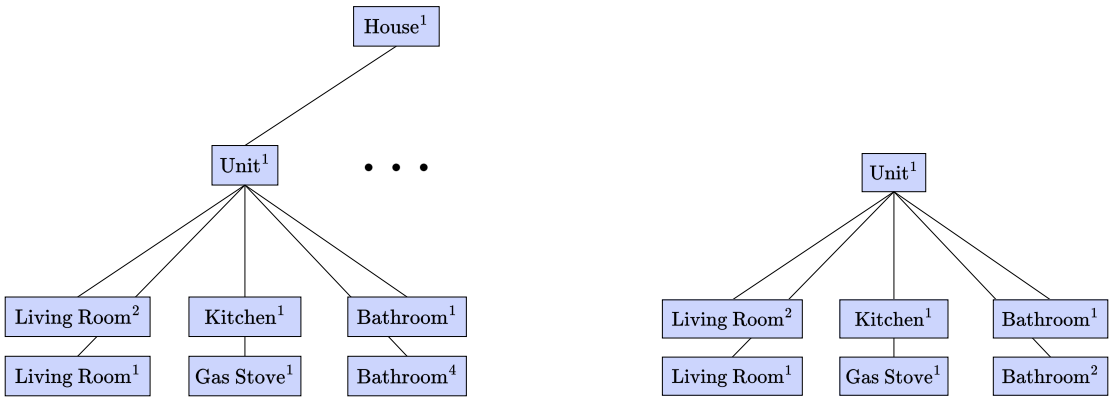
$$\prec|_{\texttt{Unit}^1} = \{\, (\rho(a), \rho(b)) \mid (a, b) \in \prec,\ a, b \in \mathrm{Inst}(M)|_{\texttt{Unit}^1} \,\}.$$

In this example, only $\texttt{Bathroom}^4$ is affected by renumbering. Hence, the pair ($\texttt{Unit}^1$, $\texttt{Bathroom}^4$) in $\prec$ becomes ($\texttt{Unit}^1, \texttt{Bathroom}^2$) in $\prec|_{\texttt{Unit}^1}$. All other pairs remain unchanged.

Result. The resulting sub-configuration

$$K_{\texttt{Unit}^1} = \left(M|_{\texttt{Unit}^1},\ \prec|_{\texttt{Unit}^1}\right)$$

is shown in Figure 3.4b. All feature instances are now consecutively enumerated, and the extracted subtree forms a valid instance-based configuration.



(a) Original configuration (excerpt). (b) Sub-configuration at $\texttt{Unit}^1$.

Figure 3.4: Construction of the sub-configuration $K_{\texttt{Unit}^1}$ in the house CFM. (a) Excerpt of a valid instance-based configuration K containing non-consecutively indexed bathroom instances within the subtree of $\texttt{Unit}^1$. (b) The sub-configuration $K_{\texttt{Unit}^1}$ after restricting the multiplicity function and renumbering feature instances to restore consecutive indexing.

Validity of Sub-Configurations

Let $K = (M, \prec)$ be a valid instance-based configuration of a CFM $\mathcal{M}$, and let $f^i \in \text{Inst}(M)$ be an arbitrary feature instance. In general, the sub-configuration K_{f^i} is *not* a valid configuration of the corresponding sub-model $\mathcal{M}_f$.

The main reason is that cross-tree constraints in a CFM are interpreted *globally*. Their satisfaction depends on the complete set of feature instances in the configuration and not only on a local subtree. Consequently, extracting a subtree may lead to a violation of cross-tree constraints, even though the original configuration satisfies them.

For CFMs *without* cross-tree constraints, this issue does not arise. In this case, every sub-configuration of a valid configuration is again a valid configuration of the corresponding sub-model.

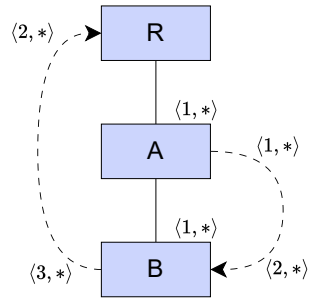
The converse statement does not hold in general. That is, not every valid configuration of a sub-model arises as the sub-configuration of some valid configuration of the full model. This can fail both in the presence and in the absence of cross-tree constraints. The underlying reason is the possible existence of *dead features*, i.e., features that never occur in any valid configuration of the full model [10].

For example, consider a feature f with $\lambda_I^F(f) = \langle 0, 0\rangle$. Then no valid configuration of the full model can contain an instance of f. However, in the corresponding sub-model $\mathcal{M}_f$, the feature-instance cardinality is fixed to $\lambda_I^F|_f(f) = \langle 1, 1\rangle$ by definition

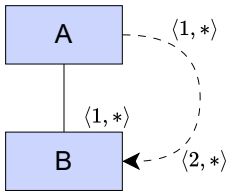
(see Definition 3.1). Thus, $\mathcal{M}_f$ admits valid configurations that can never occur as sub-configurations of the full model.

Nevertheless, in the absence of cross-tree constraints, a weaker but useful statement holds. If $f \in F$ is *not* dead, meaning that it appears in at least one valid configuration of the full model, then every valid configuration of the sub-model $\mathcal{M}_f$ arises as the sub-configuration of some valid configuration of the full model.

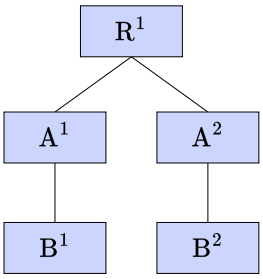
Figure 3.5 illustrates these relationships. It shows a CFM together with its sub-model rooted at feature A, as well as configurations of both models. In particular, Figure 3.5d depicts a sub-configuration that is invalid in the sub-model due to global cross-tree constraints, while Figure 3.5e shows a valid configuration of the sub-model that cannot occur as a sub-configuration of any valid configuration of the full model.



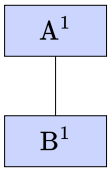
(a) Model $\mathcal{M}$.



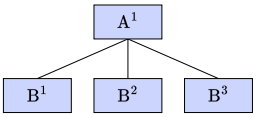
(b) Sub-model $\mathcal{M}_{\mathrm{A}}$.



(c) Configuration K.



(d) Sub-config. K_{A^1}.



(e) Config. of $\mathcal{M}_{\mathrm{A}}$.

Figure 3.5: Relationship between a CFM, its sub-model, and their configurations. (a) A CFM $\mathcal{M}$ over $F = \{\mathtt{R}, \mathtt{A}, \mathtt{B}\}$. (b) The sub-model $\mathcal{M}_{\mathrm{A}}$ rooted at feature A. (c) A valid instance-based configuration K of $\mathcal{M}$. (d) The sub-configuration K_{A^1} obtained from (c), which is *invalid* in $\mathcal{M}_{\mathrm{A}}$ due to the global interpretation of cross-tree constraints. (e) A valid configuration of the sub-model $\mathcal{M}_{\mathrm{A}}$ that cannot arise as a sub-configuration of any valid configuration of the full model $\mathcal{M}$.

3.1.4 Merging Instance-Based Configurations

After introducing sub-configurations and child configurations, we now focus exclusively on child configurations. Concretely, we want to investigate whether we can obtain (instance-based) configurations of the full model in a bottom-up manner by merging or combining child configurations. In contrast to sub-configurations, which are extracted from an existing configuration of the full model, child configurations

are configurations of sub-models defined independently of any particular global configuration. In particular, attempting to merge sub-configurations would be circular, since sub-configurations presuppose the existence of a configuration of the full model.

To formalize this question, we first introduce the notion of *child sets*, defined both at the model level and at the configuration level.

Definition 3.4 (Child Set)**.** Let $\mathcal{M} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \phi_R, \phi_X)$ be a CFM over a feature set F, and let $f \in F$. The *child set* of f is

$$C_{\mathcal{M}}(f) \coloneqq \{\, g \in F \mid f \prec_F g \,\}.$$

For a configuration $K = (M, \prec)$ of $\mathcal{M}$ and a feature instance $f^i \in \mathrm{Inst}(M)$, the *child set* of f^i is

$$C_K(f^i) \coloneqq \{\, g^j \in \mathrm{Inst}(M) \mid f^i \prec g^j \,\}.$$

For a feature $g \in F$, we further define

$$C_K(f^i, g) \coloneqq \{\, a \in C_K(f^i) \mid \mathrm{feat}(a) = g \,\},$$

that is, the set of children of f^i whose feature type is g.

As discussed in Section 3.1.3, sub-configurations and sub-models do not compose well in the presence of cross-tree constraints, since such constraints are interpreted globally and cannot be checked locally within a subtree. The same limitation applies to child configurations. For this reason, we restrict ourselves to CFMs *without* cross-tree constraints.

Let $\mathcal{M}$ be a CFM over the feature set F *without* cross-tree constraints, with root feature f_r and

$$C_{\mathcal{M}}(f_r) = \{c_1, \ldots, c_n\}.$$

Intuitively, constructing a configuration of $\mathcal{M}$ from configurations of its children should proceed in two phases.

Phase 1: Choose multiplicities. For each child feature c_i, select a number $k_i \in \mathbb{N}_0$ such that the cardinality constraints at the root are satisfied:

- $k_i \in \lambda_I^F(c_i)$,
- $\sum_i k_i \in \lambda_I^G(f_r)$,
- $|\{\, i \mid k_i > 0 \,\}| \in \lambda_T^G(f_r)$.

Phase 2: Choose child configurations. For each i and for each of the k_i required instances of c_i, select a valid child configuration of c_i, that is, a configuration of $\mathcal{M}_{c_i}$ (see Definition 3.3). Our goal is to combine all selected child configurations into a single configuration of the full model.

At first sight, this may appear straightforward. Since there are no cross-tree constraints, each subtree can be constructed independently, and the only remaining task is to glue them together below a fresh root instance.

However, a technical obstacle arises. *Feature-instance indices must not collide.* Two independently chosen configurations of $\mathcal{M}_{c_i}$ both use instance names such as $g^1, g^2, \ldots$. A naive union would therefore produce ambiguous identifiers.

To avoid this, we introduce a *shifting operator* that re-indexes feature instances before combining configurations.

Definition 3.5 (Shifted Configuration)**.** Let $K = (M, \prec)$ be an instance-based configuration and let $\Delta\colon F \to \mathbb{N}_0$ be an arbitrary multiplicity function. The *shifted configuration* $K \oplus \Delta$ is defined as

$$K \oplus \Delta \coloneqq (M^{\Delta}, \prec^{\Delta}),$$

where

$$M^{\Delta}(g) \coloneqq M(g) + \Delta(g)$$

and $\prec^{\Delta}$ is obtained by shifting every instance index by the corresponding offset. Concretely, if $(a, b) \in \prec$ with $a = g^j$ and $b = h^\ell$, then

$$(a, b)^{\Delta} \coloneqq \left(g^{\,j+\Delta(g)},\, h^{\,\ell+\Delta(h)}\right),$$

and

$$\prec^{\Delta} \coloneqq \{\, (a, b)^{\Delta} \mid (a, b) \in \prec \,\}.$$

The operator $\oplus$ ensures that independently constructed child configurations can be made disjoint with respect to their instance identifiers. However, $K \oplus \Delta$ is *not* always a configuration in the sense of Definition 2.6, since for $\Delta \neq 0$, $(\mathrm{Inst}(M^{\Delta}), \prec^{\Delta})$ is not a tree.

Merge Algorithm

With the shift operator $\oplus$ in place, we can now define the algorithm Merge, which takes, for each child c_i of the root, a collection of instance-based child configurations and combines them into a single instance-based configuration of the full model $\mathcal{M}$.

Concretely, the procedure Merge, shown in Algorithm 3.1, proceeds as follows.

It maintains

- a multiplicity function $R\colon F \to \mathbb{N}_0$ that records how many instances of each feature have been introduced so far, and
- a set E collecting all decomposition edges accumulated during the construction.

Initially, $R(f) = 0$ for all $f \in F$, and $E = \emptyset$. The algorithm then processes the selected child configurations one by one.

For each child feature c_i of f_r and for each chosen child configuration $K_{i,j}$, the following steps are performed:

1. *Shift.* The configuration $K_{i,j}$ is shifted by the current multiplicity function R, i.e. $\widetilde{K_{i,j}} \coloneqq K_{i,j} \oplus R$. This ensures that all feature-instance indices in $\widetilde{K_{i,j}}$ are strictly larger than those already present in the accumulated structure, thereby preventing identifier collisions.

2. *Accumulate instances.* The feature multiplicities of $K_{i,j}$ are added to R, so that R continues to reflect the total number of instances introduced so far.

3. *Accumulate edges.* The shifted decomposition edges are added to E.

After all child configurations have been processed in this manner, the algorithm introduces the root instance f_r^1 (by setting $R(f_r) = 1$) and connects it to every instance of its child features. Formally, for each $g \in C_{\mathcal{M}}(f_r)$ and each $1 \leq j \leq R(g)$, the edge (f_r^1, g^j) is added to E.

Finally, the merged configuration is obtained as

$$(M, \prec) \coloneqq (R, E).$$

Algorithm 3.1 MERGE$\Big((K_i)_{i=1}^n, \mathcal{M}\Big)$: Algorithm for merging instance-based child configurations of a CFM

Input:
- CFM $\mathcal{M} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \emptyset, \emptyset)$ over a feature set F, root feature f_r and $C_{\mathcal{M}}(f_r) = \{c_1, \ldots, c_n\}$
- For each child $c_i \in C_{\mathcal{M}}(f_r)$, a list of instance-based child configurations
$$K_i = [K_{i,1},\ K_{i,2},\ \ldots,\ K_{i,k_i}]$$
(i.e. configurations of the sub-model $\mathcal{M}_{c_i}$)

Output: A merged instance-based configuration $(M, \prec)$ of the full model $\mathcal{M}$

Initialization

$R(g) \leftarrow 0$ for all $g \in F$ ▷ Accumulate number of feature instances
$E \leftarrow \emptyset$ ▷ Accumulate decomposition edges

Merge all chosen child configurations

for each child c_i of f_r **do**
  for $j = 1$ to k_i **do**
    $(M_{i,j},\ \prec^{i,j}) \leftarrow K_{i,j}$
    $\widetilde{K}_{i,j} \leftarrow K_{i,j} \oplus R$ ▷ Shift according to accumulated offsets
    Let $\widetilde{K}_{i,j} = (_,\ \widetilde{\prec}^{i,j})$

    Merge instances and edges
    for each $g \in F$ **do**
      $R(g) \leftarrow R(g) + M_{i,j}(g)$
    end for
    $E \leftarrow E \cup \widetilde{\prec}^{i,j}$
  end for
end for

Add the root instance and connect it

$R(f_r) \leftarrow 1$
$E \leftarrow E \cup \Big\{ (f_r^1,\ g^j) \ \Big|\ g \in C_{\mathcal{M}}(f_r),\ 1 \le j \le R(g) \Big\}$

Construct final configuration

$M \leftarrow R$
$\prec \leftarrow E$
return $(M,\ \prec)$

Besides the merge algorithm, we also introduce the *merge property*, which is a property that any reasonable configuration semantics should satisfy. We also refer to this property as *closure under merge*.

Definition 3.6 (Merge Property)**.** Let $\mathcal{C}$ be a configuration semantics for CFMs. We say that $\mathcal{C}$ satisfies the *merge property* (or is *closed under merge*) if valid configurations of child models can be merged into valid configurations of the full model, and every valid configuration of the full model arises in this way.

More precisely, let $\mathcal{M}$ be a CFM *without* cross-tree constraints, with root feature f_r and child set

$$C_{\mathcal{M}}(f_r) = \{c_1, \ldots, c_n\}.$$

Then $\mathcal{C}$ satisfies the merge property if there exists a merge procedure appropriate for $\mathcal{C}$ such that the following hold:

1. **Soundness.** Whenever multiplicities k_i satisfy the root-group cardinality constraints and the corresponding feature-instance cardinality constraints, and for each $i \in \{1, \ldots, n\}$ and each $j \in \{1, \ldots, k_i\}$ we choose a valid child configuration
$$K_{i,j} \in \mathcal{C}(\mathcal{M}_{c_i}),$$
there exists an order of these child configurations whose merge yields a valid configuration in $\mathcal{C}(\mathcal{M})$.

2. **Completeness.** For every valid configuration $K \in \mathcal{C}(\mathcal{M})$, there exist multiplicities k_i satisfying the root-group cardinality constraints and the corresponding feature-instance cardinality constraints, as well as valid child configurations
$$K_{i,j} \in \mathcal{C}(\mathcal{M}_{c_i}),$$
such that K is obtained by merging these child configurations.

Here, the merge procedure need not be the concrete algorithm Merge itself; rather, it may be any procedure analogous to Merge and adapted to the notation and representation of the semantics $\mathcal{C}$.

Soundness of the Instance-Based Semantics

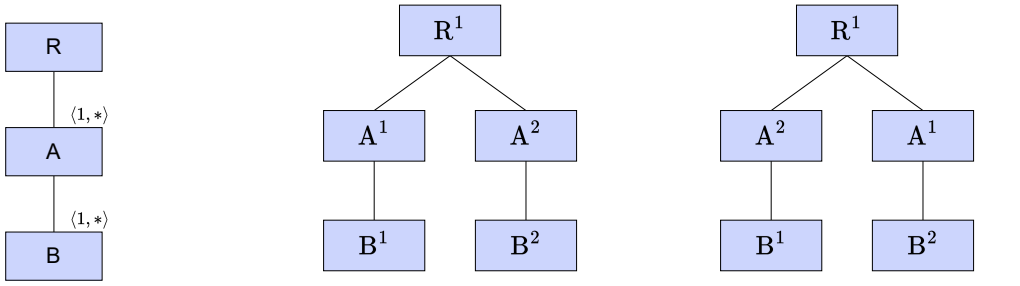
If all selected child configurations are valid and the multiplicities k_i satisfy the root constraints, then Merge always produces a valid instance-based configuration of the full model. This holds because we assume that the model has no cross-tree constraints. Hence, validity reduces to checking cardinalities and tree structure within each feature instance group, both of which are preserved by Merge.

Completeness of the Instance-Based Semantics

Although the instance-based configuration semantics satisfies the soundness condition of the merge property, since merging valid instance-based child configurations always yields a valid configuration of the full model, it does not satisfy the completeness condition. Specifically, not every valid instance-based configuration of the full model can be obtained by merging valid child configurations.

The reason lies in the fine-grained nature of instance-based configurations. The procedure introduces feature instances in a strictly increasing and sorted manner, ensuring that instances from earlier subtrees receive smaller indices than those from later ones. In particular, subtree index inversions between different child configurations cannot arise.

This phenomenon is illustrated in Figure 3.6. The configuration in Figure 3.6b can be produced by MERGE. In contrast, Figure 3.6c is a valid configuration of the same model, but it contains a subtree index inversion and is therefore not reachable by MERGE.



(a) CFM $\mathcal{M}$.

(b) Configuration of $\mathcal{M}$ reachable by MERGE.

(c) Configuration of $\mathcal{M}$ not reachable by MERGE.

Figure 3.6: Limitation of the MERGE procedure. (a) A CFM $\mathcal{M}$ without cross-tree constraints. (b) A valid instance-based configuration that can be obtained by merging valid child configurations. (c) A valid instance-based configuration of the same model that contains a subtree index inversion and therefore cannot be produced by MERGE.

Why not repair this by permuting indices? A natural idea is to post-process merged configurations by applying arbitrary permutations to the indices of feature instances of the same type.

This, however, introduces a deeper problem. There exist non-trivial permutations that leave a configuration invariant. For example, in Figure 3.6b, simultaneously swapping $\mathtt{A}^1$ with $\mathtt{A}^2$ and $\mathtt{B}^1$ with $\mathtt{B}^2$ yields exactly the same configuration, since the parent-child relationships are preserved (from a group-theoretic perspective, such configurations have a non-trivial stabilizer [70]).

From a combinatorial perspective, this means that the map from tuples of the form (choice of child configurations, permutation) to instance-based configurations is not injective. Different tuples may yield the same instance-based configuration, and the number of such representing tuples may vary from one configuration to another.

This ambiguity becomes a serious obstacle once we aim to perform *uniform random sampling*. A naive generation strategy, where we first generate child configurations, then merge them, and finally apply a random permutation of indices, does not induce a uniform distribution on the set of instance-based configurations. The reason is that configurations with more representing tuples, that is, configurations that can be obtained from more combinations of child configurations and permutations, are generated with higher probability than configurations with fewer such representations.

3.1.5 Conclusion

The instance-based configuration semantics provides a precise and explicit representation of configurations by enumerating feature instances and recording their decomposition relation. While this representation is convenient because it is easy to define and faithful to the general idea of CFMs and the intended structure of concrete products, it introduces a level of granularity that turns out to be problematic. Specifically, the instance-based configuration semantics exhibits two index anomalies which not only inflate the configuration space but also interfere with compositional reasoning.

In particular, as shown in Section 3.1.4, merging instance-based child configurations always yields a valid instance-based configuration of the full model. However, configurations exhibiting index anomalies cannot be reached by this construction. Consequently, the instance-based configuration semantics does not satisfy the merge property.

This observation motivates the introduction of a coarser notion of configuration that identifies configurations solely by their structural properties rather than by arbitrary index assignments. We call these new notions of configuration *structural* and *semi-structural* configurations to emphasize their focus on the underlying structure. In particular, we will show that both notions satisfy the desirable merge property: merging valid child configurations yields a valid full configuration, and every valid full configuration can be obtained in this way.

3.2 Structural Configurations

In the previous section, we observed that instance-based configurations distinguish between configurations that are identical at the structural level but differ only in their feature-instance indices. These artificial degrees of freedom complicate the systematic construction of configurations and obstruct uniform random sampling.

To eliminate this artefact, we introduce a coarser notion of configuration that abstracts from concrete instance indices while preserving the hierarchical structure. The central idea is to group together instance-based configurations that differ only by a consistent renaming of feature instances. The resulting equivalence classes are called *structural configurations*.

This section develops the theory of structural configurations in three steps. First, we define them as equivalence classes of instance-based configurations under a suitable isomorphism. Second, we introduce an index-free tree representation that makes their structure explicit. Third, we define canonical configurations, which serve as unique instance-based representatives of these equivalence classes.

3.2.1 Equivalence Class Representation

We formalise structural configurations by identifying instance-based configurations that differ only in their feature-instance indices. To this end, we introduce the notion of a tree isomorphism between instance-based configurations.

Definition 3.7 (Tree Isomorphism)**.** Let $\mathcal{M}$ be a CFM over a feature set F. Two instance-based configurations $K_1 = (M, \prec^{(1)})$ and $K_2 = (N, \prec^{(2)})$ are called *tree-isomorphic*, written $K_1 \cong_{\text{tree}} K_2$, if:

1. $M = N$.
2. There exists a bijection $\pi : \mathrm{Inst}(M) \to \mathrm{Inst}(M)$ such that
   (a) (*feature-wise*) for every feature $f \in F$, the restriction
   $$\pi|_{\{f^1,\dots,f^{M(f)}\}} \colon \{f^1,\dots,f^{M(f)}\} \to \{f^1,\dots,f^{M(f)}\}$$
   is a bijection;
   (b) (*edge preservation*) for all $x, y \in \mathrm{Inst}(M)$,
   $$x \prec^{(1)} y \iff \pi(x) \prec^{(2)} \pi(y).$$

A map π satisfying condition 2 is called an *tree isomorphism* between K_1 and K_2.

Remark. The above notion of a tree isomorphism coincides with the standard notion of graph isomorphism for labelled (coloured) graphs in general graph theory. Such an isomorphism is a bijection on the nodes that preserves adjacency as well as the node labels (colours) [71, 72, 73]. The definition above is the same concept applied to the domain of CFMs.

Intuitively, two configurations are tree-isomorphic if they have the same feature multiplicities and the same parent-child structure, and differ only in how the feature instances are indexed.

More precisely, the first condition $M = N$ ensures that both configurations contain exactly the same number of instances of each feature. The second condition requires that we can transform one configuration into the other simply by reassigning feature-instance indices. More precisely, there must exist a bijection π that permutes the indices of instances, but only within each feature type.

Figure 3.7 illustrates three simplified instance-based configurations of the house CFM (they are not valid, as some constraints are omitted for clarity). The configurations shown in Figures 3.7a and 3.7b are tree-isomorphic: they contain the same feature multiplicities and differ only in the assignment of feature-instance indices. In contrast, the configuration in Figure 3.7c is not tree-isomorphic to the first two, since it contains only one gas stove, whereas the other two contain two.

Concretely, a tree isomorphism transforming the configuration in Figure 3.7b into the one in Figure 3.7a is given by a bijection

$$\pi : \mathrm{Inst}(M) \to \mathrm{Inst}(M)$$

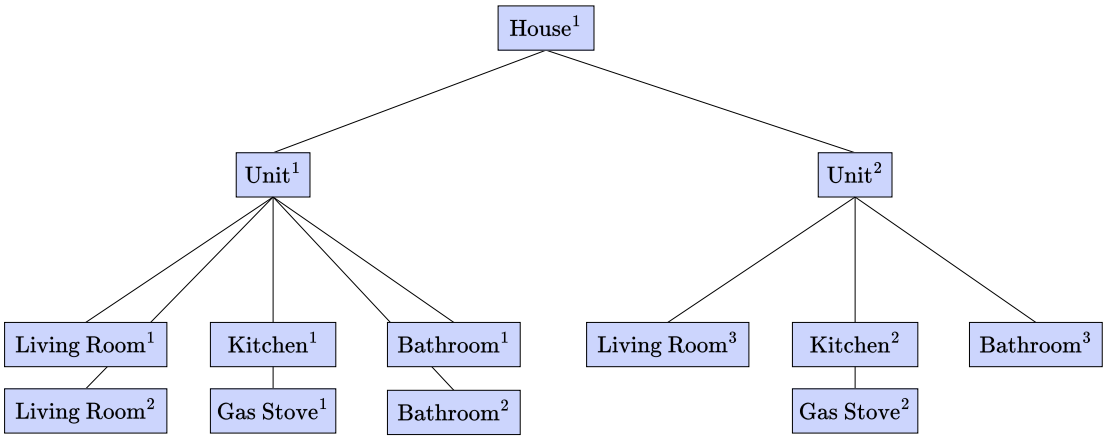
that exchanges the two `Unit` instances and two of the `LivingRoom` instances. More precisely,

$$\pi(\texttt{Unit}^1) = \texttt{Unit}^2, \qquad \pi(\texttt{Unit}^2) = \texttt{Unit}^1,$$

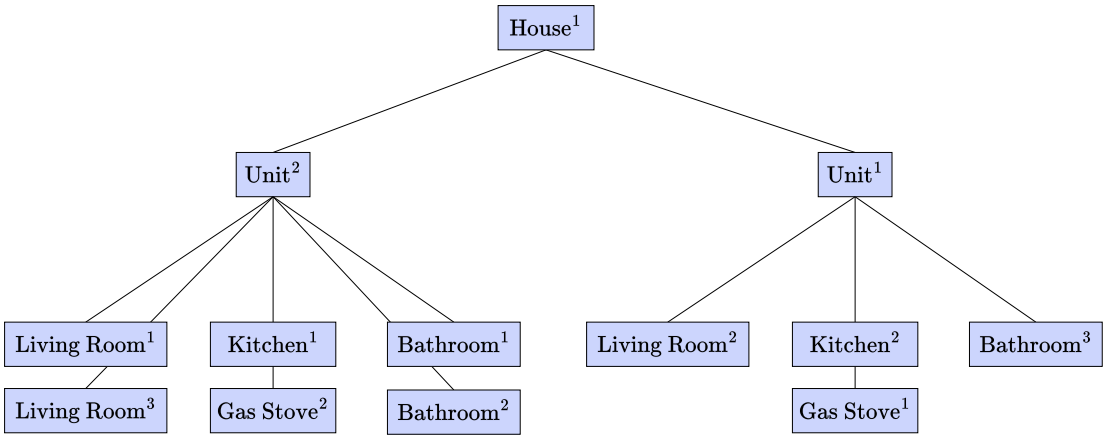
and

$$\pi(\texttt{LivingRoom}^2) = \texttt{LivingRoom}^3, \qquad \pi(\texttt{LivingRoom}^3) = \texttt{LivingRoom}^2.$$

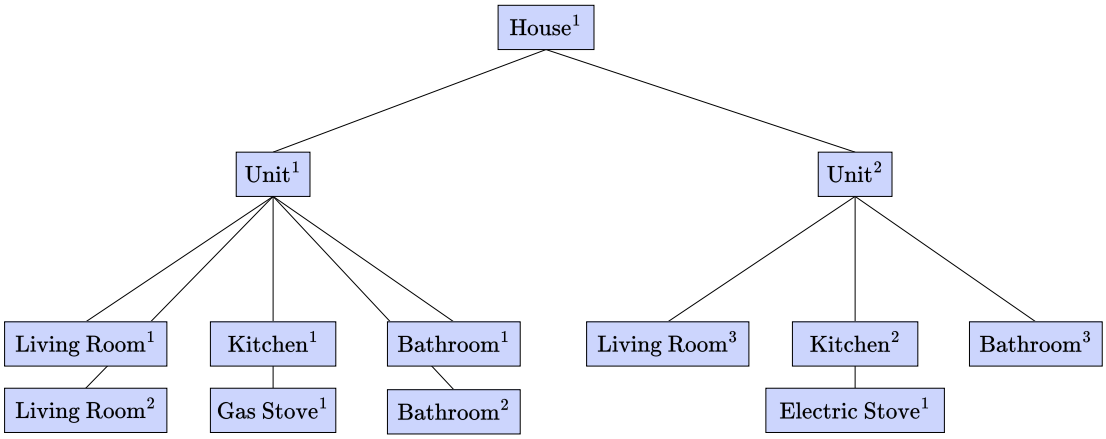
All other feature instances are mapped to themselves.



(a) Instance-based configuration K_1.



(b) Instance-based configuration K_2 (tree-isomorphic to K_1).



(c) Instance-based configuration K_3 (not tree-isomorphic to K_1 and K_2).

Figure 3.7: Three simplified instance-based configurations of the house CFM. The first two (a, b) are tree-isomorphic, as they differ only by renaming of feature instances. The third (c) differs in its feature multiplicities and is therefore not tree-isomorphic to the others.

We now show that the relation $\cong_{\text{tree}}$ is actually an equivalence relation, since only then can we define structural configurations as equivalence classes.

Lemma 3.8 (Tree Isomorphism is an Equivalence Relation)**.** For a CFM $\mathcal{M}$, the relation $\cong_{\text{tree}}$ is an equivalence relation on the set $\mathcal{C}_{\text{Inst}}(\mathcal{M})$ of all instance-based configurations of $\mathcal{M}$.

Proof. Let $K_1 = (M, \prec^{(1)})$, $K_2 = (N, \prec^{(2)})$, and $K_3 = (P, \prec^{(3)})$ be instance-based configurations of $\mathcal{M}$.

1. Reflexivity. For $K_1 = (M, \prec^{(1)})$, we have $M = M$, and the identity map on $\text{Inst}(M)$ is a bijection on every feature instance set $\{f^1, \dots, f^{M(f)}\}$, which preserves $\prec^{(1)}$. Thus $K_1 \cong_{\text{tree}} K_1$.

2. Symmetry. If $K_1 \cong_{\text{tree}} K_2$, then $M = N$ by definition. Let π be a tree isomorphism between K_1 and K_2. Since π is a bijection, it admits an inverse function π^{-1}.

Because π restricts to a bijection on every feature instance set $\{f^1, \dots, f^{M(f)}\}$, its inverse π^{-1} also restricts to a bijection on every such set.

Let $u, v \in \text{Inst}(M)$. Since π is bijective, there exist unique $x, y \in \text{Inst}(M)$ such that

$$u = \pi(x), \qquad v = \pi(y).$$

From the edge-preservation property of π we have

$$x \prec^{(1)} y \iff \pi(x) \prec^{(2)} \pi(y).$$

Substituting $u = \pi(x)$ and $v = \pi(y)$ yields

$$\pi^{-1}(u) \prec^{(1)} \pi^{-1}(v) \iff u \prec^{(2)} v.$$

Thus π^{-1} preserves the edge relation and is therefore a tree isomorphism between K_2 and K_1. Hence $K_2 \cong_{\text{tree}} K_1$.

3. Transitivity. If $K_1 \cong_{\text{tree}} K_2$ via π and $K_2 \cong_{\text{tree}} K_3$ via σ, then $M = N$ and $N = P$, so $M = P$. For every feature f, both π and σ restrict to bijections on the feature instance set $\{f^1, \dots, f^{M(f)}\}$, so their composition does as well. Moreover,

$$f^i \prec^{(1)} g^j \iff \pi(f^i) \prec^{(2)} \pi(g^j) \iff \sigma(\pi(f^i)) \prec^{(3)} \sigma(\pi(g^j)).$$

Thus $\sigma \circ \pi$ is a tree isomorphism between K_1 and K_3 and $K_1 \cong_{\text{tree}} K_3$.

Since $\cong_{\text{tree}}$ is reflexive, symmetric, and transitive, it is an equivalence relation. □

Now that we know that $\cong_{\text{tree}}$ is an equivalence relation on the set of instance-based configurations of a CFM, we can define structural configurations as the corresponding equivalence classes.

Definition 3.9 (Structural Configuration)**.** Let $\mathcal{M}$ be a CFM. A *structural configuration* of $\mathcal{M}$ is an $\cong_{\text{tree}}$-equivalence class

$$[K]_{\text{Struct}} \coloneqq \left\{ K' \in \mathcal{C}_{\text{Inst}}(\mathcal{M}) \mid K' \cong_{\text{tree}} K \right\},$$

where $K \in \mathcal{C}_{\text{Inst}}(\mathcal{M})$.

$\mathcal{C}_{\text{Struct}}(\mathcal{M})$ denotes the set of structural configurations. Formally, $\mathcal{C}_{\text{Struct}}(\mathcal{M})$ is the set of $\cong_{\text{tree}}$-equivalence classes, i.e.,

$$\mathcal{C}_{\text{Struct}}(\mathcal{M}) \coloneqq \mathcal{C}_{\text{Inst}}(\mathcal{M}) \big/ \cong_{\text{tree}} .$$

A structural configuration $[K]_{\text{Struct}}$ is called *valid* if K is valid.

The *structural configuration space* of $\mathcal{M}$, denoted $[\![\mathcal{M}]\!]_{\text{Struct}}$, is the set of valid structural configurations of $\mathcal{M}$.

In the above definition, the validity of a structural configuration is determined via an instance-based representative K of the equivalence class $[K]_{\text{Struct}}$. We now show that this notion is well-defined; that is, validity does not depend on the particular representative K chosen for $[K]_{\text{Struct}}$.

Lemma 3.10 (Validity is invariant under Tree Isomorphism)**.** Let $\mathcal{M}$ be a CFM over F and let $K_1 = (M, \prec^{(1)})$ and $K_2 = (N, \prec^{(2)})$ be instance-based configurations of $\mathcal{M}$ such that $K_1 \cong_{\text{tree}} K_2$. Then K_1 is valid if and only if K_2 is valid.

Proof. Since $K_1 \cong_{\text{tree}} K_2$, we have $M = N$ and there exists a bijection $\pi\colon \text{Inst}(M) \to \text{Inst}(M)$ that restricts to a bijection on every feature instance set $\{f^1, \dots, f^{M(f)}\}$ and satisfies

$$f^i \prec^{(1)} g^j \iff \pi(f^i) \prec^{(2)} \pi(g^j)$$

for all feature instances f^i, g^j. Thus π preserves the decomposition relation, and in particular all parent-child relationships between feature instances.

The root constraint (i.e. $M(f_r) = 1$) and all require/exclude constraints depend only on the multiplicity function M, and therefore either hold for both configurations or for neither. Since π is a bijection preserving the decomposition relation, $(\text{Inst}(M), \prec^{(1)})$ forms a directed tree with root f_r^1 if and only if $(\text{Inst}(M), \prec^{(2)})$ does.

Furthermore, since π can only replace a feature instance by another instance of the same feature, the number of *feature instances* and the number of distinct *feature types* in each feature instance group remain invariant.

Thus, the feature-instance constraint, the group-instance constraint, and the group-type constraint hold in K_2 if and only if they hold in K_1.

In total, K_1 is valid if and only if K_2 is valid. □

3.2.2 Index-Free Tree Representation

Although structural configurations are defined abstractly as equivalence classes of instance-based configurations, they admit a natural index-free representation that makes their structure explicit.

Inductive Construction

Let $K = (M, \prec)$ be an instance-based configuration such that $(\mathrm{Inst}(M), \prec)$ forms a rooted tree with root instance f_r^1. (The configuration need not be valid; only a well-formed tree structure is required.)

We define recursively, for each instance $x \in \mathrm{Inst}(M)$, an object $\Phi_K^{\mathrm{tree}}(x)$:

- If x is a leaf, i.e. $C_K(x) = \emptyset$, then
$$\Phi_K^{\mathrm{tree}}(x) \coloneqq \Big(\mathrm{feat}(x), \{\!\{\}\!\}\Big).$$
- Otherwise,
$$\Phi_K^{\mathrm{tree}}(x) \coloneqq \Big(\mathrm{feat}(x), \{\!\{\, \Phi_K^{\mathrm{tree}}(y) \mid y \in C_K(x) \,\}\!\}\Big),$$
where $\{\!\{\cdot\}\!\}$ denotes a multiset.

The index-free representation of K is then defined as
$$\Phi^{\mathrm{tree}}(K) \coloneqq \Phi_K^{\mathrm{tree}}(f_r^1).$$

Independence of Representatives

If K_1 and K_2 are tree-isomorphic via π, then
$$\Phi_{K_1}^{\mathrm{tree}}(x) = \Phi_{K_2}^{\mathrm{tree}}(\pi(x)) \qquad \text{for all } x \in \mathrm{Inst}(M).$$

This follows directly, since π preserves feature labels and only swaps instance indices. Hence, for a structural configuration $[K]_{\mathrm{Struct}}$, we may define
$$\Phi^{\mathrm{tree}}([K]_{\mathrm{Struct}}) \coloneqq \Phi^{\mathrm{tree}}(K),$$
and this is independent of the chosen representative.

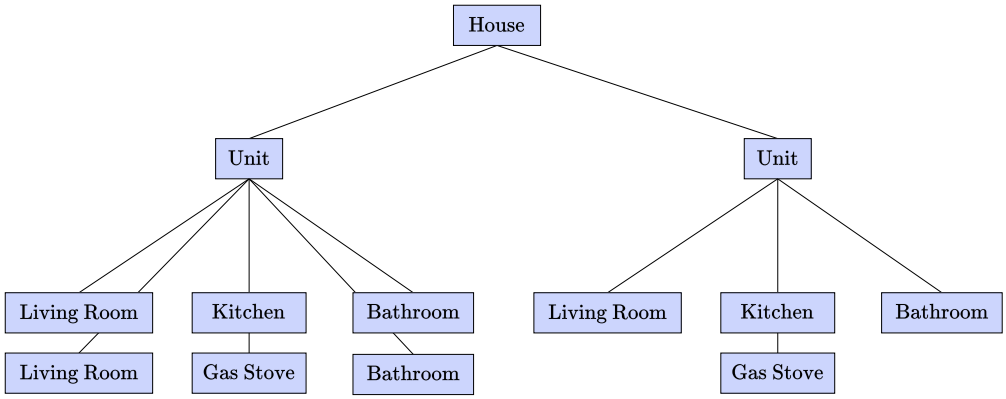
Conversely, every object of the form $\Phi^{\mathrm{tree}}([K]_{\mathrm{Struct}})$ can be converted back into a structural configuration by assigning arbitrary instance indices to its nodes, thereby obtaining an instance-based configuration K, and then taking its equivalence class $[K]_{\mathrm{Struct}}$. Since different index assignments yield tree-isomorphic configurations, the resulting structural configuration is independent of the chosen numbering.

Graphical Interpretation

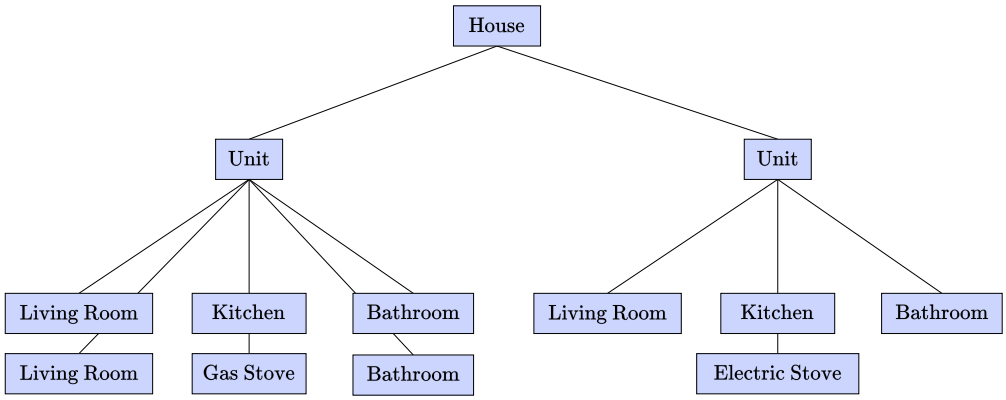
The recursive object $\Phi^{\mathrm{tree}}([K]_{\mathrm{Struct}})$ can simply be viewed as a directed tree whose nodes are labelled by features. Since feature-instance indices are ignored, multiple nodes may carry the same feature label.

This representation is particularly convenient for implementations. Instead of dealing with feature-instance indices or equivalence classes explicitly, a structural configuration can be represented directly as a basic tree data structure whose nodes store a feature identifier and a collection of child nodes.

Figure 3.8 shows the structural configurations induced by the instance-based configurations in Figure 3.7. The configurations in Figures 3.7a and 3.7b collapse to the same structural configuration, shown in Figure 3.8a because they are tree-isomorphic. In contrast, the configuration in Figure 3.7c yields the distinct structural configuration shown in Figure 3.8b.



(a) Structural configuration corresponding to Figures 3.7a and 3.7b.



(b) Structural configuration corresponding to Figure 3.7c.

Figure 3.8: Structural configurations obtained from the instance-based configurations in Figure 3.7. Tree-isomorphic configurations yield the same structural configurations.

3.2.3 Canonical Representation

While both the equivalence class representation and the index-free tree representation deliberately abstract from concrete feature-instance indices, there are situations in which this abstraction becomes inconvenient.

In particular, we lose the ability to refer directly to individual feature instances. In the index-free tree representation shown in Figure 3.8, for example, the configuration contains two instances of `Unit`, each of which contains a `Kitchen`. If we want to refer to a specific one, we can no longer simply write "`Unit`^{1}" or "`Kitchen`^{2}", since such identifiers no longer exist.

Instead, we would have to describe a node indirectly via its surrounding structure. For example, we might say: "the unit whose kitchen contains a gas stove".

This is cumbersome in practice and may even become ambiguous if two subtrees are structurally identical, for instance if both units are configured in exactly the same way.

We therefore introduce the notion of a *canonical (instance-based) configuration*. A canonical configuration is a *valid* instance-based configuration satisfying a special property (canonical criterion) that makes it unique within its equivalence class and thus yields a system of representatives for *valid* structural configurations. This not only resolves the issues discussed above by restoring stable and explicit instance identifiers, but, as we will see, it also reveals interesting connections with respect to the index anomalies discussed in Section 3.1.1.

3.2.3.1 Requirements for Canonical Configurations

In Section 3.1.1, we introduced two types of index anomalies in instance-based configurations: subtree index inversion and sibling index inversion. With the notion of tree-isomorphic configurations, we now gain additional insight into these phenomena.

Figure 3.9 shows a subtree index inversion. The two shown configurations are tree-isomorphic and therefore belong to the same structural configuration. Consequently, any canonical criterion that selects exactly one representative from each equivalence class must distinguish between such configurations. If it does not, it fails to single out a unique canonical element.

A natural requirement to exclude subtree index inversions is to enforce that instances with smaller indices occur below parent instances with smaller indices. We call this condition *subtree index monotonicity*. Under this rule, the configuration on the right-hand side is excluded, since it violates the monotonicity condition.

Figure 3.10 shows a sibling index inversion. Again, both configurations are tree-isomorphic and thus represent the same structural configuration. A canonical criterion must therefore also distinguish between such sibling permutations.

One way to prevent this anomaly is to enforce a fixed priority among competing subtrees. For instance, we may require that a subtree containing an instance of `B` must always be placed below a smaller-indexed `A` instance than a subtree containing an instance of `C`. More generally, whenever multiple sibling instances of the same

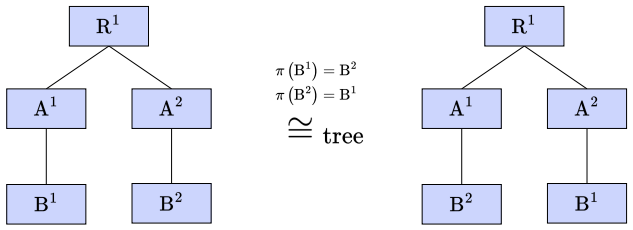


Figure 3.9: Index anomaly: Subtree index inversion

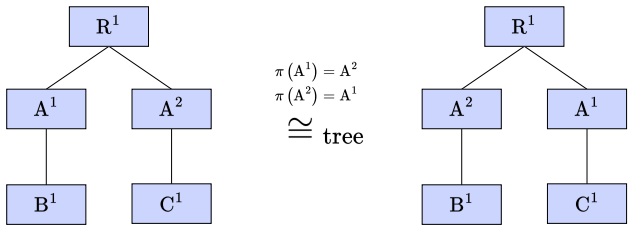


Figure 3.10: Index anomaly: Sibling index inversion

feature occur, higher-priority subtrees must be assigned to smaller indices. We call this condition *sibling index monotonicity*.

While subtree index monotonicity and sibling index monotonicity are sufficient to eliminate the anomalies in the depicted examples, it is not yet obvious that they suffice to uniquely determine a single instance-based configuration in every structural equivalence class. We will show that this is indeed the case.

To do so, we must formalize these conditions. In particular, sibling index monotonicity requires a systematic way of assigning priorities to subtrees. Hence, we first need a notion that allows us to compare and rank subtrees in a canonical manner.

3.2.3.2 Instance Order

Since we need a systematic way to compare feature instances and, in particular, to assign priorities to sibling instances when multiple instances of the same feature occur, we now introduce the notion of an *instance order*. The instance order is defined as a total preorder [74] on the feature instances of a configuration that canonically ranks them according to the structure of their subtrees. Intuitively, any two instances can be compared, and these comparisons are consistent; however, unlike total orders (e.g., $\leq$ on $\mathbb{R}$), distinct elements may still be equivalent with respect to the preorder. This is important because configurations can contain different instances with identical subtrees, and such instances should naturally receive the same rank.

The construction of this preorder is inspired by the algorithm of Aho, Hopcroft, and Ullman (AHU) for solving the tree isomorphism problem (in the sense of general graph isomorphism, rather than configuration isomorphism) [75, 76]. As in the AHU algorithm, the order is defined inductively; however, instead of computing a string encoding of the tree, we directly construct a preorder on the nodes.

Before introducing the formal definition of the instance order, we first define the *lexicographic preorder*. It is similar to the usual lexicographic order [77], but assumes

only a preorder on the underlying set and therefore induces merely a preorder on finite sequences over that set.

Definition 3.11 (Lexicographic Preorder)**.** Let $\trianglelefteq$ be a total preorder on a set X. That is, $\trianglelefteq \subseteq X \times X$ is a binary relation that is:

(i) Reflexive: $x \trianglelefteq x$ for all $x \in X$.

(ii) Transitive: $x \trianglelefteq y$ and $y \trianglelefteq z$ imply $x \trianglelefteq z$ for all $x, y, z \in X$.

(iii) Total: $x \trianglelefteq y$ or $y \trianglelefteq x$ for all $x, y \in X$.

In particular, in contrast to an order, it is not antisymmetric (that is, $x \trianglelefteq y$ and $y \trianglelefteq x$ do not imply $x = y$).

The *lexicographic preorder* induced by $\trianglelefteq$ on finite sequences over X is the relation

$$\trianglelefteq_{\text{lex}} \subseteq X^* \times X^*$$

defined as follows. For sequences $a = (a_1, \ldots, a_n)$ and $b = (b_1, \ldots, b_k)$, we set

$$(a_1, \ldots, a_n) \trianglelefteq_{\text{lex}} (b_1, \ldots, b_k) \text{ iff}$$

- either $n \leq k$ and, for all $i \in \{1, \ldots, n\}$, we have $a_i \trianglelefteq b_i$ and $b_i \trianglelefteq a_i$ (i.e. $(a_1, \ldots, a_n)$ is a prefix of $(b_1, \ldots, b_k)$ up to equivalence), or
- there exists a smallest index $\ell \in \{1, \ldots, \min(n, k)\}$ such that $a_\ell \trianglelefteq b_\ell$ and $b_\ell \not\trianglelefteq a_\ell$, and for all $i < \ell$ we have $a_i \trianglelefteq b_i$ and $b_i \trianglelefteq a_i$.

The empty sequence is defined to be lexicographically least, i.e.,

$$() \trianglelefteq_{\text{lex}} b$$

for all sequences b.

With this definition, the relation $\trianglelefteq_{\text{lex}}$ is again a total preorder.

The instance order is now defined as follows.

Definition 3.12 (Instance Order)**.** Let $\mathcal{M}$ be a CFM over the *ordered* feature set $(F, \leq_F)$, and let $K = (M, \prec)$ be a valid instance-based configuration of $\mathcal{M}$.

The *instance order* induced by K is the total preorder

$$\trianglelefteq \subseteq \text{Inst}(M) \times \text{Inst}(M)$$

defined by structural induction on the tree $(\text{Inst}(M), \prec)$.

Base case (leaves). Let $p, q \in \text{Inst}(M)$ be leaf instances with $\text{feat}(p) = g$ and $\text{feat}(q) = h$. Define

$$p \trianglelefteq q \quad :\Longleftrightarrow \quad g \leq_F h.$$

In particular, if $g = h$, then p and q are equivalent (i.e. both $p \trianglelefteq q$ and $q \trianglelefteq p$ hold).

Inductive step. Assume $\trianglelefteq$ is already defined for all children of p and q, and let $g = \text{feat}(p)$ and $h = \text{feat}(q)$.

If $g \neq h$, set

$$p \trianglelefteq q \quad :\Longleftrightarrow \quad g \leq_F h.$$

If $g = h$, consider the child sets $C_K(p)$ and $C_K(q)$. At least one of these sets is non-empty, since otherwise both p and q would be leaves and thus already covered by the base case. Since $\trianglelefteq$ is defined on all children by the induction hypothesis, we can enumerate these sets as sequences

$$(a_1, \ldots, a_n) \quad \text{and} \quad (b_1, \ldots, b_k)$$

such that each sequence is non-decreasing with respect to $\trianglelefteq$ (ties may be broken arbitrarily; the resulting preorder is independent of this choice).

We then set

$$p \trianglelefteq q \;\; :\Longleftrightarrow \;\; (a_1, \ldots, a_n) \trianglelefteq_{\text{lex}} (b_1, \ldots, b_k),$$

where $\trianglelefteq_{\text{lex}}$ denotes the lexicographic preorder induced by $\trianglelefteq$.

Example Computation

Figure 3.11 shows three instance-based configurations. Two of them were previously used in Section 3.1.1 to illustrate index anomalies. We now compute the induced instance order for each configuration.

Assume the feature set F is equipped with the order

$$\mathtt{R} \leq_F \mathtt{A} \leq_F \mathtt{B} \leq_F \mathtt{C}.$$

First configuration. For the configuration shown in Figure 3.11a, the instance order satisfies

$$\mathtt{R}^1 \trianglelefteq \mathtt{A}^1, \mathtt{A}^2 \trianglelefteq \mathtt{B}^1 \trianglelefteq \mathtt{C}^1.$$

This follows directly from the definition: instances with different underlying features are compared according to the feature order $\leq_F$.

The more interesting question is how $\mathtt{A}^1$ and $\mathtt{A}^2$ relate. Since they are instances of the same feature, their comparison is determined by their respective child sequences. The instance $\mathtt{A}^1$ has child $\mathtt{C}^1$, whereas $\mathtt{A}^2$ has child $\mathtt{B}^1$. Because

$$(\mathtt{B}^1) \trianglelefteq_{\text{lex}} (\mathtt{C}^1),$$

we obtain

$$\mathtt{A}^2 \trianglelefteq \mathtt{A}^1.$$

Thus, although both instances are $\mathtt{A}$-instances, the structure of their subtrees determines their relative position in the preorder.

This example also illustrates that the instance order depends crucially on the chosen feature order $\leq_F$. In particular, if we had $\mathtt{C} \leq_F \mathtt{B}$ instead, then the comparison of the child instances would reverse, and we would obtain

$$\mathtt{A}^1 \trianglelefteq \mathtt{A}^2.$$

Second configuration. In contrast, in the configuration shown in Figure 3.11b, the children of $\mathtt{A}^1$ and $\mathtt{A}^2$ are $\mathtt{B}^1$ and $\mathtt{B}^2$, respectively. Since these child instances are equivalent with respect to $\trianglelefteq$, we have

$$(\mathtt{B}^1) \trianglelefteq_{\text{lex}} (\mathtt{B}^2) \quad \text{and} \quad (\mathtt{B}^2) \trianglelefteq_{\text{lex}} (\mathtt{B}^1).$$

Consequently,

$$\mathtt{A}^1 \trianglelefteq \mathtt{A}^2 \quad \text{and} \quad \mathtt{A}^2 \trianglelefteq \mathtt{A}^1,$$

so the two instances are equivalent with respect to the instance order. This matches the intuition: their subtrees are structurally identical and are therefore treated equally by the preorder.

Third configuration. Finally, consider the configuration shown in Figure 3.11c. To compare $\mathtt{A}^1$ and $\mathtt{A}^2$, we again examine their child sequences.

The instance $\mathtt{A}^1$ has children $\mathtt{B}^1, \mathtt{C}^1, \mathtt{C}^2$, yielding the sequence

$$(\mathtt{B}^1, \mathtt{C}^1, \mathtt{C}^2),$$

whereas $\mathtt{A}^2$ has children $\mathtt{B}^2, \mathtt{C}^3$, yielding

$$(\mathtt{B}^2, \mathtt{C}^3).$$

Since all these instances are leaves, their comparison is determined solely by the feature order $\leq_F$. Consequently,

$$(\mathtt{B}^2, \mathtt{C}^3) \trianglelefteq_{\text{lex}} (\mathtt{B}^1, \mathtt{C}^1, \mathtt{C}^2),$$

and therefore

$$\mathtt{A}^2 \trianglelefteq \mathtt{A}^1.$$

3.2.3.3 Properties of Instance Order

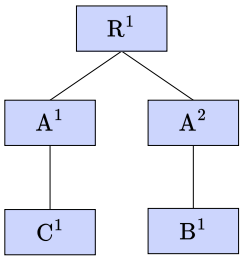
Although the definition of the instance order is straightforward, its intuitive meaning and, in particular, its interaction with tree isomorphisms are not immediately apparent.

This section collects several fundamental properties of the instance order that will later be used in the formal definition of canonical configurations.

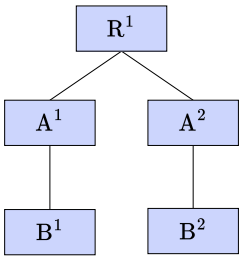
In particular, Theorem 3.13 shows that the instance order is preserved under tree isomorphisms; that is, it is invariant under renaming of instances and therefore compatible with configuration isomorphism.

Furthermore, Lemma 3.14 establishes basic structural properties of feature instances that are equivalent with respect to the instance order. In particular, such instances share the same feature type and possess pairwise equivalent children with identical multiplicities.

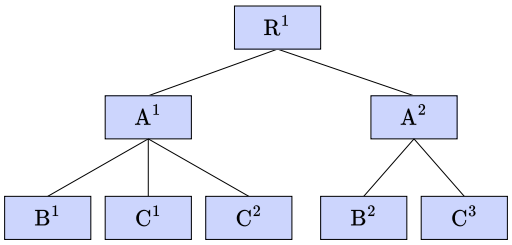
Finally, Theorem 3.15 provides a precise characterisation of equivalence in terms of tree isomorphism of the induced sub-configurations. This result directly links the notion of the instance order to the structural concept of sub-configurations.



(a) Configuration with non-tree-isomorphic A-subtrees, yielding $\mathtt{A}^2 \trianglelefteq \mathtt{A}^1$.



(b) Configuration with tree-isomorphic A-subtrees, yielding $\mathtt{A}^1$ and $\mathtt{A}^2$ $\trianglelefteq$-equivalent.



(c) Configuration with non-tree-isomorphic A-subtrees where $\mathtt{A}^2 \trianglelefteq \mathtt{A}^1$, since the sequence $(\mathtt{B}^2, \mathtt{C}^3)$ is lexicographically smaller than $(\mathtt{B}^1, \mathtt{C}^1, \mathtt{C}^2)$.

Figure 3.11: Example computations of the instance order. Configuration (a) yields a strict comparison between the two A-instances due to different child features, whereas (b) yields equivalence because the corresponding subtrees are tree-isomorphic. Configuration (c) illustrates a lexicographic comparison of longer child sequences.

Theorem 3.13 (Preservation of the Instance Order under Tree Isomorphism)**.** Let $K = (M, \prec)$ be a valid instance-based configuration of a CFM over an ordered feature set $(F, \leq_F)$, and let $\trianglelefteq$ denote the instance order induced by K.

Let $K' = (M, \prec')$ be a configuration that is tree-isomorphic to K (in particular it is also valid by Lemma 3.10), and let $\pi\colon \mathrm{Inst}(M) \to \mathrm{Inst}(M)$ be a tree isomorphism between them. Let $\trianglelefteq'$ denote the instance order induced by K'.

Then the instance order is invariant under π. That is, for all feature instances $p, q \in \mathrm{Inst}(M)$,

$$p \trianglelefteq q \iff \pi(p) \trianglelefteq' \pi(q).$$

Proof. We prove the theorem by structural induction.

Base case (leaves). Let $p, q \in \mathrm{Inst}(M)$ be leaf instances of features g and h, respectively (i.e. $\mathrm{feat}(p) = g$ and $\mathrm{feat}(q) = h$). By definition of $\trianglelefteq$,

$$p \trianglelefteq q \iff g \leq_F h.$$

Since π is a tree isomorphism, it preserves feature labels, that is, $\mathrm{feat}(\pi(p)) = g$ and $\mathrm{feat}(\pi(q)) = h$. Hence,

$$g \leq_F h \iff \mathrm{feat}(\pi(p)) \leq_F \mathrm{feat}(\pi(q)) \iff \pi(p) \trianglelefteq' \pi(q).$$

Thus, the claim holds for leaves.

Inductive step. Let $p, q \in \mathrm{Inst}(M)$ and assume that the statement holds for all children of p and q. Let $g = \mathrm{feat}(p)$ and $h = \mathrm{feat}(q)$.

If $g \neq h$, then by definition of the instance order,

$$p \trianglelefteq q \iff g \leq_F h.$$

As in the base case, π preserves feature labels, and therefore

$$g \leq_F h \iff \mathrm{feat}(\pi(p)) \leq_F \mathrm{feat}(\pi(q)) \iff \pi(p) \trianglelefteq' \pi(q).$$

Hence, the claim holds in the case $g \neq h$.

Now assume $g = h$. Let

$$C_K(p) = \{a_1, \ldots, a_n\}, \qquad C_K(q) = \{b_1, \ldots, b_k\},$$

where both sets are enumerated in non-decreasing order with respect to $\trianglelefteq$ in K (ties broken arbitrarily).

By the inductive hypothesis, for all children x, y of p or q,

$$x \trianglelefteq y \iff \pi(x) \trianglelefteq' \pi(y).$$

Since π is a tree isomorphism, it maps children of p bijectively to children of $\pi(p)$ and likewise for q. Consequently, the sequences

$$(\pi(a_1), \ldots, \pi(a_n)), \qquad (\pi(b_1), \ldots, \pi(b_k))$$

are precisely the child sets of $\pi(p)$ and $\pi(q)$, enumerated in non-decreasing order with respect to $\trianglelefteq'$ in K'.

By definition of the instance order in the case of equal features,

$$p \trianglelefteq q \iff (a_1, \ldots, a_n) \trianglelefteq_{\text{lex}} (b_1, \ldots, b_k).$$

Using the inductive hypothesis pointwise, this lexicographic comparison is preserved under π, and thus

$$(a_1, \ldots, a_n) \trianglelefteq_{\text{lex}} (b_1, \ldots, b_k) \iff (\pi(a_1), \ldots, \pi(a_n)) \trianglelefteq'_{\text{lex}} (\pi(b_1), \ldots, \pi(b_k)),$$

where $\trianglelefteq_{\text{lex}}$ and $\trianglelefteq'_{\text{lex}}$ denote the lexicographic preorders induced by $\trianglelefteq$ in K and by $\trianglelefteq'$ in K', respectively.

By the definition of $\trianglelefteq'$, this is equivalent to

$$\pi(p) \trianglelefteq' \pi(q).$$

Hence, in the case $g = h$, we obtain

$$p \trianglelefteq q \iff \pi(p) \trianglelefteq' \pi(q),$$

which completes the proof. □

Lemma 3.14 (Properties of Equivalent Feature Instances)**.** Let $K = (M, \prec)$ be a valid instance-based configuration of a CFM over an ordered feature set $(F, \leq_F)$, and let $\trianglelefteq$ be the instance order induced by K.

Let $p, q \in \text{Inst}(M)$ be two feature instances such that

$$p \trianglelefteq q \qquad \text{and} \qquad q \trianglelefteq p.$$

That is, p and q are equivalent with respect to $\trianglelefteq$. Then the following statements hold:

(1) p and q are instances of the same feature, i.e., $\text{feat}(p) = \text{feat}(q)$.

(2) p and q have the same number of children. Moreover, if

$$C_K(p) = \{a_1, \ldots, a_m\}, \qquad C_K(q) = \{b_1, \ldots, b_n\},$$

are enumerated in non-decreasing order with respect to $\trianglelefteq$ (ties broken arbitrarily), then $m = n$ and for every $\ell \in \{1, \ldots, m\}$, we have

$$a_\ell \trianglelefteq b_\ell \qquad \text{and} \qquad b_\ell \trianglelefteq a_\ell.$$

In other words, the children of p and q can be matched positionwise so that corresponding children are equivalent with respect to $\trianglelefteq$.

(3) For every feature $g \in F$, p and q have the same number of g-children, i.e.,

$$\big|C_K(p, g)\big| = \big|C_K(q, g)\big|.$$

Proof. Assume $p \trianglelefteq q$ and $q \trianglelefteq p$.

(1). Suppose, towards a contradiction, that $\text{feat}(p) \neq \text{feat}(q)$. Then, by the definition of the instance order for distinct features, we obtain

$$\text{feat}(p) \leq_F \text{feat}(q),$$

since $p \trianglelefteq q$, and

$$\text{feat}(q) \leq_F \text{feat}(p),$$

since $q \trianglelefteq p$. As $\leq_F$ is antisymmetric, this implies $\text{feat}(p) = \text{feat}(q)$, a contradiction. Hence $\text{feat}(p) = \text{feat}(q)$.

(2). If both p and q are leaves, then the claim holds trivially. If exactly one of them is a leaf, then p and q cannot be equivalent by the definition of $\trianglelefteq$ for equal features (the empty child sequence is strictly lexicographically smaller than any non-empty one). Thus, we may assume in the following that both $C_K(p)$ and $C_K(q)$ are non-empty.

Let

$$C_K(p) = \{a_1, \ldots, a_m\}, \qquad C_K(q) = \{b_1, \ldots, b_n\},$$

be enumerations in non-decreasing order with respect to $\trianglelefteq$.

By **(1)** we have $\text{feat}(p) = \text{feat}(q)$, and therefore the definition of $\trianglelefteq$ yields

$$p \trianglelefteq q \iff (a_1, \ldots, a_m) \trianglelefteq_{\text{lex}} (b_1, \ldots, b_n),$$

and

$$q \trianglelefteq p \iff (b_1, \ldots, b_n) \trianglelefteq_{\text{lex}} (a_1, \ldots, a_m).$$

Hence, the sequences $(a_1, \ldots, a_m)$ and $(b_1, \ldots, b_n)$ are mutually comparable in both directions under $\trianglelefteq_{\text{lex}}$, i.e., they are equivalent with respect to the lexicographic preorder.

By the definition of $\trianglelefteq_{\text{lex}}$, this is only possible if $m = n$ and, for every $\ell \in \{1, \ldots, m\}$,

$$a_\ell \trianglelefteq b_\ell \qquad \text{and} \qquad b_\ell \trianglelefteq a_\ell.$$

This proves **(2)**.

(3). By **(2)**, the child sequences $(a_1, \ldots, a_m)$ and $(b_1, \ldots, b_m)$ have the same length and consist of pairwise equivalent instances. By **(1)**, each pair (a_ℓ, b_ℓ) consists of instances of the same feature, i.e., $\text{feat}(a_\ell) = \text{feat}(b_\ell)$ for all ℓ.

Fix any feature $g \in F$. Then the number of indices ℓ with $\text{feat}(a_\ell) = g$ equals the number of indices ℓ with $\text{feat}(b_\ell) = g$. Equivalently,

$$\big|C_K(p, g)\big| = \big|C_K(q, g)\big|.$$

□

Theorem 3.15 (Characterisation of Equivalent Feature Instances)**.** Let $\mathcal{M}$ be a CFM, and let $K = (M, \prec)$ be a valid instance-based configuration of $\mathcal{M}$ with induced instance order $\trianglelefteq$.

For any two feature instances $f^i, f^j \in \text{Inst}(M)$, the following statements are equivalent:

(1) f^i and f^j are equivalent with respect to $\trianglelefteq$, i.e.

$$f^i \trianglelefteq f^j \qquad \text{and} \qquad f^j \trianglelefteq f^i.$$

(2) The induced sub-configurations rooted at f^i and f^j are tree-isomorphic, i.e.

$$K_{f^i} \cong_{\text{tree}} K_{f^j}.$$

Proof. We prove both implications separately.

(1) ⇒ (2)

Let $f^i, f^j \in \text{Inst}(M)$ be two feature instances that are equivalent with respect to the instance order $\trianglelefteq$. We construct an explicit tree isomorphism to show

$$K_{f^i} \cong_{\text{tree}} K_{f^j}.$$

The construction proceeds in two steps. First, we define a tree isomorphism between the original subtrees of K rooted at f^i and f^j. Second, we transfer this isomorphism to the renumbered sub-configurations via the renumbering maps from Definition 3.2.

Step 1: Recursive construction of a tree isomorphism between the rooted subtrees. We work in the original configuration $K = (M, \prec)$ and define a map

$$\pi\colon \text{Inst}(M)|_{f^i} \longrightarrow \text{Inst}(M)|_{f^j}$$

by recursion on the tree structure. The construction is top-down: we start at the roots f^i and f^j and extend π level by level towards the leaves.

Initial step (root). We set

$$\pi(f^i) \coloneqq f^j.$$

Recursive invariant. Whenever π is defined for node $p \in \text{Inst}(M)|_{f^i}$, we maintain

$$p \trianglelefteq \pi(p) \qquad \text{and} \qquad \pi(p) \trianglelefteq p.$$

This holds initially since f^i and f^j are equivalent by assumption.

Recursive step. Let $p \in \text{Inst}(M)|_{f^i}$ be such that $\pi(p)$ is already defined and the invariant holds. We extend π to the children of p.

Let the child sets of p and $\pi(p)$ in K be

$$C_K(p) = \{a_1, \ldots, a_m\}, \qquad C_K(\pi(p)) = \{b_1, \ldots, b_n\},$$

each enumerated in non-decreasing order with respect to $\trianglelefteq$ (ties broken arbitrarily). Since p and $\pi(p)$ are equivalent, Lemma 3.14 implies that $m = n$ and that, for every $\ell \in \{1, \ldots, m\}$, the instances a_ℓ and b_ℓ are equivalent with respect to $\trianglelefteq$.

We therefore extend π by setting

$$\pi(a_\ell) \coloneqq b_\ell \qquad \text{for all } \ell = 1, \ldots, m,$$

and continue recursively with each pair (a_ℓ, b_ℓ). By construction, the recursive invariant holds for all newly mapped children.

Termination. Since the subtree rooted at f^i is finite and each recursive step descends by one level, the procedure terminates after finitely many steps and defines π on all of $\text{Inst}(M)|_{f^i}$.

Properties of π. By construction, π has the following properties:

- **Bijectivity.** For every $p \in \mathrm{Inst}(M)|_{f^i}$, the restriction

$$\pi|_{C_K(p)} \colon C_K(p) \to C_K(\pi(p))$$

is a bijection. Since every node in the subtree has a unique parent, these local bijections imply that π is a bijection from $\mathrm{Inst}(M)|_{f^i}$ to $\mathrm{Inst}(M)|_{f^j}$. In particular, for every feature $g \in F$, the number of g-instances in both subtrees coincides, and thus

$$M|_{f^i} = M|_{f^j}.$$

- **Preservation of edges.** If $a \prec b$ in the induced subtree below f^i, then b is a child of a at some recursion step, and by construction $\pi(b)$ is a child of $\pi(a)$. Hence,

$$a \prec b \implies \pi(a) \prec \pi(b).$$

Conversely, since π is bijective and matches all children at every level, every edge below f^j is the image of exactly one edge below f^i. Therefore,

$$a \prec b \iff \pi(a) \prec \pi(b).$$

Step 2: Transfer to the renumbered sub-configurations. By Definition 3.2, the sub-configurations K_{f^i} and K_{f^j} are obtained from the induced subtrees via the renumbering bijections

$$\rho_i \colon \mathrm{Inst}(M)|_{f^i} \to \mathrm{Inst}(M|_{f^i}), \qquad \rho_j \colon \mathrm{Inst}(M)|_{f^j} \to \mathrm{Inst}(M|_{f^j}).$$

We define

$$\tilde{\pi} \coloneqq \rho_j \circ \pi \circ \rho_i^{-1} \colon \mathrm{Inst}(M|_{f^i}) \to \mathrm{Inst}(M|_{f^j}).$$

Since ρ_i and ρ_j preserve feature labels and edges, $\tilde{\pi}$ inherits the required properties:

- **Feature-wise bijection.** For each feature $g \in F$, the map π sends g-instances below f^i bijectively to g-instances below f^j, and ρ_i, ρ_j only renumber instances within each feature. Hence, for every $g \in F$,

$$\tilde{\pi}|_{\{g^1,\dots,g^{M|_{f^i}(g)}\}} \colon \{g^1,\dots,g^{M|_{f^i}(g)}\} \to \{g^1,\dots,g^{M|_{f^j}(g)}\}$$

is a bijection.

- **Preservation of edges.** Let (u, v) be an edge in K_{f^i}, and set $a = \rho_i^{-1}(u)$ and $b = \rho_i^{-1}(v)$. Then $a \prec b$ holds in the induced subtree below f^i. Since π preserves edges, we have $\pi(a) \prec \pi(b)$ below f^j. Applying ρ_j yields the edge

$$\tilde{\pi}(u) \prec_{K_{f^j}} \tilde{\pi}(v).$$

By bijectivity, the converse direction holds as well. Hence,

$$u \prec_{K_{f^i}} v \iff \tilde{\pi}(u) \prec_{K_{f^j}} \tilde{\pi}(v).$$

Thus $\tilde{\pi}$ satisfies the conditions of a tree isomorphism between K_{f^i} and K_{f^j}, and therefore

$$K_{f^i} \cong_{\mathrm{tree}} K_{f^j}.$$

(2) ⇒ (1)

We prove this direction by induction on the height of the sub-configurations K_{f^i} and K_{f^j}, denoted $\mathrm{ht}(K_{f^i})$ and $\mathrm{ht}(K_{f^j})$. This is possible since tree-isomorphic configurations have the same tree height. Let

$$h \coloneqq \mathrm{ht}(K_{f^i}) = \mathrm{ht}(K_{f^j}).$$

Base case ($h = 0$). If $h = 0$, then K_{f^i} and K_{f^j} consist only of their root instance. Hence f^i and f^j are leaves in K. By the definition of $\trianglelefteq$ for leaves, we obtain

$$f^i \trianglelefteq f^j \qquad \text{and} \qquad f^j \trianglelefteq f^i,$$

so f^i and f^j are equivalent.

Inductive step. Let $h \in \mathbb{N}_0$ and assume that the claim holds for all feature instances $u, v \in \mathrm{Inst}(M)$ such that

$$K_u \cong_{\text{tree}} K_v \quad \text{and} \quad \mathrm{ht}(K_u) = \mathrm{ht}(K_v) \leq h.$$

Now let $f^i, f^j \in \mathrm{Inst}(M)$ be feature instances satisfying

$$K_{f^i} \cong_{\text{tree}} K_{f^j} \qquad \text{and} \qquad \mathrm{ht}(K_{f^i}) = \mathrm{ht}(K_{f^j}) = h + 1,$$

and let π be a tree isomorphism between K_{f^i} and K_{f^j}.

Since π is bijective and preserves the decomposition relation, the root instance (which is f^1 in both sub-configurations after renumbering) has the same number of children in both configurations. Let

$$C_{K_{f^i}}(f^1) = \{a_1, \ldots, a_m\} \qquad \text{and} \qquad C_{K_{f^j}}(f^1) = \{b_1, \ldots, b_m\}.$$

Moreover, π induces a bijection between these child sets, so for each $k \in \{1, \ldots, m\}$ we have $\pi(a_k) \in \{b_1, \ldots, b_m\}$, and the induced sub-configurations satisfy

$$(K_{f^i})_{a_k} \cong_{\text{tree}} (K_{f^j})_{\pi(a_k)}.$$

Thus, we can group the children into pairs

$$(a_1, b_{i_1}), \ldots, (a_m, b_{i_m}),$$

such that

$$(K_{f^i})_{a_k} \cong_{\text{tree}} (K_{f^j})_{b_{i_k}}$$

for all k, and each of these sub-sub-configurations has height at most h.

At this point, we cannot apply the induction hypothesis directly to a_k and b_{i_k}, since a_k and b_{i_k} are instances inside the renumbered sub-configurations, and in general $(K_{f^i})_{a_k} \neq K_{a_k}$ (this happens because of renumbering). We therefore transfer the situation back to the original configuration K.

Let

$$\rho_i \colon \mathrm{Inst}(M)|_{f^i} \to \mathrm{Inst}(M|_{f^i}), \qquad \rho_j \colon \mathrm{Inst}(M)|_{f^j} \to \mathrm{Inst}(M|_{f^j})$$

be the renumbering maps from Definition 3.2. For each k, define the corresponding original children in K by

$$x_k \coloneqq \rho_i^{-1}(a_k), \qquad y_{i_k} \coloneqq \rho_j^{-1}(b_{i_k}).$$

By definition, x_k and y_{i_k} are children of f^i and f^j in K, and

$$C_K(f^i) = \{x_1, \dots, x_m\}, \qquad C_K(f^j) = \{y_{i_1}, \dots, y_{i_m}\}.$$

Furthermore, by construction of sub-configurations,

$$(K_{f^i})_{a_k} = K_{x_k}, \qquad (K_{f^j})_{b_{i_k}} = K_{y_{i_k}}.$$

Since $(K_{f^i})_{a_k} \cong_{\text{tree}} (K_{f^j})_{b_{i_k}}$, we obtain $K_{x_k} \cong_{\text{tree}} K_{y_{i_k}}$. Moreover, these sub-configurations have height at most h. Hence, by the induction hypothesis,

$$x_k \trianglelefteq y_{i_k} \qquad \text{and} \qquad y_{i_k} \trianglelefteq x_k$$

for all k.

We now compare f^i and f^j in the global instance order. Without loss of generality, assume that the children of f^i are enumerated in non-decreasing order with respect to $\trianglelefteq$, i.e.

$$x_1 \trianglelefteq \cdots \trianglelefteq x_m.$$

Since each y_{i_k} is equivalent to x_k, it follows that

$$y_{i_1} \trianglelefteq \cdots \trianglelefteq y_{i_m},$$

that is, the children of f^j can be enumerated in non-decreasing order so that they are pairwise equivalent to the corresponding children of f^i.

Therefore, the two sorted child sequences are equivalent under the lexicographic preorder induced by $\trianglelefteq$. By the definition of the instance order for equal features, this implies

$$f^i \trianglelefteq f^j \qquad \text{and} \qquad f^j \trianglelefteq f^i,$$

and thus f^i and f^j are equivalent with respect to $\trianglelefteq$. □

3.2.3.4 Canonicality Criterion

We have now developed all ingredients required for a formal definition of canonical configurations. As motivated in Section 3.2.3.1, the canonicality criterion consists of two conditions: *subtree index monotonicity* and *sibling index monotonicity*.

Definition 3.16 (Canonical Configuration)**.** Let $\mathcal{M}$ be a CFM over an ordered feature set $(F, \leq_F)$, and let $K = (M, \prec)$ be a *valid* instance-based configuration. Let $\trianglelefteq$ be the instance order induced by K.

We call K *canonical* if for every feature $f \in F$ with instances $f^1, \dots, f^{M(f)} \in \text{Inst}(M)$, the following hold for all indices $i, j \in \{1, \dots, M(f)\}$:

1. **Sibling index monotonicity.** If f^i and f^j have the same parent instance (i.e. they are siblings), then

$$i \leq j \implies f^i \trianglelefteq f^j.$$

That is, sibling instances of the same feature appear in non-decreasing order with respect to $\trianglelefteq$.

2. **Subtree index monotonicity.** For every feature $p \in F$, if $p^r, p^s \in \mathrm{Inst}(M)$ are ancestors of f^i and f^j, respectively, then

$$i \leq j \implies r \leq s.$$

In particular, if $r < s$, then for every feature $g \in F$, all g-instances in the subtree below p^r receive strictly smaller indices than the g-instances in the subtree below p^s.

We denote by $[\![\mathcal{M}]\!]_{\mathrm{Canon}}$ the set of canonical configurations of $\mathcal{M}$.

An immediate structural consequence of these two canonicality conditions is that child instances of a fixed feature must appear in well-defined, contiguous index blocks below their respective parent instances. This block structure is made precise in the following lemma.

Lemma 3.17 (Consecutive Block Decomposition of Child Indices)**.** Let $\mathcal{M}$ be a CFM over an ordered feature set $(F, \leq_F)$, and let $K = (M, \prec)$ be a canonical configuration of $\mathcal{M}$.

Fix a feature $p \in F$ and one of its child features $f \in F$. For each instance p^i of p, let

$$n_i \coloneqq \left|C_K(p^i, f)\right|,$$

i.e. the number of children of p^i that are instances of f.

For each $i \in \{1, \ldots, M(p)\}$, define

$$k_i \coloneqq 1 + \sum_{j<i} n_j.$$

Then, for every i with $n_i > 0$, the set of f-children of p^i, $C_K(p^i, f)$, equals

$$\left\{f^{k_i}, f^{k_i+1}, \ldots, f^{k_i+n_i-1}\right\}.$$

In other words, for fixed feature $p, f \in F$, the f-children of the feature instances $p^1, p^2, \ldots$ form disjoint, consecutive index blocks, ordered according to the index of the parent.

Proof. Fix features $p, f \in F$. For each $i \in \{1, \ldots, M(p)\}$, let

$$S_i \coloneqq \{\, k \in \{1, \ldots, M(f)\} \mid p^i \prec f^k \,\}$$

be the set of indices of the f-children of p^i.

By subtree index monotonicity in canonical configurations, if $i < j$ then every instance of f below p^i has a strictly smaller index than every instance of f below p^j. Thus, for all $i < j$ and all $k \in S_i$, $\ell \in S_j$ we have $k < \ell$. Since every instance of f has a unique parent instance of feature p, the sets $S_1, \ldots, S_{M(p)}$ form a partition of $\{1, \ldots, M(f)\}$ into strictly ordered, non-interleaving blocks.

Since $n_i = |S_i|$ and S_i is the i-th block, it must consist exactly of the consecutive indices $\{k_i, k_i + 1, \ldots, k_i + n_i - 1\}$, which proves the claim. □

With this lemma, we can now show that every equivalence class of valid instance-based configurations contains exactly one canonical configuration. In particular, canonical configurations form a system of representatives for the structural configuration space.

Theorem 3.18 (Existence and Uniqueness of Canonical Representatives)**.** Let $\mathcal{M}$ be a CFM over the ordered feature set $(F, \leq_F)$. Then the following statements hold:

(1) **Existence.** For every configuration $K = (M, \prec) \in [\![\mathcal{M}]\!]_{\text{Inst}}$, there exists a canonical configuration $K^{\text{can}} = (M, \prec^{\text{can}}) \in [\![\mathcal{M}]\!]_{\text{Inst}}$ such that $K \cong_{\text{tree}} K^{\text{can}}$.

(2) **Uniqueness.** If $K_1 = (M, \prec^{(1)})$ and $K_2 = (M, \prec^{(2)})$ are canonical configurations and $K_1 \cong_{\text{tree}} K_2$, then $K_1 = K_2$.

Hence, every equivalence class in $[\![\mathcal{M}]\!]_{\text{Inst}}$ under $\cong_{\text{tree}}$ contains exactly one canonical configuration. Consequently, the map

$$K \longmapsto [K]_{\text{Struct}}$$

induces a bijection between the canonical configuration space $[\![\mathcal{M}]\!]_{\text{Canon}}$ and the structural configuration space $[\![\mathcal{M}]\!]_{\text{Struct}}$.

Proof. We prove (1) and (2) separately.

(1) Existence

Let $K = (M, \prec)$ be an arbitrary valid instance-based configuration. We construct a canonical configuration K^{can} by renaming instance indices via a depth-first traversal of the configuration tree.

Concretely, we traverse the tree $(\text{Inst}(M), \prec)$ as follows:

- Start at the root instance f_r^1.
- When visiting a feature instance $p \in \text{Inst}(M)$, sort its children in increasing order with respect to the total preorder $\trianglelefteq$. If there are distinct children $a, b \in C_K(p)$ such that $a \trianglelefteq b$ and $b \trianglelefteq a$ (i.e. a and b are equivalent under $\trianglelefteq$), break ties arbitrarily among them.
- Recurse depth-first into the children in this order.

For each feature $f \in F$, introduce a counter

$$\alpha_f \coloneqq 0.$$

When we visit an instance f^i during the traversal, we increment

$$\alpha_f \coloneqq \alpha_f + 1$$

and set

$$\sigma_f(i) \coloneqq \alpha_f.$$

Since each instance of f is visited exactly once, σ_f is a bijection on $\{1, \dots, M(f)\}$.

After the traversal we define

$$f^{\sigma_f(i)} \prec^{\text{can}} g^{\sigma_g(j)} \quad :\Longleftrightarrow \quad f^i \prec g^j.$$

Thus, by construction, the configuration $K^{\text{can}} = (M, \prec^{\text{can}})$ is tree-isomorphic to K. Concretely, the tree isomorphism is given by

$$\pi \colon \operatorname{Inst}(M) \to \operatorname{Inst}(M), \qquad \pi(f^i) \coloneqq f^{\sigma_f(i)}.$$

Since validity is invariant under tree isomorphism (Lemma 3.10), and K is valid, it follows that K^{can} is also valid.

It remains to show that K^{can} is canonical.

Let $\trianglelefteq^{\text{can}}$ be the instance order induced by K^{can}.

(a) Sibling index monotonicity. Let $p \in \operatorname{Inst}(M)$ be a parent instance with children $f^i, f^j \in \operatorname{Inst}(M)$ in K^{can} and $i \leq j$.

Their preimages $\pi^{-1}(f^i)$ and $\pi^{-1}(f^j)$ are children of $\pi^{-1}(p)$ in K. Since children of $\pi^{-1}(p)$ are visited in increasing order with respect to $\trianglelefteq$, we have

$$\pi^{-1}(f^i) \trianglelefteq \pi^{-1}(f^j), \tag{1}$$

because otherwise $\pi^{-1}(f^j)$ would have been visited earlier and would therefore have received the smaller index.

By preservation of the instance order under tree isomorphism (Theorem 3.13), eq. (1) implies

$$f^i = \pi\Big(\pi^{-1}(f^i)\Big) \trianglelefteq^{\text{can}} \pi\Big(\pi^{-1}(f^j)\Big) = f^j.$$

Thus, sibling index monotonicity holds.

(b) Subtree index monotonicity. Let f^i and f^j be feature instances in K^{can} with ancestors $p^r, p^s \in \operatorname{Inst}(M)$, and assume $i < j$.

Since π is a tree isomorphism, $\pi^{-1}(p^r)$ is an ancestor of $\pi^{-1}(f^i)$ in K and $\pi^{-1}(p^s)$ is an ancestor of $\pi^{-1}(f^j)$ in K.

Because indices are assigned according to the order in which instances are first visited during the depth-first traversal, the inequality $i < j$ implies that the instance $\pi^{-1}(f^i)$ is visited strictly before $\pi^{-1}(f^j)$.

In a depth-first traversal, whenever a node is visited, the entire subtree rooted at that node is explored before the traversal returns to visit a later sibling or any node in its subtree. Consequently, if $\pi^{-1}(f^i)$ is visited before $\pi^{-1}(f^j)$, then every ancestor of $\pi^{-1}(f^i)$ is visited no later than every ancestor of $\pi^{-1}(f^j)$.

In particular, $\pi^{-1}(p^r)$ is visited no later than $\pi^{-1}(p^s)$. Since the indices of p-instances are assigned in the order in which they are visited during the traversal, this implies

$$r \leq s.$$

Thus, subtree index monotonicity holds, and therefore K^{can} is canonical.

(2) Uniqueness

Let $K_1 = (M, \prec^{(1)})$ and $K_2 = (M, \prec^{(2)})$ be canonical configurations such that $K_1 \cong_{\text{tree}} K_2$. Let $\trianglelefteq^{(1)}$ and $\trianglelefteq^{(2)}$ denote their respective global instance orders.

By assumption, there is a tree isomorphism π which, for each feature $f \in F$, induces a bijection

$$\sigma_f : \{1, \ldots, M(f)\} \to \{1, \ldots, M(f)\}, \quad \text{with} \quad \pi(f^i) = f^{\sigma_f(i)}.$$

Since π is a tree isomorphism, it preserves the number of edges, so

$$\left|\prec^{(1)}\right| \;=\; \left|\prec^{(2)}\right|.$$

Therefore, to prove $\prec^{(1)}{=}\prec^{(2)}$, it suffices to show that every edge in K_1 is also an edge in K_2, i.e. that

$$\forall\, f^i, g^j \in \text{Inst}(M): \qquad f^i \prec^{(1)} g^j \implies f^i \prec^{(2)} g^j. \tag{$*$}$$

Step 1: π only permutes equivalent feature instances. We show that for every feature $f \in F$ and every instance $f^a \in \text{Inst}(M)$, if

$$\pi(f^a) = f^b,$$

then f^a and f^b are equivalent with respect to $\trianglelefteq^{(1)}$. The proof proceeds by induction on the depth of f in the feature tree $(F, \prec_F)$.

Base case. The only feature of depth 0 is the root feature f_r. Since both configurations are valid, $M(f_r) = 1$, so there is only one root instance f_r^1. Hence,

$$\pi(f_r^1) = f_r^1,$$

and f_r^1 is trivially equivalent to itself.

Inductive step. Let f be a feature of depth $d > 0$ and let p be its parent feature in the feature tree. Assume that the statement holds for all features of depth smaller than d.

Let f^a be an instance of f in K_1 and suppose

$$\pi(f^a) = f^b.$$

Let p^r be the parent of f^a in K_1. Since π is a tree isomorphism between K_1 and K_2, the parent of f^b in K_2 is $\pi(p^r)$, so we write

$$\pi(p^r) = p^s.$$

By the induction hypothesis, p^r and p^s are equivalent with respect to $\trianglelefteq^{(1)}$.

Now consider the f-children of these parent instances. Let

$$C_{K_1}(p^r, f) = \{a_1, \ldots, a_m\}$$

be the set of f-children of p^r in K_1, enumerated in non-decreasing order with respect to $\trianglelefteq^{(1)}$. Since K_1 is canonical, the natural index order of these siblings (that is, the order on $\mathbb{N}_0$) is such an enumeration. Likewise, let

$$C_{K_2}(p^s, f) = \{b_1, \ldots, b_m\}$$

be the set of f-children of p^s in K_2, enumerated in non-decreasing order with respect to $\trianglelefteq^{(2)}$. Again, since K_2 is canonical, the natural index order of these siblings provides such an enumeration.

Both sets have the same cardinality m, since π is a tree isomorphism, $\pi(p^r) = p^s$, and thus it maps the f-children of p^r in K_1 bijectively to the f-children of p^s in K_2.

Moreover, since π preserves the instance order (Theorem 3.13), it preserves the relative ordering of feature instances. In particular, if two instances are equivalent with respect to the instance order in K_1, then their images under π are equivalent in K_2, and if one instance strictly precedes another in the instance order of K_1, then the same holds for their images in K_2.

Consequently, when the f-children of p^r in K_1 and the f-children of p^s in K_2 are listed in non-decreasing order with respect to their respective instance orders, the map π preserves the grouping of elements into maximal equivalence blocks and preserves the order of these blocks.

More precisely, let

$$(a_1, \ldots, a_m)$$

be decomposed into maximal consecutive equivalence blocks under $\trianglelefteq^{(1)}$,

$$A_1, \ldots, A_u,$$

and similarly decompose

$$(b_1, \ldots, b_m)$$

into maximal consecutive equivalence blocks under $\trianglelefteq^{(2)}$,

$$B_1, \ldots, B_v.$$

Because π preserves the preorder, if $x, x' \in A_i$ then $\pi(x)$ and $\pi(x')$ are equivalent in K_2, so $\pi(A_i) \subseteq B_j$ for some j. Furthermore, if $x \in A_i$ and $y \in A_{i'}$ with $i < i'$, then

$$x \trianglelefteq^{(1)} y \qquad \text{and} \qquad y \not\trianglelefteq^{(1)} x,$$

which implies

$$\pi(x) \trianglelefteq^{(2)} \pi(y) \qquad \text{and} \qquad \pi(y) \not\trianglelefteq^{(2)} \pi(x).$$

Hence, the block containing $\pi(x)$ precedes the block containing $\pi(y)$. Thus π induces an order-preserving injection from the A-blocks to the B-blocks. Applying the same argument to π^{-1} shows that this correspondence is bijective, so $u = v$ and $\pi(A_i) = B_i$ for all i.

On the other hand, since p^r and p^s are equivalent with respect to $\trianglelefteq^{(1)}$ (induction hypothesis), Lemma 3.14 implies that when the f-children of p^r and p^s are enumerated in non-decreasing order with respect to $\trianglelefteq^{(1)}$ (or enumerated non-decreasingly with respect to their index order, since K_1 is canonical), corresponding positions contain pairwise equivalent instances.

Combining these observations shows that f^a and $\pi(f^a) = f^b$ occupy corresponding positions within matching equivalence blocks, and hence

$$f^a \trianglelefteq^{(1)} f^b \qquad \text{and} \qquad f^b \trianglelefteq^{(1)} f^a.$$

Thus f^a and $\pi(f^a)$ are equivalent with respect to $\trianglelefteq^{(1)}$, which completes the inductive step.

Step 2: K_1 and K_2 have identical block structure. To apply the consecutive-block lemma (Lemma 3.17) in both configurations, we must show that the block sizes are the same. For this, fix a parent feature $p \in F$ and a child feature $f \in F$, and define

$$n_i^{(1)} := \left|C_{K_1}(p^i, f)\right|, \qquad n_i^{(2)} := \left|C_{K_2}(p^i, f)\right|.$$

We now show that $n_i^{(1)} = n_i^{(2)}$ for every $i \in \{1, \dots, M(p)\}$.

By Step 1, π may only permute equivalent p-instances, that is, if $\pi(p^j) = p^i$, then p^j and p^i are equivalent with respect to $\trianglelefteq^{(1)}$.

We now compute:

$$n_i^{(2)} = \left|C_{K_2}(p^i, f)\right| \overset{(1)}{=} \left|C_{K_2}(\pi(p^j), f)\right| = \left|C_{K_1}(p^j, f)\right| \overset{(2)}{=} \left|C_{K_1}(p^i, f)\right| = n_i^{(1)},$$

where in (1) we used that $\pi(p^j) = p^i$, so the f-children of p^i in K_2 are exactly the f-children of $\pi(p^j)$ in K_2, and in (2) we used Lemma 3.14 (3), since p^j and p^i are equivalent in K_1.

Because the block boundaries in a canonical configuration are defined by

$$k_i = 1 + \sum_{t<i} n_t,$$

it follows immediately that $k_i^{(1)} = k_i^{(2)}$ for all i.

Thus, the block structure of the f-children of p is identical in K_1 and K_2.

Step 3: Every edge of K_1 is also an edge of K_2. Let $p^i \prec^{(1)} f^j$ be an arbitrary edge in K_1. We show that $p^i \prec^{(2)} f^j$ in K_2.

By the consecutive block lemma (Lemma 3.17)

$$j \in \{\, k_i, \ldots, k_i + n_i - 1 \,\},$$

where $n_i = |C_{K_1}(p^i, f)|$ and $k_i = 1 + \sum_{t<i} n_t$.

By Step 2, the values n_i and k_i are the same in K_1 and K_2. Therefore, j lies in the f-child block of p^i in K_2 as well, i.e. $p^i \prec^{(2)} f^j$.

As $p^i \prec^{(1)} f^j$ was arbitrary, this proves $(*)$. Since the two edge relations have the same cardinality and $\prec^{(1)} \subseteq \prec^{(2)}$ by $(*)$, we conclude that

$$\prec^{(1)} = \prec^{(2)} .$$

□

3.2.4 Merging Canonical Configurations

In Section 3.1.4, we showed that instance-based configurations are not closed under merge (that is, they do not satisfy the merge property). For canonical configurations, and consequently for structural configurations, this property does hold. If we merge canonical child configurations using MERGE, the resulting configuration is again canonical, provided that, for each child feature, the corresponding child configurations are sorted in increasing order with respect to the instance order. Moreover, every canonical configuration of the full model can be obtained in this way.

Note that the instance order is defined only for a single configuration, whereas in the following we consider several configurations whose feature instances may overlap. Nevertheless, two canonical configurations can be compared by applying the same recursive structural comparison used in the definition of the instance order (see Definition 3.12). In particular, whenever we write "let $K_1, \ldots, K_n$ be configurations of $\mathcal{M}$ sorted increasingly with respect to the instance order", we mean that they are ordered according to this structural comparison.

Theorem 3.19 (Merging Canonical Configurations)**.** Let $\mathcal{M}$ be a CFM without cross-tree constraints with root feature f_r over an ordered feature set $(F, \leq_F)$, and let

$$C_{\mathcal{M}}(f_r) = \{c_1, \ldots, c_n\}.$$

For each child c_i, let

$$K_{i,1},\ K_{i,2},\ \ldots,\ K_{i,k_i}$$

be canonical configurations of the sub-model $\mathcal{M}_{c_i}$ (i.e. child configurations). If, for each fixed i, the configurations $K_{i,1}, \ldots, K_{i,k_i}$ are sorted increasingly with respect to the instance order, and if the multiplicities $k_1, \ldots, k_n$ satisfy the feature-instance cardinalities as well as the group-instance and group-type cardinalities at the root feature f_r, then the output configuration of MERGE is a canonical configuration of the full model $\mathcal{M}$.

Proof. Let $(M, \prec)$ be the configuration returned by MERGE, and let $\trianglelefteq$ denote the instance order induced by $(M, \prec)$.

Since $\mathcal{M}$ has no cross-tree constraints and the multiplicities $k_1, \ldots, k_n$ satisfy the feature-instance, group-instance, and group-type cardinality constraints at the root feature f_r, and since each $K_{i,j}$ is a valid configuration of the sub-model $\mathcal{M}_{c_i}$, MERGE yields a valid configuration $(M, \prec)$ of the full model $\mathcal{M}$. It therefore remains to show that $(M, \prec)$ is canonical.

1. Sibling index monotonicity in $(M, \prec)$

First, observe that MERGE does not modify the tree structure of any child configuration $K_{i,j}$. It only shifts feature-instance indices and then takes the disjoint union of all shifted trees before adding the root instance f_r^1. Since shifting only adds feature-wise constant offsets to instance indices, the relative order of sibling indices inside each subtree is preserved. Hence, all sibling relations that lie inside a shifted configuration $\widetilde{K}_{i,j}$ remain sibling-consistent (i.e. sibling index monotonicity holds) because each $K_{i,j}$ is canonical.

The only sibling relations that lie outside a shifted configuration are the new relations we add at the root instance f_r^1. For a fixed child feature c_i, these siblings are exactly the instances

$$(c_i)^1, \ldots, (c_i)^{k_i}$$

created by the successive shifting of the configurations $K_{i,1}, \ldots, K_{i,k_i}$. By assumption of the theorem, these instances are sorted in increasing order with respect to the global instance order $\trianglelefteq$, that is,

$$(c_i)^1 \trianglelefteq \cdots \trianglelefteq (c_i)^{k_i}.$$

Hence, sibling index monotonicity also holds for all children of the root feature.

2. Subtree index monotonicity in $(M, \prec)$

We must show that for every feature f and every feature p, if p^r and p^s are instances of p such that p^r is an ancestor of f^i and p^s is an ancestor of f^j and $i \leq j$, then $r \leq s$.

Again, we distinguish two situations.

Case 1: p^r **and** p^s **lie in the same shifted child configuration.** Since each feature is shifted by a feature-wise constant offset and the original child configurations are canonical, subtree index monotonicity continues to hold inside each shifted configuration.

Case 2: p^r **and** p^s **lie in different shifted child configurations.** If $p^r = p^s = f_r^1$, then $r \leq s$ holds trivially, since $r = s = 1$. Otherwise, in a valid merged configuration the ancestor p^r lies in the same shifted child configuration as f^i. Likewise, p^s lies in the same shifted child configuration as f^j. Since MERGE shifts and adds whole child configurations at once, it follows that if $i \leq j$, then $r \leq s$. □

We now show that every canonical configuration of the full model can be obtained by applying Merge to a suitable family of canonical configurations of its sub-models. A prerequisite for this result is that canonicality is inherited by sub-configurations. The following theorem shows exactly this property.

Theorem 3.20 (Canonicality is preserved by Sub-Configurations)**.** Let $\mathcal{M}$ be a CFM without cross-tree constraints over an ordered feature set $(F, \leq_F)$, and let $K = (M, \prec)$ be a canonical configuration of $\mathcal{M}$. Then, for every feature $f \in F$ and every instance $f^i \in \mathrm{Inst}(M)$, the sub-configuration K_{f^i} is a canonical configuration of the sub-model $\mathcal{M}_f$. In particular, canonicality is inherited by sub-configurations.

Proof. Fix a feature $f \in F$ and an instance $f^i \in \mathrm{Inst}(M)$. We must show that K_{f^i} satisfies the two canonicality conditions from Definition 3.16.

1. Sibling index monotonicity

Let

$$g^r,\ g^s \in \mathrm{Inst}(M|_{f^i})$$

be two sibling instances in K_{f^i} with the same parent instance $p^t \in \mathrm{Inst}(M|_{f^i})$, and assume $r \leq s$.

By Definition 3.2, these instances arise by renumbering the original feature instances

$$\rho^{-1}(g^r),\ \rho^{-1}(g^s),\ \rho^{-1}(p^t) \in \mathrm{Inst}(M)|_{f^i}$$

via the renumbering map

$$\rho\colon\ \mathrm{Inst}(M)|_{f^i} \to \mathrm{Inst}(M|_{f^i}).$$

Let $g^{i_r}, g^{i_s}, p^{i_t} \in \mathrm{Inst}(M)|_{f^i}$ be the unique feature instances satisfying

$$\rho(g^{i_r}) = g^r, \qquad \rho(g^{i_s}) = g^s, \qquad \text{and} \qquad \rho(p^{i_t}) = p^t.$$

By construction of ρ, feature instances of the same feature are renumbered in increasing order of their original indices. Hence, $r \leq s$ implies $i_r \leq i_s$.

Since K is canonical, sibling index monotonicity in K yields

$$g^{i_r} \trianglelefteq g^{i_s},$$

where $\trianglelefteq$ is the instance order induced by K.

Since ρ preserves feature types and the tree structure and only renumbers feature instances, the instance-order comparison is preserved, and we obtain

$$g^r \trianglelefteq_{K_{f^i}} g^s,$$

where $\trianglelefteq_{K_{f^i}}$ is the instance order induced by K_{f^i}.

2. Subtree index monotonicity

Let $g \in F$, and let $g^r, g^s \in \mathrm{Inst}(M|_{f^i})$ be two instances of g in K_{f^i}. Assume that there exist instances $p^u, p^v \in \mathrm{Inst}(M|_{f^i})$ such that p^u is an ancestor of g^r and p^v is an ancestor of g^s in K_{f^i}, and that $r \leq s$.

Let $g^{i_r}, g^{i_s}, p^{i_u}, p^{i_v} \in \mathrm{Inst}(M)|_{f^i}$ be the corresponding feature instances in K. Then p^{i_u} is an ancestor of g^{i_r} and p^{i_v} is an ancestor of g^{i_s} in K, and by construction of ρ we have $r \leq s$ implies $i_r \leq i_s$.

Since K is canonical, subtree index monotonicity in K yields

$$i_r \leq i_s \implies i_u \leq i_v.$$

Translating back to the renumbered configuration K_{f^i}, this implies $u \leq v$, and thus subtree index monotonicity holds in K_{f^i}.

Since the model has no cross-tree constraints, K_{f^i} is valid and therefore a canonical configuration. □

With this theorem established, we can now describe how any canonical configuration can be obtained using the MERGE algorithm.

Theorem 3.21 (Every Canonical Configuration is obtainable by MERGE)**.** Let $\mathcal{M}$ be a CFM without cross-tree constraints with root feature f_r over an ordered feature set $(F, \leq_F)$, and let $K = (M, \prec)$ be a canonical instance-based configuration of $\mathcal{M}$.

Then there exist canonical configurations

$$K_{i,1}, \ldots, K_{i,k_i}$$

of the sub-models $\mathcal{M}_{c_i}$, for each child $c_i \in C_{\mathcal{M}}(f_r)$ with $k_i = M(c_i)$, such that:

1. For each fixed i, the configurations $K_{i,1}, \ldots, K_{i,k_i}$ are sorted increasingly with respect to the instance order.
2. Applying MERGE to the family $\{K_{i,j}\}$ yields K.

Proof. Let $K = (M, \prec)$ be a canonical configuration of $\mathcal{M}$, and let f_r be the root feature. For each child feature $c_i \in C_{\mathcal{M}}(f_r)$, the corresponding child instances of f_r^1 are

$$c_i^1, \ldots, c_i^{M(c_i)}.$$

By sibling index monotonicity, these instances satisfy

$$c_i^1 \trianglelefteq \cdots \trianglelefteq c_i^{M(c_i)}$$

with respect to the instance order induced by K.

For each $j \in \{1, \ldots, M(c_i)\}$, define

$$K_{i,j} \coloneqq K_{c_i^j}.$$

By Theorem 3.20, each $K_{i,j}$ is a canonical configuration of the sub-model $\mathcal{M}_{c_i}$.

By the consecutive block decomposition lemma (Lemma 3.17), for every feature $g \in F$ the instances of g in K form contiguous index blocks below the root instance, ordered according to the indices of their parent instances. Thus, the concrete numerical values of the instance indices do not carry additional structural information beyond the block decomposition.

Therefore, when passing to the family of sub-configurations $\{K_{i,j}\}$, no essential information is lost, since the block structure is preserved, although the original indices are discarded. When applying MERGE to the family $\{K_{i,j}\}$, ordered by increasing j for each fixed i, the algorithm shifts entire sub-configurations and reassigns instance indices in the unique order required for canonicality. Thus, MERGE reconstructs exactly the canonical configuration K. □

3.3 Semi-Structural Configurations

A key characterisation of structural configurations is that they correspond to canonical configurations, that is, instance-based configurations which satisfy subtree index monotonicity and sibling index monotonicity (see Definition 3.16).

These properties are crucial, as they ensure that canonical configurations and therefore structural configurations are closed under merge.

However, sibling index monotonicity (i.e. the requirement that the indices of sibling instances of the same feature are assigned increasingly with respect to the instance order) is not necessary for this merge property. In particular, a broader class of instance-based configurations exists that is not necessarily canonical but still exhibits the merge property.

This observation leads us to a second type of configuration, which we call *semi-structural configurations*.

The structure of this section closely mirrors the introduction of structural configurations (Section 3.2). In particular, we first define semi-structural configurations as equivalence classes, then discuss their visual representation, and finally describe a system of representatives of instance-based configurations for the semi-structural configuration space.

3.3.1 Equivalence Class Representation

Similarly to structural configurations, we first define semi-structural configurations via equivalence classes induced by a suitable notion of isomorphism.

Definition 3.22 (Order-Preserving Isomorphism)**.** Let $\mathcal{M}$ be a CFM over F. Two instance-based configurations $K_1 = (M, \prec^{(1)})$ and $K_2 = (N, \prec^{(2)})$ are called *order-isomorphic*, written $K_1 \cong_{\text{order}} K_2$, if:

1. $M = N$.
2. There exists a bijection $\pi : \text{Inst}(M) \to \text{Inst}(M)$ such that:
   (a) (*feature-wise*) for every feature $f \in F$, the restriction
   $$\pi|_{\{f^1,\dots,f^{M(f)}\}} \colon \{f^1, \dots, f^{M(f)}\} \to \{f^1, \dots, f^{M(f)}\}$$
   is a bijection;

(b) (*edge preservation*) for all $x, y \in \mathrm{Inst}(M)$,

$$x \prec^{(1)} y \iff \pi(x) \prec^{(2)} \pi(y);$$

(c) (*sibling-order preservation*) for every parent instance $p \in \mathrm{Inst}(M)$ and every feature $f \in F$, and for all $a, b \in C_{K_1}(p, f)$, if $\mathrm{index}(a) < \mathrm{index}(b)$, then

$$\mathrm{index}(\pi(a)) < \mathrm{index}(\pi(b)).$$

A map π satisfying condition 2 is called an *order isomorphism* between K_1 and K_2.

The above definition of an order isomorphism is very similar to the definition of a tree isomorphism (see Definition 3.7). In particular, conditions 1 and 2(a)–(b) are identical. However, condition 2(c) is new. Intuitively, this condition requires that within each feature instance group, the relative order of feature instances determined by their indices is preserved.

For example, consider the configurations shown in Figure 3.10. Both configurations are tree-isomorphic but not order-isomorphic, since swapping the indices of the `A` instances violates sibling-order preservation.

Thus, order isomorphism is a *stronger* notion of isomorphism between instance-based configurations: every order isomorphism is a tree isomorphism, but not vice versa. Consequently, it induces finer equivalence classes.

Lemma 3.23 (Order Isomorphism is an Equivalence Relation)**.** For a fixed CFM $\mathcal{M}$, the relation $\cong_{\mathrm{order}}$ is an equivalence relation on the set $\mathcal{C}_{\mathrm{Inst}}(\mathcal{M})$ of all instance-based configurations of $\mathcal{M}$.

Proof. Let $K_1 = (M, \prec^{(1)})$, $K_2 = (N, \prec^{(2)})$, and $K_3 = (P, \prec^{(3)})$ be instance-based configurations of $\mathcal{M}$.

Since an order isomorphism satisfies conditions 1 and 2(a)–(b) of Definition 3.7, every order isomorphism is, in particular, a tree isomorphism. Hence, reflexivity, symmetry, and transitivity already hold for conditions 1 and 2(a)–(b) by Lemma 3.8. It therefore remains to verify that condition 2(c), namely sibling-order preservation, is preserved under identity, inversion, and composition.

1. Reflexivity. The identity map on $\mathrm{Inst}(M)$ clearly preserves index order within every sibling group: if $a, b \in C_{K_1}(p, f)$ and $\mathrm{index}(a) < \mathrm{index}(b)$, then also

$$\mathrm{index}(\mathrm{id}(a)) < \mathrm{index}(\mathrm{id}(b)).$$

2. Symmetry. Assume $K_1 \cong_{\mathrm{order}} K_2$ via π. Since π is in particular a tree isomorphism, its inverse π^{-1} satisfies conditions 1 and 2(a)–(b). It remains to verify condition 2(c) for π^{-1}.

Fix a parent instance $q_2 \in \mathrm{Inst}(M)$, a feature $f \in F$, and two siblings $b, c \in C_{K_2}(q_2, f)$ such that

$$\mathrm{index}(b) < \mathrm{index}(c).$$

Set

$$q_1 \coloneqq \pi^{-1}(q_2), \qquad a \coloneqq \pi^{-1}(b), \qquad d \coloneqq \pi^{-1}(c).$$

By edge preservation, $a, d \in C_{K_1}(q_1, f)$.

We claim that $\text{index}(a) < \text{index}(d)$. Suppose, for contradiction, that this is not the case. Since $a \neq d$ and both belong to the same sibling group for feature f, we must have

$$\text{index}(d) < \text{index}(a).$$

Applying condition 2(c) for π to the siblings $d, a \in C_{K_1}(q_1, f)$ yields

$$\text{index}(\pi(d)) < \text{index}(\pi(a)).$$

But $\pi(d) = c$ and $\pi(a) = b$, so this gives

$$\text{index}(c) < \text{index}(b),$$

contradicting $\text{index}(b) < \text{index}(c)$. Hence $\text{index}(a) < \text{index}(d)$, that is,

$$\text{index}(\pi^{-1}(b)) < \text{index}(\pi^{-1}(c)).$$

Thus π^{-1} preserves strict index order within every sibling group.

3. Transitivity. Assume $K_1 \cong_{\text{order}} K_2$ via π and $K_2 \cong_{\text{order}} K_3$ via σ. Since both maps are tree isomorphisms, their composition $\sigma \circ \pi$ satisfies conditions 1 and 2(a)–(b). It remains to verify condition 2(c).

Fix $p \in \text{Inst}(M)$, a feature $f \in F$, and siblings $a, b \in C_{K_1}(p, f)$ such that

$$\text{index}(a) < \text{index}(b).$$

By condition 2(c) for π, we obtain

$$\text{index}(\pi(a)) < \text{index}(\pi(b)).$$

Moreover, since π preserves features and edges, the instances $\pi(a)$ and $\pi(b)$ are siblings in $C_{K_2}(\pi(p), f)$. Applying condition 2(c) for σ therefore yields

$$\text{index}(\sigma(\pi(a))) < \text{index}(\sigma(\pi(b))).$$

Hence,

$$\text{index}((\sigma \circ \pi)(a)) < \text{index}((\sigma \circ \pi)(b)),$$

so $\sigma \circ \pi$ preserves strict index order within every sibling group.

Thus condition 2(c) is preserved under identity, inversion, and composition. Therefore, $\cong_{\text{order}}$ is reflexive, symmetric, and transitive, and hence an equivalence relation. □

Since $\cong_{\text{order}}$ is an equivalence relation, it induces a partition of $\mathcal{C}_{\text{Inst}}(\mathcal{M})$ into equivalence classes. Intuitively, two configurations lie in the same class if they differ only by a renaming of instances that preserves the relative order of siblings. In contrast to tree isomorphism, sibling permutations are no longer identified.

Definition 3.24 (Semi-Structural Configuration)**.** Let $\mathcal{M}$ be a CFM. A *semi-structural configuration* of $\mathcal{M}$ is an $\cong_{\text{order}}$-equivalence class

$$[K]_{\text{SemiStruct}} \coloneqq \Big\{ K' \in \mathcal{C}_{\text{Inst}}(\mathcal{M}) \mid K' \cong_{\text{order}} K \Big\},$$

where $K \in \mathcal{C}_{\text{Inst}}(\mathcal{M})$.

$\mathcal{C}_{\text{SemiStruct}}(\mathcal{M})$ denotes the set of semi-structural configurations. Formally, it is the set of $\cong_{\text{order}}$-equivalence classes, i.e.,

$$\mathcal{C}_{\text{SemiStruct}}(\mathcal{M}) \coloneqq \mathcal{C}_{\text{Inst}}(\mathcal{M}) \Big/ \cong_{\text{order}} .$$

A semi-structural configuration $[K]_{\text{SemiStruct}}$ is *valid* if K is valid.

The *semi-structural configuration space* of $\mathcal{M}$, denoted $[\![\mathcal{M}]\!]_{\text{SemiStruct}}$, is the set of valid semi-structural configurations of $\mathcal{M}$.

As before, we must show that validity is well-defined for semi-structural configurations, i.e., independent of the chosen representative K.

Lemma 3.25 (Validity is invariant under Order Isomorphism)**.** Let $K_1, K_2 \in \mathcal{C}_{\text{Inst}}(\mathcal{M})$. If $K_1 \cong_{\text{order}} K_2$, then K_1 is valid if and only if K_2 is valid.

Proof. Every order isomorphism is, in particular, a tree isomorphism. Hence, $K_1 \cong_{\text{order}} K_2$ implies $K_1 \cong_{\text{tree}} K_2$. By Lemma 3.10 (invariance of validity under tree isomorphism), validity is preserved. □

Remark. The notion of an order isomorphism is closely related to the standard notion of isomorphism for ordered labelled (coloured) trees in general graph theory. Such an isomorphism is a bijection on the nodes that preserves adjacency, node labels (colours), and the order of siblings [72, 78]. The definition above is the same concept applied to the domain of CFMs.

3.3.2 Index-Free Tree Representation

As in Section 3.2.2, semi-structural configurations admit a natural index-free representation. The construction is identical to the structural case, except that sibling order is retained. Consequently, children are represented as *sequences* rather than multisets.

Inductive Construction

Let $K = (M, \prec)$ be an instance-based configuration such that $(\text{Inst}(M), \prec)$ forms a rooted tree with root f_r^1.

We recursively define an object $\Phi_K^{\text{order}}(x)$ for each instance $x \in \text{Inst}(M)$:

- If x is a leaf, we set
$$\Phi_K^{\text{order}}(x) \coloneqq \Big(\text{feat}(x), ()\Big),$$
where $()$ denotes the empty sequence.

- Otherwise, for each feature $f \in F$, let

$$(y_1^{(f)}, \ldots, y_{m_f}^{(f)})$$

be the sequence of elements of $C_K(x, f)$ ordered increasingly with respect to their indices. We define

$$\Phi_K^{\text{order}}(x) := \left(\text{feat}(x), \left(\Phi_K^{\text{order}}(y_1^{(f)}), \ldots, \Phi_K^{\text{order}}(y_{m_f}^{(f)})\right)_{f \in F}\right).$$

The index-free representation of K is then defined as

$$\Phi^{\text{order}}(K) := \Phi_K^{\text{order}}(f_r^1).$$

Independence of Representatives

If K_1 and K_2 are order-isomorphic via π, then

$$\Phi_{K_1}^{\text{order}}(x) = \Phi_{K_2}^{\text{order}}(\pi(x)) \qquad \text{for all } x \in \text{Inst}(M).$$

This follows because π preserves feature labels and the parent-child relation, and by sibling-order preservation maps each child to the corresponding position within its sibling list. Hence, for a semi-structural configuration $[K]_{\text{SemiStruct}}$, we may define

$$\Phi^{\text{order}}([K]_{\text{SemiStruct}}) := \Phi^{\text{order}}(K),$$

and this is independent of the chosen representative.

Conversely, every object of the form $\Phi^{\text{order}}([K]_{\text{SemiStruct}})$ can be converted back into a semi-structural configuration as follows. We reconstruct an instance-based configuration by traversing the tree top-down. For each feature $f \in F$, we maintain a global counter $\alpha_f \in \mathbb{N}_0$, initially 0.

Whenever we encounter a node labelled f, we increment α_f and assign to that node the instance name f^{α_f}. In particular, if a node x has m children of feature f given by a sequence $(y_1, \ldots, y_m)$, then these children receive consecutive indices $f^{\alpha_f+1}, \ldots, f^{\alpha_f+m}$ in that left-to-right order, and α_f is updated accordingly. Thus, the assigned indices respect the given sibling order.

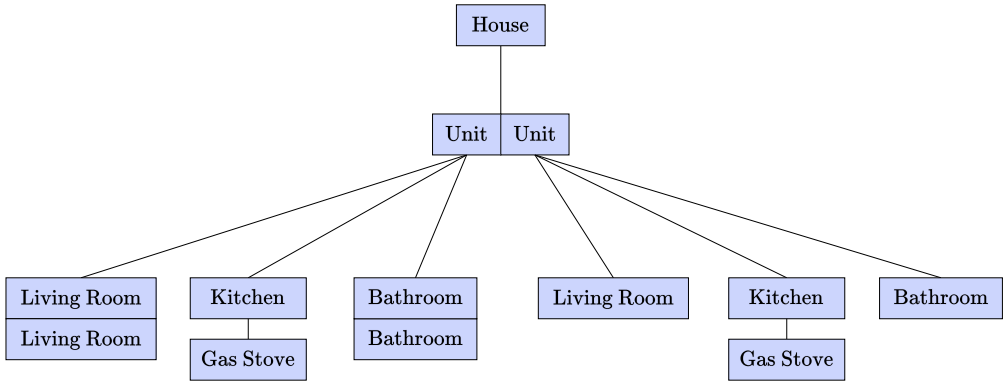
This yields an instance-based configuration K, and we then take its equivalence class $[K]_{\text{SemiStruct}}$. By construction, the parent-child relation and all sibling sequences coincide with those encoded in $\Phi^{\text{order}}([K]_{\text{SemiStruct}})$; hence the induced semi-structural configuration coincides with the original one.

Graphical Interpretation

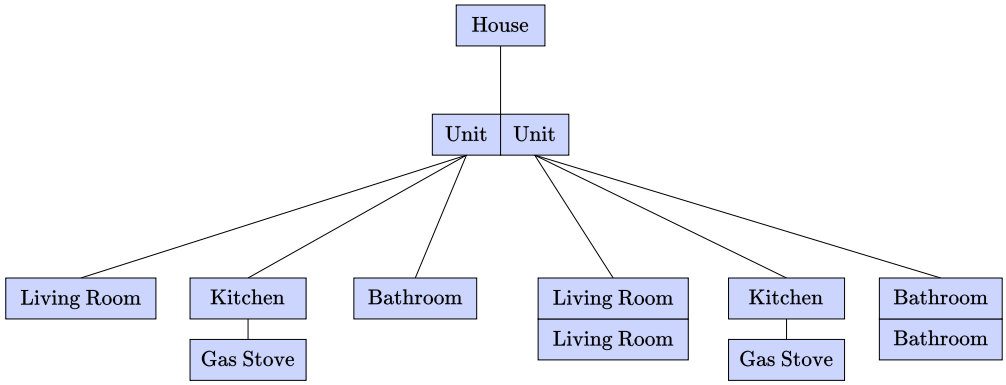
The object $\Phi^{\text{order}}([K]_{\text{SemiStruct}})$ can be viewed as a directed, feature-labelled *ordered tree* [78]. For every node and every feature $f \in F$, the children corresponding to feature f form a finite sequence.

Thus, sibling order is encoded directly in the tree layout rather than by feature-instance indices. No local or global enumeration of instances is required: the position of a node within its sibling list uniquely determines its order.

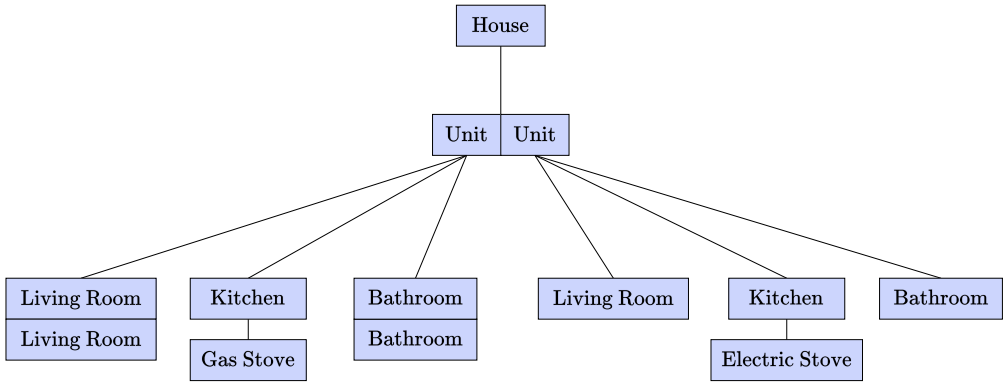
Figure 3.12 shows the semi-structural configurations obtained from the instance-based configurations in Figure 3.7. In contrast to the structural case, sibling nodes with the same feature are placed directly next to each other (horizontally or vertically) to indicate that they form an ordered sequence, read from left to right or from top to bottom. Thus, the spatial arrangement of siblings encodes their order. Moreover, unlike in the structural case, all three instance-based configurations yield distinct semi-structural configurations.



(a) Semi-structural configuration corresponding to Figure 3.7a.



(b) Semi-structural configuration corresponding to Figure 3.7b.



(c) Semi-structural configuration corresponding to Figure 3.7c.

Figure 3.12: Semi-structural configurations obtained from the instance-based configurations in Figure 3.7.

3.3.3 Semi-Canonical Representation

Canonical configurations (Definition 3.16) are a distinguished subclass of valid instance-based configurations that form a system of representatives for the structural configuration space $[\![\mathcal{M}]\!]_{\text{Struct}}$: each $\cong_{\text{tree}}$-equivalence class contains exactly one canonical configuration.

We can proceed analogously for semi-structural configurations. The key difference is that semi-structural configurations refine the underlying notion of isomorphism by *distinguishing the sibling order* of instances of the same feature (Definition 3.22). Consequently, the induced equivalence classes are finer than in the structural case, so we obtain more classes and therefore require fewer constraints to select unique representatives. In particular, sibling index monotonicity is no longer needed; it suffices to enforce subtree index monotonicity.

Definition 3.26 (Semi-Canonical Configuration)**.** Let $\mathcal{M}$ be a CFM and let $K = (M, \prec)$ be a *valid* instance-based configuration of $\mathcal{M}$. We call K *semi-canonical* if it satisfies subtree index monotonicity (see Definition 3.16).

We denote by $[\![\mathcal{M}]\!]_{\text{SemiCanon}}$ the set of semi-canonical configurations of $\mathcal{M}$.

Theorem 3.27 (Existence and Uniqueness of Semi-Canonical Representatives)**.** Let $\mathcal{M}$ be a CFM over the feature set F. Then the following statements hold:

(1) **Existence.** For every configuration $K = (M, \prec) \in [\![\mathcal{M}]\!]_{\text{Inst}}$, there exists a semi-canonical configuration $K^{\text{scan}} = (M, \prec^{\text{scan}}) \in [\![\mathcal{M}]\!]_{\text{Inst}}$ such that

$$K \cong_{\text{order}} K^{\text{scan}}.$$

(2) **Uniqueness.** If $K_1 = (M, \prec^{(1)})$ and $K_2 = (M, \prec^{(2)})$ are semi-canonical configurations and

$$K_1 \cong_{\text{order}} K_2,$$

then $K_1 = K_2$.

Consequently, every $\cong_{\text{order}}$-equivalence class contains exactly one semi-canonical configuration, and the quotient map induces a bijection

$$K \longmapsto [K]_{\text{SemiStruct}},$$

between the semi-canonical configuration space $[\![\mathcal{M}]\!]_{\text{SemiCanon}}$ and the semi-structural configuration space $[\![\mathcal{M}]\!]_{\text{SemiStruct}}$.

Proof. We prove (1) and (2) separately.

(1) Existence

Let $K = (M, \prec)$ be valid. We define K^{scan} by renumbering feature instances in a way that preserves the given sibling order and makes subtree index monotonicity true by construction.

Concretely, traverse the feature instance tree $(\text{Inst}(M), \prec)$ as follows:

- The root instance f_r^1 is visited first.
- When visiting a feature instance $p \in \mathrm{Inst}(M)$, we consider, for each child feature f of p, the sibling set

  $$C_K(p, f) = \{ f^{i_1}, \ldots, f^{i_m} \}.$$

  We order this set increasingly by their indices:

  $$i_1 < i_2 < \cdots < i_m.$$

  We then recursively (depth-first) visit the subtrees rooted at

  $$f^{i_1}, f^{i_2}, \ldots, f^{i_m}$$

  in this order.

  (The different child features of p may be processed in arbitrary order.)

For each feature $f \in F$, introduce a counter

$$\alpha_f := 0.$$

Whenever an instance f^i is visited, we increment

$$\alpha_f := \alpha_f + 1$$

and set

$$\sigma_f(i) := \alpha_f.$$

Since each instance of f is visited exactly once, σ_f is a bijection on $\{1, \ldots, M(f)\}$.

After the traversal we define

$$f^{\sigma_f(i)} \prec^{\mathrm{scan}} g^{\sigma_g(j)} \quad :\Longleftrightarrow \quad f^i \prec g^j.$$

Thus, by construction, the configuration $K^{\mathrm{scan}} = (M, \prec^{\mathrm{scan}})$ is order-isomorphic to K. Concretely, the order isomorphism is given by

$$\pi \colon \mathrm{Inst}(M) \to \mathrm{Inst}(M), \qquad \pi(f^i) := f^{\sigma_f(i)}.$$

Furthermore, K^{scan} satisfies subtree index monotonicity and is valid. The argument is identical to the corresponding part in the proof of existence of canonical configurations (Theorem 3.18), where we used a similar depth-first search (DFS)-traversal but sorted the children with respect to the instance order, instead of the index order.

(2) Uniqueness

Let $K_1 = (M, \prec^{(1)})$ and $K_2 = (M, \prec^{(2)})$ be semi-canonical configurations such that

$$K_1 \cong_{\mathrm{order}} K_2$$

via an order isomorphism π.

We show π is the identity by induction on the tree depth, which implies $\prec^{(1)} = \prec^{(2)}$ and hence $K_1 = K_2$.

Base case. The root feature f_r has exactly one instance f_r^1 in any valid configuration. Since π is feature-wise, $\pi(f_r^1) = f_r^1$.

Inductive step. Assume that for all instances of depth at most d in K_1, π fixes them.

Let p be any instance of depth d in K_1.

Fix a feature $f \in F$ and consider the f-children of p in K_1:

$$C_{K_1}(p, f) = \{f^a, f^{a+1}, \ldots, f^{a+m-1}\},$$

where the block form follows from Lemma 3.17. Although that lemma is stated for canonical configurations, its proof uses only subtree index monotonicity and does not rely on sibling index monotonicity. Hence, it applies equally to the semi-canonical setting of K_1.

Likewise, in K_2 the f-children of $p = \pi(p)$ form a consecutive block

$$C_{K_2}(p, f) = \{f^b, f^{b+1}, \ldots, f^{b+m-1}\},$$

with the *same length* m because π is a bijection between these child sets.

Now apply sibling-order preservation (Definition 3.22, 2(c)): the restriction of π to $C_{K_1}(p, f)$ is a strictly increasing bijection onto $C_{K_2}(p, f)$. Since both are consecutive blocks of the same size, the only strictly increasing bijection is the map

$$\pi(f^{a+t}) = f^{b+t} \qquad (t = 0, \ldots, m-1).$$

It remains to show $a = b$, which forces π to be the identity on this block. But a and b are determined by the block-boundary formula of Lemma 3.17:

$$a = 1 + \sum_{q \text{ is a feat}(p)\text{-instance with index smaller than index}(p)} |C_{K_1}(q, f)|,$$

and analogously for b in K_2. All the parent instances q appearing in these sums have depth d, hence are fixed by the induction hypothesis; and for each such fixed q, the cardinality $|C_{\mathcal{M}}(q, f)|$ is preserved because π is edge-preserving and feature-wise. Therefore, the sums agree, so $a = b$.

Thus π fixes every f-child of p, for every feature f, meaning it fixes all instances of depth $d + 1$.

By induction, π is the identity on all of $\mathrm{Inst}(M)$ and therefore $K_1 = K_2$. □

3.3.4 Merging Semi-Canonical Configurations

We now show that semi-canonical configurations and therefore semi-structural configurations are closed under merge. In particular, similarly to the canonical case (Section 3.2.4), merging semi-canonical child configurations yields a semi-canonical configuration of the full model and every semi-canonical configuration of the full model is obtainable in this way.

Theorem 3.28 (Merging Semi-Canonical Configurations)**.** Let $\mathcal{M}$ be a CFM without cross-tree constraints with root feature f_r, and let

$$C_{\mathcal{M}}(f_r) = \{c_1, \ldots, c_n\}.$$

For each child c_i, let

$$K_{i,1}, \ldots, K_{i,k_i}$$

be semi-canonical configurations of the sub-model $\mathcal{M}_{c_i}$. If the multiplicities $k_1, \ldots, k_n$ satisfy the feature-instance, group-instance, and group-type cardinalities at the root feature, then the output of MERGE is a semi-canonical configuration of $\mathcal{M}$. In particular, in contrast to the canonical case (Section 3.2.4), we do not need to sort the child configurations.

Proof. Let $(M, \prec)$ be the configuration returned by MERGE. Validity follows exactly as in the proof of Theorem 3.19.

For semi-canonicality, observe that the proof of Theorem 3.19 establishes subtree index monotonicity independently of the sorting assumption; the sorting is used only to guarantee sibling index monotonicity. Since semi-canonical configurations require only subtree index monotonicity, the result follows immediately. □

Again, we show that semi-canonicality is inherited by its sub-configurations.

Theorem 3.29 (Semi-Canonicality is preserved by Sub-Configurations)**.** Let $\mathcal{M}$ be a CFM without cross-tree constraints over the feature set F and let $K = (M, \prec)$ be a semi-canonical configuration of $\mathcal{M}$. Then, for every feature $f \in F$ and every instance $f^i \in \mathrm{Inst}(M)$, the sub-configuration K_{f^i} is a semi-canonical configuration of the sub-model $\mathcal{M}_f$. In particular, semi-canonicality is inherited by sub-configurations.

Proof. Fix $f \in F$ and $f^i \in \mathrm{Inst}(M)$. Since K is semi-canonical, it satisfies subtree index monotonicity. Hence, it suffices to show that subtree index monotonicity is preserved when passing from K to the renumbered sub-configuration K_{f^i}.

This is exactly the argument of Part 2 in the proof of Theorem 3.20: that part does not use sibling index monotonicity and relies only on (i) subtree index monotonicity in K and (ii) the fact that the renumbering map ρ from Definition 3.2 preserves the relative order of indices within each feature (it renumbers instances of each feature in increasing order of their original indices). Therefore, subtree index monotonicity holds in K_{f^i}, so K_{f^i} is semi-canonical (again K_{f^i} is valid because the model has no cross-tree constraints). □

Finally, we show again that every semi-canonical configuration can be obtained by MERGE.

Theorem 3.30 (Every Semi-Canonical Configuration is obtainable by MERGE)**.** Let $\mathcal{M}$ be a CFM without cross-tree constraints with root feature f_r over a feature set F, and let $K = (M, \prec)$ be a semi-canonical instance-based configuration of $\mathcal{M}$.

Then there exist semi-canonical configurations

$$K_{i,1}, \ldots, K_{i,k_i}$$

of the sub-models $\mathcal{M}_{c_i}$, for each child $c_i \in C_{\mathcal{M}}(f_r)$ with $k_i = M(c_i)$, such that applying MERGE to the family $\{K_{i,j}\}$ yields K.

Proof. The argument follows the proof of Theorem 3.21, but without the parts that rely on sibling index monotonicity.

Let $K = (M, \prec)$ be semi-canonical, and let f_r be the root feature. For each child feature $c_i \in C_{\mathcal{M}}(f_r)$, let

$$(c_i)^1, \ldots, (c_i)^{M(c_i)}$$

be the c_i-children of the root instance f_r^1 in K. For each $j \in \{1, \ldots, M(c_i)\}$, define

$$K_{i,j} \coloneqq K_{(c_i)^j}.$$

By Theorem 3.29, each $K_{i,j}$ is a semi-canonical configuration of the sub-model $\mathcal{M}_{c_i}$.

Since Merge inserts the child configurations sequentially in the order in which they are given as input and only shifts indices, the subtree rooted at each child instance c_i^j in the merged configuration is exactly the corresponding subtree in the original configuration K.

Therefore, the output of Merge is identical to K. □

3.4 Summary and Comparison of Configuration Notions

Up to this point, we have introduced four novel configuration notions: structural, semi-structural, canonical, and semi-canonical configurations. In the following, we summarize the main definitions, results, and properties of these configuration types and clarify how they relate to one another both formally and intuitively.

Moreover, we examine their relationships to previously existing configuration notions in the literature and explain why the introduced distinctions are necessary. As we will see, these distinctions are crucial for the development of uniform random sampling algorithms for CFMs, which is the main objective of this thesis.

3.4.1 Relationships and Properties

Figure 3.13 illustrates all previously introduced configuration types (multiset-based and instance-based), as well as the four new configuration types (structural, semi-structural, canonical, and semi-canonical) developed in this chapter. In particular, the following formal relationships hold among these configuration types.

Instance-based configurations. Instance-based configurations form the foundational representation. They explicitly model feature instances and how they are arranged hierarchically.

Semi-structural configurations. Semi-structural configurations abstract from instance-based configurations by ignoring the concrete index assignments. Formally, they are defined as $\cong_{\text{order}}$-equivalence classes, where $\cong_{\text{order}}$ is an equivalence relation on the set $\mathcal{C}_{\text{Inst}}(\mathcal{M})$ of instance-based configurations. In particular, this relation identifies instance-based configurations that differ only in the assignment of feature-instance indices, but *do not* reorder sibling instances of the same feature. Thus, if two or more instances of the same feature occur under the same parent, their relative order is preserved, and it remains clear which instance, together with its subtree, is considered the “first”, “second”, and so on.

Structural configurations. Structural configurations are even more coarse-grained. They abstract entirely from feature-instance indices and retain only the hierarchical structure. If several instances of the same feature occur as siblings, they are treated as unordered and indistinguishable. Formally, structural configurations are defined as $\cong_{\text{tree}}$-equivalence classes over $\mathcal{C}_{\text{Inst}}(\mathcal{M})$. Compared to $\cong_{\text{order}}$, the relation $\cong_{\text{tree}}$ is weaker, as it identifies more instance-based configurations.

Multiset-based configurations. Multiset-based configurations abstract further by discarding the hierarchical structure altogether. They retain only the global feature multiplicities and thus represent the coarsest configuration notion considered here. Formally, the projection π, defined by

$$\pi \colon \mathcal{C}_{\text{Inst}}(\mathcal{M}) \to \mathcal{C}_{\text{Multiset}}(\mathcal{M}), \quad (M, \prec) \mapsto M,$$

projects the set of instance-based configurations to the set of multiset-based configurations. π is surjective but, in general, not injective, since different instance-based configurations may have the same multiplicity function.

Canonical and semi-canonical configurations. Canonical and semi-canonical configurations do not introduce new configuration objects. Rather, they are *valid* instance-based configurations that satisfy a (semi-)canonicality condition. They serve as unique representatives of structural and semi-structural equivalence classes, respectively, allowing us to work with concrete instance-based configurations instead of equivalence classes.

In particular, there is a bijection between the semi-canonical configuration space $[\![\mathcal{M}]\!]_{\text{SemiCanon}}$ and the semi-structural configuration space $[\![\mathcal{M}]\!]_{\text{SemiStruct}}$. Analogously, the canonical configuration space $[\![\mathcal{M}]\!]_{\text{Canon}}$ is in bijection with the structural configuration space $[\![\mathcal{M}]\!]_{\text{Struct}}$.

Moreover, these representative systems provide a precise characterization of the respective configuration types. Semi-structural configurations eliminate subtree index inversions by identifying configurations that differ only by such inversions. Structural configurations extend this identification further by additionally eliminating sibling index inversions.

Configuration Space Size

Since there is a one-to-one correspondence between the (semi-)canonical and (semi-)structural configuration spaces, and since

$$[\![\mathcal{M}]\!]_{\text{Canon}} \subseteq [\![\mathcal{M}]\!]_{\text{SemiCanon}} \subseteq [\![\mathcal{M}]\!]_{\text{Inst}},$$

the sizes of the configuration spaces satisfy

$$\left|[\![\mathcal{M}]\!]_{\text{Struct}}\right| \leq \left|[\![\mathcal{M}]\!]_{\text{SemiStruct}}\right| \leq \left|[\![\mathcal{M}]\!]_{\text{Inst}}\right|.$$

Every valid structural configuration is induced by at least one valid instance-based configuration. Moreover, by definition of multiset validity, every valid instance-based configuration induces a valid multiset-based configuration, since a multiset-based configuration is valid whenever it can be extended to a valid instance-based one.

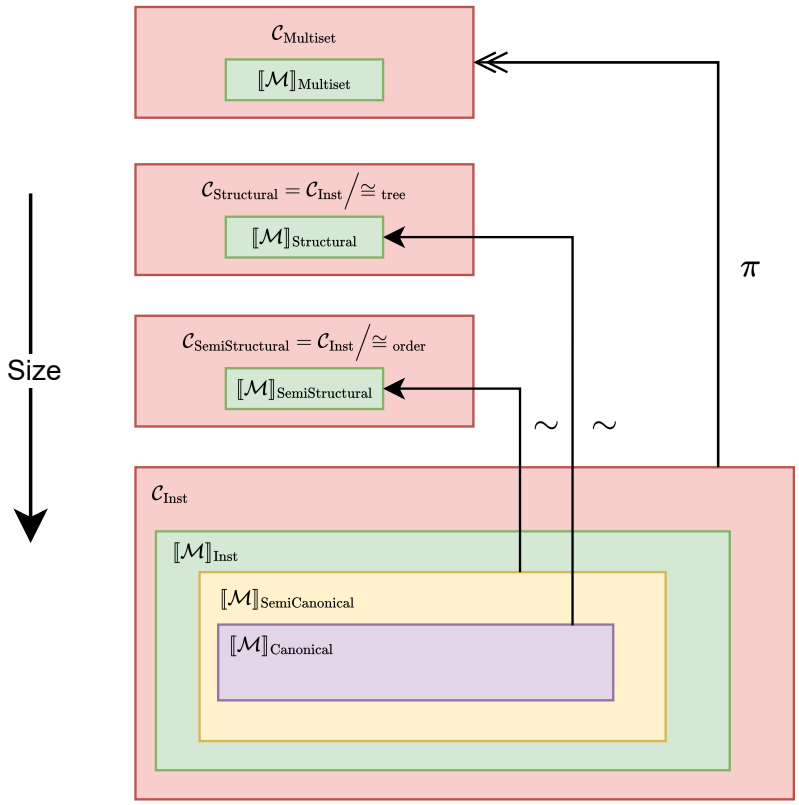


Figure 3.13: Overview of the configuration types considered in this work and their relationships. Instance-based configurations form the most detailed representation. Semi-structural and structural configurations arise as equivalence classes of instance-based configurations under order-isomorphism ($\cong_{\text{order}}$) and tree-isomorphism ($\cong_{\text{tree}}$), respectively. Multiset-based configurations arise by forgetting the hierarchical structure and retaining only global feature multiplicities. The arrow $\twoheadrightarrow$ labelled by π denotes the surjective projection $\pi : \mathcal{C}_{\text{Inst}}(\mathcal{M}) \to \mathcal{C}_{\text{Multiset}}(\mathcal{M})$ that maps an instance-based configuration $(M, \prec)$ to its multiplicity function M. Arrows annotated with $\sim$ indicate bijections between configuration spaces.

Consequently, every valid structural configuration also induces a valid multiset-based configuration. It follows that the multiset-based configuration space is the smallest among these configuration spaces.

Intuitively, this matches our expectations. Multiset-based configurations abstract away entirely from the hierarchical structure and retain only the global feature multiplicities. They therefore discard strictly more information than structural or semi-structural configurations and thus give rise to the smallest configuration space. Altogether, we obtain

$$\left|[\![\mathcal{M}]\!]_{\text{Multiset}}\right| \leq \left|[\![\mathcal{M}]\!]_{\text{Struct}}\right| \leq \left|[\![\mathcal{M}]\!]_{\text{SemiStruct}}\right| \leq \left|[\![\mathcal{M}]\!]_{\text{Inst}}\right|.$$

Properties

Table 3.1 summarizes the central properties of the considered configuration notions and highlights their structural differences. In particular, the table illustrates the gradual refinement of information from multiset-based to instance-based configurations, as well as the different symmetries that are identified at each level.

A key property of the structural and semi-structural configuration semantics is that they are closed under Merge. In particular, this means that merging valid child configurations always yields a valid configuration of the full model, and that every valid configuration of the full model can be obtained in this way. In contrast, the instance-based configuration semantics is not closed under merge (see Section 3.1.4). For multiset-based configurations, closure under Merge has not yet been investigated and may be an interesting direction for future work.

3.4.2 Interpretation of Configuration Semantics

So far, we have focused on the formal definitions of the different configuration notions and their theoretical properties. However, we have not yet discussed how the resulting configurations can be interpreted as concrete products, depending on the chosen configuration semantics. To illustrate one possible interpretation, we consider a simple parking-garage product line.

Figure 3.14 shows the corresponding CFM together with one valid configuration of each type introduced in this thesis. The depicted configurations are chosen such that they correspond to one another under the respective projection and quotient constructions. For example, the structural configuration (c) is obtained as the tree-isomorphism equivalence class of the instance-based configuration (e).

The underlying CFM models a simplified parking garage, for example, of a car manufacturer. A garage may contain an arbitrary number of cars. Each car consists of exactly four tires, and each tire is either a summer tire or a winter tire. Depending on the chosen configuration notion, the resulting configurations convey different semantic information, even though they describe the same underlying product.

Multiset-based configuration. The multiset-based configuration shown in Figure 3.14b records only global feature multiplicities. It specifies how many times each feature occurs in total, but abstracts from the hierarchical decomposition. In particular, although we observe four summer tires and four winter tires, we cannot

Table 3.1: Comparison of configuration notions and their properties

Property	Multiset	Structural	Semi-Struct.	Instance-Based
Information Preserved				
Records global multiplicities	✓	✓	✓	✓
Represents hierarchical decomposition	✗	✓	✓	✓
Distinguishes sibling order	✗	✗	✓	✓
Distinguishes global instance identities	✗	✗	✗	✓
Symmetry Elimination				
Eliminates subtree index inversions	✓	✓	✓	✗
Eliminates sibling index inversions	✓	✓	✗	✗
Relation to $\mathcal{C}_{\text{Inst}}$	projection $\pi(\mathcal{C}_{\text{Inst}})$	$\mathcal{C}_{\text{Inst}}/\cong_{\text{tree}}$	$\mathcal{C}_{\text{Inst}}/\cong_{\text{order}}$	$\mathcal{C}_{\text{Inst}}/\text{id}$
Systems of Representatives and Operations				
Has a canonical system of representatives	✗	✓	✓	✓
Closed under MERGE	–	✓	✓	✗

determine how they are distributed across the cars. Thus, we cannot tell whether one car uses only winter tires, only summer tires, or a mixture of both.

One possible use case for this configuration type arises when the manufacturer is concerned only with the total number of required components, rather than with how they are assembled or distributed across individual vehicles.

Structural configuration. The structural configuration in Figure 3.14c refines this information by explicitly modelling how tire instances are assigned to cars. We can now see that each car contains two summer and two winter tires. However, we cannot distinguish which tire occupies which position (e.g. front-left or rear-right), and the cars themselves are not ordered. Thus, the configuration determines the assembled structure but not positional or ordering information.

A possible application scenario for this configuration type is production planning, where it is relevant how components are grouped into individual products, but not which specific component occupies which physical position.

Semi-structural configuration. The semi-structural configuration shown in Figure 3.14d additionally preserves the order among sibling instances. Consequently, we can determine the exact tire type at each position. In the depicted configuration, the first car has summer tires in the front positions and winter tires in the rear positions, while for the second car the situation is reversed. Moreover, the cars themselves are ordered.

A possible interpretation is that the parking garage consists of numbered slots that are filled consecutively. In this case, the position of a car becomes semantically relevant. More generally, the notion of semi-structural configurations is appropriate in scenarios where positional information matters. For example, in a simulation of the deceleration behaviour of cars, it is essential to capture the concrete distribution of summer and winter tires across the wheel positions. Likewise, assembly instructions, safety constraints, or quality checks may depend on the exact placement of components within a product.

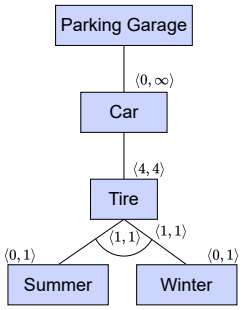
Instance-based configuration. Finally, the instance-based configuration in Figure 3.14e enumerates all feature instances globally. Each tire is identified not only by its type and structural position, but also by a concrete instance index.

In contrast to the previous configuration notions, for which a natural application-oriented interpretation was available, we are currently not aware of any fully satisfactory interpretation of the instance-based representation in its most general form.

For the concrete configuration shown in Figure 3.14e, one may interpret feature-instance indices as part-specific serial numbers assigned during manufacturing. Under this interpretation, internal structural indices encode positional information (e.g. first car, second car, front-left tire, front-right tire), while feature-instance indices represent global serial identifiers. For example, the first car could be understood as containing the second ever manufactured summer tire in its front-left position.

However, this only works because the example deliberately avoids using “exotic” index assignments such as a vehicle whose tires carry the instance identifiers 8,1,2,3, where tire 8 has the summer-tire instance 1.

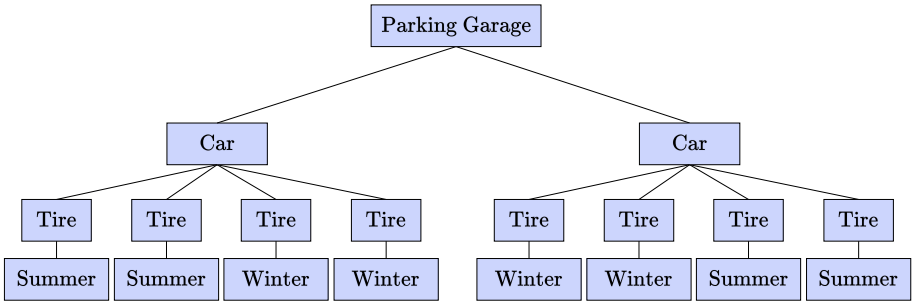
Note also that in the above example we implicitly assume an interpretation of tire order as corresponding to fixed physical positions (e.g. front-left, front-right, rear-left, rear-right). This positional interpretation is not imposed by the configuration formalism itself, but arises from a meta-level interpretation of the configuration semantics.



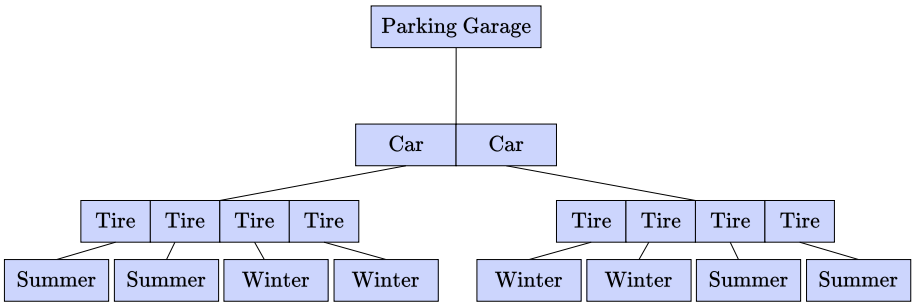
(a) Parking garage CFM.



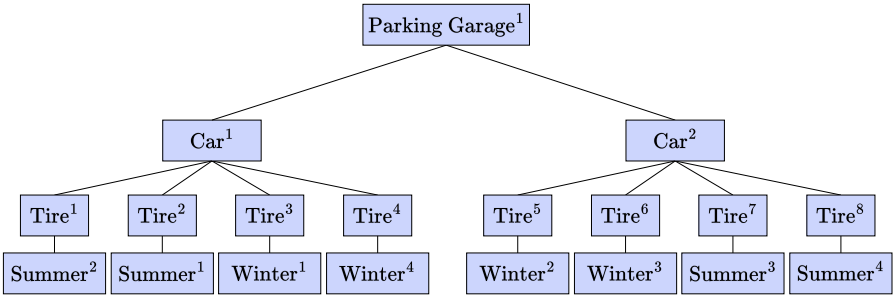
(b) Multiset-based configuration.



(c) Structural configuration.

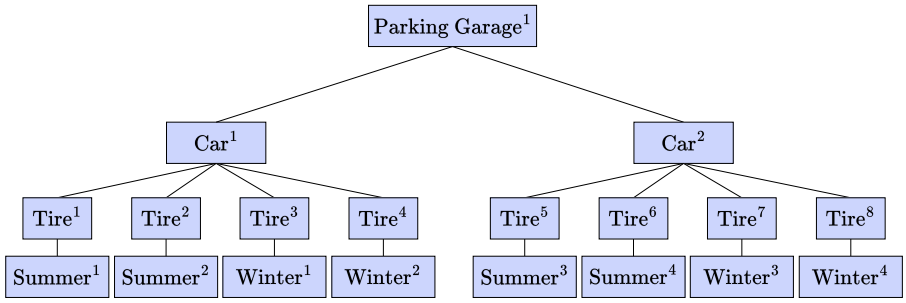


(d) Semi-structural configuration.

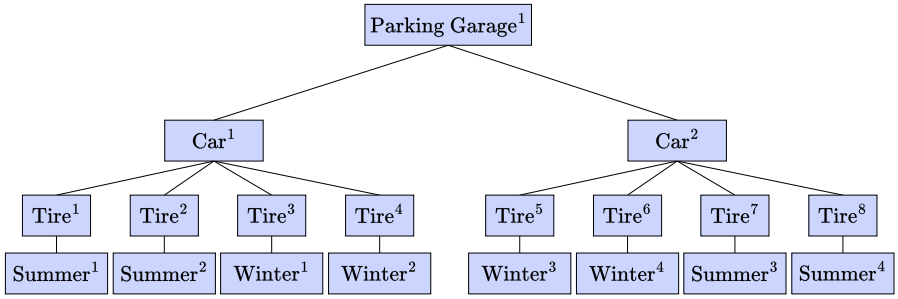


(e) Instance-based configuration.

Figure 3.14: A parking garage CFM together with corresponding valid configurations for all configuration types introduced in this thesis. Each configuration induces the next more abstract one via the respective projection or quotient construction.



(f) Canonical configuration.



(g) Semi-canonical configuration.

Figure 3.14: Parking garage CFM together with corresponding valid configurations for all configuration types introduced in this thesis (continued). Canonical and semi-canonical configurations are shown with respect to the feature order `Summer` $\leq_F$ `Winter`.

3.4.3 Configuration Semantics in the Literature

Throughout this chapter and the thesis so far, we have presented instance-based and multiset-based configurations as the baseline configuration semantics established in the literature, and structural, semi-structural, canonical, and semi-canonical configurations as new notions introduced in this work.

However, this presentation requires some clarification, since several configuration semantics for CFMs have been proposed in the literature over the past 25 years. In particular, the structural configuration semantics is not new at all, but in fact one of the earliest configuration types developed for CFMs. The key difference to our presentation lies in how it is formally defined.

Structural Configurations

One of the earliest formal configuration semantics for CFMs was proposed by Czarnecki et al. [2]. Their semantics is conceptually close to our notion of structural configurations. In particular, their configurations capture the full hierarchical structure of feature instances, but do not introduce explicit global instance identifiers.

Technically, Czarnecki et al. [2] achieve this by translating the model into a context-free grammar, and thereby into a formal language. Each word of the formal language corresponds to exactly one valid configuration, and can be read as a recursively nested pair of the form

$$(f,\ \mathcal{M}),$$

where f is a feature label and $\mathcal{M}$ is a *multiset* of child configurations.

This representation was later refined by Michel et al. [14], who introduced a clone-based representation of configurations of the form

$$\mathit{Clone} \in \mathit{Feature} \times \mathit{Multiset}(\mathit{Clone})$$

that is, a feature label together with a multiset of child clones. This representation is precisely the index-free tree representation of structural configurations introduced in Section 3.2.2. In particular, it does not have an order among the children, since they are stored in a multiset.

While conceptually these semantics coincide with our notion of structural configurations, the corresponding works employ a slightly different CFM syntax. In particular, the explicit distinction between *group-type* and *group-instance* cardinalities was later introduced by Weckesser et al. [10], whose CFM syntax we adopt in this thesis.

However, instead of continuing the clone-based, index-free representation of configurations introduced by Michel et al. [14], Weckesser et al. propose an *instance-based* representation in which a configuration is given as a tuple

$$(M, \prec),$$

consisting of a multiplicity function M and a feature-instance decomposition relation

$$\prec \subseteq \mathrm{Inst}(M) \times \mathrm{Inst}(M).$$

As already discussed in Section 2.2.2.3, Weckesser et al. [10] formally write the decomposition relation as

$$\prec \subseteq M \times M,$$

which is mathematically imprecise.

Although their formal definition deviates from the clone-based (structural) semantics, judging from their figures, examples, and accompanying explanations, it appears that the intended semantics still corresponds to the clone-based view of Michel et al. [14]. In particular, their figures never explicitly display feature-instance indices; instead, configurations are depicted exactly in the form we use for structural configurations, namely as feature-labelled, unordered trees.

As a consequence, the index-related anomalies identified in Section 3.1 are never discussed in their presentation.

Multiset-based Configurations

Another widely used abstraction is the multiset-based (or flat) configuration semantics, which records only the global multiplicity of each feature. Such configurations are commonly used in the literature [2, 10, 11, 28, 31].

Although this abstraction is used repeatedly, only Güthing et al. [11] define multiset-based configurations explicitly as the projection of instance-based configurations onto their multiplicity function. In contrast, other works such as Weckesser et al. [10], Safilian and Maibaum [28], and Schnabel et al. [31] characterize valid multisets indirectly via sets of arithmetic constraints that eliminate impossible multiplicity assignments.

While such constraint systems are useful for pruning the search space of multiplicity functions, they do not fully capture the projection-based semantics adopted in this thesis. In particular, they do not explicitly require that a multiset can be extended to a valid instance-based configuration. Instead, they merely rule out some inconsistent multiplicity assignments and therefore cannot guarantee that every remaining multiset corresponds to a realizable hierarchical configuration [21].

Semi-Structural Configurations

Although never explicitly discussed in the literature, several recent solver-based encodings implicitly represent what we call *semi-structural configurations*. In particular, the encodings proposed by Zhang [18] and Eger [21] aim to represent structural configurations using SMT, ILP, or CSP. However, due to the way configurations are represented in these encodings, the resulting objects correspond to semi-structural rather than structural configurations.

In the structural configuration semantics introduced in Section 3.2, the children of a feature instance are modelled as *multisets*. Consequently, there is no order among child instances of the same feature.

In contrast, solver encodings typically represent feature instances by introducing a fixed set of variables, each corresponding to a potential feature instance. Since these variables must be distinct, the corresponding feature instances become inherently distinguishable. The solver then assigns a child configuration to each of these

variables. As a result, the solver encoding actually models children as *sequences* rather than multisets, and permuting subtree assignments between siblings leads to distinct solver solutions.

For example, suppose a feature instance x has two child instances y_1 and y_2 with configurations

$$\Phi_K^{\text{tree}}(y_1) = T_1 \qquad \text{and} \qquad \Phi_K^{\text{tree}}(y_2) = T_2.$$

Under the structural configuration semantics, the index-free tree representation of the configuration rooted at x is

$$\Phi_K^{\text{tree}}(x) = \Big(\text{feat}(x), \{\!\{T_1, T_2\}\!\}\Big).$$

Consequently,

$$\Big(\text{feat}(x), \{\!\{T_1, T_2\}\!\}\Big) = \Big(\text{feat}(x), \{\!\{T_2, T_1\}\!\}\Big),$$

since multisets do not distinguish the order of their elements.

In contrast, solver encodings typically assign child configurations to a fixed sequence of variables. This corresponds to an ordered representation of the form

$$\Phi_K^{\text{order}}(x) = \Big(\text{feat}(x), (T_1, T_2)\Big),$$

where the two child positions are distinguished. In this representation,

$$(\text{feat}(x), (T_1, T_2)) \neq (\text{feat}(x), (T_2, T_1)),$$

so the two permutations correspond to distinct assignments in the solver encoding.

3.4.4 Implications for Uniform Random Sampling

One of the main reasons why we have studied the various configuration semantics in this chapter so rigorously, and why we have distinguished them so carefully, is their role in uniform random sampling. In particular, the theoretical distinctions developed above allow us to identify a substantial semantic gap in the existing sampling approach.

The uniform random sampling algorithm proposed by Zhang [18], shown in Algorithm 2.1, is not uniform with respect to structural configurations (or equivalently canonical configurations), even if every randomly chosen cardinality is weighted by the number of structural configurations it admits.

The reason is the following. Once a cardinality $r \in \lambda_I^F(f')$ has been drawn for a child feature f', the procedure GenerateChildren is invoked recursively r times for that feature. Since these recursive calls are independent, they produce a list of r child configurations. These configurations are then appended using the same shift mechanism introduced in Section 3.1.4 for the Merge algorithm. However, this process produces a *semi-canonical* configuration, since the resulting list of child configurations is inserted without enforcing any canonical ordering.

One might attempt to resolve this issue by sorting the child configurations afterwards in order to obtain canonical representations. While this indeed ensures that the resulting configurations correspond to canonical (and hence structural) configurations, it unfortunately breaks uniformity.

The reason is that sorting collapses multiple sequences of child configurations into the same canonical representation. The number of such sequences depends on how many duplicates occur among the child configurations. Consequently, configurations containing many duplicates correspond to fewer sequences, whereas configurations with fewer duplicates correspond to more sequences. As a result, configurations with many duplicates are underrepresented, while configurations with few duplicates are overrepresented.

Consider the CFM shown in Figure 3.15 together with its three structural configurations, each represented by its canonical configuration with respect to the feature order $\texttt{B} \leq_F \texttt{C}$. In this model, the only admissible feature-instance cardinality of feature `A` is 2.

If we now generate two child configurations by invoking a uniform random sampler twice, the algorithm returns one of the lists

$$[\texttt{B}^1, \texttt{B}^1], \quad [\texttt{B}^1, \texttt{C}^1], \quad [\texttt{C}^1, \texttt{B}^1], \quad [\texttt{C}^1, \texttt{C}^1],$$

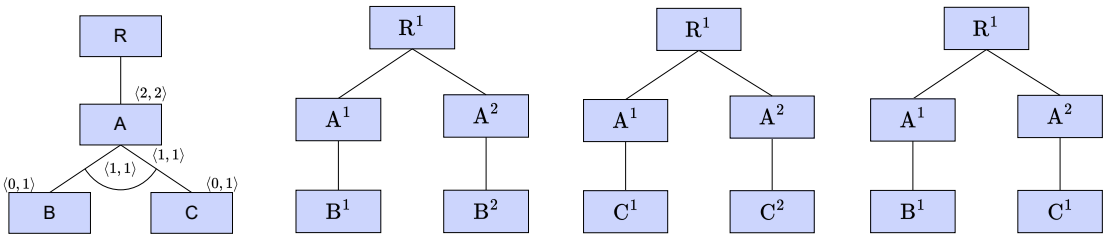
each with probability 0.25.

After sorting these lists to obtain canonical representations, only the sequences

$$[\texttt{B}^1, \texttt{B}^1], \quad [\texttt{B}^1, \texttt{C}^1], \quad [\texttt{C}^1, \texttt{C}^1]$$

remain. However, the resulting probabilities are no longer uniform: the first and third configurations each occur with probability 0.25, while the middle configuration occurs with probability 0.5.

A solution to this problem will be developed later in Chapter 4, where we introduce uniform random sampling algorithms for the structural configuration space.



(a) (b) (c) (d)

Figure 3.15: CFM (a), together with its structural configuration space (b–d), represented as canonical configurations with the feature order $\texttt{B} \leq_F \texttt{C}$.

Overall, these observations show that it is not sufficient to simply ignore the index-related anomalies of the instance-based configuration semantics and reason informally as if all configurations were structural (or clone-based). Doing so is not only mathematically imprecise, but can also lead to substantial technical errors. In particular, as the previous discussion illustrates, this semantic conflation has already led in the literature to solver encodings that do not, in fact, represent structural configurations, and to sampling algorithms that are not uniform for the intended configuration space. This is precisely why the careful semantic distinctions introduced in this chapter are necessary.

3.4.5 Outlook: Tailoring Configuration Semantics

Beyond their role in uniform random sampling, the constructions developed in this chapter suggest a more general perspective on configuration semantics. In particular, the transition from instance-based configurations to semi-structural and structural configurations can be understood as the deliberate formation of equivalence classes that abstract from information considered irrelevant for a given modelling purpose.

Concretely, structural configurations discard the order among sibling instances for *all* features, whereas semi-structural configurations preserve this order for *all* features. This suggests that more refined configuration notions may be possible in which such distinctions are handled only selectively. For example, one could preserve sibling order only for certain features, only at certain depths of the feature tree, only for leaf or internal features, or only for *abstract features*, that is, features that are used solely during modelling and do not occur in the final real-world product [20, 79]. Such notions would then lie between the structural and the semi-structural semantics.

Similarly, one could investigate configuration notions that relax *subtree index monotonicity* only for selected classes of features, for example depending again on whether a feature is abstract, internal, or a leaf. In such semantics, feature-instance indices would still be required to behave monotonically in some parts of the configuration, but not in others. For example, in the parking-garage interpretation discussed in Section 3.4.2, one might require monotone indices for the `Tire` instances of each car because these indices encode the wheel positions, while allowing more flexible index assignments for the `Summer` and `Winter` instances, if those are interpreted as serial identifiers of tire types. Such semantics would no longer lie between the structural and semi-structural views, but between semi-structural and fully instance-based configurations because subtree index monotonicity is exactly the property that separates these two semantics.

More generally, this raises the prospect of developing configuration semantics in a systematic and application-driven way: depending on the system to be modelled, one may choose explicitly which distinctions should be preserved and which symmetries should be factored out, thereby adjusting the expressiveness of the semantics to the desired application.

A particular advantage of this approach is that the syntax of CFMs would remain unchanged, and validity could still be defined in terms of valid instance-based configurations, while only the granularity of the induced equivalence classes would vary. In this sense, the techniques used in this chapter may provide the basis for a more general methodology for tailoring configuration semantics to the desired modelling setting. Exploring this idea in a broader and more systematic way is an interesting direction for future work.

3.4.6 Overview of Definitions, Theorems, and Lemmas

We conclude this chapter by summarizing and collecting all definitions, theorems, and lemmas introduced in this chapter.

In Section 3.1, we introduced the problems associated with instance-based configurations. In particular, we identified two types of index anomalies that can occur

in such configurations: subtree index inversion and sibling index inversion. These anomalies not only inflate the configuration space, but also prevent the application of an intuitive merge algorithm in which a configuration is constructed incrementally from smaller parts of the CFM.

To formalize this merge property, we introduced sub-models (Definition 3.1) and sub-configurations (Definition 3.2). Sub-models restrict the feature tree of the CFM to the subtree rooted at a given feature, while sub-configurations extract the corresponding subtree of the feature instance tree of a configuration rooted at a given feature instance. Moreover, we introduced the notions of *child models* and *child configurations* (Definition 3.3). A child model is the sub-model rooted at a child feature of the feature currently under consideration, and a child configuration is a configuration of such a child model. These notions allow us to formalize both the merge property and the corresponding merge algorithm.

For convenience, we defined child sets (Definition 3.4) for both CFMs and instance-based configurations. We further introduced shifted configurations (Definition 3.5), which are instance-based configurations in which feature-instance indices are shifted by a certain offset for each feature. With these concepts in place, we were able to define the merge algorithm (Algorithm 3.1) and the corresponding merge property (Definition 3.6). The merge property is a general condition on a configuration semantics requiring that merging valid child configurations yields a valid configuration of the full model, and that every valid configuration of the full model can be obtained in this way.

Finally, we observed that instance-based configurations do not satisfy the merge property. This observation motivated the introduction of structural and semi-structural configurations.

In Section 3.2, we introduced structural configurations. The initial definition was given in terms of equivalence classes (Definition 3.9) induced by tree isomorphism (Definition 3.7). In particular, we showed that this isomorphism induces an equivalence relation on the set of instance-based configurations (Lemma 3.8). Moreover, we proved that configuration validity is invariant under tree isomorphism (Lemma 3.10), which allows us to define the validity of a structural configuration by selecting an arbitrary instance-based representative of the corresponding equivalence class.

After this formal definition, we introduced an index-free tree representation that allows structural configurations to be interpreted as rooted, directed, unordered feature-labelled trees. A drawback of this representation is that nodes no longer have unique identifiers. To address this issue, we introduced canonical configurations, which provide a unique instance-based representative for each valid equivalence class based on a suitable canonicality criterion.

This criterion must distinguish configurations exhibiting subtree index inversion or sibling index inversion, since such configurations are tree-isomorphic and therefore belong to the same equivalence class. To eliminate subtree index inversions, we introduced the notion of *subtree index monotonicity*, which requires that smaller indices occur under smaller indices in the feature instance tree. Similarly, to eliminate sibling index inversions, we introduced *sibling index monotonicity*, which requires that, for sibling instances of the same feature, their subtrees are assigned according to a fixed priority.

To formalize this priority, we defined the *instance order* (Definition 3.12), a preorder on the feature instances of a configuration. This preorder compares instances either by their feature type (if the underlying features differ) or by the lexicographic preorder (Definition 3.11) induced by their children.

After defining the instance order, we established several fundamental properties. In particular, we showed that the instance order is invariant under tree isomorphisms (Theorem 3.13) and that feature instances equivalent under the instance order share several structural properties, such as having the same feature and the same number of child instances of each feature type (Lemma 3.14). Finally, we proved that two feature instances are equivalent with respect to the instance order if and only if their sub-configurations are tree-isomorphic (Theorem 3.15).

Using the instance order, we were then able to formally define the canonicality criteria, namely subtree index monotonicity, and sibling index monotonicity, as well as the notion of a canonical configuration (Definition 3.16). We further showed that the indices in canonical configurations exhibit a block structure (Lemma 3.17). This property allowed us to prove the existence and uniqueness of canonical configurations within each valid tree-isomorphism equivalence class (Theorem 3.18). Consequently, the canonical configurations form a unique system of representatives for the structural configuration space.

Finally, we showed that canonical configurations, and therefore structural configurations, satisfy the merge property. In particular, merging sorted lists of canonical child configurations yields a canonical configuration of the full model (Theorem 3.19). Furthermore, the sub-configuration of a canonical configuration is again canonical for the corresponding sub-model (Theorem 3.20), and every canonical configuration can be obtained by merging a suitable set of canonical child configurations (Theorem 3.21). All of these results hold for CFMs *without* cross-tree constraints.

In Section 3.3, we introduced semi-structural configurations. The construction proceeds similarly to the structural case. We first defined order-preserving isomorphisms between instance-based configurations (Definition 3.22), which again induce an equivalence relation on the set of instance-based configurations (Lemma 3.23). Semi-structural configurations are then defined as equivalence classes under this order isomorphism (Definition 3.24), and validity is preserved under this relation (Lemma 3.25).

Semi-structural configurations also admit an index-free representation, this time as rooted, directed, *ordered* feature-labelled trees. As in the structural case, valid semi-structural configurations admit a system of representatives consisting of instance-based configurations, called *semi-canonical configurations* (Definition 3.26). The existence and uniqueness of these representatives were proved in Theorem 3.27.

Moreover, semi-canonical configurations, and thus also semi-structural configurations, satisfy the merge property. In particular, merging semi-canonical child configurations (this time in arbitrary order) yields a semi-canonical configuration (Theorem 3.28). Furthermore, sub-configurations of semi-canonical configurations are again semi-canonical configurations of the corresponding sub-model (Theorem 3.29), and every semi-canonical configuration can be obtained by merging a suitable set of semi-canonical child configurations (Theorem 3.30). As before, these results hold only for CFMs *without* cross-tree constraints.

4. Uniform Random Sampling

This chapter develops algorithms for generating uniform random samples from the configuration space of a CFM. More precisely, we sample from the *structural configuration space* (equivalently, the canonical configuration space; see Chapter 3). In particular, we do not work with the instance-based configuration space, since it is not closed under Merge (see Section 3.1). However, the algorithms and techniques presented in this chapter can be adapted easily to semi-structural configurations. We briefly discuss this adaptation in the conclusion (see Section 4.3).

The chapter proceeds in two stages. First, in Section 4.1, we consider CFMs *without* cross-tree constraints. In this setting, we present two algorithms for uniform random sampling:

- UniSample-BT, which constructs configurations recursively using weighted random choices guided by specially constructed dynamic programming (DP) tables, and
- UniSample-UR, which samples configurations by means of a ranking and unranking scheme (see Section 2.4.3).

In the second stage, presented in Section 4.2, we extend these algorithms to CFMs with cross-tree constraints. This extension is generic: any sampling algorithm that generates canonical configurations uniformly for models without cross-tree constraints can be extended to a uniform random sampling algorithm for CFMs with cross-tree constraints.

Throughout this chapter, we assume that all feature-instance cardinalities are bounded. This ensures that the configuration space of every sub-model considered by the algorithms is finite. In particular, it is not sufficient for the configuration space to become finite only after applying cross-tree constraints, since the algorithms operate bottom-up on the relaxed model without cross-tree constraints and therefore require every sub-model to admit only finitely many configurations.

If a model contains unbounded feature-instance cardinalities, these must therefore be bounded before applying the algorithms presented in this chapter. In practice,

this can be achieved using the Big-M method introduced in Section 2.2.3, which ensures that all non-convex regions of the configuration space are preserved.

4.1 Uniform Random Sampling without Cross-Tree Constraints

One of the key properties of canonical (and therefore structural) configurations is that they are closed under the Merge operation. More precisely, by Theorems 3.19 and 3.21, every canonical configuration of a model can be obtained by merging canonical configurations of child models.

This property allows us to reduce the problem of uniformly sampling from the canonical configuration space recursively along the feature tree. Starting at the root feature, we generate configurations for the sub-models rooted at its children and then combine them using Merge. Applying this idea recursively decomposes the sampling problem into smaller and smaller subproblems, until we reach leaf models, i.e., models consisting solely of the root feature. For such models, sampling is trivial, since they admit exactly one valid configuration (independent of the chosen configuration semantics).

The main difficulty of this approach lies in determining how to perform the transitions during the merging process. In particular, given an algorithm that generates uniform random samples for the child models, we must determine how these samples should be combined using Merge so that the resulting configuration of the full model remains uniformly distributed. As explained in the conclusion of Chapter 3, ensuring this property requires careful consideration.

To address this problem, we first study how to count the number of canonical (and therefore structural) configurations of a CFM without cross-tree constraints. As discussed in Section 2.3, counting the number of solutions to a combinatorial problem is often closely related to the problem of uniform random sampling.

4.1.1 Counting Canonical Configurations via Dynamic Programming

As in the case of uniform random sampling, directly counting the number of canonical configurations (or configurations in general) of a CFM is difficult, whereas for a leaf model, the counting problem is trivial.

To address this problem, we employ dynamic programming (DP). DP is a technique for solving complex problems by decomposing them into simpler subproblems. However, in contrast to the divide-and-conquer technique, these subproblems overlap.

There are two common approaches for storing and reusing the solutions of these subproblems. In the *top-down* approach (also called *memoization*), the problem is solved recursively and the results of already solved subproblems are stored and reused whenever they are needed again. In the *bottom-up* approach (also called *tabulation*), the solutions are stored in a table-like data structure that is filled iteratively [80, 81, 82].

In our setting, we use the bottom-up (tabulation) approach because it is often more efficient [82] and because we require the full DP tables later for our sampling

algorithms. Concretely, we start at the leaf features and compute the number of canonical configurations for each leaf model. We then traverse the feature tree upwards, combining the configuration counts of the child models in a suitable way to obtain the counts for the full model.

To formalise this approach, we first introduce the notion of *realised root-group cardinalities*. These quantities describe which concrete cardinalities of the root feature group are realised by a given configuration.

Definition 4.1 (Realised Root-Group Cardinalities)**.** Let $\mathcal{M}$ be a CFM over a feature set F with root feature f_r, and let K be a configuration of $\mathcal{M}$ (either multiset-based, instance-based, structural or semi-structural). We define two quantities for the root feature group:

- The *realised group-type cardinality* of K, denoted $\gamma_T(K)$, is the number of distinct child feature types of f_r in K.
- The *realised group-instance cardinality* of K, denoted $\gamma_I(K)$, is the number of child instances of f_r in K.

Formally, let M be the multiset associated with K (i.e. $K = M$, $K = (M, \prec)$, $K = [\tilde{K}]_{\text{Struct}}$ or $K = [\tilde{K}]_{\text{SemiStruct}}$ with $\tilde{K} = (M, \prec)$). Then

$$\gamma_T(K) \coloneqq \Big|\{\, f \in C_{\mathcal{M}}(f_r) \mid M(f) > 0 \,\}\Big|,$$
$$\gamma_I(K) \coloneqq \sum_{f \in C_{\mathcal{M}}(f_r)} M(f).$$

Remark. The definition of the realised root-group cardinalities is well-defined for all types of configurations. In particular, it is well-defined for structural and semi-structural configurations, since tree- or order-isomorphic instance-based configurations share the same underlying multiset.

Moreover, we define realised cardinalities only for the root group for the following reason. In every valid instance-based configuration there is exactly one instance of the root feature and therefore exactly one root feature instance group. For all other features, however, multiple feature instances may occur, and consequently several instances of their associated feature groups may be present. In such cases one would have to specify to which group instance the realised cardinalities refer.

4.1.1.1 DP Variables

Using the notion of realised root-group cardinalities, we can now define the concrete subproblems and which DP variables exist.

For each feature $f \in F$ and each pair $(c, k) \in \mathbb{N}_0 \times \mathbb{N}_0$, we consider the number of canonical configurations K of the sub-model $\mathcal{M}_f$ whose realised root-group cardinalities satisfy

$$\gamma_T(K) = c \qquad \text{and} \qquad \gamma_I(K) = k.$$

We denote this number by $C_f(c, k)$. Thus, $C_f(c, k)$ counts the canonical configurations of $\mathcal{M}_f$ that realise exactly c distinct child feature types and k feature instances in the root group.

The variables $C_f(c,k)$ already capture how root-group cardinalities affect the number of canonical configurations. However, they do not distinguish between different children of f. In particular, if the number of canonical configurations in a child model of f changes, we cannot locate at which child this change happens, and how it affects $C_f(c,k)$.

To capture this information, we refine the DP variables as follows. For a feature $f \in F$ with children $c_1, \ldots, c_n$, we define $C_f^i(c,k)$ as the number of canonical configurations of $\mathcal{M}_f$ that realise $\gamma_T(K) = c$ and $\gamma_I(K) = k$, under the restriction that only the first i children $c_1, \ldots, c_i$ of f may contribute instances. That is, these configurations contain no feature instances of the children $c_{i+1}, \ldots, c_n$.

Intuitively, these variables $C_f^i(c,k)$ should now be related in the following sense. If we take a canonical configuration K' that is counted by $C_f^i(c,k)$ and add r child configurations of c_i, we obtain a canonical configuration K that is counted by $C_f^{i+1}(c+\mathbb{1}_{\{r>0\}}, k+r)$. Note that we have to add the new instances in such a way that sibling index monotonicity and subtree index monotonicity are maintained in order to obtain a canonical configuration (see Section 3.2.4).

However, this assumption is false because $C_f^i(c,k)$ counts canonical configurations of $\mathcal{M}_f$, i.e., only valid configurations. Since cardinality intervals may contain gaps, it can occur that $(c,k) \in \lambda_T^G(f) \times \lambda_I^G(f)$ while $(c + \mathbb{1}_{\{r>0\}},\, k+r) \notin \lambda_T^G(f) \times \lambda_I^G(f)$. In this case, adding r instances of a child feature yields a configuration that violates the root-group constraints and is therefore not counted by the DP variables, even though this violation may disappear once we add instances of the children $c_{i+2}, \ldots, c_n$. By contrast, choosing a value of r that exceeds the upper bounds of the cardinality intervals leads to violations that can never be repaired by adding further children. Such cases correspond to genuinely invalid constructions and must be excluded altogether.

In order to resolve this issue, we introduce the notion of a *filled sub-model*. In this model, all gaps in the group-type and group-instance cardinality intervals of f are filled, while the original upper bounds are preserved. That is, the cardinalities are allowed to take any value between 0 and their respective maxima.

Definition 4.2 (Filled Sub-Model)**.** Let $\mathcal{M} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \phi_R, \phi_X)$ be a CFM over a feature set F, and let $f \in F$. The *filled sub-model* of $\mathcal{M}$ at f, denoted $\mathcal{M}_f^{\text{filled}}$, is the CFM obtained from the sub-model $\mathcal{M}_f$ by removing all gaps in the root-group cardinality constraints at f.

Concretely, $\mathcal{M}_f^{\text{filled}}$ is identical to $\mathcal{M}_f$ in all aspects, except we set the group-type cardinality interval of f to

$$\psi_T^G(f) \coloneqq \left\langle 0, \max \lambda_T^G(f) \right\rangle,$$

and the group-instance cardinality interval of f to

$$\psi_I^G(f) \coloneqq \left\langle 0, \max \lambda_I^G(f) \right\rangle.$$

Using the filled sub-model, we now define $C_f^i(c,k)$ as the number of canonical configurations of $\mathcal{M}_f^{\text{filled}}$ that realise $\gamma_T(K) = c$ and $\gamma_I(K) = k$, under the restriction that

only the first i children $c_1, \ldots, c_i$ of f may contribute instances. This relaxation ensures that all intermediate states arising from iteratively adding child configurations are captured by the DP variables.

In total, we obtain the following DP variables.

Definition 4.3 (DP Variables)**.** Let $\mathcal{M}$ be a CFM over an ordered feature set $(F, \leq_F)$ with no cross-tree constraints.

For a feature $f \in F$, $(c, k) \in \mathbb{Z}^2$ and $i \in \{0, \ldots, |C_{\mathcal{M}}(f)|\}$, we define:

T_f	Total number of canonical configurations of the sub-model rooted at f. Thus, $T_f = \lvert[\![\mathcal{M}_f]\!]_{\text{Canon}}\rvert$.
$C_f(c, k)$	Number of canonical configurations K of the sub-model $\mathcal{M}_f$ such that the realised group-type and group-instance cardinalities satisfy $\gamma_T(K) = c$ and $\gamma_I(K) = k$. In other words, only configurations with exactly c distinct child feature types and exactly k child instances under f are counted.
$C_f^i(c, k)$	Number of canonical configurations K of the filled sub-model $\mathcal{M}_f^{\text{filled}}$ with $(\gamma_T(K), \gamma_I(K)) = (c, k)$, under the restriction that only the first i children (first with respect to $\leq_F$) may contribute instances. That is, if $C_{\mathcal{M}}(f) = \{c_1, \ldots, c_n\}$ (enumerated with respect to $\leq_F$), K contains no feature instances from the children $c_{i+1}, \ldots, c_n$. In particular, if $i = 0$, no children are allowed and if $i = n$, all children are allowed.

Additionally, the *DP table* of $f \in F$ is defined as the collection of values $C_f^i(c, k)$ indexed by $i \in \{0, \ldots, |C_{\mathcal{M}}(f)|\}$ and $(c, k) \in \mathbb{Z}^2$.

The following lemma collects some basic properties of the DP variables.

Lemma 4.4 (Basic Properties of the DP Variables)**.** Let $\mathcal{M} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \emptyset, \emptyset)$ be a CFM over an ordered feature set $(F, \leq_F)$ with no cross-tree constraints. Let $f \in F$ have $n \in \mathbb{N}_0$ children. Then the following holds.

1. For all $i \in \{0, \ldots, n\}$ and all $(c, k) \in \mathbb{Z}^2$, we have
$$C_f^i(c, k) = 0 \qquad \text{whenever } c \notin \psi_T^G(f) \text{ or } k \notin \psi_I^G(f).$$
Moreover, for $i = 0$,
$$C_f^0(c, k) \;=\; \begin{cases} 1, & \text{if } (c, k) = (0, 0), \\ 0, & \text{otherwise.} \end{cases}$$
2. For all $(c, k) \in \lambda_T^G(f) \times \lambda_I^G(f)$,
$$C_f^n(c, k) = C_f(c, k).$$
3. The total number T_f of canonical configurations of the sub-model $\mathcal{M}_f$ satisfies
$$T_f = \left|[\![\mathcal{M}_f]\!]_{\text{Canon}}\right| = \sum_{c \in \lambda_T^G(f)} \sum_{k \in \lambda_I^G(f)} C_f(c, k).$$

Proof. We prove the three statements separately.

(1). $C_f^i(c,k) = 0$, for $c \notin \psi_T^G(f)$ or $k \notin \psi_I^G(f)$, follows immediately because there are no valid configurations that satisfy the root-group constraints.

For $i = 0$ no child of f is allowed to contribute instances. Thus, any canonical configuration counted by $C_f^0(c,k)$ must contain no child instances, and therefore has realised root-group cardinalities $(c,k) = (0,0)$. Furthermore, there is exactly one such canonical configuration, namely the configuration consisting solely of the feature instance f^1, and it is counted by $C_f^0(c,k)$ since it is a canonical configuration of $\mathcal{M}_f^{\text{filled}}$ ($(0,0)$ is an allowed root-group cardinality for $\mathcal{M}_f^{\text{filled}}$).

(2). If $n = 0$ (i.e. f is a leaf), then $C_f^n(c,k) = C_f^0(c,k)$. By **(1)**, we have $C_f^0(c,k) = 1$ if $(c,k) = (0,0)$ and $C_f^0(c,k) = 0$ otherwise. Since f is a leaf, the sub-model $\mathcal{M}_f$ admits exactly one canonical configuration, namely the configuration consisting solely of the feature instance f^1, whose realised root-group cardinalities are $(0,0)$. Consequently, $C_f(c,k)$ takes the same values as $C_f^0(c,k)$, and therefore

$$C_f^n(c,k) = C_f(c,k) \quad \text{for all } (c,k) \in \mathbb{Z}^2.$$

If $n \geq 1$ (i.e. f is not a leaf), let $C_{\mathcal{M}}(f) = \{c_1, \ldots, c_n\}$, enumerated with respect to $\leq_F$. By definition, $C_f^n(c,k)$ counts the canonical configurations of the filled sub-model $\mathcal{M}_f^{\text{filled}}$ with realised root-group cardinalities (c,k), under the restriction that only the first n children may contribute instances. Since f has exactly n children, this restriction is vacuous. Hence, $C_f^n(c,k)$ counts all canonical configurations of $\mathcal{M}_f^{\text{filled}}$ with realised root-group cardinalities (c,k).

If $(c,k) \in \lambda_T^G(f) \times \lambda_I^G(f)$, these configurations coincide with the canonical configurations of the original sub-model $\mathcal{M}_f$ with realised root-group cardinalities (c,k). Consequently,

$$C_f^n(c,k) = C_f(c,k) \quad \text{for all } (c,k) \in \lambda_T^G(f) \times \lambda_I^G(f).$$

(3). By definition, $C_f(c,k)$ counts the number of canonical configurations $K \in [\![\mathcal{M}_f]\!]_{\text{Canon}}$ whose realised root-group cardinalities satisfy

$$\gamma_T(K) = c \qquad \text{and} \qquad \gamma_I(K) = k.$$

For any canonical configuration $K = (M, \prec)$, the values c and k are uniquely determined by the underlying multiset M. Consequently, if $(c,k) \neq (c',k')$, no canonical configuration can be counted by both $C_f(c,k)$ and $C_f(c',k')$. Hence, for distinct pairs $(c,k) \in \mathbb{N}_0 \times \mathbb{N}_0$, the sets of canonical configurations counted by $C_f(c,k)$ are pairwise disjoint, which yields

$$T_f \geq \sum_{c \in \lambda_T^G(f)} \sum_{k \in \lambda_I^G(f)} C_f(c,k).$$

Conversely, every canonical configuration $K \in [\![\mathcal{M}_f]\!]_{\text{Canon}}$ has well-defined realised cardinalities $c = \gamma_T(K)$ and $k = \gamma_I(K)$ and is therefore counted by exactly one value $C_f(c,k)$. Hence, every canonical configuration contributes to the above sum, which implies

$$T_f \leq \sum_{c \in \lambda_T^G(f)} \sum_{k \in \lambda_I^G(f)} C_f(c,k).$$

Combining both inequalities yields

$$T_f = \left|[\![\mathcal{M}_f]\!]_{\text{Canon}}\right| = \sum_{c \in \lambda_T^G(f)} \sum_{k \in \lambda_I^G(f)} C_f(c,k).$$

□

4.1.1.2 Recursion Formula

Having defined the DP variables, we now establish how they are related. In particular, the following theorem expresses the variables $C_f^i(c,k)$ in terms of the variables $C_f^{i-1}(c',k')$. It therefore characterises how the number of canonical configurations changes when configurations are allowed to contain instances from one additional child.

This stepwise characterisation allows us to count canonical configurations incrementally by considering the children of f one by one.

Theorem 4.5 (DP Recursion Formula)**.** Let $\mathcal{M} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \emptyset, \emptyset)$ be a CFM over an ordered feature set $(F, \leq_F)$ with no cross-tree constraints, and let $f \in F$ be a non-leaf feature with children $c_1, \ldots, c_n$ (sorted increasingly with respect to $\leq_F$). For $i \in \{1, \ldots, n\}$ and $(c,k) \in \psi_T^G(f) \times \psi_I^G(f)$, we have

$$C_f^i(c,k) = \sum_{r \in \lambda_I^F(c_i)} C_f^{i-1}\big(c - \mathbb{1}_{\{r>0\}},\, k - r\big) \cdot \left(\!\!\binom{T_{c_i}}{r}\!\!\right).$$

Proof. Let $f \in F$ be a non-leaf feature with children $c_1, \ldots, c_n$, and let $(c,k) \in \psi_T^G(f) \times \psi_I^G(f)$.

Every canonical configuration $K = (M, \prec)$ counted by $C_f^i(c,k)$ contains a well-defined number

$$r \coloneqq M(c_i) \in \lambda_I^F(c_i)$$

of instances of the child feature c_i. Hence, the configurations counted by $C_f^i(c,k)$ can be partitioned according to this value. For each $r \in \lambda_I^F(c_i)$ let

$$\mathcal{Z}_r \coloneqq \left\{ K = (M, \prec) \in \left[\!\!\left[\mathcal{M}_f^{\text{filled}}\right]\!\!\right]_{\text{Canon}} \;\middle|\; K \text{ is counted by } C_f^i(c,k) \text{ and } M(c_i) = r \right\}.$$

These sets are pairwise disjoint, and their union is exactly the set of configurations counted by $C_f^i(c,k)$. Consequently,

$$C_f^i(c,k) = \sum_{r \in \lambda_I^F(c_i)} |\mathcal{Z}_r|.$$

Fix $r \in \lambda_I^F(c_i)$. We now show that

$$|\mathcal{Z}_r| = C_f^{i-1}\big(c - \mathbb{1}_{\{r>0\}},\, k - r\big) \cdot \left(\!\!\binom{T_{c_i}}{r}\!\!\right). \qquad (*)$$

To this end, let $\mathcal{A}_r$ denote the set of pairs (K', V) such that

- K' is a canonical configuration counted by $C_f^{i-1}\big(c - \mathbb{1}_{\{r>0\}},\, k - r\big)$, and
- V is an r-multiset of canonical configurations of $\mathcal{M}_{c_i}$.

Since there are $\left(\!\!\binom{T_{c_i}}{r}\!\!\right)$ r-multisets over $[\![\mathcal{M}_{c_i}]\!]_{\text{Canon}}$, whose cardinality is T_{c_i}, we obtain

$$|\mathcal{A}_r| = C_f^{i-1}\big(c - \mathbb{1}_{\{r>0\}},\, k - r\big) \cdot \left(\!\!\binom{T_{c_i}}{r}\!\!\right).$$

Note that this formula also correctly handles the case $T_{c_i} = 0$: by the definition of $\left(\!\!\binom{n}{k}\!\!\right)$ (Definition 2.12) we have

$$\left(\!\!\binom{0}{0}\!\!\right) = 1 \quad \text{and} \quad \left(\!\!\binom{0}{r}\!\!\right) = 0 \text{ for } r > 0,$$

since the empty set admits exactly one multiset of size 0 (the empty multiset) and no multisets of positive size.

To show $(*)$, we construct maps

$$\Phi_r \colon \mathcal{A}_r \to \mathcal{Z}_r \qquad \text{and} \qquad \Psi_r \colon \mathcal{Z}_r \to \mathcal{A}_r$$

and show that they are inverse to each other.

Given $(K', V) \in \mathcal{A}_r$, order the configurations in V non-decreasingly with respect to the instance order $\trianglelefteq$ and iteratively merge them into K' using a slight modification of the merge algorithm (Algorithm 3.1), in which the root instance already present in K' is preserved and only additional child configurations are attached.

Since K' contains no instances of c_i and each configuration in V contributes exactly one instance of c_i, the resulting configuration has realised root-group cardinalities (c, k). Moreover, by the same argument as in Theorem 3.19, the resulting configuration is canonical. Thus, this construction defines a map

$$\Phi_r \colon \mathcal{A}_r \to \mathcal{Z}_r.$$

Conversely, let $K = (M, \prec) \in \mathcal{Z}_r$. If $c - \mathbb{1}_{\{r>0\}} < 0$ or $k - r < 0$, then $\mathcal{Z}_r = \emptyset$, because there is no configuration with r instances of a child feature such that the realised group-type cardinality is less than $\mathbb{1}_{\{r>0\}}$ or the group-instance cardinality is less than r. Hence, we can assume $c - \mathbb{1}_{\{r>0\}} \geq 0$ and $k - r \geq 0$.

We now remove all r instances of c_i together with all their descendants from K, which yields a configuration K' whose realised root-group cardinalities are

$$(\gamma_T(K'), \gamma_I(K')) = \big(c - \mathbb{1}_{\{r>0\}},\, k - r\big).$$

By construction, K' contains no instances of c_i and no instances of any child c_j with $j > i$. Moreover, since removing all instances of a feature together with their descendants preserves canonicity, K' is counted by

$$C_f^{i-1}\big(c - \mathbb{1}_{\{r>0\}},\, k - r\big).$$

Finally, the r removed c_i-instances together with their descendants each yield a canonical configuration of $\mathcal{M}_{c_i}$ (possibly after shifting indices). Together, they form a multiset V of size r over $[\![\mathcal{M}_{c_i}]\!]_{\text{Canon}}$. Thus, we obtain a tuple $(K', V) \in \mathcal{A}_r$ from $K \in \mathcal{Z}_r$, which defines a map

$$\Psi_r \colon \mathcal{Z}_r \to \mathcal{A}_r.$$

It remains to show that Φ_r and Ψ_r are inverse to each other. Starting from $(K', V) \in \mathcal{A}_r$, the map Φ_r merges exactly the configurations contained in the multiset V into K'. Applying Ψ_r to the resulting configuration removes all c_i-subtrees again and recovers the same residual configuration K' together with the same multiset V. Hence,

$$\Psi_r\Big(\Phi_r(K', V)\Big) = (K', V) \qquad \text{for all } (K', V) \in \mathcal{A}_r.$$

Conversely, starting from $K \in \mathcal{Z}_r$, the map Ψ_r removes all c_i-subtrees and records them as a multiset V, while retaining the remaining configuration K'. Applying Φ_r then reattaches exactly these removed canonical child configurations to K' in canonical order, and therefore reconstructs K. Thus,

$$\Phi_r\Big(\Psi_r(K)\Big) = K \qquad \text{for all } K \in \mathcal{Z}_r.$$

In total,

$$\Psi_r \circ \Phi_r = \mathrm{id}_{\mathcal{A}_r} \qquad \text{and} \qquad \Phi_r \circ \Psi_r = \mathrm{id}_{\mathcal{Z}_r},$$

which shows that Φ_r and Ψ_r are inverse bijections. In particular,

$$|\mathcal{A}_r| = |\mathcal{Z}_r|,$$

which proves $(*)$ and hence the DP recursion formula. □

Remark. The proof crucially relies on choosing $\mathcal{A}_r$ as the set of pairs whose first component is a canonical configuration counted by

$$C_f^{i-1}\Big(c - \mathbb{1}_{\{r>0\}},\, k - r\Big)$$

and whose second component is a r-multiset of canonical configurations of $\mathcal{M}_{c_i}$. In particular, if we were to use sequences of length r of canonical configurations instead, then Ψ_r would still be injective, but Φ_r would no longer be injective. Indeed, multiple sequences would be mapped to the same canonical configuration, since the child configurations are sorted before they are merged.

Moreover, in the recursion formula of Theorem 4.5, the arguments

$$c - \mathbb{1}_{\{r>0\}} \qquad \text{and} \qquad k - r$$

may become negative for certain values of r. This is not problematic, since in such cases

$$C_f^{i-1}\Big(c - \mathbb{1}_{\{r>0\}},\, k - r\Big) = 0,$$

by Lemma 4.4.

4.1.1.3 Computation of the DP Tables

Using the recursion formula from Theorem 4.5, we can now compute the DP tables incrementally. Moreover, by Lemma 4.4, once the DP tables have been computed, we can also compute the total configuration counts.

Since the entries of the DP table of a feature f depend on the total configuration counts of its child features, these values must be available before f is processed. Hence, the computation proceeds bottom-up using a post-order traversal of the feature tree, which guarantees that the DP tables and total configuration counts of all child features are available when processing f [78]. The complete procedure is shown in Algorithm 4.1.

For a feature $f \in F$, the algorithm proceeds as follows.

If f is a leaf feature, then the sub-model $\mathcal{M}_f$ admits exactly one instance-based configuration. This configuration is canonical, and hence

$$T_f = 1.$$

If f has children $c_1, \ldots, c_n$, then their DP tables and total configuration counts have already been computed, since the traversal is performed in post-order. For each child index $i = 1, \ldots, n$ and each pair $(c, k) \in \psi_T^G(f) \times \psi_I^G(f)$, the value $C_f^i(c, k)$ is computed using the recursion formula from Theorem 4.5. The values $C_f^i(c, k)$ for $i = 0$ or $(c, k) \notin \psi_T^G(f) \times \psi_I^G(f)$ need not be computed explicitly, since they are the same for all features by Lemma 4.4.

Afterwards, the total configuration count T_f is computed via

$$T_f = \sum_{c \in \lambda_T^G(f)} \sum_{k \in \lambda_I^G(f)} C_f(c, k) = \sum_{c \in \lambda_T^G(f)} \sum_{k \in \lambda_I^G(f)} C_f^n(c, k).$$

Algorithm 4.1 ComputeDP($\mathcal{M}$): Algorithm for computing DP tables that count canonical configurations in a CFM without cross-tree constraints using bottom-up dynamic programming

Input: CFM $\mathcal{M} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \emptyset, \emptyset)$ over ordered feature set $(F, \leq_F)$

Output: DP tables $C_f^i(c,k)$ and total configuration counts T_f for all $f \in F$

1: **for all** features $f \in F$ in post-order **do**
2: **if** f is a leaf **then**
3: $T_f \leftarrow 1$
4: **continue**
5: **end if**
6: **for** $i = 1$ to $|C_{\mathcal{M}}(f)|$ **do**
7: **for all** $(c,k) \in \psi_T^G(f) \times \psi_I^G(f)$ **do**
8: $$C_f^i(c,k) \leftarrow \sum_{r \in \lambda_I^F(c_i)} C_f^{i-1}\Big(c - \mathbb{1}_{\{r>0\}},\, k - r\Big) \cdot \left(\!\!\binom{T_{c_i}}{r}\!\!\right)$$
9: **end for**
10: **end for**
11: $$T_f \leftarrow \sum_{(c,k) \in \lambda_T^G(f) \times \lambda_I^G(f)} C_f(c,k)$$
12: **end for**

4.1.2 The UniSample-BT Algorithm

Using the precomputed DP tables, we now introduce the algorithm UniSample-BT, which performs uniform random sampling from the canonical (structural) configuration space of a CFM *without* cross-tree constraints.

At a high level, our sampling algorithm follows the same fundamental approach as the random sampling algorithm of Zhang [18], presented in Section 2.3.2. In particular, it constructs a configuration recursively, starting at the root feature. For each feature, the algorithm first selects concrete values for the group-type and group-instance cardinalities and then determines how many feature instances each child contributes to the selected target cardinalities. Afterwards, the algorithm is called recursively to generate child configurations for each child.

In contrast to Zhang, however, our algorithm does not select these cardinalities uniformly at random. Instead, each possible choice is weighted by the number of canonical configurations that are compatible with it. As a result, choices that admit many canonical configurations are selected with higher probability than those that admit only a few. This weighting is the essential modification which ensures that the final outcome is a uniform sample from the canonical configuration space.

To determine these weights, we use the DP tables $C_f^i(c,k)$. In particular, they are designed in such a way that they allow us to answer questions such as "how many canonical configurations realise a given pair of root-group cardinalities (c,k)?" or "if we select r instances of feature c_i, in how many ways can we choose instances of the children $c_1, \ldots, c_{i-1}$, together with their child configurations, so that the resulting configuration has realised root-group cardinalities (c,k)?".

The former question can be answered directly via $C_f(c, k)$, while the latter can be answered using

$$C_f^{i-1}\big(c_{\text{rem}} - \mathbb{1}_{\{r>0\}},\, k_{\text{rem}} - r\big) \cdot \left(\!\!\binom{T_{c_i}}{r}\!\!\right),$$

since extending each configuration counted by $C_f^{i-1}(c_{\text{rem}} - \mathbb{1}_{\{r>0\}}, k_{\text{rem}} - r)$ with a multiset of r canonical configurations of the sub-model $\mathcal{M}_{c_i}$ yields a canonical configuration whose realised root-group cardinalities match the desired target values.

The full algorithm is shown in Algorithm 4.2, and it proceeds in three steps.

Step 1: Sampling root-group cardinalities. For the current feature f, the algorithm first samples a pair of root-group cardinalities (c, k), where c denotes the number of distinct child feature types and k the total number of child instances under f. The pair (c, k) is chosen according to the probability distribution

$$\mathbb{P}(c, k) = \frac{C_f(c, k)}{T_f}.$$

Thus, the pair (c, k) is selected with probability proportional to the number of canonical configurations that realise these root-group cardinalities. This ensures that cardinalities that lead to many canonical configurations are chosen more often on average than cardinalities that lead to fewer configurations.

Step 2: Sampling child multiplicities. Given the selected root-group cardinalities (c, k), the algorithm determines how many instances each child feature contributes. This is done by iterating backwards over the ordered children of f and sampling a multiplicity $r_i \in \mathbb{N}_0$ for each child c_i.

At the beginning of the iteration for child c_i, let $(c_{\text{rem}}, k_{\text{rem}})$ denote the remaining group-type and group-instance cardinalities. Thus, they describe how many distinct feature types and feature instances can still be chosen. The multiplicity r_i is sampled according to the probability

$$\mathbb{P}(r_i \mid c_{\text{rem}}, k_{\text{rem}}) = \frac{C_f^{i-1}\big(c_{\text{rem}} - \mathbb{1}_{\{r_i>0\}},\, k_{\text{rem}} - r_i\big) \cdot \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right)}{C_f^{i}(c_{\text{rem}}, k_{\text{rem}})}.$$

This choice follows directly from the DP recursion formula (Theorem 4.5). In particular, for a fixed value r_i, the product

$$C_f^{i-1}\big(c_{\text{rem}} - \mathbb{1}_{\{r_i>0\}},\, k_{\text{rem}} - r_i\big) \cdot \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right)$$

is exactly the number of canonical configurations that realise $(c_{\text{rem}}, k_{\text{rem}})$ and in which the child c_i contributes precisely r_i instances. Thus r_i is chosen with probability proportional to the number of configurations that are compatible with this choice.

After sampling r_i, the remaining cardinalities $(c_{\text{rem}}, k_{\text{rem}})$ are updated accordingly.

Step 3: Recursive construction and merging. In the final step, the algorithm recursively generates r_i canonical configurations of the sub-model rooted at c_i for each child c_i. This is done by invoking UniSample-BT r_i times for each child c_i, yielding a list of r_i child configurations. Afterwards, this list is sorted with respect to the instance order $\trianglelefteq$ and passed to Merge.

However, this naive approach, where every generated list is accepted, sorted, and then passed to Merge, would not yield a uniform sampling algorithm. The reason is that sorting identifies multiple distinct lists with the same multiset of configurations. For example, the lists $[A, B]$ and $[B, A]$ both result in the same sorted list $[A, B]$ and therefore correspond to the same multiset $\{A, B\}$.

What we actually want to generate uniformly is a multiset of configurations, whereas sequentially calling UniSample-BT yields a list of configurations uniformly. The reason sorting such lists breaks uniformity is that the number of lists that correspond to the same multiset depends on the multiset itself. More concretely, consider a multiset of size r_i that contains k distinct configurations with multiplicities $m_1, \ldots, m_k$, where $m_1 + \cdots + m_k = r_i$. By the ordered-sequence representation of multisets (see Definition 2.13), this multiset corresponds to a unique non-decreasing sequence of length r_i. Every list that sorts to this sequence is a permutation of its elements. Since permutations that only exchange identical configurations do not produce distinct lists, the number of lists corresponding to the multiset equals the number of distinct permutations of this sequence, namely

$$\frac{r_i!}{m_1! \cdots m_k!}.$$

As a consequence, multisets with few duplicates (i.e. where the m_j are close to 1) are overrepresented. For instance, the multiset $\{A, B\}$ corresponds to the lists $[A, B]$ and $[B, A]$, whereas the multiset $\{A, A\}$ corresponds only to the list $[A, A]$. Thus, if we generate lists uniformly, the multiset $\{A, B\}$ is twice as likely as $\{A, A\}$.

To compensate for this bias, the algorithm applies a rejection step. If the generated list contains k distinct configurations with multiplicities $m_1, \ldots, m_k$, it is accepted with probability

$$\frac{m_1! \cdots m_k!}{r_i!}.$$

This acceptance probability exactly cancels the number of permutations that lead to the same multiset, ensuring that every multiset of size r_i is sampled with equal probability.

Once a list is accepted, it is sorted according to the instance order $\trianglelefteq$. Finally, the sorted child configurations of all children are combined using Merge, yielding a canonical configuration.

Algorithm 4.2 UniSample-BT$(f, \mathcal{M})$: Algorithm for uniform random sampling in a CFM without cross-tree constraints via DP-guided recursive construction

Input:
- CFM $\mathcal{M} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \emptyset, \emptyset)$ over an ordered feature set $(F, \leq_F)$ together with precomputed DP tables $C_f^i(c,k)$ and total configuration counts T_f for all $f \in F$
- Feature f with children $\{c_1, \ldots, c_n\}$ (ordered w.r.t. $\leq_F$)

Output: Canonical configuration $(M, \prec)$ of $\mathcal{M}_f$

Step 1: Sample group cardinalities (c,k) at node f

Sample $(c,k) \in \lambda_T^G(f) \times \lambda_I^G(f)$ with probability

$$\mathbb{P}(c,k) = \frac{C_f(c,k)}{T_f}$$

$(c_{\text{rem}}, k_{\text{rem}}) \leftarrow (c,k)$

Step 2: Backward sampling of child multiplicities

for $i = n$ down to 1 **do**

  Sample $r_i \in \lambda_I^F(c_i)$ with probability

$$\mathbb{P}(r_i \mid c_{\text{rem}}, k_{\text{rem}}) = \frac{C_f^{i-1}\big(c_{\text{rem}} - \mathbb{1}_{\{r_i>0\}},\ k_{\text{rem}} - r_i\big) \cdot \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right)}{C_f^i(c_{\text{rem}}, k_{\text{rem}})}$$

  $c_{\text{rem}} \leftarrow c_{\text{rem}} - \mathbb{1}_{\{r_i>0\}}$

  $k_{\text{rem}} \leftarrow k_{\text{rem}} - r_i$

end for

Step 3: Construct canonical configuration recursively
for $i = 1$ to n **do**
  $L_i \leftarrow [\,]$ ▷ list of sampled configurations of $\mathcal{M}_{c_i}$
  if $r_i = 0$ **then**
    continue
  end if
  repeat
    $L_i \leftarrow [\,]$
    for $j = 1$ to r_i **do**
      $(M_j, \prec^j) \leftarrow$ UniSample-BT$(c_i, \mathcal{M})$
      append $(M_j, \prec^j)$ to L_i
    end for
    Determine the distinct configurations $D_1, \ldots, D_k$ in L_i and their multiplicities $m_1, \ldots, m_k$
    accept L_i with probability

$$p_{\text{acc}} \coloneqq \frac{m_1! \cdots m_k!}{r_i!},$$

  until the list is accepted
  Sort L_i increasingly with respect to $\trianglelefteq$
end for

$(M, \prec) \leftarrow$ Merge$((L_1, \ldots, L_n), \mathcal{M}_f)$
return $(M, \prec)$

Naming. The suffix BT stands for *backtracking*. The algorithm constructs a configuration by making a sequence of random choices that are guided by the DP tables. Starting from the root feature, it first samples the realised root-group cardinalities and then iteratively determines how many instances each child contributes. At each step, the algorithm selects a value that is consistent with the remaining cardinalities and the DP counts, thereby effectively following a path through the DP tables from the root state to a leaf state. In this sense, the algorithm performs a backtracking-style exploration of the configuration space, where the DP tables guide the choices so that the final configuration is sampled uniformly.

4.1.3 The UniSample-UR Algorithm

Using the same precomputed DP tables as before, we now describe UniSample-UR, an alternative uniform sampling algorithm for CFMs *without* cross-tree constraints. In contrast to UniSample-BT, which constructs a configuration incrementally by making random local choices guided by the DP tables, UniSample-UR follows a fundamentally different paradigm based on *ranking and unranking* (see Section 2.4.3).

As discussed in Section 2.3.1, there exists a close relationship between counting and uniform random sampling. In particular, if the number of solutions of a combinatorial problem is known, then one can assign each solution a unique integer rank and obtain a bijection between the solution space and the rank set $\{0, \ldots, N-1\}$, where N denotes the total number of solutions. Uniform random sampling can

then be performed by drawing a rank uniformly at random and reconstructing the corresponding solution using an *unranking* procedure.

The same idea can be applied to canonical configurations of a CFM. Since the DP framework introduced in the previous section allows us to compute the total number of canonical configurations of the model, we can assign each configuration a unique rank in the set

$$\{0, \dots, N-1\},$$

where $N = T_{f_r}$ denotes the total number of canonical configurations of the model rooted at f_r.

With this, the corresponding sampling algorithm becomes very simple. We draw a rank

$$R \in \{0, \dots, N-1\}$$

uniformly at random and reconstruct the corresponding configuration via the unranking procedure. The complete algorithm is shown in Algorithm 4.3. In particular, the suffix UR stands for *unranking*, which is the underlying technique used by the algorithm.

Algorithm 4.3 UniSample-UR($\mathcal{M}$): Algorithm for uniform random sampling in a CFM without cross-tree constraints via unranking

Input:
- CFM $\mathcal{M}$ over an ordered feature set $(F, \leq_F)$ with root feature f_r
- precomputed DP tables $C_f^i(c,k)$ and total configuration counts T_f for all $f \in F$

Output: A canonical configuration $(M, \prec)$ of $\mathcal{M}$

1: Draw $R \leftarrow \text{Uniform}\big(\{0, \dots, T_{f_r} - 1\}\big)$
2: **return** UnrankConfig($f_r, R, \mathcal{M}$)

4.1.3.1 Ranking a Configuration

The only difficult part of the unranking approach is deciding which rank is assigned to which configuration in such a way that an algorithm can efficiently reconstruct a configuration from a given rank. By efficiency, we specifically mean that unranking can be performed algorithmically without relying on a global lookup table that stores the rank of every canonical configuration. Concretely, we define the ranking via (strong) induction over the height h of the feature tree.

Base Case $h = 0$ (Leaf Model)

Let $\mathcal{M} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \emptyset, \emptyset)$ be a CFM over an ordered feature set $(F, \leq_F)$ with root feature $f_r \in F$ and tree height 0 (i.e. f_r is a leaf). Because f_r is a leaf, the only canonical configuration of $\mathcal{M}$ is the configuration consisting only of the feature instance f_r^1. We assign this configuration the rank 0.

Inductive Step

Now let $h \in \mathbb{N}_0$ and assume that the ranking is already defined for CFMs with tree height at most h. Using these existing rankings, we now define a ranking for a CFM with tree height $h+1$.

The general idea is to proceed hierarchically. Since we know that the ranks lie in $\{0, \dots, T_{f_r} - 1\}$, we can iteratively partition this set into contiguous subsets of ranks (i.e. each subset contains ranks of the form $i, i+1, i+2, \dots$ with no gaps). At each step, we specify for each partition which ranks it contains and to which configurations these ranks correspond. However, we do not define a concrete mapping between ranks and configurations inside the partition. Thus, we can only say which ranks each partition contains and which set of configurations is described by those ranks. As the partition is refined further, the number of ranks in each partition becomes smaller and the associated sets of configurations more specific.

Concretely, we partition the rank set in four steps. To distinguish the partitions arising in different steps, we use specific names for them. In the first step, the rank set is partitioned according to the realised root-group cardinalities, and the resulting partitions are called *blocks*. In the second step, each block is further partitioned into *grids*, based on the multiplicity of the last child feature c_n in the root group. In the third step, each grid is split into *fibers*, using a decomposition property of configurations. Finally, in the fourth step, we explicitly assign individual ranks to individual configurations.

Throughout this construction, we describe ranks relative to the current partition using *local ranks*. A local rank is only meaningful within its partition and can be translated to a rank at a higher level by adding the sizes of all preceding partitions at that level. For example, a fiber-local rank is converted to a grid-local rank by adding the sizes of all fibers preceding the current fiber within the same grid. Similarly, a grid-local rank is converted to a block-local rank by adding the sizes of all grids preceding the current grid within the block. Finally, the global rank is obtained by adding the sizes of all blocks preceding the current block to the block-local rank.

For the concrete construction of the partitions, we again use induction, but this time over the number of child features $n \in \mathbb{N}$ in the root group ($n \geq 1$, since the tree height is $h+1 \geq 1$). The reason this is necessary lies in the way grids are partitioned into fibers, which will be discussed in Step 3.

Base Case $n = 1$

Let $\mathcal{M} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \emptyset, \emptyset)$ be a CFM over an ordered feature set $(F, \leq_F)$ with root feature $f_r \in F$, tree height $h+1$, and $C_{\mathcal{M}}(f_r) = \{c_1\}$. We partition the rank set $\{0, \dots, T_{f_r} - 1\}$ in four steps. Each step is formulated to be as independent as possible of the number of root children n so that it can be reused in the inductive step.

Step 1: Partition into blocks. The first criterion for partitioning the rank set is the realised root-group cardinalities. We know that T_{f_r} satisfies

$$T_{f_r} = \sum_{(c,k) \in \lambda_T^G(f_r) \times \lambda_I^G(f_r)} C_{f_r}(c,k).$$

Hence, we can partition $\{0, \dots, T_{f_r} - 1\}$ into $|\lambda_T^G(f_r) \times \lambda_I^G(f_r)|$ contiguous *blocks*, where each block corresponds to a unique tuple $(c, k) \in \lambda_T^G(f_r) \times \lambda_I^G(f_r)$ and has size $C_{f_r}(c, k)$. More concretely, we assume a fixed but arbitrary traversal $(c^1, k^1), \dots, (c^m, k^m)$ of $\lambda_T^G(f_r) \times \lambda_I^G(f_r)$, where $m \coloneqq |\lambda_T^G(f_r) \times \lambda_I^G(f_r)|$. With this traversal, the first block (c^1, k^1) contains the ranks

$$\left\{0, \dots, C_{f_r}(c^1, k^1) - 1\right\},$$

the second block (c^2, k^2) contains the ranks

$$\left\{C_{f_r}(c^1, k^1), \dots, C_{f_r}(c^1, k^1) + C_{f_r}(c^2, k^2) - 1\right\},$$

and, more generally, the block (c^s, k^s) contains the ranks

$$\left\{\sum_{j=1}^{s-1} C_{f_r}(c^j, k^j), \dots, \sum_{j=1}^{s} C_{f_r}(c^j, k^j) - 1\right\}.$$

For a configuration $K \in [\![\mathcal{M}]\!]_{\text{Canon}}$, we require its rank to lie in the block indexed by its realised root-group cardinalities, that is, in the block $(\gamma_T(K), \gamma_I(K))$. This is possible since K is a valid configuration and therefore

$$(\gamma_T(K), \gamma_I(K)) \in \lambda_T^G(f_r) \times \lambda_I^G(f_r).$$

Furthermore, by construction, the block (c, k) contains exactly $C_{f_r}(c, k)$ ranks, and $C_{f_r}(c, k)$ is precisely the number of canonical configurations of $\mathcal{M}$ with realised root-group cardinalities (c, k). Consequently, each configuration can be assigned a unique rank within its block.

Step 2: Partition blocks into grids. After partitioning $\{0, \dots, T_{f_r} - 1\}$ into blocks according to the realised root-group cardinalities, we next need a method to assign ranks to configurations within a fixed block. Since all configurations whose ranks lie inside the same block share the same realised root-group cardinalities, a natural next partition criterion is the multiplicity of the child feature $c_n = c_1$ $(n = 1)$.

Concretely, for a fixed block (c^s, k^s), we partition the set of block-local ranks

$$\{0, \dots, C_{f_r}(c^s, k^s) - 1\}$$

into

$$t \coloneqq |\lambda_I^F(c_n)|$$

contiguous *grids*, where each grid is identified by a multiplicity $r \in \lambda_I^F(c_n)$ and contains

$$W\left(r, c^s, k^s\right) \coloneqq C_{f_r}^{n-1}\left(c^s - \mathbb{1}_{\{r>0\}},\, k^s - r\right) \cdot \left(\!\!\binom{T_{c_n}}{r}\!\!\right)$$

ranks. This partition is possible because, by the DP recursion formula (Theorem 4.5), we know

$$\begin{aligned} C_{f_r}\left(c^s, k^s\right) = C_{f_r}^{n}\left(c^s, k^s\right) &= \sum_{r \in \lambda_I^F(c_n)} C_{f_r}^{n-1}\left(c^s - \mathbb{1}_{\{r>0\}},\, k^s - r\right) \cdot \left(\!\!\binom{T_{c_n}}{r}\!\!\right) \\ &= \sum_{r \in \lambda_I^F(c_n)} W\left(r, c^s, k^s\right). \end{aligned}$$

We now again assume a fixed but arbitrary ordering of the multiplicities in $\lambda_I^F(c_n)$, say $r_n^1, \dots, r_n^t$. This ordering determines the layout of the grids within the block. Concretely, the first grid r_n^1 contains the block-local ranks

$$\left\{0, \dots, W(r_n^1, c^s, k^s) - 1\right\},$$

the second grid r_n^2 contains the block-local ranks

$$\left\{W(r_n^1, c^s, k^s), \dots, W(r_n^1, c^s, k^s) + W(r_n^2, c^s, k^s) - 1\right\},$$

and, more generally, the grid r_n^j contains the block-local ranks

$$\left\{\sum_{\ell=1}^{j-1} W(r_n^\ell, c^s, k^s), \dots, \sum_{\ell=1}^{j} W(r_n^\ell, c^s, k^s) - 1\right\}.$$

Similarly to the block decomposition, we require that the rank of a configuration $K = (M, \prec) \in [\![\mathcal{M}]\!]_{\text{Canon}}$ lies in the block $(\gamma_T(K), \gamma_I(K))$ and, within this block, in the grid indexed by $M(c_n)$. Again this is possible, since there are exactly

$$C_{f_r}^{n-1}\left(c^s - \mathbb{1}_{\{r_n^j > 0\}},\, k^s - r_n^j\right) \cdot \left(\!\!\binom{T_{c_n}}{r_n^j}\!\!\right)$$

configurations with realised root-group cardinalities (c^s, k^s) and r_n^j instances of the child c_n, which matches exactly the number of ranks in grid r_n^j of block (c^s, k^s).

Step 3: Partition grids into fibers. After partitioning the set of all ranks into blocks and each block into grids, we need a way to assign ranks to configurations within a fixed grid. Fix a block (c^s, k^s) and a grid r_n^j. Let P denote the set of canonical configurations counted by

$$C_{f_r}^{n-1}\left(c^s - \mathbb{1}_{\{r_n^j > 0\}},\, k^s - r_n^j\right).$$

That is, P consists exactly of those canonical configurations of $\mathcal{M}_{f_r}^{\text{filled}}$ whose realised root-group cardinalities are

$$\left(c^s - \mathbb{1}_{\{r_n^j > 0\}},\, k^s - r_n^j\right)$$

and which contain no instances of the child feature c_n.

Since the grid r_n^j has size

$$C_{f_r}^{n-1}\left(c^s - \mathbb{1}_{\{r_n^j > 0\}},\, k^s - r_n^j\right) \cdot \left(\!\!\binom{T_{c_n}}{r_n^j}\!\!\right),$$

the set of grid-local ranks is given by

$$\left\{0, \dots, C_{f_r}^{n-1}\left(c^s - \mathbb{1}_{\{r_n^j > 0\}},\, k^s - r_n^j\right) \cdot \left(\!\!\binom{T_{c_n}}{r_n^j}\!\!\right) - 1\right\}$$

$$= \left\{0, \dots, |P| \cdot \left(\!\!\binom{T_{c_n}}{r_n^j}\!\!\right) - 1\right\}.$$

We partition this set into $|P|$ contiguous *fibers*, where each fiber corresponds to exactly one configuration in P. In contrast to the block and grid decompositions, where the sizes of blocks and grids may vary, all fibers in this step have the same size, namely

$$\left(\!\!\binom{T_{c_n}}{r_n^j}\!\!\right).$$

As before, we must specify the layout of the fibers inside the grid, that is, determine which fiber contains the grid-local ranks

$$\left\{0, \ldots, \left(\!\!\binom{T_{c_n}}{r_n^j}\!\!\right) - 1\right\},$$

which fiber contains

$$\left\{\left(\!\!\binom{T_{c_n}}{r_n^j}\!\!\right), \ldots, 2\left(\!\!\binom{T_{c_n}}{r_n^j}\!\!\right) - 1\right\},$$

and so on. However, unlike in the block and grid decompositions, where we can assume that there exists a fixed traversal order for $\lambda_T^G(f_r) \times \lambda_I^G(f_r)$ and $\lambda_I^F(c_n)$, respectively, we cannot assume a traversal order for the configurations in P. This is because defining such an order is precisely the goal of the ranking construction we are currently defining. Indeed, if a traversal order of these configurations were already available, it would immediately induce a ranking by taking the position of each configuration in that order, which would make our ranking construction superfluous.

To solve this problem, we now use the fact that $n = 1$. By Lemma 4.4 we have

$$\begin{aligned}
|P| &= C_{f_r}^{n-1}\left(c^s - \mathbb{1}_{\{r_n^j > 0\}},\, k^s - r_n^j\right) \\
&= C_{f_r}^{0}\left(c^s - \mathbb{1}_{\{r_n^j > 0\}},\, k^s - r_n^j\right) \\
&= \begin{cases} 1, & \text{if } \left(c^s - \mathbb{1}_{\{r_n^j > 0\}},\, k^s - r_n^j\right) = (0,0), \\ 0, & \text{otherwise.} \end{cases}
\end{aligned}$$

Hence, every grid is either empty or consists of exactly one fiber. In the latter case, the fiber ordering is trivial.

For the inductive case $n > 1$, we use a structural property of canonical configurations that follows directly from the DP recursion formula and its proof (see Theorem 4.5). Every canonical configuration K whose rank lies in grid r_n^j of block (c^s, k^s) admits a unique decomposition into two components, called the *fiber decomposition* of K:

- a configuration $K' \in P$, and
- a multiset V of size r_n^j whose elements are canonical configurations of the sub-model $\mathcal{M}_{c_n}$.

Conversely, every such pair (K', V) uniquely determines a canonical configuration whose rank lies in grid r_n^j of block (c^s, k^s). This correspondence follows directly from the proof of the DP recursion formula (Theorem 4.5), where we constructed a bijection between the set of all configurations counted by $C_f^i(c, k)$ with r instances of c_i and the set of pairs of the form (K', V), where

- K' is a canonical configuration counted by $C_f^{i-1}\big(c - \mathbb{1}_{\{r>0\}},\, k - r\big)$, and
- V is a multiset of size r whose elements are canonical configurations of the sub-model $\mathcal{M}_{c_i}$.

Using this decomposition, we now assign fibers as follows. Let $K = (M, \prec) \in [\![\mathcal{M}]\!]_{\text{Canon}}$ be a canonical configuration, and let (K', V) be its fiber decomposition. We require that the rank of K lies in the fiber corresponding to K' within the grid indexed by $M(c_n)$ inside the block $(\gamma_T(K), \gamma_I(K))$.

Step 4: Fiber-local ranking. At the level of fibers, we finally define an explicit bijection between configurations and fiber-local ranks. Each fiber of grid r_n^j within block (c^s, k^s) has size

$$\left(\!\!\binom{T_{c_n}}{r_n^j}\!\!\right),$$

and hence the set of fiber-local ranks is

$$\left\{0, \ldots, \left(\!\!\binom{T_{c_n}}{r_n^j}\!\!\right) - 1\right\}.$$

Since configurations within a fiber differ only in the multiset of child configurations of c_n, it suffices to rank these multisets. By the outer induction hypothesis on the tree height, every canonical configuration of the sub-model $\mathcal{M}_{c_n}$ already has a rank in

$$\{0, \ldots, T_{c_n} - 1\}.$$

We therefore apply the standard multiset ranking procedure RankMultiset (Section 2.4.3.2) to assign each r_n^j-multiset of c_n child configurations a unique fiber-local rank.

Final ranking. Combining Steps 1–4, the rank of a configuration $K = (M, \prec) \in [\![\mathcal{M}]\!]_{\text{Canon}}$ with fiber decomposition (K', V) is obtained as follows. First, compute the fiber-local rank of K using RankMultiset. This rank is then successively converted to a grid-local, block-local, and finally global rank by adding the sizes of all preceding fibers within the grid, all preceding grids within the block, and all preceding blocks, respectively.

Inductive Step

Let $n \in \mathbb{N} \setminus \{1\}$ and assume that a ranking has already been defined for every CFM with no cross-tree constraints, tree height at most $h+1$, and whose root has at most $n-1$ children (we formulate the inductive step from $n-1$ to n, rather than from n to $n+1$, to obtain simpler expressions). Let $\mathcal{M} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \emptyset, \emptyset)$ be a CFM over an ordered feature set $(F, \leq_F)$ with root feature $f_r \in F$, tree height $h+1$, and $C_{\mathcal{M}}(f_r) = \{c_1, \ldots, c_n\}$.

As in the base case $n = 1$, we partition the set of ranks

$$\{0, \ldots, T_{f_r} - 1\},$$

into blocks, the blocks into grids, and the grids into fibers. Steps 1 and 2 are unchanged, since their construction does not depend on n. The only place where $n = 1$ was used is Step 3, namely to justify a traversal order of the set P. We now show how to obtain such a traversal for general n.

Concretely, we use the *fiber decomposition* introduced in Step 3. In particular, we showed that every configuration whose rank lies in grid r_n^j of block (c^s, k^s) admits a unique decomposition into

$$(K', V),$$

where

- $K' \in P$ is a canonical configuration counted by

$$C_{f_r}^{n-1}\left(c^s - \mathbb{1}_{\{r_n^j > 0\}},\, k^s - r_n^j\right),$$

  and

- V is a multiset of size r_n^j whose elements are canonical configurations of the sub-model $\mathcal{M}_{c_n}$.

Thus, the fibers of grid r_n^j are naturally indexed by the configurations in P. To define the layout of the fibers within the grid, it therefore suffices to define a traversal order of the configurations in P.

To obtain such an order, we construct from $\mathcal{M}$ a derived CFM whose canonical configuration space is exactly P. Concretely, we remove the child c_n from $\mathcal{M}$ together with all of its descendants, and restrict the root group-type cardinality interval to

$$\left\langle c^s - \mathbb{1}_{\{r_n^j > 0\}}, c^s - \mathbb{1}_{\{r_n^j > 0\}} \right\rangle$$

and the root group-instance cardinality interval to

$$\left\langle k^s - r_n^j, k^s - r_n^j \right\rangle.$$

We call the resulting model $\mathcal{M}'$ the *child-pruned model* of $\mathcal{M}$ (with respect to (c^s, k^s) and r_n^j). By construction, $\mathcal{M}'$ is a CFM with $n-1$ children, tree height at most $h+1$, and its canonical configuration space is exactly P. Hence, by the induction hypothesis, the configurations in P admit a unique ranking. We may therefore assume a traversal

$$K_1, \ldots, K_{|P|}$$

of the configurations in P induced by this ranking.

Using this traversal, we arrange the fibers within the grid as follows: for each $i \in \{1, \ldots, |P|\}$, the fiber corresponding to K_i contains the grid-local ranks

$$\left\{(i-1)\left(\!\!\binom{T_{c_n}}{r_n^j}\!\!\right), \ldots, i\left(\!\!\binom{T_{c_n}}{r_n^j}\!\!\right) - 1\right\}.$$

Consequently, for a canonical configuration $K = (M, \prec)$ with fiber decomposition (K', V), the fiber containing the rank of K is exactly the fiber corresponding to K'

inside grid $M(c_n)$ and block $(\gamma_T(K), \gamma_I(K))$. The final rank is then obtained by computing the fiber-local rank RankMultiset(V) and converting it to a global rank by adding the corresponding fiber, grid, and block offsets.

Note that the traversal $K_1, \ldots, K_{|P|}$ may differ between grids, even within the same block, since the set P and therefore the child-pruned model $\mathcal{M}'$ depend on the chosen block (c^s, k^s) and grid r_n^j.

Overview of the Rank Decomposition

Figure 4.1 provides a visual overview of the resulting hierarchical decomposition of the rank set into blocks, grids, and fibers. In particular, it highlights the following properties:

- each block contains the same number of grids, namely $|\lambda_I^F(c_n)|$;
- grid sizes may vary both within and across blocks;
- corresponding grids (i.e. grids corresponding to the same multiplicity $r_n^j \in \lambda_I^F(c_n)$) have identical fiber sizes, given by

$$\left(\!\!\binom{T_{c_n}}{r_n^j}\!\!\right);$$

- every configuration whose rank lies in grid r_n^j of block (c^s, k^s) decomposes into a configuration counted by

$$C_{f_r}^{n-1}\left(c^s - \mathbb{1}_{\{r_n^j > 0\}},\, k^s - r_n^j\right)$$

  and a multiset of child configurations of c_n.

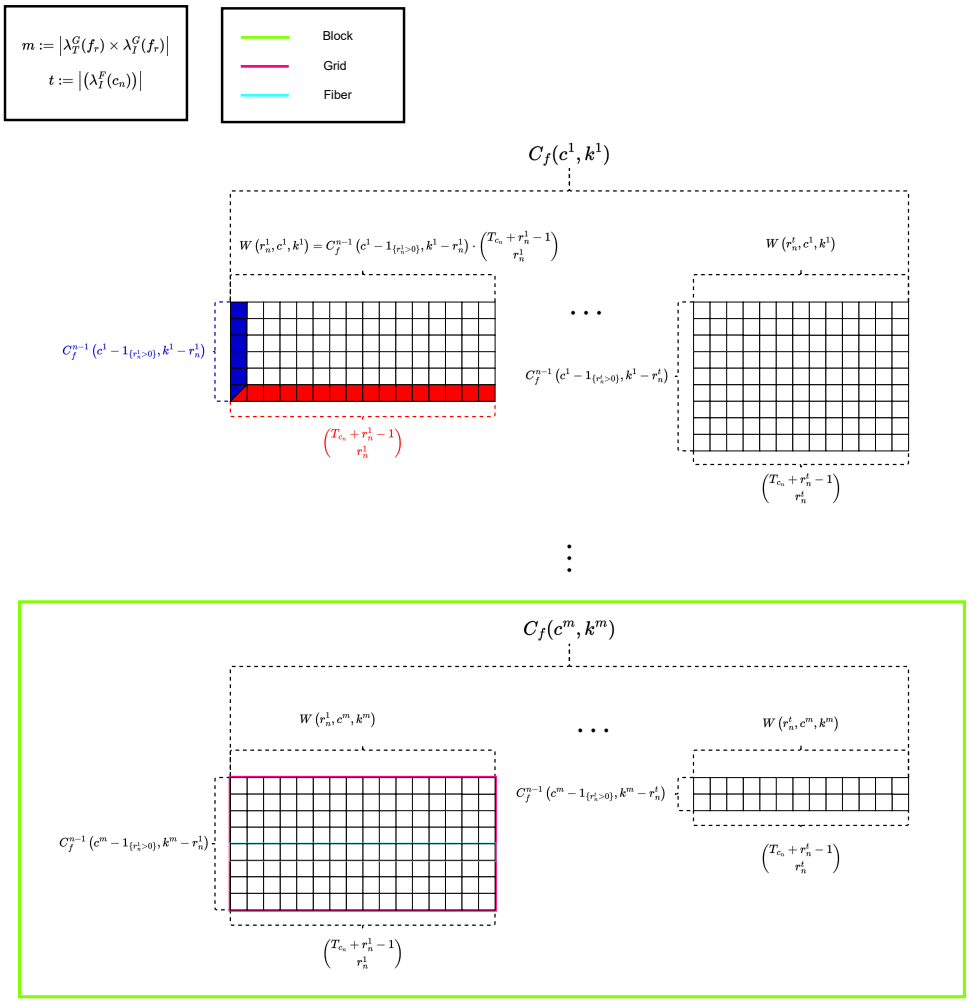


Figure 4.1: Hierarchical decomposition of the rank set into blocks, grids, and fibers

4.1.3.2 Unranking a Configuration

One key requirement of the configuration ranking scheme introduced in the previous section is that it supports an efficient unranking procedure. That is, given a rank $R \in \{0, \ldots, T_f - 1\}$, the procedure must reconstruct the unique canonical configuration corresponding to R without relying on a global lookup table. To this end, we present the algorithm UnrankConfig shown in Algorithm 4.4, which inverts the inductively defined ranking scheme. It does so by exploiting two fundamental properties of the ranking construction.

The first property is that the ranking was defined by repeatedly partitioning the rank interval into contiguous segments (blocks, grids, fibers) according to structural properties of configurations. Each such partition corresponds to a concrete structural property of the configuration: the block determines the realised root-group cardinalities, the grid determines the multiplicity of the last child feature, and the fiber determines the configuration of the child-pruned model, which is extended by adding the corresponding multiset of child configurations. Consequently, once it is known which partition contains a given rank, the corresponding properties of the configuration are uniquely determined. For example, if we know that rank R is contained in the block (c, k), then the realised root-group cardinalities of the configuration with rank R are (c, k).

The second property is that the size and layout of all partitions are known explicitly from the DP tables $C_f^i(c, k)$ and the total configuration counts T_f. This makes it possible to efficiently locate the block, grid, and fiber containing a given rank.

Building on these properties, UnrankConfig proceeds in the following steps.

Step 1: Determine block (c, k). Given the rank R, UnrankConfig first determines the unique block $(c, k) \in \lambda_T^G(f) \times \lambda_I^G(f)$ that contains R.

Since the block (c^s, k^s) contains the ranks

$$\left\{ \sum_{j=1}^{s-1} C_f(c^j, k^j), \ldots, \sum_{j=1}^{s} C_f(c^j, k^j) - 1 \right\},$$

a given rank $R \in \{0, \ldots, T_f - 1\}$ belongs to the block (c^s, k^s) if and only if

$$S \coloneqq \sum_{j=1}^{s-1} C_f(c^j, k^j) \;\leq\; R \;<\; \sum_{j=1}^{s} C_f(c^j, k^j),$$

where $(c^1, k^1), \ldots, (c^m, k^m)$ is the traversal order of $\lambda_T^G(f) \times \lambda_I^G(f)$. The algorithm therefore iterates over $\lambda_T^G(f) \times \lambda_I^G(f)$ in that order and keeps track of the cumulative sum to determine the block containing R. Afterwards, subtracting the cumulative sum S from R yields the block-local rank R'.

Step 2: Determine child multiplicities and child configurations. After identifying the block (c, k) and the corresponding block-local rank R', UnrankConfig proceeds by determining, for each child feature, its multiplicity and the associated child configurations. More specifically, it iterates from n down to 1, and in iteration i determines the multiplicity and child configurations of c_i.

At first glance, this approach differs from the inductive ranking construction, where only the last child c_n was treated explicitly, and the remaining children were handled recursively via a child-pruned model. In the unranking procedure, however, this recursion can be unfolded into a simple loop. The reason is that the root-group cardinality intervals of the child-pruned model allow exactly one root-group cardinality pair (c, k). Consequently, the rank set of the child-pruned model consists of a single block, and Step 1 of the unranking procedure would be trivial for every recursive call. It is therefore more convenient to process all children iteratively.

Formally, at the beginning of iteration i, the algorithm is effectively operating on the i-th child-pruned model, which has the children $c_1, \ldots, c_i$ and admits a single feasible pair of root-group cardinalities, namely $(c_{\mathrm{rem}}, k_{\mathrm{rem}})$, which is updated at the end of every iteration (Step 2e). This iterative approach is possible because the DP tables required for unranking in the child-pruned model are already contained in the DP tables of the original model. In particular, we have

$$C_f^i(a,b) = \widetilde{C}_f^i(a,b) \quad \text{for } (a,b) \in \langle 0, c_{\mathrm{rem}} \rangle \times \langle 0, k_{\mathrm{rem}} \rangle \,,$$

where $\widetilde{C}_f^i(a,b)$ are the DP variables of the i-th child-pruned model and count canonical configurations of the filled child-pruned model.

(2a) Determine grid r_i for child c_i. In iteration i, the algorithm determines the multiplicity r_i of the child feature c_i by identifying the grid that contains the current block-local rank R'. Since in iteration i the algorithm operates on the i-th child-pruned model, which has the children $c_1, \ldots, c_i$ and the only admissible root-group cardinalities $(c_{\mathrm{rem}}, k_{\mathrm{rem}})$, the rank set is

$$\left\{0, \ldots, \widetilde{C}_f^i(c_{\mathrm{rem}}, k_{\mathrm{rem}}) - 1\right\} = \left\{0, \ldots, C_f^i(c_{\mathrm{rem}}, k_{\mathrm{rem}}) - 1\right\}.$$

This is also the set of block-local ranks because the child-pruned model has only the single block $(c_{\mathrm{rem}}, k_{\mathrm{rem}})$.

By construction of the ranking, the set of block-local ranks is partitioned into grids indexed by the possible multiplicities of c_i. Since the grid r_i^j contains the block-local ranks

$$\left\{ \sum_{\ell=1}^{j-1} C_f^{i-1}\left(c_{\mathrm{rem}} - \mathbb{1}_{\{r_i^\ell > 0\}},\, k_{\mathrm{rem}} - r_i^\ell\right) \cdot \left(\!\!\binom{T_{c_i}}{r_i^\ell}\!\!\right), \ldots, \right.$$

$$\left. \sum_{\ell=1}^{j} C_f^{i-1}\left(c_{\mathrm{rem}} - \mathbb{1}_{\{r_i^\ell > 0\}},\, k_{\mathrm{rem}} - r_i^\ell\right) \cdot \left(\!\!\binom{T_{c_i}}{r_i^\ell}\!\!\right) - 1 \right\},$$

a block-local rank

$$R' \in \{0, \ldots, C_f^i(c_{\mathrm{rem}}, k_{\mathrm{rem}}) - 1\}$$

belongs to the grid r_i^j if and only if

$$S' \coloneqq \sum_{\ell=1}^{j-1} C_f^{i-1}\left(c_{\mathrm{rem}} - \mathbb{1}_{\{r_i^\ell > 0\}},\, k_{\mathrm{rem}} - r_i^\ell\right) \cdot \left(\!\!\binom{T_{c_i}}{r_i^\ell}\!\!\right) \;\leq\; R'$$

$$< \sum_{\ell=1}^{j} C_f^{i-1}\left(c_{\mathrm{rem}} - \mathbb{1}_{\{r_i^\ell > 0\}},\, k_{\mathrm{rem}} - r_i^\ell\right) \cdot \left(\!\!\binom{T_{c_i}}{r_i^\ell}\!\!\right),$$

where $r_i^1, \ldots, r_i^{|\lambda_I^F(c_i)|}$ is the traversal order of $\lambda_I^F(c_i)$. Thus, UnrankConfig iterates over the values in $\lambda_I^F(c_i)$ in this order until the cumulative sum exceeds R'. The identified grid uniquely determines the multiplicity r_i of the child feature c_i. Furthermore, subtracting S' from R' yields the grid-local rank R''.

(2b) Split R'' into a fiber index and fiber-local rank. After identifying the grid r_i and the corresponding grid-local rank R'', UnrankConfig determines the fiber and the fiber-local rank.

The set of grid-local ranks is

$$\left\{0, \ldots, C_f^{i-1}\Big(c_{\text{rem}} - \mathbb{1}_{\{r_i>0\}},\, k_{\text{rem}} - r_i\Big) \cdot \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right) - 1\right\},$$

and is partitioned into

$$C_f^{i-1}\Big(c_{\text{rem}} - \mathbb{1}_{\{r_i>0\}},\, k_{\text{rem}} - r_i\Big)$$

fibers, each of equal size

$$B \coloneqq \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right).$$

Consequently, the grid-local rank R'' admits a unique decomposition

$$R'' = B \cdot R' + R''',$$

where

$$R' \coloneqq \left\lfloor \frac{R''}{B} \right\rfloor \qquad \text{and} \qquad R''' \coloneqq R'' \bmod B.$$

Here, R' is the *fiber index* and identifies the position of the fiber within the grid. By construction of the ranking, this index is exactly the rank of a configuration of the next child-pruned model (i.e. the child-pruned model obtained after removing c_i). Since child-pruned models have only one block, this is also the block-local rank of that model. The remainder R''' is the *fiber-local rank* and uniquely encodes the choice of the r_i-multiset of canonical configurations of the sub-model $\mathcal{M}_{c_i}$.

(2c) Unrank multiset. The fiber-local rank R''' corresponds to an r_i-multiset whose elements are ranks of canonical configurations of the sub-model $\mathcal{M}_{c_i}$. Applying the standard multiset unranking procedure UnrankMultiset yields a sequence $(\rho_1, \ldots, \rho_{r_i})$, where each $\rho_j \in \{0, \ldots, T_{c_i} - 1\}$ is the rank of a canonical configuration of $\mathcal{M}_{c_i}$.

(2d) Recursively unrank child configurations. Next, the ranks from the sequence $(\rho_1, \ldots, \rho_{r_i})$ obtained in Step 2c are processed in order. For each $j \in \{1, \ldots, r_i\}$, the algorithm constructs the canonical configuration $K_{i,j}$ corresponding to rank ρ_j by invoking UnrankConfig recursively.

Since the sequence $(\rho_1, \ldots, \rho_{r_i})$ is sorted and may contain duplicate ranks, the algorithm checks whether $j > 1$ and $\rho_j = \rho_{j-1}$. If this is the case, the previously constructed configuration $K_{i,j-1}$ is reused by setting $K_{i,j} \leftarrow K_{i,j-1}$, which avoids expensive and unnecessary unranking calls.

The resulting configurations are collected in a list

$$L_i \leftarrow [K_{i,1}, \ldots, K_{i,r_i}],$$

which is then sorted with respect to the instance order $\trianglelefteq$.

This sorting step ensures that, when the child configurations are later combined using MERGE, the resulting configuration is canonical.

(2e) Update parameters for the child-pruned model. After processing child c_i, the algorithm must continue unranking using the fiber index R', which represents the rank of the corresponding configuration in the next child-pruned model (i.e. the child-pruned model obtained after removing c_i from the current child-pruned model). Given the already determined multiplicity r_i, the only admissible root-group cardinalities for this model are

$$\Big(c_{\mathrm{rem}} - \mathbb{1}_{\{r_i>0\}},\ k_{\mathrm{rem}} - r_i\Big).$$

Consequently, the algorithm updates the remaining cardinalities

$$c_{\mathrm{rem}} \leftarrow c_{\mathrm{rem}} - \mathbb{1}_{\{r_i>0\}}, \qquad k_{\mathrm{rem}} \leftarrow k_{\mathrm{rem}} - r_i,$$

and proceeds to iteration $i-1$ to unrank R', which is the block-local rank of the $(i-1)$-th child-pruned model.

Final merge. After all children $c_1, \ldots, c_n$ have been processed, the algorithm has constructed for each child c_i a sorted list L_i of canonical child configurations. These lists are finally combined using the MERGE procedure,

$$(M, \prec) \leftarrow \textsc{Merge}((L_1, \ldots, L_n), \mathcal{M}_f),$$

which yields the unique canonical configuration of $\mathcal{M}_f$ corresponding to the rank R.

Algorithm 4.4 UnrankConfig$(f, R, \mathcal{M})$: Algorithm for unranking canonical configurations in a CFM without cross-tree constraints

Input:
- CFM $\mathcal{M} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \emptyset, \emptyset)$ over an ordered feature set $(F, \leq_F)$ together with precomputed DP tables $C_f^i(c,k)$ and total configuration counts T_f for all $f \in F$
- feature f with children $\{c_1, \ldots, c_n\}$ (ordered w.r.t. $\leq_F$)
- rank $R \in \{0, \ldots, T_f - 1\}$

Output: The canonical configuration $(M, \prec)$ of $\mathcal{M}_f$ with rank R

if f is a leaf **then**
  return the unique canonical configuration of $\mathcal{M}_f$
end if

Step 1: Determine block (c,k)
Let $S \leftarrow 0$
for all $(c,k) \in \lambda_T^G(f) \times \lambda_I^G(f)$ **do**
  if $S \leq R < S + C_f(c,k)$ **then**
    break
  end if
  $S \leftarrow S + C_f(c,k)$
end for

$R' \leftarrow R - S$ ▷ Block-local rank
$(c_{\text{rem}}, k_{\text{rem}}) \leftarrow (c,k)$

Step 2: Determine child multiplicities and child configurations
for $i = n$ down to 1 **do**
  (2a) Determine grid r_i **for child** c_i
  Let $S' \leftarrow 0$
  for all $r_i \in \lambda_I^F(c_i)$ **do**
    $W \leftarrow C_f^{i-1}\Big(c_{\text{rem}} - \mathbb{1}_{\{r_i>0\}},\ k_{\text{rem}} - r_i\Big) \cdot \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right)$
    if $S' \leq R' < S' + W$ **then**
      break
    end if
    $S' \leftarrow S' + W$
  end for
  $R'' \leftarrow R' - S'$ ▷ Grid-local rank

(2b) Split R'' into a fiber index and fiber-local rank

$B \leftarrow \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right)$

$R''' \leftarrow R'' \bmod B$ ▷ Fiber-local rank

$R' \leftarrow \left\lfloor \frac{R''}{B} \right\rfloor$ ▷ Fiber index

(2c) Unrank multiset

$(\rho_1, \ldots, \rho_{r_i}) \leftarrow \textsc{UnrankMultiset}(T_{c_i}, r_i, R''')$

(2d) Recursively unrank child configurations

for $j = 1$ to r_i **do**

  if $j = 1$ **or** $\rho_j \neq \rho_{j-1}$ **then** ▷ Only unrank configuration once

    $K_{i,j} \leftarrow \textsc{UnrankConfig}(c_i, \rho_j, \mathcal{M})$

  else

    $K_{i,j} \leftarrow K_{i,j-1}$

  end if

end for

$L_i \leftarrow [K_{i,1}, \ldots, K_{i,r_i}]$

Sort L_i increasingly with respect to $\trianglelefteq$

(2e) Update parameters for the child-pruned model

$c_{\text{rem}} \leftarrow c_{\text{rem}} - \mathbb{1}_{\{r_i > 0\}}$

$k_{\text{rem}} \leftarrow k_{\text{rem}} - r_i$

end for

$(M, \prec) \leftarrow \textsc{Merge}((L_i)_{i=1}^{n}, \mathcal{M}_f)$

return $(M, \prec)$

4.2 Uniform Random Sampling with Cross-Tree Constraints

In Section 4.1, we introduced two algorithms for uniform sampling from the canonical configuration space of a CFM *without* cross-tree constraints (CTCs), namely UniSample-BT and UniSample-UR. Both algorithms generate canonical configurations uniformly at random once the DP tables and total configuration counts have been computed.

In the following, we show how either of these algorithms can be extended to handle CFMs *with* cross-tree constraints. The construction is generic and relies solely on the property that the underlying sampling algorithm generates canonical configurations uniformly for CFMs without cross-tree constraints. Consequently, it can be applied to any sampling algorithm that satisfies this uniformity property.

Let $\mathcal{M}$ be a CFM with cross-tree constraints (ϕ_R, ϕ_X). Since cross-tree constraints only restrict the configuration space by forbidding certain configurations, we can use rejection sampling [55]. More precisely, we generate candidate configurations uniformly at random using a sampling algorithm for CFMs without cross-tree constraints and accept only those candidates that satisfy the cross-tree constraints (ϕ_R, ϕ_X). The complete procedure is shown in Algorithm 4.5.

Algorithm 4.5 UniSample-CTC($\mathcal{M}$, BaseSample): Algorithm for uniform random sampling in a CFM with cross-tree constraints using rejection sampling

Input:
- CFM $\mathcal{M} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \phi_R, \phi_X)$ over ordered feature set $(F, \leq_F)$
- Precomputed DP tables $C_f^i(c, k)$ and total configuration counts T_f of the relaxed model $\widetilde{\mathcal{M}} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \emptyset, \emptyset)$
- Sampling algorithm BaseSample for CFMs with no cross-tree constraints

Output: Canonical configuration $(M, \prec)$ of $\mathcal{M}$.

```
1: while true do
2:     (M, ≺) ← BaseSample(M̃)
3:     if (M, ≺) satisfies the CTCs (φ_R, φ_X) then
4:         break
5:     end if
6: end while
7: return (M, ≺)
```

4.3 Summary and Outlook

In this chapter, we presented algorithms for uniformly sampling canonical, and therefore structural, configurations of CFMs. We first considered the case without cross-tree constraints and introduced two sampling algorithms: the backtracking-based algorithm UniSample-BT (Algorithm 4.2) and the unranking-based algorithm UniSample-UR (Algorithm 4.3). Both algorithms rely on DP tables that count canonical configurations in a bottom-up manner.

We then showed how these sampling algorithms can be extended to handle CFMs with cross-tree constraints. The resulting algorithm UniSample-CTC (Algorithm 4.5) uses rejection sampling: candidate configurations are generated using a base sampler for the relaxed model without cross-tree constraints and are accepted only if they satisfy the cross-tree constraints.

Empty Configuration Spaces

The assumption that the configuration space is non-empty is required only for the sampling algorithms themselves. The DP framework used to compute configuration counts remains well-defined even when the CFM admits no valid configurations. In that case, the computed values simply satisfy $T_{f_r} = 0$, indicating that the configuration space is empty.

Furthermore, for non-root features, dead features and empty sub-models (i.e., sub-models with an empty configuration space, also called void models [20, 83]) do not pose a problem for the sampling algorithms. If a non-root feature is dead, or if the sub-model rooted at a non-root feature admits no canonical configurations (i.e., $T_f = 0$), then the DP tables ensure that no instances of this feature are selected during sampling. Consequently, the sampling algorithms never attempt to construct child configurations for such features. This does not, of course, apply to the root feature, since the sub-model rooted at the root is the full model itself, which is assumed to have a non-empty configuration space.

Note that, for CFMs with no cross-tree constraints, if the sub-model rooted at a feature is empty, then that feature is necessarily dead; however, the converse does not necessarily hold.

Next Chapter

In the next chapter, we provide a theoretical analysis of the algorithms introduced in this chapter. In particular, we prove their correctness by establishing termination and uniformity of the generated samples. Furthermore, we analyse their space and time complexity. This analysis also allows us to directly compare the algorithms UniSample-BT and UniSample-UR and to identify the situations in which one approach is preferable to the other.

Future Extensions

Beyond this analysis, several extensions of the presented techniques are possible. For example, the sampling algorithms can be adapted with only minor modifications to operate on *semi-structural configurations*. Since semi-structural configurations explicitly distinguish configurations based on the order of their children, this adaptation can be achieved by replacing all occurrences of "an r-multiset of child configurations" with "a list of r child configurations". Accordingly, the combinatorial terms $\left(\!\!\binom{T_{c_i}}{r}\!\!\right)$ are replaced by $T_{c_i}^r$. In particular, in the backtracking-based algorithm UniSample-BT, the rejection step used for sampling multisets is no longer necessary. Instead, a uniform random list of r child configurations can be generated directly by performing r independent sampling calls.

More generally, the DP tables and the unranking machinery developed in this chapter are useful not only for uniform random sampling, but also for counting and efficiently enumerating canonical configurations. This suggests that the same framework may be applicable to a much broader range of analysis tasks for FMs and CFMs. In the Boolean setting, such tasks are often delegated to SAT and #SAT solvers, and more generally to CSP, SMT, or ILP solvers [10, 20, 21]. By contrast, the DP-based framework developed here exploits the tree structure of CFMs directly and therefore has the potential to be significantly more efficient.

In particular, for CFMs without cross-tree constraints, the computed configuration counts and the ability to enumerate configurations make it possible to derive a variety of analyses directly, including the detection of dead cardinalities, dead features, and void (empty) models [10, 20, 21].

For models with cross-tree constraints, one can in principle combine the same ideas with exhaustive enumeration of the relaxed (unconstrained) configuration space together with filtering by the cross-tree constraints. However, the practical efficiency of this approach depends on the size of the unconstrained space and therefore requires further investigation.

Finally, the DP framework can be extended to compute additional statistics of the configuration space. In particular, it is possible to derive a DP-based algorithm for computing the expected configuration size in a CFM without cross-tree constraints, where the expectation is taken with respect to the uniform distribution over canonical configurations. Although we do not discuss how this can be done in detail, the algorithm is implemented in the CFMTOOLS framework described in Chapter 6.

5. Theoretical Evaluation

This chapter analyses the theoretical properties of the sampling algorithms introduced in the previous chapter.

In particular, in Section 5.1 we establish the correctness of the algorithms UniSample-BT, UniSample-UR, and CTC-UniSample. We then study their computational complexity in Section 5.2, including the time and space required for constructing the DP tables with ComputeDP.

As in Chapter 4, we assume throughout this chapter that all CFMs have bounded feature-instance cardinalities. This ensures that the configuration space considered by the algorithms is finite. Dead features and empty sub-models (for non-root features) are explicitly allowed; only the sampling algorithms themselves require that the configuration space is non-empty (see Section 4.3).

5.1 Correctness

In this section, we establish the correctness of the sampling algorithms introduced in the previous chapter. Correctness includes not only that the generated samples are uniformly distributed over the canonical configuration space, but also that the algorithms terminate almost surely (i.e. with probability 1), provided the configuration space is non-empty.

5.1.1 UniSample-BT

The following theorem and its proof establish the correctness of the algorithm UniSample-BT (Algorithm 4.2). Beyond establishing correctness, the proof also provides insight into how the individual components of the algorithm interact. In particular, the argument essentially follows the execution of the algorithm step by step and derives the probability distributions induced by each of its sampling decisions.

Theorem 5.1 (Correctness of UniSample-BT)**.** Let $\mathcal{M}$ be a CFM over an ordered feature set $(F, \leq_F)$ with root feature f_r, without cross-tree constraints, and with a non-empty configuration space. Then UniSample-BT(f_r) terminates almost surely

(with probability 1) and outputs a canonical configuration. Moreover, each canonical configuration is generated with probability

$$\frac{1}{|[\![\mathcal{M}]\!]_{\text{Canon}}|}.$$

Proof. We first show that UniSample-BT terminates with probability 1 and that every configuration it generates is canonical.

Since the algorithm performs a finite recursion over the feature tree of the CFM, which itself is finite, only finitely many calls to UniSample-BT are made. The only potentially unbounded loop is the repeat-loop in Step 3. For a fixed feature c_i with multiplicity $r_i > 0$, this loop terminates as soon as a list of r_i child configurations is accepted. In each iteration, the acceptance probability

$$p_{\text{acc}} = \frac{m_1! \cdots m_k!}{r_i!} \geq \frac{1}{r_i!} > 0$$

is strictly positive for every possible list, and hence the probability that the loop eventually terminates is 1. Since only finitely many such repeat-loops are executed, the entire algorithm terminates with probability 1.

We next observe that every configuration generated by the algorithm is canonical. Validity follows from the fact that, when sampling cardinalities, each cardinality choice is weighted by the number of canonical configurations that are compatible with this choice. Cardinalities that lead to invalid configurations therefore receive probability 0. Moreover, since the child configurations are merged in sorted order using the Merge algorithm, the resulting configuration is canonical (see Section 3.2.4).

We now prove that UniSample-BT generates each canonical configuration with the same probability, by strong induction on the height of the feature tree $(F, \prec_F)$. Concretely, we have the following induction hypothesis.

Induction Hypothesis

For all $h \in \mathbb{N}_0$ the following holds. Let $\mathcal{M}$ be a CFM over an ordered feature set $(F, \leq_F)$ of tree height at most h, without cross-tree constraints, and with non-empty configuration space. Let f_r denote the root feature of $\mathcal{M}$. Then the algorithm UniSample-BT(f_r) outputs each canonical configuration $K \in [\![\mathcal{M}]\!]_{\text{Canon}}$ with probability

$$\frac{1}{|[\![\mathcal{M}]\!]_{\text{Canon}}|} = \frac{1}{T_{f_r}}.$$

Base Case $h = 0$ (Leaf Model)

If f_r is a leaf (i.e. it has no children), then the only valid instance-based configuration of $\mathcal{M}$ is the configuration that consists solely of the root feature instance f_r^1. This configuration is trivially canonical. Since f_r has no children ($n = 0$), both loops in UniSample-BT are skipped and the algorithm calls Merge with no input lists. By definition of Merge, this returns exactly the configuration containing only the root feature instance f_r^1. Hence, the claim holds.

Inductive Step

Let $h \in \mathbb{N}_0$ and assume that the induction hypothesis holds for height h. Let $\mathcal{M}$ be a CFM of height $h+1$ without cross-tree constraints and with non-empty configuration space, and let f_r denote its root feature with children $c_1, \ldots, c_n$ ($n \geq 1$ since the tree height is $h+1$). Furthermore, let $K^* = (M, \prec) \in [\![\mathcal{M}]\!]_{\text{Canon}}$ be a fixed canonical configuration.

We now compute the probability that UniSample-BT(f_r) produces exactly K^*. To do so, we introduce a random variable for each relevant variable of the algorithm.

Let $\mathbf{C}$ and $\mathbf{K}$ denote the random variables corresponding to the root-group cardinalities (c, k), let $\mathbf{R} = (\mathbf{R}_1, \ldots, \mathbf{R}_n)$ denote the random vector containing the child multiplicities and let $\mathbf{C}^i_{\text{rem}}$ and $\mathbf{K}^i_{\text{rem}}$ denote the running state of $(c_{\text{rem}}, k_{\text{rem}})$ at the beginning of the for-loop iteration in Step 2, with index i.

Note that although these random variables are not constant, there are deterministic relationships between them. Concretely,

$$(\mathbf{C}^n_{\text{rem}}, \mathbf{K}^n_{\text{rem}}) = (\mathbf{C}, \mathbf{K})$$

and

$$(\mathbf{C}^i_{\text{rem}}, \mathbf{K}^i_{\text{rem}}) = (\mathbf{C}^{i+1}_{\text{rem}} - \mathbb{1}_{\{\mathbf{R}_{i+1}>0\}}, \mathbf{K}^{i+1}_{\text{rem}} - \mathbf{R}_{i+1}) \quad \text{for } i \in \{0, \ldots, n-1\}, \qquad (*)$$

since after selecting a multiplicity r_{i+1} in the for-loop with index $i+1$, we adjust the remaining group-cardinalities $(c_{\text{rem}}, k_{\text{rem}})$ accordingly.

Leaf configuration. If K^* contains only the root feature instance f_r^1, then

$$\gamma_T(K^*) = 0, \qquad \gamma_I(K^*) = 0.$$

The first step in computing the probability that the algorithm generates K^* is to compute the probability that $(\mathbf{C}, \mathbf{K}) = (0, 0)$ is chosen in Step 1, since all other root-group cardinalities lead to different configurations. This probability is

$$\mathbb{P}(\mathbf{C} = 0, \mathbf{K} = 0) = \frac{C_{f_r}(0, 0)}{T_{f_r}}.$$

By definition, $C_{f_r}(0, 0)$ counts the number of canonical configurations with $c = 0$ and $k = 0$. Since there is only one such configuration (namely K^*), we have

$$\mathbb{P}(\mathbf{C} = 0, \mathbf{K} = 0) = \frac{1}{T_{f_r}}.$$

Next, conditioned on $(\mathbf{C}, \mathbf{K}) = (0, 0)$, the algorithm enters Step 2 with $(\mathbf{C}^n_{\text{rem}}, \mathbf{K}^n_{\text{rem}}) = (0, 0)$. Under this condition, the distribution of $\mathbf{R}_n$ is given by

$$\mathbb{P}(\mathbf{R}_n = r_n \mid \mathbf{C}^n_{\text{rem}} = 0, \mathbf{K}^n_{\text{rem}} = 0) = \frac{C^{n-1}_{f_r}\left(0 - \mathbb{1}_{\{r_n>0\}},\ 0 - r_n\right) \cdot \left(\!\!\binom{T_{c_n}}{r_n}\!\!\right)}{C^n_{f_r}(0, 0)}.$$

Since $C^{n-1}_{f_r}(0, 0) = 1$ (the configuration consisting only of the root feature instance f_r^1) and for all $r_n > 0$ we have

$$C_{f_r}^{n-1}\Big(0 - \mathbb{1}_{\{r_n>0\}},\ 0 - r_n\Big) = C_{f_r}^{n-1}(-1, -r_n) = 0,$$

whereas for $r_n = 0$ we have

$$C_{f_r}^{n-1}(0,0) = 1,$$

it follows immediately that

$$\mathbb{P}(\mathbf{R}_n = 0 \mid \mathbf{C}_{\text{rem}}^n = 0, \mathbf{K}_{\text{rem}}^n = 0) = 1 \quad \text{and} \quad \mathbb{P}(\mathbf{R}_n > 0 \mid \mathbf{C}_{\text{rem}}^n = 0, \mathbf{K}_{\text{rem}}^n = 0) = 0.$$

Moreover, if $\mathbf{R}_n = 0$, $(*)$ yields

$$(\mathbf{C}_{\text{rem}}^{n-1}, \mathbf{K}_{\text{rem}}^{n-1}) = (0,0).$$

Applying the same argument inductively, we obtain for all $i \in \{1, \dots, n\}$

$$\mathbb{P}(\mathbf{R}_i = 0 \mid \mathbf{C}_{\text{rem}}^i = 0, \mathbf{K}_{\text{rem}}^i = 0) = 1 \quad \text{and} \quad \mathbb{P}(\mathbf{R}_i > 0 \mid \mathbf{C}_{\text{rem}}^i = 0, \mathbf{K}_{\text{rem}}^i = 0) = 0.$$

Hence, in Step 2 all multiplicities satisfy $\mathbf{R}_i = 0$, with probability 1. Consequently, in Step 3 for all $i \in \{1, \dots, n\}$, the list $\mathbf{L}_i$ is initialised to the empty list, and no child configurations are generated for child c_i. Hence, the family of lists passed to MERGE is

$$(\mathbf{L}_1, \dots, \mathbf{L}_n) = (\,[], \dots, []\,).$$

Since MERGE applied to empty lists returns exactly the canonical configuration consisting only of the root instance f_r^1, the algorithm generates K^* with probability

$$\mathbb{P}(K^*) = \frac{1}{T_{f_r}}.$$

Non-leaf configuration. We now consider the case where K^* does not consist only of the root feature instance f_r^1.

For each $i \in \{1, \dots, n\}$, let

$$r_i^{\text{target}} \coloneqq M(c_i)$$

be the number of instances of the feature c_i in K^*. Furthermore, let

$$c_{\text{target}} \coloneqq \gamma_T(K^*), \qquad k_{\text{target}} \coloneqq \gamma_I(K^*)$$

be the realised group-type and group-instance cardinalities of K^* at the root feature f_r.

We again compute the probability that UNISAMPLE-BT(f_r) produces exactly the canonical configuration K^*. Note that in order to generate K^*, the algorithm has to choose exactly the root-group cardinalities $(c_{\text{target}}, k_{\text{target}})$ and the child multiplicities $r_1^{\text{target}}, \dots, r_n^{\text{target}}$.

Step 1: Sampling (c,k). In the first step, the algorithm chooses $(\mathbf{C}, \mathbf{K})$ with probability

$$\mathbb{P}(\mathbf{C} = c, \mathbf{K} = k) = \frac{C_{f_r}(c,k)}{T_{f_r}}.$$

Thus, the probability of choosing the correct root-group cardinalities $(c_{\text{target}}, k_{\text{target}})$ equals

$$\mathbb{P}(\mathbf{C} = c_{\text{target}}, \mathbf{K} = k_{\text{target}}) = \frac{C_{f_r}(c_{\text{target}}, k_{\text{target}})}{T_{f_r}}.$$

Step 2: Sampling the multiplicities $(r_1, \dots, r_n)$**.** Conditioned on $(\mathbf{C}, \mathbf{K}) = (c, k)$, the algorithm proceeds in Step 2 and, for each $i \in \{1, \dots, n\}$, samples a multiplicity $\mathbf{R}_i \in \lambda_I^F(c_i)$ for the child c_i. Given the current remaining cardinalities $(\mathbf{C}^i_{\text{rem}}, \mathbf{K}^i_{\text{rem}}) = (a_i, b_i)$, the conditional distribution of $\mathbf{R}_i$ is

$$\mathbb{P}\Big(\mathbf{R}_i = r_i \,\Big|\, \mathbf{C}^i_{\text{rem}} = a_i,\ \mathbf{K}^i_{\text{rem}} = b_i\Big) = \frac{C_{f_r}^{i-1}\big(a_i - \mathbb{1}_{\{r_i>0\}},\ b_i - r_i\big)\left(\!\!\binom{T_{c_i}}{r_i}\!\!\right)}{C^i_{f_r}(a_i, b_i)}. \qquad (\bigstar)$$

Using this, we can compute the joint probability distribution of the whole vector $\mathbf{R} = (\mathbf{R}_1, \dots, \mathbf{R}_n)$. Concretely, applying the chain rule for conditional probabilities [59, 84] yields

$$\begin{aligned}&\mathbb{P}(\mathbf{R}_1 = r_1, \dots, \mathbf{R}_n = r_n \mid \mathbf{C} = c, \mathbf{K} = k)\\ &\qquad = \prod_{i=1}^{n} \mathbb{P}\left(\mathbf{R}_i = r_i \mid \mathbf{R}_{i+1} = r_{i+1}, \dots, \mathbf{R}_n = r_n, \mathbf{C} = c, \mathbf{K} = k\right).\end{aligned}$$

By $(*)$, the event $\{\mathbf{R}_{i+1} = r_{i+1}, \dots, \mathbf{R}_n = r_n, \mathbf{C} = c, \mathbf{K} = k\}$ completely determines $(\mathbf{C}^i_{\text{rem}}, \mathbf{K}^i_{\text{rem}})$, since these remaining cardinalities can be obtained by successive application of $(*)$. Therefore,

$$\begin{aligned}&\mathbb{P}\left(\mathbf{R}_i = r_i | \mathbf{R}_{i+1} = r_{i+1}, \dots, \mathbf{R}_n = r_n, \mathbf{C} = c, \mathbf{K} = k\right)\\ &\qquad = \mathbb{P}\left(\mathbf{R}_i = r_i \middle| \mathbf{C}^i_{\text{rem}} = a_i, \mathbf{K}^i_{\text{rem}} = b_i\right).\end{aligned}$$

Using $(\bigstar)$, we obtain

$$\begin{aligned}&\mathbb{P}\left(\mathbf{R}_i = r_i | \mathbf{R}_{i+1} = r_{i+1}, \dots, \mathbf{R}_n = r_n, \mathbf{C} = c, \mathbf{K} = k\right)\\ &\qquad = \frac{C_{f_r}^{i-1}\big(a_i - \mathbb{1}_{\{r_i>0\}},\ b_i - r_i\big)\left(\!\!\binom{T_{c_i}}{r_i}\!\!\right)}{C^i_{f_r}(a_i, b_i)},\end{aligned}$$

and therefore

$$\mathbb{P}(\mathbf{R}_1 = r_1, \dots, \mathbf{R}_n = r_n \mid \mathbf{C} = c, \mathbf{K} = k) = \prod_{i=1}^{n} \frac{C_{f_r}^{i-1}\big(a_i - \mathbb{1}_{\{r_i>0\}},\ b_i - r_i\big)\left(\!\!\binom{T_{c_i}}{r_i}\!\!\right)}{C^i_{f_r}(a_i, b_i)}.$$

Since

$$(a_{i-1}, b_{i-1}) = \big(a_i - \mathbb{1}_{\{r_i>0\}},\, b_i - r_i\big) \qquad \text{for all } i \in \{1, \dots, n\},$$

this is a telescoping product which simplifies to

$$\frac{C^0_{f_r}(a_0, b_0)}{C^n_{f_r}(a_n, b_n)} \cdot \prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right),$$

Next, conditioned on $(\mathbf{C}, \mathbf{K}) = (c, k)$, Step 2 is entered with $(\mathbf{C}^n_{\text{rem}}, \mathbf{K}^n_{\text{rem}}) = (c, k)$, we have $(a_n, b_n) = (c, k)$. Moreover, since $C^0_{f_r}(a, b)$ is strictly positive only for

$(a, b) = (0, 0)$ (see Lemma 4.4), in the last iteration of the loop the only value of $\mathbf{R}_1$ with non-zero probability is the one satisfying $(a_0, b_0) = (0, 0)$. Thus, we obtain

$$\begin{aligned}\mathbb{P}(\mathbf{R}_1 = r_1, \dots, \mathbf{R}_n = r_n \mid \mathbf{C} = c, \mathbf{K} = k) &= \frac{C^0_{f_r}(a_0, b_0)}{C^n_{f_r}(a_n, b_n)} \cdot \prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right) \\ &= \frac{C^0_{f_r}(0, 0)}{C^n_{f_r}(c, k)} \cdot \prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right) \\ &= \frac{1}{C_{f_r}(c, k)} \cdot \prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right).\end{aligned}$$

In particular

$$\begin{aligned}&\mathbb{P}\left(\mathbf{R}_1 = r_1^{\text{target}}, \dots, \mathbf{R}_n = r_n^{\text{target}} \mid \mathbf{C} = c_{\text{target}}, \mathbf{K} = k_{\text{target}}\right) \\ &\qquad = \frac{1}{C_{f_r}(c_{\text{target}}, k_{\text{target}})} \cdot \prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i^{\text{target}}}\!\!\right).\end{aligned}$$

Step 3: Sampling child configurations and merging. Next, conditioned on $\mathbf{R} = (r_1, \dots, r_n)$, the algorithm makes r_i independent recursive calls to generate r_i configurations for each child c_i.

For $i \in \{1, \dots, n\}$ and $j \in \{1, \dots, r_i\}$, we define

$$K^{\text{target}}_{i,j} \coloneqq K^*_{c_i^j}.$$

Thus, $K^{\text{target}}_{i,j}$ is the sub-configuration of K^* rooted at the feature instance c_i^j. Since K^* is canonical, all $K^{\text{target}}_{i,j}$ are canonical as well (Theorem 3.20).

Fix an index $i \in \{1, \dots, n\}$, and let $\mathbf{L}_i$ be the random variable corresponding to the list of configurations for the child c_i produced in Step 3, that is, the list returned upon termination of the rejection loop. In order for UniSample-BT to generate the canonical configuration K^*, the list $\mathbf{L}_i$ must correspond exactly to the multiset

$$M^{\text{target}}_i \coloneqq \{K^{\text{target}}_{i,1}, \dots, K^{\text{target}}_{i,r_i}\}.$$

To this end, we first derive the probability distribution of the list $\mathbf{L}_i$ produced by the rejection loop. Having obtained this distribution, the probability that $\mathbf{L}_i$ corresponds to the target multiset M^{target}_i is then computed by summing the probabilities of all lists L_i whose underlying multiset equals M^{target}_i.

Let $L_i = [K_{i,1}, \dots, K_{i,r_i}]$ be a concrete list of canonical configurations of $\mathcal{M}_{c_i}$, and let $m_1, \dots, m_k$ denote the multiplicities of the distinct configurations occurring in L_i.

Single iteration t. We first compute the probability that the concrete list L_i is proposed and accepted in a fixed iteration $t \in \mathbb{N}$ of the rejection loop.

In iteration t, the algorithm generates a candidate list via r_i independent recursive calls to UniSample-BT$(c_i, \mathcal{M})$. We may apply the induction hypothesis to the

sub-model $\mathcal{M}_{c_i}$, since it has tree height at most h, inherits the absence of cross-tree constraints from $\mathcal{M}$, and has non-empty canonical configuration space whenever $r_i > 0$. Indeed, if $[\![\mathcal{M}_{c_i}]\!]_{\text{Canon}} = \emptyset$, then $T_{c_i} = 0$, and therefore

$$\left(\!\!\binom{T_{c_i}}{r_i}\!\!\right) = \left(\!\!\binom{0}{r_i}\!\!\right) = 0 \qquad \text{for all } r_i > 0.$$

Hence, every choice with $r_i > 0$ has probability 0 in Step 2, so the case under consideration can only occur when $[\![\mathcal{M}_{c_i}]\!]_{\text{Canon}} \neq \emptyset$.

Therefore, by the induction hypothesis, each recursive call returns a canonical configuration of $\mathcal{M}_{c_i}$ uniformly. Hence,

$$\mathbb{P}(\text{propose } L_i \text{ in iteration } t) = \frac{1}{T_{c_i}^{r_i}}.$$

Since the multiplicities of the distinct configurations occurring in L_i are $m_1, \ldots, m_k$, it is accepted with probability

$$\frac{m_1! \cdots m_k!}{r_i!}.$$

In total, the probability that L_i is both proposed and accepted in iteration t is

$$\mathbb{P}(\text{propose and accept } L_i \text{ in iteration } t) = \frac{1}{T_{c_i}^{r_i}} \cdot \frac{m_1! \cdots m_k!}{r_i!}.$$

Termination in iteration t with list L_i. We now compute the probability that the algorithm exits the rejection loop in iteration $t \in \mathbb{N}$ and returns the concrete list L_i. For this to happen, the algorithm must reject all proposed lists in iterations $1, \ldots, t-1$ and then propose and accept L_i in iteration t.

Let

$$q_t \coloneqq \mathbb{P}(\text{accept proposed list in iteration } t).$$

Since each iteration of the rejection loop follows the same distribution, we have

$$q_t = q \qquad \text{for all } t \in \mathbb{N}, \qquad q \coloneqq q_1.$$

By independence of the iterations, it follows that

$$\begin{aligned}
&\mathbb{P}\Big(\text{the loop terminates in iteration } t \text{ and returns } L_i\Big) \\
&\quad = \mathbb{P}\Big(\text{reject in iterations } 1, \ldots, t-1, \text{ and propose and accept } L_i \text{ in iteration } t\Big) \\
&\quad = \prod_{j=1}^{t-1} \mathbb{P}(\text{reject in iteration } j) \cdot \mathbb{P}\Big(\text{propose and accept } L_i \text{ in iteration } t\Big) \\
&\quad = (1-q)^{t-1} \cdot \frac{1}{T_{c_i}^{r_i}} \cdot \frac{m_1! \cdots m_k!}{r_i!}.
\end{aligned}$$

Any iteration. Summing over all $t \in \mathbb{N}$ yields the total probability that the repeat-loop eventually terminates with the concrete list L_i:

$$\begin{aligned}\mathbb{P}(\mathbf{L}_i = L_i) &= \sum_{t=1}^{\infty} \mathbb{P}(\text{the loop terminates in iteration } t \text{ and returns } L_i) \\ &= \frac{1}{T_{c_i}^{r_i}} \cdot \frac{m_1! \cdots m_k!}{r_i!} \sum_{t=1}^{\infty} (1-q)^{t-1} \\ &= \frac{1}{T_{c_i}^{r_i}} \cdot \frac{m_1! \cdots m_k!}{r_i!} \cdot \frac{1}{q}.\end{aligned}$$

Computing q. It remains to compute q, that is, the probability that a single iteration of the rejection loop accepts the proposed list. To achieve this, we group all possible lists by the underlying multisets of canonical configurations to which they correspond.

Let M be a r_i-multiset over $[\![\mathcal{M}_{c_i}]\!]_{\text{Canon}}$, and let $m_1, \ldots, m_k$ denote the multiplicities of its distinct elements. We write $\mathcal{L}_M$ for the set of all lists whose underlying multiset is exactly M. Then

$$|\mathcal{L}_M| = \frac{r_i!}{m_1! \cdots m_k!},$$

since this is exactly the number of distinct permutations of the multiset M [57].

From the single-iteration analysis above, we know that for any fixed list $L \in \mathcal{L}_M$, the probability that L is proposed and accepted in a given iteration equals

$$\frac{1}{T_{c_i}^{r_i}} \cdot \frac{m_1! \cdots m_k!}{r_i!}.$$

Summing over all lists whose underlying multiset equals M yields

$$\begin{aligned}\mathbb{P}(\text{propose and accept a list with multiset } M) &= \sum_{L \in \mathcal{L}_M} \frac{1}{T_{c_i}^{r_i}} \cdot \frac{m_1! \cdots m_k!}{r_i!} \\ &= \frac{r_i!}{m_1! \cdots m_k!} \cdot \frac{1}{T_{c_i}^{r_i}} \cdot \frac{m_1! \cdots m_k!}{r_i!} \\ &= \frac{1}{T_{c_i}^{r_i}}.\end{aligned}$$

Since there are exactly

$$\left(\!\!\binom{T_{c_i}}{r_i}\!\!\right)$$

distinct multisets of size r_i over $[\![\mathcal{M}_{c_i}]\!]_{\text{Canon}}$, summing over all such multisets yields

$$q = \mathbb{P}(\text{accept a proposed list in one iteration}) = \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right) \cdot \frac{1}{T_{c_i}^{r_i}}.$$

Substituting this expression into the formula for $\mathbb{P}(\mathbf{L}_i = L_i)$ yields

$$\mathbb{P}(\mathbf{L}_i = L_i) = \frac{1}{T_{c_i}^{r_i}} \cdot \frac{m_1! \cdots m_k!}{r_i!} \cdot \frac{1}{q} = \frac{m_1! \cdots m_k!}{r_i!} \cdot \frac{1}{\left(\!\!\binom{T_{c_i}}{r_i}\!\!\right)}.$$

Generating the multiset. With the probability distribution of $\mathbf{L}_i$, we can now compute the probability that $\mathbf{L}_i$ corresponds to the multiset M_i^{target}. Namely, for a r_i-multiset M, with multiplicities $m_1, \dots, m_k$, summing over all lists $L \in \mathcal{L}_M$ (i.e. lists that realise M) yields

$$\mathbb{P}(\mathbf{L}_i \text{ corresponds to } M) = \sum_{L \in \mathcal{L}_M} \mathbb{P}(\mathbf{L}_i = L) = \frac{m_1! \cdots m_k!}{r_i!} \cdot \frac{1}{\left(\!\binom{T_{c_i}}{r_i}\!\right)} \cdot \frac{r_i!}{m_1! \cdots m_k!}$$

$$= \frac{1}{\left(\!\binom{T_{c_i}}{r_i}\!\right)},$$

since

$$|\mathcal{L}_M| = \frac{r_i!}{m_1! \cdots m_k!}.$$

In particular, the probability that $\mathbf{L}_i$ corresponds to a multiset M depends only on its size r_i and not on the specific choice of M. Consequently, $\mathbf{L}_i$ is uniformly distributed over all multisets of size r_i over $[\![\mathcal{M}_{c_i}]\!]_{\text{Canon}}$, and therefore

$$\mathbb{P}(\mathbf{L}_i \text{ corresponds to } M_i^{\text{target}}) = \frac{1}{\left(\!\binom{T_{c_i}}{r_i}\!\right)}.$$

Since the recursive sampling calls for different children are independent and depend only on the respective multiplicities r_i, the probability that, in Step 3, all lists $\mathbf{L}_i$ correspond exactly to the target multisets

$$\{K_{i,1}^{\text{target}}, \dots, K_{i,r_i}^{\text{target}}\} \quad \text{for all } i \in \{1, \dots, n\}$$

is given by

$$\prod_{i=1}^{n} \frac{1}{\left(\!\binom{T_{c_i}}{r_i}\!\right)}.$$

Combining Steps 1-3. Combining the three steps, the probability that UniSample-BT outputs exactly the fixed canonical configuration K^* is

$$\mathbb{P}(K^* \text{ is sampled}) = \underbrace{\frac{C_{f_r}(c_{\text{target}}, k_{\text{target}})}{T_{f_r}}}_{\text{choose}(c_{\text{target}}, k_{\text{target}})} \cdot \underbrace{\frac{1}{C_{f_r}(c_{\text{target}}, k_{\text{target}})} \prod_{i=1}^{n} \left(\!\binom{T_{c_i}}{r_i^{\text{target}}}\!\right)}_{\text{choose multiplicities } r^{\text{target}}}$$

$$\cdot \underbrace{\prod_{i=1}^{n} \frac{1}{\left(\!\binom{T_{c_i}}{r_i^{\text{target}}}\!\right)}}_{\text{choose child configurations for each } c_i}$$

$$= \frac{1}{T_{f_r}}.$$

This value does not depend on the particular canonical configuration K^*. Hence, every canonical configuration of $\mathcal{M}$ is produced with probability $1/T_{f_r}$. □

5.1.2 UniSample-UR

The following theorem and its proof establish the correctness of the algorithm UniSample-UR (Algorithm 4.3). In contrast to the correctness proof of UniSample-BT, the arguments here are considerably shorter, since randomness occurs only in the initial choice of a rank, while the ranking and unranking procedures are deterministic.

Theorem 5.2 (Correctness of UniSample-UR)**.** Let $\mathcal{M}$ be a CFM over an ordered feature set $(F, \leq_F)$ with root feature f_r, without cross-tree constraints, and with a non-empty configuration space. Then UniSample-UR(f_r) always terminates (in particular also almost surely) and returns a canonical configuration. Moreover, each canonical configuration is generated with probability

$$\frac{1}{|[\![\mathcal{M}]\!]_{\text{Canon}}|}.$$

Proof. UniSample-UR samples a rank uniformly at random from $\{0, \dots, T_{f_r} - 1\}$ and returns the configuration obtained by unranking this rank. Since ranking and unranking define a bijection between ranks and canonical configurations, each canonical configuration is produced with equal probability. Moreover, termination is guaranteed, since drawing a rank requires finite time and once a rank is chosen, the time for unranking is deterministic and finite. □

5.1.3 UniSample-CTC

The following theorem and its proof establish the correctness of the algorithm CTC-UniSample (Algorithm 4.5).

Theorem 5.3 (Correctness of CTC-UniSample)**.** Let $\mathcal{M}$ be a CFM over an ordered feature set $(F, \leq_F)$ with root feature f_r and a non-empty configuration space. Moreover, let $\widetilde{\mathcal{M}}$ be the CFM obtained from $\mathcal{M}$ by removing all cross-tree constraints and let BaseSample be a sampling algorithm that samples uniformly from $[\![\widetilde{\mathcal{M}}]\!]_{\text{Canon}}$. Then CTC-UniSample($\mathcal{M}$) terminates almost surely (with probability 1) and samples uniformly from $[\![\mathcal{M}]\!]_{\text{Canon}}$.

Proof. We first observe that removing cross-tree constraints can only enlarge the configuration space. Therefore,

$$[\![\mathcal{M}]\!]_{\text{Canon}} \subseteq [\![\widetilde{\mathcal{M}}]\!]_{\text{Canon}}.$$

Furthermore, the probability that a uniformly drawn canonical configuration of $\widetilde{\mathcal{M}}$ satisfies all cross-tree constraints of $\mathcal{M}$ is

$$p \coloneqq \frac{|[\![\mathcal{M}]\!]_{\text{Canon}}|}{|[\![\widetilde{\mathcal{M}}]\!]_{\text{Canon}}|} > 0,$$

which is well-defined, since $[\![\mathcal{M}]\!]_{\text{Canon}}$ is non-empty and therefore also $[\![\widetilde{\mathcal{M}}]\!]_{\text{Canon}}$.

Let $(\mathbf{X}_i)_{i \in \mathbb{N}}$ denote the sequence of independent samples drawn by BaseSample inside CTC-UniSample. By assumption, each $\mathbf{X}_i$ is uniformly distributed over $[\![\widetilde{\mathcal{M}}]\!]_{\text{Canon}}$, that is,

$$\mathbb{P}(\mathbf{X}_i = K) = \frac{1}{|[\![\widetilde{\mathcal{M}}]\!]_{\text{Canon}}|} \qquad \text{for all } K \in [\![\widetilde{\mathcal{M}}]\!]_{\text{Canon}}.$$

In particular,

$$\mathbb{P}(\mathbf{X}_i \in [\![\mathcal{M}]\!]_{\text{Canon}}) = p, \qquad \mathbb{P}(\mathbf{X}_i \notin [\![\mathcal{M}]\!]_{\text{Canon}}) = 1 - p.$$

Using these observations, we now prove termination and uniformity of CTC-UniSample.

Termination. The rejection loop of CTC-UniSample terminates as soon as an index $i \in \mathbb{N}$ occurs with $\mathbf{X}_i \in [\![\mathcal{M}]\!]_{\text{Canon}}$. Since the events $\{\mathbf{X}_i \in [\![\mathcal{M}]\!]_{\text{Canon}}\}_{i\in\mathbb{N}}$ are independent and each has probability $p > 0$, the probability that the algorithm terminates exactly at iteration i is

$$\begin{aligned}\mathbb{P}\Big(\mathbf{X}_1, \dots, \mathbf{X}_{i-1} \notin [\![\mathcal{M}]\!]_{\text{Canon}},\, \mathbf{X}_i \in [\![\mathcal{M}]\!]_{\text{Canon}}\Big)& \\ = \prod_{j=1}^{i-1} \mathbb{P}\Big(\mathbf{X}_j \notin [\![\mathcal{M}]\!]_{\text{Canon}}\Big) \cdot \mathbb{P}\Big(\mathbf{X}_i \in [\![\mathcal{M}]\!]_{\text{Canon}}\Big) &= (1-p)^{i-1}\, p.\end{aligned}$$

Summing over all $i \in \mathbb{N}$ yields

$$\begin{aligned}\mathbb{P}(\text{termination}) = \sum_{i=1}^{\infty} (1-p)^{i-1} p &= p \sum_{i=1}^{\infty} (1-p)^{i-1} \\ &= p \sum_{i=0}^{\infty} (1-p)^{i} = p \cdot \frac{1}{p} = 1.\end{aligned}$$

Hence, CTC-UniSample terminates with probability 1.

Uniformity of the output. Let $K^* \in [\![\mathcal{M}]\!]_{\text{Canon}}$ be an arbitrary canonical configuration of $\mathcal{M}$. The algorithm outputs K^* if and only if there exists an index $i \in \mathbb{N}$ such that

$$\mathbf{X}_1, \dots, \mathbf{X}_{i-1} \notin [\![\mathcal{M}]\!]_{\text{Canon}} \quad \text{and} \quad \mathbf{X}_i = K^*.$$

Again, by independence of the samples, the probability of this event for a fixed i is

$$\begin{aligned}\mathbb{P}\Big(\mathbf{X}_1, \dots, \mathbf{X}_{i-1} \notin [\![\mathcal{M}]\!]_{\text{Canon}},\, \mathbf{X}_i = K^*\Big)& \\ = \prod_{j=1}^{i-1} \mathbb{P}\Big(\mathbf{X}_j \notin [\![\mathcal{M}]\!]_{\text{Canon}}\Big) \cdot \mathbb{P}\Big(\mathbf{X}_i = K^*\Big)& \\ = (1-p)^{i-1} \cdot \frac{1}{|[\![\widetilde{\mathcal{M}}]\!]_{\text{Canon}}|}.&\end{aligned}$$

Summing over all possible indices $i \in \mathbb{N}$, we obtain

$$\begin{aligned}\mathbb{P}(\text{CTC-UniSample outputs } K^*) &= \sum_{i=1}^{\infty} (1-p)^{i-1} \cdot \frac{1}{|[\![\widetilde{\mathcal{M}}]\!]_{\text{Canon}}|} \\ &= \frac{1}{|[\![\widetilde{\mathcal{M}}]\!]_{\text{Canon}}|} \sum_{i=0}^{\infty} (1-p)^{i} \\ &= \frac{1}{|[\![\widetilde{\mathcal{M}}]\!]_{\text{Canon}}|} \cdot \frac{1}{p} \\ &= \frac{1}{|[\![\widetilde{\mathcal{M}}]\!]_{\text{Canon}}|} \cdot \frac{|[\![\widetilde{\mathcal{M}}]\!]_{\text{Canon}}|}{|[\![\mathcal{M}]\!]_{\text{Canon}}|} = \frac{1}{|[\![\mathcal{M}]\!]_{\text{Canon}}|}.\end{aligned}$$

Since K^* was arbitrary, every canonical configuration of $\mathcal{M}$ is returned with the same probability. Therefore, CTC-UniSample samples uniformly from $[\![\mathcal{M}]\!]_{\text{Canon}}$. □

5.2 Time and Space Complexity

With the correctness of the algorithms established, we now analyse the time complexity of UniSample-BT, UniSample-UR, and CTC-UniSample. We also consider the additional cost of constructing and storing the DP tables using ComputeDP, since all sampling algorithms require these tables as input.

After deriving runtime bounds for each sampling algorithm, we conclude the section with a comparison of their runtime behaviour and discuss under which circumstances one algorithm may be preferable to another. We also discuss how the naive implementations can be improved with respect to runtime efficiency and practical implementation aspects.

Throughout this section, we use the random access machine (RAM) model as our model of computation. In this model, integers whose values are bounded by a polynomial in the input size can be stored in a single word. Memory accesses as well as basic arithmetic operations, including addition, subtraction, multiplication, and division, operate on such words and take constant time [80, 82, 85].

Unless stated otherwise, we also assume that all integer values occurring during the execution of the algorithms are sufficiently small to fit into a single word. Moreover, we assume that binomial coefficients can be computed in constant time.

We will see that the time complexity of the algorithms depends directly on the cardinality intervals of the CFM, in particular, on the maximum value $\max(I)$ of a cardinality interval I. Consequently, the time and space complexity of our algorithms can be artificially inflated by dead cardinalities.

We therefore assume that all trivial dead cardinalities have been removed using the normalisation method described in Section 2.2.3.

5.2.1 ComputeDP

Let $\mathcal{M} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \emptyset, \emptyset)$ be a CFM over an ordered feature set $(F, \leq_F)$. In the following we analyse the space and time complexity of ComputeDP (Algorithm 4.1).

5.2.1.1 Space Complexity

We first analyse the space requirements of the DP tables and the total configuration counts.

DP table size. For a fixed feature $f \in F$ with child set $C_{\mathcal{M}}(f)$, the DP table of f consists of the values $C_f^i(c,k)$ indexed by $i \in \{0, \ldots, |C_{\mathcal{M}}(f)|\}$ and $(c,k) \in \mathbb{Z}^2$.

By Lemma 4.4, the values for $i = 0$ are fixed, namely

$$C_f^0(c,k) = \begin{cases} 1, & \text{if } (c,k) = (0,0), \\ 0, & \text{otherwise,} \end{cases}$$

and for all $i \in \{0, \ldots, |C_{\mathcal{M}}(f)|\}$ we have

$$C_f^i(c,k) = 0 \qquad \text{whenever} \qquad (c,k) \notin \psi_T^G(f) \times \psi_I^G(f).$$

Consequently, neither the values C_f^0 nor any entries outside $\psi_T^G(f) \times \psi_I^G(f)$ need to be stored or computed. Thus, the DP table of f contains

$$|C_{\mathcal{M}}(f)| \cdot |\psi_T^G(f)| \cdot |\psi_I^G(f)|$$

entries.

Summing over all features in the model, the total space required to store all DP tables is

$$\sum_{f \in F} |C_{\mathcal{M}}(f)| \cdot |\psi_T^G(f)| \cdot |\psi_I^G(f)|.$$

Total configuration counts (space). In addition to the DP tables, ComputeDP stores one integer value T_f for each feature $f \in F$, which requires $O(|F|)$ additional space.

Overall, the space complexity of ComputeDP is

$$O\left(\sum_{f \in F} |C_{\mathcal{M}}(f)| \cdot |\psi_T^G(f)| \cdot |\psi_I^G(f)| \; + \; |F| \right).$$

5.2.1.2 Time Complexity

We now turn to the runtime of ComputeDP.

DP table construction time. For fixed $f \in F$, $i \in \{1, \ldots, |C_{\mathcal{M}}(f)|\}$, and $(c, k) \in \psi_T^G(f) \times \psi_I^G(f)$, ComputeDP computes $C_f^i(c, k)$ via the recursion formula

$$C_f^i(c,k) = \sum_{r \in \lambda_I^F(c_i)} C_f^{i-1}\big(c - \mathbb{1}_{\{r>0\}},\, k - r\big) \cdot \left(\!\!\binom{T_{c_i}}{r}\!\!\right).$$

Under the RAM model, all basic arithmetic operations on word-sized integers take constant time. Moreover, as assumed throughout this section, all integer values fit in a single word and binomial coefficients can be evaluated in constant time. Consequently, computing $C_f^i(c, k)$ requires

$$O\big(|\lambda_I^F(c_i)|\big)$$

time.

For a fixed feature f, ComputeDP iterates over all child indices $i = 1, \ldots, |C_{\mathcal{M}}(f)|$ and, for each such i, over all pairs $(c, k) \in \psi_T^G(f) \times \psi_I^G(f)$. Thus, the total time required to compute the DP table of f is

$$O\left(|\psi_T^G(f)| \cdot |\psi_I^G(f)| \cdot \sum_{i=1}^{|C_{\mathcal{M}}(f)|} |\lambda_I^F(c_i)| \right).$$

Summing over all features yields the overall time complexity

$$O\left(\sum_{f \in F} |\psi_T^G(f)| \cdot |\psi_I^G(f)| \cdot \sum_{i=1}^{|C_{\mathcal{M}}(f)|} |\lambda_I^F(c_i)| \right).$$

Total configuration counts (time). After all DP tables are computed, ComputeDP determines the total configuration count T_f for each feature $f \in F$ via

$$T_f = \sum_{(c,k)\in\lambda_T^G(f)\times\lambda_I^G(f)} C_f^{|C_{\mathcal{M}}(f)|}(c,k).$$

This requires

$$O\left(\sum_{f\in F} |\lambda_T^G(f)| \cdot |\lambda_I^G(f)|\right)$$

time.

The overall time complexity of ComputeDP is therefore

$$O\left(\sum_{f\in F} |\psi_T^G(f)| \cdot |\psi_I^G(f)| \cdot \sum_{i=1}^{|C_{\mathcal{M}}(f)|} |\lambda_I^F(c_i)| \;+\; \sum_{f\in F} |\lambda_T^G(f)| \cdot |\lambda_I^G(f)|\right).$$

5.2.1.3 Simplified Bounds via Global Maxima

To obtain more compact asymptotic bounds, we replace feature-dependent quantities by global maxima over all features. Concretely, let

$$\Lambda_T^G \coloneqq \max_{f\in F} \max\Big(\lambda_T^G(f)\Big), \quad \Lambda_I^G \coloneqq \max_{f\in F} \max\Big(\lambda_I^G(f)\Big), \quad \Lambda_I^F \coloneqq \max_{f\in F} \max\Big(\lambda_I^F(f)\Big).$$

Similarly, we define the maximum sizes of the corresponding cardinality intervals as

$$\Xi_T^G \coloneqq \max_{f\in F}\Big|\lambda_T^G(f)\Big|, \quad \Xi_I^G \coloneqq \max_{f\in F}\Big|\lambda_I^G(f)\Big|, \quad \Xi_I^F \coloneqq \max_{f\in F}\Big|\lambda_I^F(f)\Big|.$$

By Definition 4.2, the filled intervals are defined as

$$\psi_T^G(f) = \left\langle 0, \max(\lambda_T^G(f))\right\rangle, \qquad \psi_I^G(f) = \left\langle 0, \max(\lambda_I^G(f))\right\rangle.$$

Hence, they satisfy

$$|\psi_T^G(f)| \le \Lambda_T^G + 1, \qquad |\psi_I^G(f)| \le \Lambda_I^G + 1 \qquad \text{for all } f \in F.$$

Using these global bounds, we can bound the space complexity as follows:

$$\begin{aligned}
&O\left(\sum_{f\in F} |C_{\mathcal{M}}(f)| \cdot |\psi_T^G(f)| \cdot |\psi_I^G(f)| \;+\; |F|\right) \\
\subseteq\; &O\left((\Lambda_T^G+1)\cdot(\Lambda_I^G+1)\cdot\sum_{f\in F}|C_{\mathcal{M}}(f)| \;+\; |F|\right) \\
=\; &O\Big((\Lambda_T^G+1)\cdot(\Lambda_I^G+1)\cdot|F| + |F|\Big) \\
=\; &O\Big(|F|\cdot(\Lambda_T^G+1)\cdot(\Lambda_I^G+1)\Big),
\end{aligned}$$

where we used that $(F, \prec_F)$ forms a directed tree, and hence $\sum_{f\in F}|C_{\mathcal{M}}(f)| = |F|-1$.

Similarly, for the time complexity,

$$
\begin{aligned}
&O\left(\sum_{f\in F} |\psi_T^G(f)| \cdot |\psi_I^G(f)| \cdot \sum_{i=1}^{|C_{\mathcal{M}}(f)|} |\lambda_I^F(c_i)| \;+\; \sum_{f\in F} |\lambda_T^G(f)| \cdot |\lambda_I^G(f)|\right) \\
&\subseteq O\left(\sum_{f\in F} (\Lambda_T^G+1)\cdot(\Lambda_I^G+1) \cdot \sum_{i=1}^{|C_{\mathcal{M}}(f)|} \Xi_I^F \;+\; |F|\cdot \Xi_T^G \cdot \Xi_I^G\right) \\
&= O\left((\Lambda_T^G+1)\cdot(\Lambda_I^G+1)\cdot \Xi_I^F \cdot \sum_{f\in F} |C_{\mathcal{M}}(f)| \;+\; |F|\cdot \Xi_T^G \cdot \Xi_I^G\right) \\
&= O\Big((\Lambda_T^G+1)\cdot(\Lambda_I^G+1)\cdot \Xi_I^F \cdot |F| \;+\; |F|\cdot \Xi_T^G \cdot \Xi_I^G\Big) \\
&= O\Big(|F|\cdot(\Lambda_T^G+1)\cdot(\Lambda_I^G+1)\cdot \Xi_I^F\Big),
\end{aligned}
$$

where in the last step we used that

$$
\Xi_T^G \le \Lambda_T^G + 1, \qquad \Xi_I^G \le \Lambda_I^G + 1, \qquad \Xi_I^F \ge 1.
$$

Note that these bounds are asymptotically tight in the worst case. Indeed, there exist CFMs in which, for every feature, the maxima of the group-type, group-instance, and feature-instance intervals equal Λ_T^G, Λ_I^G, and Λ_I^F, respectively, while the sizes of all corresponding intervals simultaneously attain Ξ_T^G, Ξ_I^G, and Ξ_I^F.

5.2.1.4 Possible Optimizations

The asymptotic bounds derived above describe the worst-case complexity of ComputeDP when implemented in a straightforward manner. In practice, however, the runtime can be reduced considerably by exploiting structural properties of the DP recursion and of the cardinality constraints. The implementation used in the cfmtools framework (see Chapter 6) incorporates several such optimizations. We briefly summarise the most important ones.

First-Child Initialization

In the theoretical algorithm, the DP recursion is applied uniformly for all child indices $i = 1, \ldots, |C_{\mathcal{M}}(f)|$. However, the first iteration can be simplified considerably. Recall that the base layer of the DP table satisfies

$$
C_f^0(c,k) = \begin{cases} 1, & \text{if } (c,k) = (0,0), \\ 0, & \text{otherwise.} \end{cases}
$$

Consequently, when processing the first child c_1, every term in the recursion

$$
C_f^1(c,k) = \sum_{r\in\lambda_I^F(c_1)} C_f^0\big(c - \mathbb{1}_{\{r>0\}},\, k-r\big) \cdot \left(\!\!\binom{T_{c_1}}{r}\!\!\right)
$$

is zero except when

$$
(c - \mathbb{1}_{\{r>0\}},\, k-r) = (0,0).
$$

Hence, the only non-zero states in layer $i = 1$ are those with

$$(c, k) = \left(\mathbb{1}_{\{r>0\}},\, r\right) \qquad \text{for } r \in \lambda_I^F(c_1),$$

and their values are simply

$$C_f^1\left(\mathbb{1}_{\{r>0\}},\, r\right) = \left(\!\!\binom{T_{c_1}}{r}\!\!\right).$$

Thus, the first DP layer can be constructed directly by iterating over $r \in \lambda_I^F(c_1)$ and inserting these values, thereby avoiding the iteration over all pairs $(c, k) \in \psi_T^G(f) \times \psi_I^G(f)$.

Bounded Iteration Ranges

In the theoretical algorithm (Algorithm 4.1), the loops over (c, k) range over $\psi_T^G(f) \times \psi_I^G(f)$ when filling the DP tables. However, many of these states are unreachable. For example, during iteration i of the outer loop we compute the number of canonical configurations of the filled model in which instances can only be selected from the first i children.

Consequently, the realised group-type cardinality can never exceed i, and the realised group-instance cardinality cannot exceed the sum of the maximum feature-instance cardinalities of the first i children. Hence, the loops can be restricted to

$$c \leq i \qquad \text{and} \qquad k \leq \sum_{j=1}^{i} \max(\lambda_I^F(c_j)),$$

since all other entries $C_f^i(c, k)$ are necessarily zero.

Batch Computation of Multiset Coefficients

The term $\left(\!\!\binom{T_{c_i}}{r}\!\!\right)$ appears repeatedly for the same child feature c_i. Instead of recomputing this value for every pair $(c, k) \in \psi_T^G(f) \times \psi_I^G(f)$, the required values can be precomputed once per child. In particular, we can compute

$$\left\{ \left(\!\!\binom{T_{c_i}}{r}\!\!\right) \;\middle|\; r \in \lambda_I^F(c_i) \right\}$$

in a single batch.

Moreover, when computing the values in a batch, we can traverse the values $r \in \lambda_I^F(c_i)$ in increasing order. Since

$$\left(\!\!\binom{n}{r}\!\!\right) = \binom{n+r-1}{r},$$

and

$$\binom{n+t}{t+1} = \binom{n+t-1}{t} \cdot \frac{n+t}{t+1},$$

we can compute $\left(\!\!\binom{n}{r}\!\!\right)$ from $\left(\!\!\binom{n}{r-1}\!\!\right)$, or more generally from $\left(\!\!\binom{n}{r-k}\!\!\right)$, using $O(k)$ elementary operations.

In practice, this incremental computation yields substantial performance improvements, since binomial coefficients are considerably more expensive to compute than assumed in the theoretical analysis (in particular, their evaluation is not constant-time).

Forward State Updates

The theoretical algorithm computes $C_f^i(c,k)$ using the recursion

$$C_f^i(c,k) = \sum_{r \in \lambda_I^F(c_i)} C_f^{i-1}\Big(c - \mathbb{1}_{\{r>0\}},\, k-r\Big) \cdot \left(\!\!\binom{T_{c_i}}{r}\!\!\right),$$

which corresponds conceptually to *backward updates*: for each target state (c,k), all predecessor states are inspected. In practice, it is often more efficient to perform the updates in the opposite direction. Instead of computing each state from its predecessors, we iterate over all states (c,k) in layer i and propagate their contributions forward to the next layer. Concretely, for every state (c,k) and every multiplicity $r \in \lambda_I^F(c_i)$ we perform the update

$$C_f^{i+1}\Big(c + \mathbb{1}_{\{r>0\}},\, k+r\Big) \;\mathrel{+}=\; C_f^i(c,k) \cdot \left(\!\!\binom{T_{c_i}}{r}\!\!\right).$$

Thus, each state (c,k) distributes its contribution to all successor states in the next DP layer. This *forward update* formulation enables the following optimizations.

Early termination in the multiplicity loop. If the values $r \in \lambda_I^F(c_i)$ are processed in increasing order, the multiplicity loop can terminate early once the bounds

$$k + r > k_{\max} \qquad \text{or} \qquad c + \mathbb{1}_{\{r>0\}} > c_{\max}$$

are violated, where $c_{\max}$ and $k_{\max}$ denote the bounds from the iteration range optimization. Since all subsequent values of r are larger, the loop can terminate immediately.

Skipping zero states. Before propagating a state (c,k), we can check whether $C_f^i(c,k) = 0$. If this is the case, the state contributes nothing to any successor and can therefore be skipped.

5.2.2 UniSample-BT

We now analyse the runtime of the sampling algorithm UniSample-BT (Algorithm 4.2). In contrast to ComputeDP, which is deterministic, UniSample-BT performs several random choices during sampling. Consequently, the runtime is itself a random variable, and we analyse the algorithm in terms of its expected runtime. We will see that the expected runtime is closely related to the expected sample size, i.e., the expected size of a uniformly random configuration. For this reason, we also derive a formula for the expected sample size.

5.2.2.1 Expected Runtime of UniSample-BT

In addition to the RAM model assumptions, we now also assume that we can sample from a discrete probability distribution with n outcomes in $O(1)$. This is possible for example with the alias method, which requires only $O(n)$ additional space and $O(n)$ preprocessing time [86, 87, 88].

Let $f \in F$ be a feature, such that $T_f \geq 1$, and let $\mathbf{RT}(f)$ denote the (random) runtime of UniSample-BT$(f, \mathcal{M})$.

For a canonical configuration $K = (M, \prec) \in [\![\mathcal{M}_f]\!]_{\text{Canon}}$, we define its *size* as the number of feature instances:

$$\text{Size}(K) \coloneqq |M|.$$

We further define the *average canonical configuration size* of $\mathcal{M}_f$ as

$$\overline{\text{Size}}(\mathcal{M}_f) \coloneqq \frac{1}{|[\![\mathcal{M}_f]\!]_{\text{Canon}}|} \sum_{K \in [\![\mathcal{M}_f]\!]_{\text{Canon}}} \text{Size}(K) = \frac{1}{T_f} \sum_{K \in [\![\mathcal{M}_f]\!]_{\text{Canon}}} \text{Size}(K).$$

Equivalently, $\overline{\text{Size}}(\mathcal{M}_f)$ is the expected value of $\text{Size}(\mathbf{U})$, where $\mathbf{U}$ is a random configuration drawn uniformly from $[\![\mathcal{M}_f]\!]_{\text{Canon}}$ because

$$\mathbb{E}[\text{Size}(\mathbf{U})] = \sum_{K \in [\![\mathcal{M}_f]\!]_{\text{Canon}}} \text{Size}(K) \cdot \mathbb{P}(\mathbf{U} = K) = \sum_{K \in [\![\mathcal{M}_f]\!]_{\text{Canon}}} \text{Size}(K) \cdot \frac{1}{T_f}.$$

With these definitions, we can now derive an upper bound for the expected runtime of UniSample-BT$(f, \mathcal{M})$, that is, for $\mathbb{E}[\mathbf{RT}(f)]$.

Leaf case. If f is a leaf, then UniSample-BT$(f, \mathcal{M})$ returns the unique configuration consisting only of the instance f^1. The runtime for this is deterministic and $O(1)$, since the rejection-loop, which is the only non-deterministic part of the algorithm, is skipped. Hence,

$$\mathbb{E}[\mathbf{RT}(f)] \in O(1).$$

Non-leaf case. Now let f be a non-leaf feature with children $c_1, \ldots, c_n$. In contrast to the leaf case, the algorithm now performs several recursive calls to generate child configurations for its child features. The number of these calls, as well as the children on which they are performed, depends on the chosen root-group cardinalities (c, k) and the multiplicity vector $r = (r_1, \ldots, r_n)$. Therefore, let $\mathbf{C}, \mathbf{K}$ denote the random variables corresponding to the root-group cardinalities, and let $\mathbf{R} = (\mathbf{R}_1, \ldots, \mathbf{R}_n)$ denote the random vector of child multiplicities.

With these random variables, we can apply the law of total expectation [89, 90] to obtain

$$\begin{aligned}
\mathbb{E}[\mathbf{RT}(f)] &= \sum_{\substack{(c,k):\\ \mathbb{P}(\mathbf{C}=c,\mathbf{K}=k)>0}} \mathbb{P}(\mathbf{C} = c, \mathbf{K} = k) \cdot \mathbb{E}\left[\mathbf{RT}(f) \mid \mathbf{C} = c, \mathbf{K} = k\right] \\
&= \sum_{\substack{(c,k):\\ \mathbb{P}(\mathbf{C}=c,\mathbf{K}=k)>0}} \mathbb{P}(\mathbf{C} = c, \mathbf{K} = k) \sum_{\substack{r:\\ \mathbb{P}(\mathbf{R}=r|\mathbf{C}=c,\mathbf{K}=k)>0}} \\
&\qquad \cdot \mathbb{P}(\mathbf{R} = r \mid \mathbf{C} = c, \mathbf{K} = k) \cdot \mathbb{E}\left[\mathbf{RT}(f) \mid \mathbf{R} = r, \mathbf{C} = c, \mathbf{K} = k\right].
\end{aligned}$$

Note that we do not know yet if $\mathbb{E}[\mathbf{RT}(f)]$ exists (i.e. is finite). But since $\mathbf{RT}(f) \geq 0$, the use of the law of total expectation is allowed.

From the correctness proof of UniSample-BT (Theorem 5.1), we know

$$\mathbb{P}(\mathbf{C} = c, \mathbf{K} = k) = \frac{C_f(c, k)}{T_f} \qquad \text{for } (c, k) \in \lambda_T^G(f) \times \lambda_I^G(f).$$

In particular,

$$\mathbb{P}(\mathbf{C} = c, \mathbf{K} = k) > 0 \quad \iff \quad C_f(c,k) > 0.$$

Moreover, for any (c,k) with $C_f(c,k) > 0$, we have

$$\mathbb{P}(\mathbf{R} = r \mid \mathbf{C} = c, \mathbf{K} = k) = \begin{cases} \dfrac{1}{C_f(c,k)} \displaystyle\prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right), & \text{if } r \text{ realises } (c,k) \\ & \text{and } \forall i\colon r_i \in \lambda_I^F(c_i), \\ 0, & \text{otherwise.} \end{cases}$$

Thus,

$$\mathbb{P}(\mathbf{R} = r \mid \mathbf{C} = c, \mathbf{K} = k) > 0 \quad \iff \quad r \text{ realises } (c,k) \text{ and } r_i \in \lambda_I^F(c_i).$$

Plugging these values into the law of total expectation yields

$$\begin{aligned} &\mathbb{E}[\mathbf{RT}(f)] \\ &= \sum_{\substack{(c,k):\\ C_f(c,k)>0}} \frac{C_f(c,k)}{T_f} \cdot \sum_{\substack{r \text{ realises } (c,k) \\ \forall i\colon r_i \in \lambda_I^F(c_i)}} \frac{1}{C_f(c,k)} \prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right) \\ &\qquad \cdot \mathbb{E}\left[\mathbf{RT}(f) \mid \mathbf{R} = r, \mathbf{C} = c, \mathbf{K} = k\right] \\ &= \frac{1}{T_f} \cdot \sum_{\substack{(c,k):\\ C_f(c,k)>0}} \sum_{\substack{r \text{ realises } (c,k) \\ \forall i\colon r_i \in \lambda_I^F(c_i)}} \prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right) \cdot \mathbb{E}\left[\mathbf{RT}(f) \mid \mathbf{R} = r, \mathbf{C} = c, \mathbf{K} = k\right]. \end{aligned}$$

Conditional expected runtime. The only unknown term in the formula is the expected runtime, conditioned on $\mathbf{R} = r, \mathbf{C} = c$ and $\mathbf{K} = k$, where $r = (r_1, \ldots, r_n)$. With these conditions, the following steps are performed by the sampling algorithm.

Step 1+2: Sample $(\mathbf{C}, \mathbf{K})$ and $\mathbf{R}$. The first step of UniSample-BT is to sample the realised root-group cardinalities $(\mathbf{C}, \mathbf{K})$ and subsequently the child multiplicities $\mathbf{R}$.

Since we assume that we can sample from a discrete distribution in constant time, the time for sampling $(\mathbf{C}, \mathbf{K})$ and $\mathbf{R} = (\mathbf{R}_1, \ldots, \mathbf{R}_n)$ is in

$$O(1+n) = O(1 + |C_{\mathcal{M}}(f)|) = O(|C_{\mathcal{M}}(f)|).$$

Step 3: Recursively generate child configurations. Next the algorithm generates for each child c_i with $r_i > 0$, a r_i-multiset of child configurations uniformly. This is done by a rejection loop, where a list of child configurations is generated via r_i calls to UniSample-BT$(c_i, \mathcal{M})$ and accepted with a certain probability, depending on the list itself.

To analyse the runtime of this rejection loop, let $\mathbf{T}_t \geq 0$ be the random variable corresponding to the total runtime spent in the t-th iteration of the loop, and let

$\mathbf{N}$ be the random variable corresponding to the total number of iterations until termination. Then the total runtime of the rejection-loop is given by

$$\text{Runtime rejection loop} = \sum_{t=1}^{\mathbf{N}} \mathbf{T}_t,$$

and taking the expected value yields

$$\mathbb{E}[\text{Runtime rejection loop}] = \mathbb{E}\left[\sum_{t=1}^{\mathbf{N}} \mathbf{T}_t\right].$$

Since the upper limit of the sum is random, we cannot directly pull the expected value inside the sum. Instead, for each iteration $t \in \mathbb{N}$, let $\mathbf{X}_t$ denote the full random execution trace of the t-th iteration of the repeat-loop. Since each iteration uses fresh independent randomness and the same procedure is executed each time, the sequence $(\mathbf{X}_t)_{t\in\mathbb{N}}$ is independent and identically distributed (i.i.d.).

Moreover, the runtime $\mathbf{T}_t$ of iteration t is completely determined by $\mathbf{X}_t$ (formally, $\mathbf{T}_t$ is a measurable function of $\mathbf{X}_t$), and the decision whether the algorithm terminates by iteration t depends only on the outcomes of the first t iterations. Consequently, the random variable $\mathbf{N}$ is a stopping time [90] with respect to the sequence $(\mathbf{X}_t)_{t\in\mathbb{N}}$ (more precisely, with respect to the filtration generated by $(\mathbf{X}_t)_{t\in\mathbb{N}}$). Hence, we may apply Wald's identity [91] and obtain

$$\mathbb{E}\left[\sum_{t=1}^{\mathbf{N}} \mathbf{T}_t\right] = \mathbb{E}[\mathbf{N}]\,\mathbb{E}[\mathbf{T}_1].$$

From the correctness proof of UniSample-BT (Theorem 5.1), we know $\mathbf{N}$ is geometrically distributed with parameter

$$q_i = \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right) \cdot \frac{1}{T_{c_i}^{r_i}}.$$

Hence $\mathbb{E}[\mathbf{N}] = 1/q_i$.

Cost of single iteration. It remains to compute $\mathbb{E}[\mathbf{T}_1]$, i.e. the expected runtime for a single iteration of the repeat-loop. In each iteration, the algorithm generates a list $\mathbf{L}_i$ of r_i canonical-configurations via r_i independent recursive calls of UniSample-BT$(c_i, \mathcal{M})$. Hence, the expected runtime for generating $\mathbf{L}_i$ is

$$\mathbb{E}[\text{runtime generate } \mathbf{L}_i] = r_i \cdot \mathbb{E}[\mathbf{RT}(c_i)].$$

After generating the list $\mathbf{L}_i = (\mathbf{K}_{i,1}, \ldots, \mathbf{K}_{i,r_i})$, the distinct configurations and their multiplicities are computed. This is achieved by inserting all configurations of $\mathbf{L}_i$ into a hash map, where the key is the configuration itself and the value is its number of occurrences.

Inserting a configuration of size k into the hash map takes $O(k)$ time on average. Computing the hash value requires $O(k)$ time and, in the event of a hash collision, checking for equality also takes $O(k)$ time. We assume that the expected number of hash collisions per insertion is $O(1)$.

Thus, the total time for processing the list $\mathbf{L}_i$ can be written as

$$\text{Runtime inserting } \mathbf{L}_i \text{ into hash map } \in O\left(\sum_{j=1}^{r_i} \text{Size}(\mathbf{K}_{i,j})\right).$$

Taking expectations and using linearity of expectation yields

$$\mathbb{E}[\text{Runtime inserting } \mathbf{L}_i \text{ into hash map}] \in O\left(\sum_{j=1}^{r_i} \mathbb{E}\left[\text{Size}(\mathbf{K}_{i,j})\right]\right).$$

Since each configuration $\mathbf{K}_{i,j}$ is generated by a recursive call to UniSample-BT(c_i, $\mathcal{M}$), the correctness theorem of UniSample-BT(Theorem 5.1) implies that $\mathbf{K}_{i,j}$ is uniformly distributed over $[\![\mathcal{M}_{c_i}]\!]_{\text{Canon}}$. Hence,

$$\mathbb{E}\left[\text{Size}(\mathbf{K}_{i,j})\right] = \overline{\text{Size}}(\mathcal{M}_{c_i}),$$

and consequently,

$$\mathbb{E}\left[\text{Runtime inserting } \mathbf{L}_i \text{ into the hash map}\right] \in O\Big(r_i \cdot \overline{\text{Size}}(\mathcal{M}_{c_i})\Big).$$

Combining the expected runtime for generating $\mathbf{L}_i$ and inserting it into the hash map yields

$$\mathbb{E}[\mathbf{T}_1] \in O\left(r_i \cdot \overline{\text{Size}}(\mathcal{M}_{c_i}) + r_i \cdot \mathbb{E}[\mathbf{RT}(c_i)]\right)$$

Sorting configurations. The final operation performed for a child c_i is to sort the accepted list $\mathbf{L}_i = (\mathbf{K}_{i,1}, \ldots, \mathbf{K}_{i,r_i})$ of child configurations with respect to the instance order $\trianglelefteq$.

To decide whether $K_a \trianglelefteq K_b$ holds for two canonical configurations K_a, K_b, their tree representations must be compared recursively. Concretely, for two feature instances with children, this requires ordering their children increasingly with respect to $\trianglelefteq$ and subsequently comparing the resulting sequences lexicographically.

At first glance, the sorting step appears expensive, since we have to sort the children and again compare them recursively in order to compute the instance order. However, we can exploit the fact that all configurations are canonical. In particular, sibling instances of the same feature are already sorted with respect to $\trianglelefteq$. Consequently, during a comparison it suffices to group children by their features and order these groups according to the feature order.

Assuming that the feature set F is totally ordered and encoded by the integers $0, \ldots, |F| - 1$, this grouping and ordering can be implemented using bucket sort. With bucket sort, deciding whether $K_a \trianglelefteq K_b$ holds for two configurations K_a, K_b takes

$$O\Big(\min(\text{Size}(K_a), \text{Size}(K_b)) + |F|\Big)$$

time.

One might expect that a comparison-based sorting algorithm with total work $\Theta(r_i \log r_i)$ on an input of size r_i, such as heap sort or merge sort [80, 82], compares each element only $O(\log r_i)$ times on average.

This intuition is false in general. Afshani et al. [92] showed that any sorting algorithm based on binary merge sort has worst-case *fragile complexity* $\Omega(\log^2 r_i)$, that is, there exists an input on which some element participates in $\Omega(\log^2 r_i)$ comparisons. For the classical merge procedure that repeatedly compares the front elements of the two lists, the fragile complexity is even $\Theta(r_i)$.

Although there exist comparison-based sorting algorithms with $\Theta(r_i \log r_i)$ total work and $\Theta(\log r_i)$ fragile complexity, such as specialized sorting networks, their constant factors are large, and they are generally not considered practical [92].

We therefore analyse the sorting step for a routine with worst-case fragile complexity $O(r_i)$.

For each $j \in \{1, \dots, r_i\}$, let $C_{i,j}$ denote the number of comparisons involving $\mathbf{K}_{i,j}$ during the sorting of $\mathbf{L}_i$. By assumption, $C_{i,j} \in O(r_i)$ for all j. Charging the cost of each comparison to its participating elements, the sorting time is bounded by

$$\text{Runtime sorting } \mathbf{L}_i \in O\left(\sum_{j=1}^{r_i} C_{i,j}\Big(\text{Size}(\mathbf{K}_{i,j}) + |F|\Big)\right).$$

Using $C_{i,j} \in O(r_i)$ and taking the expectation yields

$$\mathbb{E}\left[\text{Runtime sorting } \mathbf{L}_i\right] \in O\left(r_i \cdot \sum_{j=1}^{r_i}\Big(\mathbb{E}[\text{Size}(\mathbf{K}_{i,j})] + |F|\Big)\right). \qquad (*)$$

We now derive the distribution of an individual configuration $\mathbf{K}_{i,j}$ in order to compute the expected value. Although, within each iteration of the rejection loop, the configurations $\mathbf{K}_{i,1}, \dots, \mathbf{K}_{i,r_i}$ are generated independently and uniformly, this alone does not imply that the same holds after conditioning on acceptance. Hence, we must show that the rejection step does not introduce any bias in the marginal distribution of the individual list entries.

From the uniformity proof of UniSample-BT (Theorem 5.1), we know that for any concrete list $L = (K_1, \dots, K_{r_i})$ of canonical configurations of $\mathcal{M}_{c_i}$ whose distinct elements occur with multiplicities $m_1, \dots, m_k$, the probability that the rejection loop terminates with exactly this list is given by

$$\mathbb{P}(\mathbf{L}_i = L) = \frac{m_1! \cdots m_k!}{r_i!} \cdot \frac{1}{\left(\!\binom{T_{c_i}}{r_i}\!\right)}.$$

In particular, this probability depends on L only through the multiplicities $m_1, \dots, m_k$ of its elements, and not on their order. Consequently, all list representations of the same multiset occur with equal probability. That is, the random list $\mathbf{L}_i$ is a uniform random permutation of its underlying multiset.

We now use this property to derive the distribution of an individual entry $\mathbf{K}_{i,j}$. Fix an index $j \in \{1, \dots, r_i\}$ and a canonical configuration K of $\mathcal{M}_{c_i}$. Let $\mathbf{M}$ denote the random multiset underlying the random list $\mathbf{L}_i$. We now condition on $\mathbf{M} = M$ and examine the conditional distribution of $\mathbf{K}_{i,j}$.

Let $K \in [\![\mathcal{M}_{c_i}]\!]_{\text{Canon}}$ be an arbitrary configuration. Since $\mathbf{L}_i$ is a uniform list representation of M, each element has the same probability of occurring at position j. Thus,

$$\mathbb{P}(\mathbf{K}_{i,j} = K \mid \mathbf{M} = M) = \frac{M(K)}{r_i}.$$

Summing over all r_i-multisets over $[\![\mathcal{M}_{c_i}]\!]_{\text{Canon}}$ yields

$$\mathbb{P}(\mathbf{K}_{i,j} = K) = \sum_{M\colon\ r_i\text{-multiset}} \mathbb{P}(\mathbf{K}_{i,j} = K \mid \mathbf{M} = M) \cdot \mathbb{P}(\mathbf{M} = M)$$

$$= \sum_{M\colon\ r_i\text{-multiset}} \frac{M(K)}{r_i} \cdot \mathbb{P}(\mathbf{M} = M).$$

By the uniformity proof of UniSample-BT (Theorem 5.1), the multiset $\mathbf{M}$ is uniformly distributed over all r_i-multisets over $[\![\mathcal{M}_{c_i}]\!]_{\text{Canon}}$. Hence,

$$\mathbb{P}(\mathbf{K}_{i,j} = K) = \frac{1}{r_i} \cdot \frac{1}{\left(\!\binom{T_{c_i}}{r_i}\!\right)} \sum_{M\colon\ r_i\text{-multiset}} M(K).$$

The expression

$$\frac{1}{\left(\!\binom{T_{c_i}}{r_i}\!\right)} \sum_{M\colon\ r_i\text{-multiset}} M(K),$$

is exactly the expected multiplicity of configuration K in a uniformly distributed r_i-multiset over $[\![\mathcal{M}_{c_i}]\!]_{\text{Canon}}$, which is equal to r_i/T_{c_i} by Lemma 2.14. Consequently,

$$\mathbb{P}(\mathbf{K}_{i,j} = K) = \frac{1}{T_{c_i}}.$$

In particular, $\mathbf{K}_{i,j}$ is uniformly distributed over $[\![\mathcal{M}_{c_i}]\!]_{\text{Canon}}$ and hence

$$\mathbb{E}[\text{Size}(\mathbf{K}_{i,j})] = \frac{1}{T_{c_i}} \sum_{K \in [\![\mathcal{M}_{c_i}]\!]_{\text{Canon}}} \text{Size}(K) = \overline{\text{Size}}(\mathcal{M}_{c_i}).$$

Plugging this into $(*)$ yields

$$\begin{aligned}\mathbb{E}\left[\text{Runtime sorting } \mathbf{L}_i\right] &\in O\left(r_i \sum_{j=1}^{r_i} \left(\overline{\text{Size}}(\mathcal{M}_{c_i}) + |F|\right)\right) \\ &= O\left(r_i^2 \cdot \left(\overline{\text{Size}}(\mathcal{M}_{c_i}) + |F|\right)\right).\end{aligned}$$

Merging configurations. Finally, after all lists $\mathbf{L}_i$ have been generated and sorted, the algorithm applies Merge to combine the child configurations. The cost of merging the configurations is linear in the size of the configurations, since we only have to shift the indices of the feature instances. Conditioned on $\mathbf{R} = (r_1, \ldots, r_n)$, the runtime of the merge step is therefore in

$$O\left(\sum_{i=1}^{n} \sum_{j=1}^{r_i} \text{Size}(\mathbf{K}_{i,j})\right).$$

Taking the expectation yields

$$\mathbb{E}[\text{Runtime of Merge} \mid \mathbf{R} = r] \in O\left(\sum_{i=1}^{n} r_i \cdot \overline{\text{Size}}(\mathcal{M}_{c_i})\right).$$

In total, the conditional expected runtime of UniSample-BT$(f, \mathcal{M})$ given $\mathbf{R} = r = (r_1, \dots, r_n)$ and $(\mathbf{C}, \mathbf{K}) = (c, k)$ satisfies

$$\mathbb{E}\left[\mathbf{RT}(f) \mid \mathbf{R} = r, \mathbf{C} = c, \mathbf{K} = k\right] \in$$

$$O\Bigg(\underbrace{|C_{\mathcal{M}}(f)|}_{\substack{\text{sample child} \\ \text{multiplicities}}} + \sum_{i=1}^{n} \underbrace{\frac{1}{q_i}}_{\text{rejection loop}} \Big(\underbrace{r_i \cdot \mathbb{E}[\mathbf{RT}(c_i)]}_{\substack{\text{generate child} \\ \text{configurations}}} + \underbrace{r_i \cdot \overline{\text{Size}}(\mathcal{M}_{c_i})}_{\text{compute multiplicities}} \Big) + \underbrace{\sum_{i=1}^{n} r_i^2 \cdot \left(\overline{\text{Size}}(\mathcal{M}_{c_i}) + |F|\right)}_{\text{sorting by } \trianglelefteq} + \underbrace{\sum_{i=1}^{n} r_i \cdot \overline{\text{Size}}(\mathcal{M}_{c_i})}_{\text{merging}} \Bigg).$$

Putting everything together. Plugging the expression into the law of total expectation yields

$$\mathbb{E}[\mathbf{RT}(f)] = \frac{1}{T_f} \sum_{\substack{(c,k): \\ C_f(c,k)>0}} \sum_{\substack{r \text{ realises } (c,k) \\ \forall i:\, r_i \in \lambda_I^F(c_i)}} \prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right)$$

$$\cdot \mathbb{E}\left[\mathbf{RT}(f) \mid \mathbf{R} = r, \mathbf{C} = c, \mathbf{K} = k\right]$$

$$\in O\left(\begin{array}{c} \frac{1}{T_f} \sum\limits_{\substack{(c,k): \\ C_f(c,k)>0}} \sum\limits_{\substack{r \text{ realises } (c,k) \\ \forall i:\, r_i \in \lambda_I^F(c_i)}} \prod\limits_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right) \\ \cdot \left(|C_{\mathcal{M}}(f)| + \sum\limits_{i=1}^{n} \frac{1}{q_i} \left(r_i \cdot \mathbb{E}[\mathbf{RT}(c_i)] + r_i \cdot \overline{\text{Size}}(\mathcal{M}_{c_i}) \right) \right. \\ \left. + \sum\limits_{i=1}^{n} r_i^2 \cdot \left(\overline{\text{Size}}(\mathcal{M}_{c_i}) + |F|\right) + \sum\limits_{i=1}^{n} r_i \cdot \overline{\text{Size}}(\mathcal{M}_{c_i}) \right) \end{array} \right), \qquad (*)$$

where, for each child c_i, the acceptance parameter is

$$q_i = \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right) \cdot \frac{1}{T_{c_i}^{r_i}}.$$

5.2.2.2 Interpretation of the Expected Runtime

The expected runtime of UniSample-BT, given in $(*)$, decomposes naturally into two parts.

Construction cost. The first part consists of all costs incurred while explicitly constructing a configuration. These include sampling child multiplicities, recursively generating child configurations, counting multiplicities, sorting child instances by instance order, and finally merging the child configurations.

For a fixed recursion level, these operations are linear or polynomial in the size of the generated child configurations and in the child multiplicities r_i. In particular, all steps scale linearly in r_i and in the expected size of the child configurations, with the sole exception of sorting, which requires r_i^2 time.

Although the corresponding terms appear only linearly in $(*)$, this does not imply that the overall runtime of UniSample-BT is linear. The reason is that all these costs accumulate along the recursive calls induced by the feature tree. If all child multiplicities r_i, as well as the expected runtime and expected size of each recursive call, are bounded by a global constant, then the total runtime becomes polynomial, where the degree of the polynomial is the tree height.

Rejection cost. The second part of the runtime stems from enforcing uniformity via the rejection loop and is captured by the factor $1/q_i$.

For a fixed child multiplicity r_i, we have

$$\begin{aligned} q_i &= \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right) \cdot \frac{1}{T_{c_i}^{r_i}} \\ &= \frac{1}{r_i!} \prod_{k=0}^{r_i-1} \frac{T_{c_i}+k}{T_{c_i}} = \frac{1}{r_i!} \prod_{k=0}^{r_i-1} \left(1 + \frac{k}{T_{c_i}}\right). \end{aligned}$$

Since $1 + \frac{k}{T_{c_i}} \geq 1$ for all $k \geq 0$, it follows that

$$q_i \geq \frac{1}{r_i!}, \qquad \text{and therefore} \qquad \frac{1}{q_i} \leq r_i!.$$

Using the inequality $1 + x \leq \exp(x)$ for all $x \in \mathbb{R}$, we furthermore obtain

$$\begin{aligned} q_i &\leq \frac{1}{r_i!} \prod_{k=0}^{r_i-1} \exp\left(\frac{k}{T_{c_i}}\right) = \frac{1}{r_i!} \exp\left(\sum_{k=0}^{r_i-1} \frac{k}{T_{c_i}}\right) \\ &= \frac{1}{r_i!} \exp\left(\frac{r_i(r_i-1)}{2\,T_{c_i}}\right). \end{aligned}$$

Consequently,

$$\frac{1}{q_i} \geq r_i! \exp\left(-\frac{r_i(r_i-1)}{2\,T_{c_i}}\right).$$

Combining both bounds yields

$$r_i! \exp\left(-\frac{r_i(r_i-1)}{2\,T_{c_i}}\right) \;\leq\; \frac{1}{q_i} \;\leq\; r_i!.$$

In practice, especially for features higher up in the feature tree, the number of canonical configurations in the subtree below c_i is typically much larger than the multiplicity r_i. In this regime we have

$$\frac{r_i(r_i-1)}{2\,T_{c_i}} \approx 0,$$

so that

$$\exp\left(-\frac{r_i(r_i-1)}{2\,T_{c_i}}\right) \approx 1.$$

Hence, the lower and upper bounds become nearly identical, implying that

$$\frac{1}{q_i} \approx r_i!.$$

This means that the rejection loop requires roughly $r_i!$ iterations in expectation. Since each iteration involves recursively generating a full set of child configurations, this results in a drastic increase in the runtime.

The weighting factor sums to 1. We have so far ignored the term

$$\frac{1}{T_f} \sum_{\substack{(c,k):\\ C_f(c,k)>0}} \sum_{\substack{r \text{ realises } (c,k)\\ \forall i\colon r_i \in \lambda_I^F(c_i)}} \prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right),$$

because it does not contribute to the asymptotic runtime. Rather, this term only forms a weighted average of the conditional runtimes.

In particular, we obtain

$$\begin{aligned}
&\frac{1}{T_f} \sum_{\substack{(c,k):\\ C_f(c,k)>0}} \sum_{\substack{r \text{ realises } (c,k)\\ \forall i\colon r_i \in \lambda_I^F(c_i)}} \prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right) \\
&\qquad = \frac{1}{T_f} \sum_{\substack{(c,k):\\ C_f(c,k)>0}} \sum_{\substack{r \text{ realises } (c,k)\\ \forall i\colon r_i \in \lambda_I^F(c_i)}} \mathbb{P}(\mathbf{R} = r \mid \mathbf{C} = c, \mathbf{K} = k) \cdot C_f(c,k) \\
&\qquad = \frac{1}{T_f} \sum_{\substack{(c,k):\\ C_f(c,k)>0}} C_f(c,k) \sum_{\substack{r \text{ realises } (c,k)\\ \forall i\colon r_i \in \lambda_I^F(c_i)}} \mathbb{P}(\mathbf{R} = r \mid \mathbf{C} = c, \mathbf{K} = k) \\
&\qquad = \frac{1}{T_f} \sum_{\substack{(c,k):\\ C_f(c,k)>0}} C_f(c,k) \cdot 1 \\
&\qquad = 1,
\end{aligned}$$

where we use the fact that

$$\mathbb{P}(\mathbf{R} = r \mid \mathbf{C} = c, \mathbf{K} = k) = \frac{1}{C_f(c,k)} \prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right),$$

and that the total number of canonical configurations is given by

$$T_f = \sum_{\substack{(c,k):\\ C_f(c,k)>0}} C_f(c,k).$$

5.2.2.3 Expected Sample Size of UniSample-BT

In the runtime analysis of UniSample-BT (Section 5.2.2.1), the expected runtime was expressed in terms of the average canonical configuration size $\overline{\text{Size}}(\mathcal{M}_f)$. While one could compute this value by enumerating all canonical configurations and calculating their sizes, there is a more efficient way, which exploits the uniformity of UniSample-BT and leads to a similar recursive formula as in the runtime analysis.

For $f \in F$ let $\mathbf{K}_f$ be the random variable corresponding to the configuration produced by UniSample-BT$(f, \mathcal{M})$. Since $\overline{\text{Size}}(\mathcal{M}_f)$ is exactly the expected value of a uniformly distributed random variable over $[\![\mathcal{M}_f]\!]_{\text{Canon}}$ (see Section 5.2.2.1) and $\mathbf{K}_f$ is uniformly distributed by the uniformity of UniSample-BT, we derive the formula by analysing $\mathbb{E}[\text{Size}(\mathbf{K}_f)]$.

Leaf case. If f is a leaf feature, then UniSample-BT$(f, \mathcal{M})$ deterministically returns the unique configuration consisting only of the root instance f^1. Consequently,

$$\mathbb{P}\Big(\text{Size}(\mathbf{K}_f) = 1\Big) = 1 \qquad \text{and hence} \qquad \mathbb{E}[\text{Size}(\mathbf{K}_f)] = 1.$$

Non-leaf case. Now let f be a non-leaf feature with children $c_1, \ldots, c_n$. As in the runtime analysis, the number of recursive calls and their distribution depend on the sampled root-group cardinalities and the child multiplicities. Let

$$(\mathbf{C}, \mathbf{K}) \quad \text{and} \quad \mathbf{R} = (\mathbf{R}_1, \ldots, \mathbf{R}_n)$$

denote the corresponding random variables.

By the law of total expectation, we obtain

$$\begin{aligned}
&\mathbb{E}\left[\text{Size}(\mathbf{K}_f)\right] \\
&= \sum_{\substack{(c,k):\\ \mathbb{P}(\mathbf{C}=c,\mathbf{K}=k)>0}} \mathbb{P}(\mathbf{C}=c, \mathbf{K}=k) \sum_{\substack{r:\\ \mathbb{P}(\mathbf{R}=r \mid \mathbf{C}=c,\mathbf{K}=k)>0}} \mathbb{P}(\mathbf{R}=r \mid \mathbf{C}=c, \mathbf{K}=k) \\
&\qquad \cdot \mathbb{E}\left[\text{Size}(\mathbf{K}_f) \mid \mathbf{R}=r, \mathbf{C}=c, \mathbf{K}=k\right].
\end{aligned}$$

Again

$$\mathbb{P}(\mathbf{C}=c, \mathbf{K}=k) = \frac{C_f(c,k)}{T_f},$$

for all (c,k), and

$$\mathbb{P}(\mathbf{R}=r \mid \mathbf{C}=c, \mathbf{K}=k) = \begin{cases} \dfrac{1}{C_f(c,k)} \displaystyle\prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right), & \text{if } r \text{ realises } (c,k) \\ & \text{and } \forall i\colon r_i \in \lambda_I^F(c_i), \\ 0, & \text{otherwise.} \end{cases}$$

Plugging these expressions into the law of total expectation yields

$$\begin{aligned}
\mathbb{E}[\text{Size}(\mathbf{K}_f)] = \frac{1}{T_f} \sum_{\substack{(c,k):\\ C_f(c,k)>0}} \sum_{\substack{r \text{ realises } (c,k)\\ \forall i\colon r_i \in \lambda_I^F(c_i)}} \prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right) \\
\cdot \mathbb{E}\left[\text{Size}(\mathbf{K}_f) \mid \mathbf{R}=r, \mathbf{C}=c, \mathbf{K}=k\right].
\end{aligned}$$

Conditional expected size. It remains to compute the conditional expectation $\mathbb{E}\left[\text{Size}(\mathbf{K}_f) \mid \mathbf{R} = r, \mathbf{C} = c, \mathbf{K} = k\right]$. Since the realised root-group cardinalities and the child multiplicities are fixed, the only remaining factor influencing the size is the choice of the child configurations. In particular, UniSample-BT generates in Step 3, for each child c_i, an r_i-multiset $\mathbf{M}_i$ with canonical configurations from $\mathcal{M}_{c_i}$. From the uniformity proof of UniSample-BT (Theorem 5.1), we know that this multiset $\mathbf{M}_i$ is uniformly distributed over all r_i-multisets of the set $[\![\mathcal{M}_{c_i}]\!]_{\text{Canon}}$.

For a canonical configuration $K \in [\![\mathcal{M}_{c_i}]\!]_{\text{Canon}}$ let $\mathbf{M}_i(K)$ be the number of occurrences of K in $\mathbf{M}_i$. The expected total size of the configurations in the multiset $\mathbf{M}_i$ is then given by

$$\begin{aligned}\mathbb{E}\left[\text{Total size of the configurations in } \mathbf{M}_i\right] &= \mathbb{E}\left[\sum_{K \in [\![\mathcal{M}_{c_i}]\!]_{\text{Canon}}} \mathbf{M}_i(K) \cdot \text{Size}(K)\right] \\ &= \sum_{K \in [\![\mathcal{M}_{c_i}]\!]_{\text{Canon}}} \mathbb{E}[\mathbf{M}_i(K)] \cdot \text{Size}(K) \\ &\overset{(1)}{=} \frac{r_i}{T_{c_i}} \sum_{K \in [\![\mathcal{M}_{c_i}]\!]_{\text{Canon}}} \text{Size}\left(K\right),\end{aligned}$$

where we used in (1) that $\mathbf{M}_i$ is a uniformly distributed r_i-multiset over $[\![\mathcal{M}_{c_i}]\!]_{\text{Canon}}$, which has T_{c_i} elements and therefore by Lemma 2.14, the expected number of occurrences of each element is r_i / T_{c_i}.

Observe that in the last expression,

$$\frac{\sum_{K \in [\![\mathcal{M}_{c_i}]\!]_{\text{Canon}}} \text{Size}\left(K\right)}{T_{c_i}},$$

is exactly the average size of a configuration in $[\![\mathcal{M}_{c_i}]\!]_{\text{Canon}}$ and therefore equal to $\mathbb{E}[\text{Size}(\mathbf{K}_{c_i})]$. Thus, we get

$$\mathbb{E}\left[\text{Total size of the configurations in } \mathbf{M}_i\right] = r_i \cdot \mathbb{E}[\text{Size}(\mathbf{K}_{c_i})].$$

With the expected total size of the configurations in $\mathbf{M}_i$, we can now compute $\mathbb{E}\left[\text{Size}(\mathbf{K}_f) \mid \mathbf{R} = r, \mathbf{C} = c, \mathbf{K} = k\right]$. In Step 3, the child configurations contained in $\mathbf{M}_i$ are combined using Merge. Since the size of the configuration produced by Merge is equal to the sum of the sizes of its input configurations, plus one for the root feature instance f^1 that is added, we obtain

$$\mathbb{E}\left[\text{Size}(\mathbf{K}_f) \mid \mathbf{R} = r, \mathbf{C} = c, \mathbf{K} = k\right] = 1 + \sum_{i=1}^{n} r_i \cdot \mathbb{E}\left[\text{Size}(\mathbf{K}_{c_i})\right].$$

Plugging this expression into the law of total expectation yields

$$\begin{aligned}\mathbb{E}[\text{Size}(\mathbf{K}_f)] = \frac{1}{T_f} \cdot \sum_{\substack{(c,k):\\ C_f(c,k)>0}} \sum_{\substack{r \text{ realises } (c,k)\\ \forall i:\, r_i \in \lambda_I^F(c_i)}} \prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right) \\ \cdot \left(1 + \sum_{i=1}^{n} r_i \cdot \mathbb{E}\left[\text{Size}(\mathbf{K}_{c_i})\right]\right).\end{aligned}$$

This equation expresses the average configuration size of $\mathcal{M}_f$ solely in terms of the corresponding quantities of its children. In particular, $\overline{\text{Size}}(\mathcal{M}_f)$ can be computed bottom-up along the feature tree without explicitly enumerating the configuration space $[\![\mathcal{M}_f]\!]_{\text{Canon}}$. However, this way of computing the average configuration size is still computationally infeasible for large models, since computing the number of vectors that realise a given (c, k) is not trivial.

A more efficient approach is to use a DP method similar to the one used for counting configurations, but adapted to compute the average configuration size instead. This approach is already implemented in the CFMTOOLS framework (see Section 4.3 and Chapter 6).

The main purpose of the formula above is therefore conceptual: it allows a direct comparison with the expected runtime formula, since both expressions have the same structural form.

5.2.3 UniSample-UR

We now analyse the expected runtime of the algorithm UniSample-UR (Algorithm 4.3). As in the analysis of UniSample-BT, we assume that sampling from a discrete probability distribution can be performed in $O(1)$ time. Since UniSample-UR itself is randomised, we again focus exclusively on its expected runtime.

For a feature $f \in F$ and a rank $\mathcal{R} \in \{0, \dots, T_f - 1\}$ let $\text{RT}(\mathcal{R}, f)$ denote the (deterministic) runtime of UnrankConfig$(f, \mathcal{R}, \mathcal{M})$. Furthermore, let $\boldsymbol{\mathcal{R}}_f$ be a random variable uniformly distributed over the rank set of $\mathcal{M}_f$, that is

$$\boldsymbol{\mathcal{R}}_f \sim \text{Unif}\Big(\{0, \dots, T_f - 1\}\Big).$$

Since UniSample-UR picks a rank uniformly and unranks it via UnrankConfig, by construction $\mathbb{E}[\text{RT}(\boldsymbol{\mathcal{R}}_{f_r}, f_r)]$ is exactly the expected runtime of UniSample-UR. Because of this, we now derive a recursive expression for $\mathbb{E}[\text{RT}(\boldsymbol{\mathcal{R}}_f, f)]$ for all $f \in F$.

Leaf case. If f is a leaf feature, then UnrankConfig$(f, \boldsymbol{\mathcal{R}}_f, \mathcal{M})$ deterministically returns the unique configuration consisting of the single instance f^1. Hence,

$$\mathbb{E}[\text{RT}(\boldsymbol{\mathcal{R}}_f, f)] \in O(1).$$

Non-leaf case. Let f be a non-leaf feature with children $c_1, \dots, c_n$. As in the analysis of UniSample-BT, let $\mathbf{C}$ and $\mathbf{K}$ denote the random variables corresponding to the realised root-group cardinalities, and let $\mathbf{R} = (\mathbf{R}_1, \dots, \mathbf{R}_n)$ denote the multiplicity vector.

The crucial difference to UniSample-BT is that we do not generate $\mathbf{C}$, $\mathbf{K}$, and $\mathbf{R}$ according to some probability distribution, but instead they are completely determined by $\boldsymbol{\mathcal{R}}_f$. In particular, once $\boldsymbol{\mathcal{R}}_f$ is known, the values of $(\mathbf{C}, \mathbf{K})$ and $\mathbf{R}$ are also known.

Nevertheless, the law of total expectation still applies and yields

$$\mathbb{E}[\mathrm{RT}(\boldsymbol{\mathcal{R}}_f, f)] = \sum_{\substack{(c,k):\\ \mathbb{P}(\mathbf{C}=c,\mathbf{K}=k)>0}} \mathbb{P}(\mathbf{C}=c,\mathbf{K}=k) \cdot \mathbb{E}[\mathrm{RT}(\boldsymbol{\mathcal{R}}_f, f) \mid \mathbf{C}=c,\mathbf{K}=k]$$

$$= \sum_{\substack{(c,k):\\ \mathbb{P}(\mathbf{C}=c,\mathbf{K}=k)>0}} \mathbb{P}(\mathbf{C}=c,\mathbf{K}=k) \sum_{\substack{r:\\ \mathbb{P}(\mathbf{R}=r\mid\mathbf{C}=c,\mathbf{K}=k)>0}}$$

$$\cdot \mathbb{P}(\mathbf{R}=r \mid \mathbf{C}=c,\mathbf{K}=k) \cdot \mathbb{E}[\mathrm{RT}(\boldsymbol{\mathcal{R}}_f, f) \mid \mathbf{R}=r,\mathbf{C}=c,\mathbf{K}=k].$$

As already mentioned, in contrast to UniSample-BT, the distributions of $\mathbf{C}$, $\mathbf{K}$, and $\mathbf{R}$ are not explicitly defined a priori and must be derived from the ranking scheme and the fact that $\boldsymbol{\mathcal{R}}_f$ is uniformly distributed over the rank set $\{0,\dots,T_f-1\}$.

In particular, the probability $\mathbb{P}(\mathbf{C}=c,\mathbf{K}=k)$ is obtained by counting the number of ranks that induce the root-group cardinalities (c,k). By definition, this number is $C_f(c,k)$, and thus

$$\mathbb{P}(\mathbf{C}=c,\mathbf{K}=k) = \frac{C_f(c,k)}{T_f}.$$

Similarly, the conditional probability

$$\mathbb{P}(\mathbf{R}=r \mid \mathbf{C}=c,\mathbf{K}=k) = \frac{\mathbb{P}(\mathbf{R}=r,\mathbf{C}=c,\mathbf{K}=k)}{\mathbb{P}(\mathbf{C}=c,\mathbf{K}=k)},$$

is obtained by counting the number of ranks that realise both the root-group cardinalities (c,k) and the multiplicity vector r, and dividing it by the number of ranks that realise (c,k).

For fixed (c,k), the number of ranks that realise (c,k) and $r=(r_1,\dots,r_n)$ equals

$$\prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right),$$

since we can choose for each child c_i a r_i-multiset over $[\![\mathcal{M}_{c_i}]\!]_{\mathrm{Canon}}$. Hence,

$$\mathbb{P}(\mathbf{R}=r \mid \mathbf{C}=c,\mathbf{K}=k) = \frac{1}{C_f(c,k)} \prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right).$$

Putting these expressions together, we obtain

$$\mathbb{E}[\mathrm{RT}(\boldsymbol{\mathcal{R}}_f, f)] = \frac{1}{T_f} \sum_{\substack{(c,k):\\ C_f(c,k)>0}} \sum_{\substack{r \text{ realises } (c,k)\\ \forall i:\, r_i \in \lambda_I^F(c_i)}} \prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right)$$

$$\cdot \mathbb{E}[\mathrm{RT}(\boldsymbol{\mathcal{R}}_f, f) \mid \mathbf{R}=r,\mathbf{C}=c,\mathbf{K}=k],$$

which is identical to the expression obtained in the runtime analysis of UniSample-BT.

Conditional expected runtime. We now derive an upper bound for the conditional expected runtime $\mathbb{E}[\mathrm{RT}(\boldsymbol{\mathcal{R}}_f, f) \mid \mathbf{R} = r, \mathbf{C} = c, \mathbf{K} = k]$. Concretely, the runtime of UnrankConfig consists of the following parts.

Step 1: Determine block (c, k). The first step of UnrankConfig determines the unique block $(c, k) \in \lambda_T^G(f) \times \lambda_I^G(f)$ containing the rank $\boldsymbol{\mathcal{R}}_f$. This is done by iterating over the blocks in a fixed order and maintaining the cumulative size S until

$$S \leq \boldsymbol{\mathcal{R}}_f < S + C_f(c, k).$$

Since the DP tables are precomputed and each iteration consists of constant-time operations (one comparison and one addition), the runtime of this step is proportional to the number of blocks inspected and therefore in $O(|\lambda_T^G(f) \times \lambda_I^G(f)|) = O(|\lambda_T^G(f)| \cdot |\lambda_I^G(f)|)$.

Note that so far, we have never specified the concrete order in which we traverse the blocks, both in ranking and unranking, as it did not matter. However, we can now see that the order directly influences the runtime of Step 1: placing large blocks late (i.e. scanning blocks in increasing order of their sizes) yields the worst case, whereas traversing blocks in decreasing order is the best case. Thus, the runtime of Step 1 can be reduced by choosing an order which traverses large blocks first.

Furthermore, instead of scanning the blocks linearly, one may precompute a prefix-sum array

$$P(j) \coloneqq \sum_{\ell=1}^{j} C_f(c_\ell, k_\ell),$$

where (c_ℓ, k_ℓ) denotes the ℓ-th block in the chosen traversal order. Then the unique block (c, k) satisfying

$$P(j-1) \leq \boldsymbol{\mathcal{R}}_f < P(j)$$

can be located by a binary search in time $O(\log(|\lambda_T^G(f)| \cdot |\lambda_I^G(f)|)) = O(\log|\lambda_T^G(f)| + \log|\lambda_I^G(f)|)$.

This optimization requires additional storage for the prefix array and incurs a one-time preprocessing cost linear in the number of blocks. However, this overhead is acceptable, since the analysis in Section 5.2.2 assumes the use of alias tables to enable constant-time sampling and does not account for their preprocessing or storage costs. In fact, we will see in Section 5.2.5, that the time required to compute the prefix arrays and the associated storage requirements match the time and space requirements needed to construct and store the alias tables.

Step 2a: Determine grid r_i for child c_i. Next, the algorithm determines for each child c_i the grid r_i by iterating over $\lambda_I^F(c_i)$. Again, for each possible value $r_i' \in \lambda_I^F(c_i)$, the algorithm performs some constant work (binomials can also be computed in $O(1)$ by assumption) and therefore the time for Step 2a is in $O(|\lambda_I^F(c_i)|)$.

As in Step 1, the traversal order matters. Traversing the grids in increasing order with respect to their sizes yields the worst case, whereas traversing them in decreasing order yields the best case. Moreover, the search can be sped up by using prefix-sum arrays.

Step 2b: Split grid-local rank. Once the grid r_i has been determined, the algorithm splits the current grid-local rank into the fiber-local rank and the fiber index. This requires only a constant number of arithmetic operations (in particular an integer division with remainder) and is therefore in $O(1)$.

Step 2c: Unrank multiset. Given the child multiplicity r_i, UnrankConfig unranks an r_i-multiset over $\{0, \dots, T_{c_i} - 1\}$ using UnrankMultiset.

As described in Section 2.4.3.2, multiset unranking is reduced to combination unranking via the shift bijection. More precisely, an r_i-multiset over a ground set of size T_{c_i} corresponds to an r_i-combination of the set $\{0, \dots, T_{c_i} + r_i - 2\}$.

Since we use the incremental binomial evaluation strategy from Section 2.4.3.1, unranking such a combination takes

$$O((T_{c_i} + r_i)\, r_i)$$

time.

After reconstructing the combination $(c_1, \dots, c_{r_i})$, the corresponding multiset $(m_1, \dots, m_{r_i})$ is obtained via the shift transformation

$$m_j = c_j - (j - 1), \qquad j = 1, \dots, r_i.$$

This transformation requires only a single subtraction per element and therefore runs in $O(r_i)$ time. Since $O(r_i) \subseteq O((T_{c_i} + r_i)\, r_i)$, the total cost for unranking the multiset remains

$$O((T_{c_i} + r_i)\, r_i)\,.$$

Step 2d: Recursively unrank child-configurations. Let $\mathbf{M}_i$ be the r_i-multiset of child ranks generated for c_i in Step 2c. The algorithm iterates over each rank in $\mathbf{M}_i$ and unranks them by calling UnrankConfig if the rank occurs for the first time in the sequence and otherwise the previously computed configuration is reused.

We first have to derive the distribution of $\mathbf{M}_i$, conditioned on $\mathbf{C} = c$, $\mathbf{K} = k$, and $\mathbf{R} = r$. Let M be an arbitrary r_i-multiset over $\{0, \dots, T_{c_i} - 1\}$.

Again

$$\mathbb{P}(\mathbf{M}_i = M \mid \mathbf{C} = c, \mathbf{K} = k, \mathbf{R} = r) = \frac{\mathbb{P}(\mathbf{M}_i = M, \mathbf{C} = c, \mathbf{K} = k, \mathbf{R} = r)}{\mathbb{P}(\mathbf{C} = c, \mathbf{K} = k, \mathbf{R} = r)},$$

can be determined by counting the number of ranks that realise (c, k), r, and M, and dividing it by the number of ranks that realise (c, k) and r.

The number of ranks that realise (c, k) and r was already calculated earlier and was given by

$$\prod_{j=1}^{n} \left(\!\!\binom{T_{c_j}}{r_j}\!\!\right).$$

Similarly, the number of ranks that realise (c, k), r, and M is given by

$$\prod_{\substack{j=1 \\ j \neq i}}^{n} \left(\!\!\binom{T_{c_j}}{r_j}\!\!\right),$$

since the multiset for the child c_i is fixed to M.

Hence,

$$\mathbb{P}(\mathbf{M}_i = M \mid \mathbf{C} = c, \mathbf{K} = k, \mathbf{R} = r) = \frac{1}{\left(\!\!\binom{T_{c_i}}{r_i}\!\!\right)}.$$

In particular, $\mathbf{M}_i$ is uniformly distributed over all r_i-multisets over $\{0, \ldots, T_{c_i} - 1\}$.

With the distribution of $\mathbf{M}_i$, we can now compute the expected runtime for unranking all child ranks in $\mathbf{M}_i$. Since each unique rank gets unranked at most once, the runtime is given by

$$\text{Time to unrank all ranks in } \mathbf{M}_i = r_i + \sum_{k=0}^{T_{c_i}-1} \mathbb{1}_{\{\mathbf{M}_i(k)>0\}} \cdot \mathrm{RT}(k, c_i),$$

Taking expectations yields

$$\begin{aligned}
\mathbb{E}[\text{Time to unrank all ranks in } \mathbf{M}_i] &= \mathbb{E}\left[r_i + \sum_{k=0}^{T_{c_i}-1} \mathbb{1}_{\{\mathbf{M}_i(k)>0\}} \cdot \mathrm{RT}(k, c_i)\right] \\
&= r_i + \sum_{k=0}^{T_{c_i}-1} \mathbb{E}\left[\mathbb{1}_{\{\mathbf{M}_i(k)>0\}}\right] \cdot \mathrm{RT}(k, c_i) \\
&= r_i + \sum_{k=0}^{T_{c_i}-1} \mathbb{P}\left(\mathbf{M}_i(k) > 0\right) \cdot \mathrm{RT}(k, c_i) \\
&\overset{(1)}{=} r_i + \frac{r_i}{T_{c_i} + r_i - 1} \sum_{k=0}^{T_{c_i}-1} \mathrm{RT}(k, c_i),
\end{aligned}$$

where in (1) we used that the probability that a given element occurs at least once in a r_i-multiset over a set with T_{c_i} elements, is $r_i/(T_{c_i} + r_i - 1)$ (Lemma 2.15).

Since

$$\frac{1}{T_{c_i}} \sum_{k=0}^{T_{c_i}-1} \mathrm{RT}(k, c_i) = \sum_{k=0}^{T_{c_i}-1} \mathrm{RT}(k, c_i) \cdot \mathbb{P}(\boldsymbol{\mathcal{R}}_{c_i} = k) = \mathbb{E}\left[\mathrm{RT}(\boldsymbol{\mathcal{R}}_{c_i}, c_i)\right],$$

we obtain

$$\mathbb{E}[\text{Time to unrank all ranks in } \mathbf{M}_i] = r_i + \frac{r_i \cdot T_{c_i}}{T_{c_i} + r_i - 1} \cdot \mathbb{E}\left[\mathrm{RT}(\boldsymbol{\mathcal{R}}_{c_i}, c_i)\right].$$

Sorting configurations. After all ranks from $\mathbf{M}_i$ have been unranked and collected in the list

$$\mathbf{L}_i = [\mathbf{K}_{i,1}, \ldots, \mathbf{K}_{i,r_i}],$$

the algorithm sorts $\mathbf{L}_i$ increasingly with respect to the instance order $\trianglelefteq$.

As in the analysis of the expected sample size for UniSample-BT (Section 5.2.2.3), the expected total size of the configurations in the underlying random r_i-multiset is

$$\mathbb{E}\left[\sum_{j=1}^{r_i} \mathrm{Size}(\mathbf{K}_{i,j})\right] = r_i \cdot \overline{\mathrm{Size}}(\mathcal{M}_{c_i}).$$

Moreover, as shown in the analysis of UniSample-BT, sorting $\mathbf{L}_i$ takes expected time

$$\mathbb{E}[\text{Runtime sorting } \mathbf{L}_i] \in O\left(r_i \cdot \mathbb{E}\left[\sum_{j=1}^{r_i}\left(\text{Size}(\mathbf{K}_{i,j}) + |F|\right)\right]\right).$$

Hence,

$$\mathbb{E}[\text{Runtime sorting } \mathbf{L}_i] \in O\left(r_i^2 \cdot \left(\overline{\text{Size}}(\mathcal{M}_{c_i}) + |F|\right)\right).$$

Combining this with the cost of recursively unranking child configurations yields that, conditioned on $\mathbf{R} = r$, $\mathbf{C} = c$, and $\mathbf{K} = k$, the expected runtime of Step 2d for a fixed child c_i is in

$$O\left(r_i + \frac{r_i \cdot T_{c_i}}{T_{c_i} + r_i - 1} \cdot \mathbb{E}[\text{RT}(\boldsymbol{\mathcal{R}}_{c_i}, c_i)] + r_i^2 \cdot \left(\overline{\text{Size}}(\mathcal{M}_{c_i}) + |F|\right)\right).$$

Merging configurations. After all children $c_1, \ldots, c_n$ have been processed, the algorithm combines the sorted lists $L_1, \ldots, L_n$ using Merge. As in the analysis of UniSample-BT, the expected cost for this step is in

$$O\left(\sum_{i=1}^{n} r_i \cdot \overline{\text{Size}}(\mathcal{M}_{c_i})\right).$$

Conditional expected runtime. Putting together Step 1, Step 2, and the merge step, we obtain the conditional expected runtime of UniSample-UR:

$$\begin{aligned}\mathbb{E}[\text{RT}(\boldsymbol{\mathcal{R}}_f, f) \mid \mathbf{R} = r, \mathbf{C} = c, \mathbf{K} = k] \in O\Bigg(&\underbrace{|\lambda_T^G(f)| \cdot |\lambda_I^G(f)|}_{\text{determine group}} \\ &+ \sum_{i=1}^{n}\Bigg(\underbrace{|\lambda_I^F(c_i)|}_{\substack{\text{determine grid}\\ \text{for } c_i}} + \underbrace{(T_{c_i} + r_i)\, r_i}_{\substack{\text{unrank}\\ \text{multiset}}} + \underbrace{r_i + \frac{r_i \cdot T_{c_i}}{T_{c_i} + r_i - 1} \cdot \mathbb{E}[\text{RT}(\boldsymbol{\mathcal{R}}_{c_i}, c_i)]}_{\substack{\text{unrank child configurations}\\ \text{for } c_i}}\Bigg) \\ &+ \underbrace{\sum_{i=1}^{n} r_i^2 \cdot \left(\overline{\text{Size}}(\mathcal{M}_{c_i}) + |F|\right)}_{\text{sort by } \trianglelefteq} + \underbrace{\sum_{i=1}^{n} r_i \cdot \overline{\text{Size}}(\mathcal{M}_{c_i})}_{\text{merge}}\Bigg).\end{aligned}$$

Putting everything together. Substituting the conditional bound into the law of total expectation yields

$$\mathbb{E}[\mathrm{RT}(\boldsymbol{\mathcal{R}}_f, f)] = \frac{1}{T_f} \sum_{\substack{(c,k):\\ C_f(c,k)>0}} \sum_{\substack{r \text{ realises } (c,k)\\ \forall i:\, r_i \in \lambda_I^F(c_i)}} \prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right)$$

$$\cdot \mathbb{E}[\mathrm{RT}(\boldsymbol{\mathcal{R}}_f, f) \mid \mathbf{R} = r, \mathbf{C} = c, \mathbf{K} = k]$$

$$\in O\left(\begin{array}{c} \frac{1}{T_f} \sum\limits_{\substack{(c,k):\\ C_f(c,k)>0}} \sum\limits_{\substack{r \text{ realises } (c,k)\\ \forall i:\, r_i \in \lambda_I^F(c_i)}} \prod\limits_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right) \cdot \left(|\lambda_T^G(f)| \cdot |\lambda_I^G(f)| \right. \\ + \sum\limits_{i=1}^{n} \left(|\lambda_I^F(c_i)| + (T_{c_i} + r_i)\, r_i + r_i + \frac{r_i \cdot T_{c_i}}{T_{c_i} + r_i - 1} \cdot \mathbb{E}[\mathrm{RT}(\boldsymbol{\mathcal{R}}_{c_i}, c_i)] \right) \\ \left. + \sum\limits_{i=1}^{n} r_i^2 \cdot \left(\overline{\mathrm{Size}}(\mathcal{M}_{c_i}) + |F| \right) + \sum\limits_{i=1}^{n} r_i \cdot \overline{\mathrm{Size}}(\mathcal{M}_{c_i}) \right) \end{array}\right).$$

5.2.4 UniSample-CTC

We now analyse the expected runtime of the algorithm CTC-UniSample (Algorithm 4.5). In contrast to UniSample-BT and UniSample-UR, this algorithm does not construct configurations directly but instead applies rejection sampling on top of a base sampler that ignores cross-tree constraints. The runtime therefore depends directly on the runtime of the underlying base sampler.

Let $\widetilde{\mathcal{M}}$ denote the CFM obtained from $\mathcal{M}$ by removing all cross-tree constraints, and let BaseSample be a sampling algorithm that produces canonical configurations of $\widetilde{\mathcal{M}}$ uniformly at random.

The probability that a uniformly sampled configuration of $\widetilde{\mathcal{M}}$ satisfies all cross-tree constraints is given by

$$p \coloneqq \frac{|[\![\mathcal{M}]\!]_{\mathrm{Canon}}|}{|[\![\widetilde{\mathcal{M}}]\!]_{\mathrm{Canon}}|}.$$

Cost per iteration. Each iteration performs two operations:

1. sampling a configuration using BaseSample, and
2. checking whether the sampled configuration satisfies all cross-tree constraints.

Let $\mathbf{T}_t$ denote the total runtime of the t-th iteration of the rejection loop, and let $\mathbf{RT}_{\mathrm{Base}}^{(t)}$ denote the random runtime of the t-th call to BaseSample.

Since cross-tree constraints are interpreted globally, verifying that a configuration satisfies all cross-tree constraints amounts to iterating once over all require-edges and exclude-edges and checking whether the multiplicities in the sampled configuration satisfy the corresponding conditions. Hence, the cost of verifying the constraints is in

$$O(|\phi_R| + |\phi_X|).$$

Therefore, for every iteration $t \in \mathbb{N}$, we have

$$\mathbf{T}_t \in O\Big(\mathbf{RT}^{(t)}_{\text{BASE}} + |\phi_R| + |\phi_X|\Big).$$

Since the iterations are independent and identical, the random variables $(\mathbf{T}_t)_{t\in\mathbb{N}}$ are i.i.d. distributed, and thus

$$\mathbb{E}[\mathbf{T}_1] \in O(\mathbb{E}[\mathbf{RT}_{\text{BASE}}] + |\phi_R| + |\phi_X|),$$

where $\mathbf{RT}_{\text{BASE}}$ denotes the random runtime of a single call to BASESAMPLE.

Expected runtime. Let $\mathbf{N}$ denote the number of iterations of the rejection loop. Then the total runtime of CTC-UNISAMPLE is given by

$$\mathbf{RT}_{\text{CTC}} = \sum_{t=1}^{\mathbf{N}} \mathbf{T}_t.$$

As in the runtime analysis of UNISAMPLE-BT (see Section 5.2.2.1), we justify the use of Wald's identity via the full execution traces of the loop iterations. Hence,

$$\mathbb{E}[\mathbf{RT}_{\text{CTC}}] = \mathbb{E}\left[\sum_{t=1}^{\mathbf{N}} \mathbf{T}_t\right] = \mathbb{E}[\mathbf{N}] \cdot \mathbb{E}[\mathbf{T}_1].$$

Because $\mathbf{N} \sim \text{Geom}(p)$, we have

$$\mathbb{E}[\mathbf{N}] = \frac{1}{p}.$$

Hence, the expected runtime of CTC-UNISAMPLE satisfies

$$\mathbb{E}[\mathbf{RT}_{\text{CTC}}] \in O\left(\frac{1}{p}\Big(\mathbb{E}[\mathbf{RT}_{\text{BASE}}] + |\phi_R| + |\phi_X|\Big)\right).$$

In particular, if the cross-tree constraints are restrictive (i.e., $p \approx 0$), the expected runtime grows rapidly.

5.2.5 Comparison of Unisample-BT and Unisample-UR

Having derived explicit runtime bounds for both UNISAMPLE-BT and UNISAMPLE-UR, we now compare the algorithms and discuss in which situations one approach may be preferable over the other.

The final runtime bound of UNISAMPLE-BT was shown to be

$$O\left(\begin{array}{c} \dfrac{1}{T_f} \displaystyle\sum_{\substack{(c,k):\\ C_f(c,k)>0}} \sum_{\substack{r \text{ realises } (c,k) \\ \forall i:\, r_i \in \lambda_I^F(c_i)}} \prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right) \\ \cdot \left(|C_{\mathcal{M}}(f)| + \displaystyle\sum_{i=1}^{n} \frac{1}{q_i} \left(r_i \cdot \mathbb{E}[\mathbf{RT}(c_i)] + r_i \cdot \overline{\text{Size}}(\mathcal{M}_{c_i}) \right) \right. \\ \left. + \displaystyle\sum_{i=1}^{n} r_i^2 \cdot (\overline{\text{Size}}(\mathcal{M}_{c_i}) + |F|) + \sum_{i=1}^{n} r_i \cdot \overline{\text{Size}}(\mathcal{M}_{c_i}) \right) \end{array}\right),$$

where for each child c_i the acceptance parameter is

$$q_i = \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right) \cdot \frac{1}{T_{c_i}^{r_i}}.$$

The corresponding runtime bound of UniSample-UR is

$$O\left(\begin{array}{c} \frac{1}{T_f} \sum\limits_{\substack{(c,k):\\ C_f(c,k)>0}} \sum\limits_{\substack{r \text{ realises } (c,k)\\ \forall i:\, r_i \in \lambda_I^F(c_i)}} \prod\limits_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right) \\ \cdot \left(|\lambda_T^G(f)| \cdot |\lambda_I^G(f)| \right. \\ \left. + \sum\limits_{i=1}^{n} \left(|\lambda_I^F(c_i)| + (T_{c_i} + r_i)\, r_i + r_i + \frac{r_i \cdot T_{c_i}}{T_{c_i} + r_i - 1} \cdot \mathbb{E}[\mathrm{RT}(\boldsymbol{\mathcal{R}}_{c_i}, c_i)] \right) \right. \\ \left. + \sum\limits_{i=1}^{n} r_i^2 \cdot (\overline{\mathrm{Size}}(\mathcal{M}_{c_i}) + |F|) + \sum\limits_{i=1}^{n} r_i \cdot \overline{\mathrm{Size}}(\mathcal{M}_{c_i}) \right) \end{array}\right).$$

Tables 5.1 and 5.2 summarise the main runtime contributions of both algorithms. As discussed in Section 5.2.2.2, the prefactor

$$\frac{1}{T_f} \sum_{\substack{(c,k):\\ C_f(c,k)>0}} \sum_{\substack{r \text{ realises } (c,k)\\ \forall i:\, r_i \in \lambda_I^F(c_i)}} \prod_{i=1}^{n} \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right)$$

only weights the individual components and sums to 1 and is therefore excluded.

	UniSample-BT	UniSample-UR
1.	$\lvert C_{\mathcal{M}}(f)\rvert$	$\lvert\lambda_T^G(f)\rvert \cdot \lvert\lambda_I^G(f)\rvert + \sum\limits_{i=1}^{n} \lvert\lambda_I^F(c_i)\rvert$
2.	$\sum\limits_{i=1}^{n} \frac{1}{q_i} \Big(r_i \cdot \mathbb{E}[\mathbf{RT}(c_i)] + r_i \cdot \overline{\mathrm{Size}}(\mathcal{M}_{c_i}) \Big)$	$\sum\limits_{i=1}^{n} \Big((T_{c_i} + r_i)\, r_i + r_i + \frac{r_i \cdot T_{c_i}}{T_{c_i} + r_i - 1} \cdot \mathbb{E}[\mathrm{RT}(\boldsymbol{\mathcal{R}}_{c_i}, c_i)] \Big)$
3.	$\sum\limits_{i=1}^{n} r_i^2\, (\overline{\mathrm{Size}}(\mathcal{M}_{c_i}) + \lvert F\rvert)$	$\sum\limits_{i=1}^{n} r_i^2\, (\overline{\mathrm{Size}}(\mathcal{M}_{c_i}) + \lvert F\rvert)$
4.	$\sum\limits_{i=1}^{n} r_i \cdot \overline{\mathrm{Size}}(\mathcal{M}_{c_i})$	$\sum\limits_{i=1}^{n} r_i \cdot \overline{\mathrm{Size}}(\mathcal{M}_{c_i})$

Table 5.1: Comparison of the main runtime components of UniSample-BT and UniSample-UR

From Table 5.1 we observe that the costs of sorting and merging (Steps 3 and 4) are identical for both algorithms. The essential differences therefore arise in Step 1 (determining the child multiplicities) and Step 2 (recursive generation of child configurations).

Step	Algorithmic step
1.	Determine child multiplicities
2.	Recursive generation of child configurations
3.	Sorting child configurations
4.	Merge configurations

Table 5.2: Description of the algorithmic steps referenced in Table 5.1

Step 1: Determining child multiplicities. For this step, UniSample-BT has a clear advantage. Its runtime contribution depends only on the number of children,

$$|C_{\mathcal{M}}(f)|,$$

whereas UniSample-UR depends on the sizes of the cardinality intervals

$$|\lambda_T^G(f)| \cdot |\lambda_I^G(f)| + \sum_{i=1}^{n} |\lambda_I^F(c_i)|.$$

Even if prefix-sum arrays are used to accelerate the block and grid search in UniSample-UR, the runtime would still contain logarithmic factors of the form

$$\log |\lambda_T^G(f)| + \log |\lambda_I^G(f)| + \sum_{i=1}^{n} \log |\lambda_I^F(c_i)|.$$

In contrast, UniSample-BT samples multiplicities using the alias method, which allows sampling from each distribution in constant time once the tables have been constructed. Consequently, the sampling step of UniSample-BT is asymptotically independent of the sizes of the cardinality intervals involved.

Step 2: Recursive generation of child configurations. The comparison of the recursive component is more subtle, since the corresponding terms differ in their structure.

One advantage of UniSample-UR is that the recursive cost is multiplied by the factor

$$\frac{r_i \cdot T_{c_i}}{T_{c_i} + r_i - 1},$$

which satisfies

$$\frac{r_i \cdot T_{c_i}}{T_{c_i} + r_i - 1} \leq \frac{r_i \cdot T_{c_i}}{T_{c_i}} = r_i.$$

This reflects an important structural difference between the algorithms. In UniSample-UR, identical child configurations are only constructed once because repeated ranks can be reused during unranking. In contrast, UniSample-BT performs a recursive sampling call for every child instance, even if several instances correspond to the same configuration.

On the other hand, the runtime bound of UniSample-UR contains additional overhead terms such as

$$(T_{c_i} + r_i)r_i + r_i,$$

which originate from the multiset-unranking procedure. If T_{c_i} is large, these terms may become significant. However, this overhead is partly an artefact of the concrete unranking implementation used in the analysis. With more specialised unranking algorithms for combinations, this part of the runtime can be reduced substantially, for example to a quadratic dependence on r_i (see Section 2.4.3.1).

In contrast, the recursive term of UniSample-BT contains an additional contribution

$$r_i \cdot \overline{\text{Size}}(\mathcal{M}_{c_i}),$$

which reflects the cost of computing the multiplicities of the sampled child configurations, i.e., determining how often each configuration appears among the r_i sampled instances. This introduces an explicit dependence on the average configuration size of the sub-model $\mathcal{M}_{c_i}$. In particular, higher up in the feature tree the values $\overline{\text{Size}}(\mathcal{M}_{c_i})$ typically increase, since the corresponding sub-models contain larger subtrees. As a consequence, the recursive cost of UniSample-BT may grow more rapidly near the root of the feature hierarchy, whereas UniSample-UR does not exhibit such a direct dependence in this step.

Effect of the rejection factor. The most significant difference between the algorithms is the rejection factor

$$\frac{1}{q_i},$$

which appears only in the runtime bound of UniSample-BT. As discussed in Section 5.2.2.2, this factor can grow as fast as $r_i!$ in the worst case. Since the rejection step occurs independently at every level of the feature tree, it can dominate the entire runtime of UniSample-BT. In particular, if a sample is rejected higher up in the tree, all rejection loops in the corresponding subtrees must be executed again.

Consequently, although UniSample-BT has a cheaper multiplicity sampling step, the rejection factor can offset this advantage. In models where repeated rejection is likely, UniSample-UR is therefore expected to perform better overall.

Overall Comparison

Summarising the discussion above, we obtain the following qualitative picture:

- UniSample-BT has a simpler and often faster procedure for determining child multiplicities.
- UniSample-UR benefits from avoiding repeated construction of identical child configurations.
- The rejection factor in UniSample-BT can grow factorially and may dominate the runtime.

Taken together, these observations suggest that UniSample-UR is typically the more robust algorithm from an asymptotic perspective, especially in cases where rejection occurs frequently. This situation arises when the multiplicity r_i is small compared to T_{c_i}, since duplicates among the sampled child configurations are then

unlikely and the acceptance probability of the rejection step approaches $1/r_i!$. Consequently, the expected number of iterations of the rejection loop grows approximately as $r_i!$. Of course, this effect becomes practically relevant only when r_i itself is moderately large, since a factor of 2! has little impact on the runtime, whereas a factor of 50! would be prohibitive.

5.2.5.1 Semi-Structural Configurations

The situation changes if we consider semi-structural configurations. As described in Section 4.3, both algorithms can be adapted to this setting. In particular, UniSample-BT then samples ordered lists of child configurations rather than multisets. As a result, the rejection loop disappears entirely. Moreover, the additional size-dependent term

$$r_i \cdot \overline{\mathrm{Size}}(\mathcal{M}_{c_i})$$

vanishes as well, since multiplicities of identical child configurations no longer have to be computed. Consequently, two of the main sources of overhead of UniSample-BT disappear in this setting. This makes the performance comparison between the two algorithms much less clear-cut and further increases the competitiveness of UniSample-BT relative to UniSample-UR.

5.2.5.2 Preprocessing Data Structures

So far, we have also ignored the time and space required to construct the auxiliary data structures used by the two samplers after the DP tables have been computed. Concretely, UniSample-BT uses alias tables in order to sample from the discrete distributions occurring in Step 1 and Step 2 in constant time, whereas UniSample-UR can use prefix-sum arrays in order to speed up the location of blocks and grids during unranking.

We now show that both types of preprocessing have the same asymptotic time and space complexity.

Alias tables for UniSample-BT. For each feature $f \in F$, UniSample-BT requires two kinds of alias tables.

First, in Step 1 the algorithm samples a pair

$$(c, k) \in \lambda_T^G(f) \times \lambda_I^G(f)$$

with probability proportional to $C_f(c, k)$. Hence, for every feature f we build one alias table over the non-zero weights

$$\left\{ C_f(c,k) \;\middle|\; (c,k) \in \lambda_T^G(f) \times \lambda_I^G(f),\ C_f(c,k) > 0 \right\}.$$

The number of entries of this table is therefore at most

$$|\lambda_T^G(f)| \cdot |\lambda_I^G(f)|.$$

Second, in Step 2 the algorithm samples, for each child index $i \in \{1, \ldots, |C_{\mathcal{M}}(f)|\}$ and each reachable state $(c_{\mathrm{rem}}, k_{\mathrm{rem}}) \in \psi_T^G(f) \times \psi_I^G(f)$, a multiplicity

$$r_i \in \lambda_I^F(c_i)$$

with probability proportional to

$$C_f^{i-1}\big(c_{\text{rem}} - \mathbb{1}_{\{r_i>0\}},\, k_{\text{rem}} - r_i\big) \cdot \left(\!\!\binom{T_{c_i}}{r_i}\!\!\right).$$

Thus, for every triple $(f, i, (c, k))$ we build one alias table whose outcomes are exactly the values

$$r \in \lambda_I^F(c_i)$$

with non-zero weight. Consequently, the number of entries in this table is at most

$$|\lambda_I^F(c_i)|.$$

Summing over all features, the total number of stored alias-table entries is therefore bounded by

$$\sum_{f\in F} |\lambda_T^G(f)| \cdot |\lambda_I^G(f)| \;+\; \sum_{f\in F} |\psi_T^G(f)| \cdot |\psi_I^G(f)| \cdot \sum_{i=1}^{|C_{\mathcal{M}}(f)|} |\lambda_I^F(c_i)|.$$

Since the alias method constructs an alias table with n outcomes in $O(n)$ time and $O(n)$ space [86, 87, 88], the total preprocessing time and space for all alias tables is

$$O\left(\sum_{f\in F} |\lambda_T^G(f)| \cdot |\lambda_I^G(f)| \;+\; \sum_{f\in F} |\psi_T^G(f)| \cdot |\psi_I^G(f)| \cdot \sum_{i=1}^{|C_{\mathcal{M}}(f)|} |\lambda_I^F(c_i)|\right). \qquad (\bigstar)$$

Prefix-sum arrays for UniSample-UR. For UniSample-UR, we may preprocess the search structures used in Step 1 and Step 2a of UnrankConfig.

For each feature f, Step 1 determines the block (c, k) containing the sampled rank. To accelerate this search, we store a prefix-sum array over the non-zero block sizes

$$C_f(c, k), \qquad (c, k) \in \lambda_T^G(f) \times \lambda_I^G(f).$$

This requires at most

$$|\lambda_T^G(f)| \cdot |\lambda_I^G(f)|$$

entries for feature f.

Similarly, for Step 2a and each child-pruned model index $i \in \{1, \ldots, |C_{\mathcal{M}}(f)|\}$ and each state $(c, k) \in \psi_T^G(f) \times \psi_I^G(f)$, we may store a prefix-sum array over the grid sizes

$$C_f^{i-1}\big(c - \mathbb{1}_{\{r>0\}},\, k - r\big) \cdot \left(\!\!\binom{T_{c_i}}{r}\!\!\right), \qquad r \in \lambda_I^F(c_i),$$

restricted again to the non-zero values. Thus, the number of entries is at most

$$|\lambda_I^F(c_i)|$$

for each such triple $(f, i, (c, k))$.

Hence, the total size of all prefix-sum arrays is bounded by exactly the same expression as in $(\bigstar)$, namely

$$O\left(\sum_{f\in F} |\lambda_T^G(f)| \cdot |\lambda_I^G(f)| \;+\; \sum_{f\in F} |\psi_T^G(f)| \cdot |\psi_I^G(f)| \cdot \sum_{i=1}^{|C_{\mathcal{M}}(f)|} |\lambda_I^F(c_i)|\right). \qquad (*)$$

Since each prefix-sum array with n entries can be built in $O(n)$ time and stored in $O(n)$ space, the total preprocessing time and space for all prefix arrays is also given by $(*)$.

Conclusion. In total, the auxiliary preprocessing required by UniSample-BT and UniSample-UR has identical asymptotic complexity. Both the alias tables of UniSample-BT and the prefix-sum arrays of UniSample-UR require

$$O\left(\sum_{f\in F}|\lambda_T^G(f)|\cdot|\lambda_I^G(f)| \;+\; \sum_{f\in F}|\psi_T^G(f)|\cdot|\psi_I^G(f)|\cdot\sum_{i=1}^{|C_{\mathcal{M}}(f)|}|\lambda_I^F(c_i)|\right)$$

time and space.

Consequently, the prefix-sum optimization in UniSample-UR does not increase the asymptotic preprocessing time or space requirements.

5.2.6 Eliminating Sorting of Child Configurations

In both UniSample-BT and UniSample-UR, the algorithms described in Section 4.1 generate, for each child feature c_i, a list

$$L_i = [K_{i,1}, \ldots, K_{i,r_i}]$$

of sampled child configurations. Before these configurations are passed to Merge, the list is sorted with respect to the instance order $\trianglelefteq$ in order to ensure that the resulting configuration is canonical. This step contributes the runtime term

$$\sum_{i=1}^{n} r_i^2 \cdot (\overline{\mathrm{Size}}(\mathcal{M}_{c_i}) + |F|),$$

which appears in the runtime bounds of both algorithms (see Table 5.1).

In addition, both algorithms incur the cost

$$\sum_{i=1}^{n} r_i \cdot \overline{\mathrm{Size}}(\mathcal{M}_{c_i}),$$

for merging the sampled child configurations. Furthermore, UniSample-BT requires an additional pass over the sampled configurations in order to determine their multiplicities when computing the acceptance probability. This introduces another size-dependent contribution of the form

$$r_i \cdot \overline{\mathrm{Size}}(\mathcal{M}_{c_i}).$$

In practice, however, all of these size-dependent contributions can be eliminated by switching from the canonical representation of structural configurations to the index-free tree representation introduced in Section 3.2.2.

5.2.6.1 Structural Node Representation

In the index-free representation, a structural configuration is a directed tree whose nodes are labelled by features. In an implementation, such a configuration can therefore be represented by a node object

$$(\mathit{feature}, \{\!\{\, c_1, \ldots, c_k \,\}\!\}),$$

where the c_j are child nodes.

Since every multiset M admits a unique compressed ordered-sequence representation (see Definition 2.13), the child multiset can equivalently be stored as a sorted list of tuples

$$(c_1, m_1), \ldots, (c_t, m_t),$$

where the c_j are distinct child nodes and m_j denotes the multiplicity of c_j.

To do so, however, we have to define an order on the elements. This is done by attaching to each node a unique identifier, where the identifier uniquely represents the subtree structure of the node. In particular, similar to the instance order (Definition 3.12), two nodes receive the same identifier if their subtrees are identical; that is, if their corresponding instance-based sub-configurations are tree-isomorphic (see Theorem 3.15).

Thus, in total, a structural node is represented by a tuple (f, id, L) where

- f is the feature label of the node,
- id is a unique identifier of the node, and
- L is a list of child nodes sorted with respect to their identifiers.

Structural interning. To assign canonical identifiers to structural nodes, we construct them bottom-up and maintain a global *interning table*. The goal of this procedure is to ensure that structurally identical (i.e. tree-isomorphic) sub-configurations receive the same identifier.

When a node with feature label f is constructed, we assume that all of its child nodes have already been interned and therefore already possess identifiers. Using these identifiers we construct a *canonical key* consisting of the feature label f together with the identifiers of its child nodes and their multiplicities.

Concretely, the children are normalised into a list

$$(c_1, m_1), \ldots, (c_t, m_t),$$

where the c_i denote the identifiers of the child nodes, sorted strictly increasingly with respect to their identifiers, and $m_i \geq 1$ denotes the multiplicity of c_i. If several children share the same identifier, they are merged into a single entry whose multiplicity records the number of occurrences. This normalised list provides a canonical representation of the child multiset, and therefore the key uniquely determines the structural node.

The interning table maintains a mapping from canonical keys to identifiers. When a node with feature label f is constructed, its canonical key is computed from the identifiers and multiplicities of its child nodes. The table is then queried with this key: if the key already exists, the associated identifier is reused; otherwise a new identifier is created and the key–identifier pair is inserted into the table. Consequently, structurally identical nodes always receive the same identifier.

Merging configurations. Using the structural interning representation described above, structural configurations are constructed bottom-up. When generating a configuration for a feature f, the sampling algorithm first recursively generates child configurations, each represented by the identifier of its root node.

Merging therefore amounts to constructing a structural node whose children are the root nodes of the sampled child configurations and interning this node to obtain the identifier of the resulting configuration.

To construct the canonical key of this node, the identifiers of the child nodes are first transformed into the normalised list

$$(c_1, m_1), \ldots, (c_t, m_t),$$

where c_j are the distinct child identifiers, sorted strictly increasingly, and m_j denotes the multiplicity of c_j. Here, t denotes the number of distinct child configurations (over all children). In the worst case no two sampled child configurations coincide, and therefore

$$t = \sum_{i=1}^{n} r_i.$$

For UniSample-BT, the sampled child configurations are generated explicitly as a list of length $\sum_{i=1}^{n} r_i$. Hence, before constructing the normalised list, all identifiers must first be scanned and their multiplicities determined. Since each configuration is represented by a node identifier, this can be done using a hash table in expected

$$O\left(\sum_{i=1}^{n} r_i\right)$$

time. Afterwards, the t distinct identifiers are sorted to obtain the canonical key, which requires

$$O(t \log t)$$

time.

For UniSample-UR, by contrast, repeated child configurations are already compressed during unranking, since each distinct rank is unranked at most once and multiplicities are tracked directly. Thus, the normalised list is available without an additional linear pass over all occurrences, and only the sorting of the t distinct identifiers contributes asymptotically, again requiring

$$O(t \log t)$$

time.

Finally, in order to evaluate cross-tree constraints efficiently, we store for each structural node the number of instances of every feature contained in the corresponding subtree. When a node is interned, these counts are computed by summing the feature counts of its children and adding one for the current feature f. Without storing these aggregated counts, checking cross-tree constraints would require traversing the entire configuration.

Computing these aggregated feature counts requires

$$O(|F| \cdot t)$$

time during the interning step, which in the worst case becomes

$$O\left(|F| \cdot \sum_{i=1}^{n} r_i\right).$$

5.2.6.2 Updated Runtime Comparison

Table 5.3 summarises the runtime components of the improved versions of UniSample-BT and UniSample-UR.

In particular, configurations are now manipulated via the identifiers of their root nodes rather than by explicitly comparing entire trees. As a consequence, the operations that previously depended on the sizes of the generated configurations disappear.

In the original algorithms, sorting and merging child configurations required comparing entire subtrees, leading to runtime terms of the form

$$\sum_{i=1}^{n} r_i^2 \cdot (\overline{\text{Size}}(\mathcal{M}_{c_i}) + |F|) \quad \text{and} \quad \sum_{i=1}^{n} r_i \cdot \overline{\text{Size}}(\mathcal{M}_{c_i}).$$

Using structural interning, these operations are replaced by counting and sorting node identifiers. For UniSample-BT, this yields the cost

$$O\left(\sum_{i=1}^{n} r_i + t \log t + |F| \cdot t\right),$$

since all sampled identifiers must first be scanned to determine their multiplicities. For UniSample-UR, the child configurations are already available in compressed form, so the corresponding cost reduces to

$$O(t \log t + |F| \cdot t).$$

In both cases, t denotes the number of distinct child configurations and in the worst case satisfies

$$t = \sum_{i=1}^{n} r_i.$$

The sorting term corresponds to ordering the distinct child identifiers when constructing the canonical key of the node, while the $|F| \cdot t$ term arises from aggregating feature counts in order to support efficient evaluation of cross-tree constraints.

Overall, the improved representation removes all runtime dependencies on the average configuration size $\overline{\text{Size}}(\mathcal{M}_{c_i})$. The remaining costs depend only on the number of sampled child configurations and the number of features.

Size of Node Identifiers

Strictly speaking, the statement that the runtime is independent of the expected configuration size requires some additional justification. In particular, the identifiers assigned by the interning procedure depend on the number of distinct nodes created during the computation and therefore on the total configuration size. If N denotes this number, then identifiers can be represented using $O(\log N)$ bits. Since

	UniSample-BT	UniSample-UR
1.	$\lvert C_{\mathcal{M}}(f)\rvert$	$\log\lvert\lambda_T^G(f)\rvert + \log\lvert\lambda_I^G(f)\rvert + \sum_{i=1}^{n}\log\lvert\lambda_I^F(c_i)\rvert$
2.	$\sum_{i=1}^{n}\frac{1}{q_i}\Big(r_i\cdot\mathbb{E}[\mathbf{RT}(c_i)]+r_i\Big)$	$\sum_{i=1}^{n}\Big((T_{c_i}+r_i)\,r_i + r_i + \frac{r_i\cdot T_{c_i}}{T_{c_i}+r_i-1}\cdot\mathbb{E}[\mathrm{RT}(\boldsymbol{\mathcal{R}}_{c_i},c_i)]\Big)$
3/4.	$O\Big(\sum_{i=1}^{n} r_i + t\log t + \lvert F\rvert\cdot t\Big)$	$O(t\log t + \lvert F\rvert\cdot t)$

Table 5.3: Comparison of the main runtime components of the improved versions of UniSample-BT and UniSample-UR

the number of nodes may grow with the size of the generated configurations, one might therefore argue that operations involving identifiers introduce a logarithmic dependency on the configuration size.

This effect, however, is irrelevant for the runtime analysis. Throughout this section, we assume that all integers occurring during the computation fit into a single machine word and can therefore, under the RAM model assumptions, be compared, copied, and hashed in constant time. Consequently, the use of node identifiers does not introduce any additional dependency on the size of the generated configurations.

5.2.6.3 Future Improvements

The above optimizations, together with prefix-sum arrays, are the only optimizations we use in our CFMTOOLS implementation (see Chapter 6). However, several further improvements are possible.

Eliminating sorting via multiset hashing. The sorting step used to canonicalise the child multiset can be eliminated entirely by indexing the hash table directly by the multiset of child identifiers rather than by its compressed ordered-sequence representation. This can be achieved using a multiset hash function, whose hash values can be computed in time proportional to the size of the multiset and are independent of the order of the elements [93, 94].

Using such a representation, node interning can be implemented in expected $O(t)$ time (or in expected $O(\sum_{i=1}^{n} r_i)$ time if the child configurations are given as an explicit list rather than already in compressed form).

Sparse aggregation of feature counts. The current implementation recomputes aggregated feature counts during the interning step by summing the feature vectors of all child nodes. Although this computation already runs in

$$O(|F|\cdot t),$$

it may be further improved by using sparse feature representations. In particular, a node corresponding to a feature instance can only contain feature instances of

features that appear in the feature tree below it. Consequently, storing feature counts sparsely and propagating only non-zero entries can reduce the aggregation cost to the number of actually occurring features.

5.3 Summary and Outlook

In this chapter, we provided a theoretical evaluation of the algorithms introduced in the previous chapter. In particular, we analysed the correctness and runtime behaviour of UniSample-BT, UniSample-UR, and CTC-UniSample, as well as the preprocessing required for computing the DP tables with ComputeDP.

Correctness. The correctness proofs in Section 5.1 were of different complexity. For UniSample-BT we showed that the algorithm terminates with probability 1, always produces canonical configurations, and samples them uniformly. Since UniSample-BT makes random choices throughout its execution, the proof followed the full control flow of the algorithm and derived the probability distributions induced by its individual sampling steps. Besides establishing correctness, this also provided additional insight into how the different components of the algorithm interact.

By contrast, the correctness proof for UniSample-UR was considerably simpler, since the only random choice occurs in the initial selection of a rank, while the remainder of the algorithm is deterministic unranking. Finally, for CTC-UniSample we showed that rejection sampling on top of a uniform base sampler preserves uniformity over the valid configurations and terminates almost surely whenever the constrained configuration space is non-empty.

Time and space complexity. The main focus of the chapter was the runtime analysis of the sampling algorithms. In particular, in Section 5.2, after deriving the time and space complexity of ComputeDP, we analysed the expected runtime of UniSample-BT, UniSample-UR, and CTC-UniSample. Throughout this analysis, particular care was taken to justify the probabilistic behaviour of the algorithms and the distributions induced by their individual sampling steps. Rather than relying on informal arguments, the analysis explicitly accounts for the effect of algorithmic components such as sorting and rejection sampling.

This careful approach not only ensures the correctness of the derived runtime bounds but also makes the analysis more robust and easier to adapt. In particular, it facilitates extending the analysis to related sampling procedures or alternative configuration semantics, such as semi-structural configurations.

The analysis itself allowed us to identify the dominant costs of the sampling algorithms. In particular, UniSample-BT incurs additional overhead due to the rejection sampling used to generate multisets, whereas UniSample-UR avoids this cost but instead requires more expensive combinatorial unranking operations. The preprocessing overhead for both algorithms, however, is exactly the same. Concretely, the time and space required for constructing the alias tables used by UniSample-BT have the same asymptotic complexity as building the prefix-sum arrays used by UniSample-UR.

For CTC-UniSample, the runtime is largely determined by the rejection rate induced by the cross-tree constraints. In particular, it depends on the restrictiveness

of these constraints, which determines the expected number of sampling attempts required before a valid configuration is obtained.

It is important to note that the presented runtime analysis is intentionally simplified. Throughout the analysis we assumed that all arithmetic operations, including binomial coefficients and multiset counts, can be computed in constant time and that all integers fit into a machine word. In practice this assumption is unrealistic. Even for relatively small CFMs the total number of configurations can grow extremely large (e.g. numbers with millions of digits). Consequently, implementations must rely on arbitrary-precision integers and expensive big-integer arithmetic, which may dominate the actual runtime of the algorithms.

Nevertheless, the simplified analysis remains valuable. It isolates the structural costs of the algorithms and makes it possible to reason about algorithmic trade-offs independent of implementation details. In particular, it led to several optimizations discussed in Section 5.2.6.

Moreover, the analysis shows that neither algorithm is universally superior. Each approach has advantages depending on the structure of the underlying model. The situation becomes even less clear when considering alternative configuration semantics such as semi-structural configurations, where UniSample-BT no longer needs a rejection loop.

Overall, the theoretical evaluation provides a solid foundation for understanding the behaviour of the algorithms and for guiding future improvements. A more realistic runtime analysis that explicitly accounts for big-integer arithmetic and combinatorial computation costs remains an interesting direction for future work.

6. Implementation

This chapter presents the concrete implementation of the algorithms and configuration semantics introduced in Chapters 3 and 4. The implementation is provided as the open-source tool CFMTOOLS, which is available at https://github.com/BaumKM/cfmtools.

We first describe the overall architecture of the system and the main design principles underlying the implementation. Subsequently, we discuss how the algorithms developed in the previous chapters are realized in practice, focusing in particular on the differences between the theoretical algorithm descriptions and their concrete implementation.

It is important to note that the code presented throughout this chapter often consists of excerpts from the actual implementation. In particular, certain parts are intentionally omitted for clarity. Moreover, comments and formatting may differ slightly from the original source code.

6.1 General Architecture and Framework

CFMTOOLS is implemented as a stand-alone, plugin-based CLI application written in Python. Conceptually, it follows a similar design philosophy to the existing CFMTOOLBOX[1], developed at the Institute for Test Validation and Analysis of Software-Intensive Systems (TVA) of the Karlsruhe Institute of Technology (KIT), but differs in several key aspects.

Like the CFMTOOLBOX, the system relies on Python entry points[2] to discover and dynamically load third-party extensions at runtime. This mechanism enables extensibility and allows external developers to integrate new functionality without modifying the core code base.

In contrast to the CFMTOOLBOX, which is built on top of the Typer[3] framework for command parsing, CFMTOOLS uses Python's built-in `argparse`[4] module combined

[1] https://github.com/KIT-TVA/cfmtoolbox
[2] https://packaging.python.org/en/latest/specifications/entry-points/
[3] https://typer.tiangolo.com/
[4] https://docs.python.org/3/library/argparse.html

with a custom parsing layer. This design allows commands to be chained in a pipeline-like manner.

At a high level, the system consists of three main components:

1. a compact core model defining abstractions for features, cardinality intervals, and CFMs,
2. a pipeline-oriented command-line interface with plugin support, and
3. a native backend for performance-critical operations.

6.1.1 Core Model Component

The core model component provides the fundamental abstractions used throughout CFMTOOLS. It defines representations for features, cardinality intervals, and cardinality-based feature models, while deliberately leaving the notion of a configuration unspecified. This design choice reflects the fact that different algorithms operate on different configuration types (e.g. instance-based or multiset-based configurations) and allows the core model to remain independent of any particular configuration semantics. As a result, the framework can be extended to support additional configuration semantics in the future without requiring modifications to the core model representation.

Features are internally represented as integers, enabling the use of array-based data structures instead of hash maps. For type safety, this implementation detail is encapsulated by the custom types `Feature` and `FeatureName`, which are defined via `NewType` based on `int` and `str`, respectively.

To store data associated with features, the implementation provides the wrapper classes `FeatureList` and `FeatureTuple`. These classes encapsulate standard Python `list` and `tuple` objects while allowing direct indexing by `Feature` identifiers. For example, as shown in Listing 6.1, the class `CFM` contains the attribute `feature_names`, which is a `FeatureTuple` storing the `FeatureName` associated with each `Feature`.

Cardinality intervals are represented by the class `CardinalityInterval`, which internally consists of a finite list of `SimpleCardinalityInterval` objects (see Listing 6.2). During construction, intervals are validated, normalized, and compactified by merging overlapping or adjacent ranges. This guarantees a canonical representation and simplifies equality checks and subsequent processing.

CFMs themselves are represented by the class `CFM`. In particular, the feature tree is stored explicitly using both directions of the parent–child relation, which enables efficient tree traversal. More concretely, as shown in Listing 6.3, the attribute `children` represents the parent-to-children relation and stores, for each feature, its child features. Conversely, the attribute `parents` represents the child-to-parent relation and stores, for each feature, its parent feature, with the root being the only feature without a parent.

The correctness of CFMs is enforced during construction. The `CFM` class checks structural properties such as acyclicity, connectivity, and the presence of a single

```python
# --- Feature types ---

Feature = NewType("Feature", int)
FeatureName = NewType("FeatureName", str)

# --- Feature containers ---

class FeatureList(Generic[T]):
    def __init__(self, values: Iterable[T]) -> None: ...
    def __len__(self) -> int: ...
    def __getitem__(self, feature: Feature) -> T: ...
    def __setitem__(self, feature: Feature, value: T) -> None: ...
    def __iter__(self) -> Iterator[T]: ...
    def as_feature_tuple(self) -> FeatureTuple[T]: ...

class FeatureTuple(Generic[T]):
    def __init__(self, values: Iterable[T]) -> None: ...
    def __len__(self) -> int: ...
    def __getitem__(self, feature: Feature) -> T: ...
    def __iter__(self) -> Iterator[T]: ...
```

Listing 6.1: CFMTOOLS Python core: Feature identifiers and feature-indexed containers

```python
@dataclass(frozen=True)
class SimpleCardinalityInterval:
    lower: int
    upper: int | None

    def contains(self, value: int) -> bool: ...
    @property
    def size(self) -> int | None: ...

@dataclass(frozen=True)
class CardinalityInterval:
    _intervals: tuple[SimpleCardinalityInterval, ...]

    def contains(self, value: int) -> bool: ...
    def values(self) -> Iterator[int]: ...
    def bound(self, max_value: int) -> CardinalityInterval: ...

    @property
    def min(self) -> int: ...
    @property
    def max(self) -> int | None: ...
    @property
    def size(self) -> int | None: ...
```

Listing 6.2: CFMTOOLS Python core: Simple and compound cardinality intervals

root, as well as cardinality constraints such as requiring group-type and group-instance cardinalities of leaf features to be $\langle 0, 0 \rangle$ (see Definition 2.5). To simplify model construction, the builder pattern is used: the `CfmBuilder` class allows models to be constructed incrementally using feature names (see Listing 6.4).

6.1.2 CLI Pipeline Component

The CLI of CFMTOOLS is organized around a pipeline abstraction. Each command-line invocation is modelled as an ordered sequence of pipeline steps that operate on a shared mutable execution state, the `PipelineContext`. This context stores global parameters as well as the currently active CFM model and is passed sequentially to all pipeline steps. This design allows multiple processing stages to be composed within a single command invocation without relying on intermediate files.

Pipeline steps are grouped into a small, fixed set of step types, namely `Load`, `Transform`, `Sample`, `Analyze`, and `Export`. Each step type represents a distinct role in the processing workflow and defines how a step is expected to interact with the `PipelineContext`. For example, load steps are responsible for loading a CFM from an arbitrary source (e.g. a database, the file system, or a hard-coded model) and storing it in the context. Transform steps modify the CFM stored in the context, while sampling and analysis steps operate on the model without modifying it. Export steps convert the CFM into an external representation (e.g. files or textual output). Grouping pipeline steps by their type not only enables the extraction of common behaviour and avoids code duplication but also allows the pipeline infrastructure to enforce meaningful execution rules, such as requiring exactly one load step at the beginning of the pipeline.

The pipeline infrastructure itself defines only the execution model and the abstract semantics of each step type. The concrete behaviour of individual pipeline steps is provided by plugins. Each plugin inherits from a step-type-specific base class, for example the `Loader` class when defining a loading step, and is registered using a custom decorator. Multiple plugins may provide implementations for the same step type, enabling support for alternative input formats, transformations, or sampling algorithms.

During startup, the main application discovers all registered plugins, exposes their implementations as command-line commands, and constructs the pipeline based on the parsed arguments.

Listings 6.5 and 6.6 illustrate two concrete pipeline step implementations provided as plugins. Each plugin class is registered using a custom decorator that specifies the command name under which the step becomes available in the CLI. For example, the loader is registered under the name `json` and can therefore be invoked using the corresponding load command. In the case of the sampler, the decorator additionally declares the configuration spaces supported by the algorithm via the `spaces` parameter. This information is used by the sampling pipeline step to determine which algorithms are applicable for a given configuration semantics.

The constructor of the `JsonLoader` further illustrates how command parameters are declared. Parameters can be annotated with metadata that influence their behaviour in the CLI, for example how they are passed on the command line or which help text is displayed. In the example shown, the `allow_implicit_root` parameter is

```python
# --- Cross-tree constraints ---

@dataclass(frozen=True)
class RequireConstraint:
    first_feature: Feature
    first_cardinality: CardinalityInterval
    second_cardinality: CardinalityInterval
    second_feature: Feature

@dataclass(frozen=True)
class ExcludeConstraint:
    first_feature: Feature
    first_cardinality: CardinalityInterval
    second_cardinality: CardinalityInterval
    second_feature: Feature

# --- CFM model ---

@dataclass(frozen=True)
class CFM:

    # Tree structure
    root: Feature
    parents: FeatureTuple[Feature | None]
    children: FeatureTuple[tuple[Feature, ...]]

    # Cardinalities
    feature_instance_cardinalities: FeatureTuple[CardinalityInterval]
    group_instance_cardinalities: FeatureTuple[CardinalityInterval]
    group_type_cardinalities: FeatureTuple[CardinalityInterval]

    # Cross-tree constraints
    require_constraints: list[RequireConstraint]
    exclude_constraints: list[ExcludeConstraint]

    # Feature name mapping
    feature_names: FeatureTuple[FeatureName]

    def features(self) -> Iterator[Feature]: ...
    def traverse_preorder(self) -> Iterator[Feature]: ...
    def traverse_postorder(self) -> Iterator[Feature]: ...

    @property
    def n_features(self) -> int: ...
    @property
    def is_finite(self) -> bool: ...
```

Listing 6.3: CFMTOOLS Python core: CFMs and cross-tree constraints

```python
class CfmBuilder:
    def __init__(self, feature_names: Iterable[str], root: str) -> None
    ↪ : ...

    # Tree construction
    def set_parent(self, child: str, parent: str | None) -> None: ...

    # Cardinalities
    def set_feature_instance_cardinality(
        self, feature: str, card: CardinalityInterval
    ) -> None: ...

    def set_group_instance_cardinality(
        self, feature: str, card: CardinalityInterval
    ) -> None: ...

    def set_group_type_cardinality(
        self, feature: str, card: CardinalityInterval
    ) -> None: ...

    # Cross-tree constraints
    def add_require_constraint(
        self,
        first_feature: str,
        first_cardinality: CardinalityInterval,
        second_cardinality: CardinalityInterval,
        second_feature: str,
    ) -> None: ...

    def add_exclude_constraint(
        self,
        first_feature: str,
        first_cardinality: CardinalityInterval,
        second_cardinality: CardinalityInterval,
        second_feature: str,
    ) -> None: ...

    # Finalization
    def build(self) -> CFM: ...
```

Listing 6.4: CFMTOOLS Python core: CFM builder for constructing a CFM

defined as a boolean flag and can therefore be enabled in the CLI using the option `--allow-implicit-root`.

6.1.3 Native Backend

For performance-critical tasks, CFMTOOLS provides an optional native backend written in Rust and integrated into the Python application via PyO3[5] bindings. PyO3 allows Rust code to be compiled into a shared library that can be imported directly by the Python interpreter. This enables performance-sensitive functionality to be invoked from Python while the underlying logic is executed as optimized native machine code.

Currently, all algorithms presented in Chapters 3 and 4 are implemented in the native Rust backend. In particular, this includes both the construction of the DP tables and the sampling algorithms themselves. Implementing these components in Rust enables a fair evaluation in Chapter 7, since other performance-critical components, such as SAT solvers, are typically implemented in low-level compiled languages such as C or C++, enabling highly efficient execution close to the hardware.

[5] https://pyo3.rs/v0.28.0/

```python
@loader("json")
class JsonLoader(Loader):
    """
    Load a CFM model from JSON.
    """

    @classmethod
    @override
    def get_command_help(cls) -> str:
        return "Load a CFM model from a JSON file."

    @classmethod
    @override
    def get_command_description(cls) -> str: ...

    def __init__(
        self,
        path: Annotated[
            Path,
            ParamHelp("Path to the input JSON file."),
        ],
        allow_implicit_root: Annotated[
            bool,
            ParamBehavior.FLAG,
            ParamHelp(
                "Allow the JSON model to omit an explicit root feature.
    ↪ "
            ),
        ] = False,
    ):
        self.path = path
        self.allow_implicit_root = allow_implicit_root

    def load(self) -> CFM:
        if not self.path.exists():
            raise FileNotFoundError(self.path)

        data = json.loads(self.path.read_text(encoding="utf-8"))
        return _parse_cfm_from_json(data, self.allow_implicit_root)
```

Listing 6.5: CFMTOOLS CLI pipeline: Loader plugin for importing CFMs from JSON files

```python
@sampler("backtracking", spaces={"structural"})
class BacktrackingSampler(SampleAlgorithm):
    """
    Uniform backtracking sampler (Rust backend).
    """

    @override
    def run_once(
        self,
        model: CFM,
        *,
        space: str,
        samples: int,
        seed: int | None,
        out_path: Path,
    ) -> None:
        raise NotImplementedError()

    @override
    def run_benchmark(
        self,
        model: CFM,
        *,
        space: str,
        samples: int,
        runs: int,
        seed: int,
        out_path: Path,
        constrained_space: bool,
    ) -> None:

        structural = model.to_native().structural()

        result = structural.benchmark_backtracking_sampler(
            runs=runs,
            samples=samples,
            seed=seed,
            calculate_constrained_space_size=constrained_space,
        )

        out_path.parent.mkdir(parents=True, exist_ok=True)

        with out_path.open("w", encoding="utf-8") as f:
            json.dump(result, f, indent=2)
```

Listing 6.6: CFMTOOLS CLI pipeline: Sampler plugin implementing UNISAMPLE-BT

6.2 Algorithm Implementations in Rust

In the following, we present the concrete implementations of the algorithms introduced in the previous chapters in Rust. While the underlying logic closely follows the algorithmic descriptions, the structure of the code does not correspond to their presentation one-to-one. Function names are adapted to language-specific conventions, and several algorithms are reorganized to improve code structure and maintainability. In particular, the algorithm CTC-UniSample (Algorithm 4.5), which was introduced as a higher-order algorithm that conceptually takes a base sampler as a parameter, is not implemented directly. Instead, its logic is integrated into the respective concrete sampling algorithms.

A key design decision of the Rust backend is its independence from the surrounding Python application. Consequently, all relevant abstractions defined in the Python core module are reimplemented in Rust. This includes, in particular, representations of CFMs, features, and cardinality intervals, as well as the builder pattern used for constructing CFMs.

Another central design goal of the backend is the ability to handle extremely large configuration spaces. Even for relatively small CFMs, the number of configurations can exceed the maximum values representable by 64-bit or even 128-bit integers. For example, the synthetic *chain* model with parameters $h = 4$ and $k = 5$ used in the evaluation dataset (see Section 7.2.3 and Figure 7.1), which has at most five features and cardinality intervals bounded by five, already exceeds this range. In fact, one CFM in the dataset has an unconstrained structural configuration space whose size is a number with more than 14 million digits.

For this reason, all algorithms that operate on configuration counts use arbitrary-precision arithmetic. Concretely, configuration counts are represented using the type `Integer` from the `rug` crate[6], which provides arbitrary-precision integers backed by the GNU Multiple Precision Arithmetic Library (GMP) library[7]. GMP is a widely used high-performance library for arbitrary-precision arithmetic that implements large integers, rational numbers, and floating-point numbers. Since `rug` relies on native GMP bindings, the current implementation of cfmtools is supported only on Linux.

6.2.1 Representation of Structural Configurations

As discussed in the theoretical evaluation, particularly in Section 5.2.6, the canonical instance-based representation of structural configurations is not ideal in terms of runtime. In particular, it requires managing feature-instance indices, sorting configurations before merging, and performing full traversals for equality checks. Instead, it is significantly more efficient to use the index-free tree representation introduced in Section 3.2.2.

In addition to the concepts described in Section 5.2.6, cfmtools employs a directed acyclic graph (DAG)-compressed representation in which identical subtrees of configurations are stored only once and shared across the configuration [76, 95, 96].

[6] https://crates.io/crates/rug
[7] https://gmplib.org/

Structural configuration objects. The main data structures used to represent structural configurations are shown in Listing 6.7. The core representation is a tree of `StructuralNode`s. Each `StructuralNode` stores the feature label, a list of child entries, and a unique identifier `NodeId`. The children are represented by `ChildEntry` objects that store a reference to the child node together with its multiplicity. This representation allows repeated subtrees to be represented compactly by storing the subtree only once together with a multiplicity counter.

Nodes are referenced using `Arc` pointers rather than embedding child nodes directly. This yields a DAG-compressed representation in which structurally identical subtrees can be shared across the configuration. Consequently, each subtree is represented by a single node that may have multiple incoming references.

The identifier `NodeId` uniquely identifies the subtree rooted at a node. In particular, two nodes with the same identifier correspond to tree-isomorphic sub-configurations. Equality checks between structural configurations therefore reduce to comparing the identifiers of their root nodes.

The struct `StructuralConfiguration` is a lightweight wrapper around the root node of such a configuration. In addition to storing the root, it maintains aggregate information about the configuration, namely the total size and a feature-indexed vector containing the multiplicity of each feature. These aggregate values are primarily used for efficient checks of cross-tree constraints. In principle, they could also be computed by traversing the configuration tree.

```rust
pub struct StructuralConfiguration {
    root: Arc<StructuralNode>,
    feature_counts: FeatureVec<usize>,
    size: usize,
}

pub struct StructuralNode {
    id: NodeId,
    feature: Feature,
    children: Vec<ChildEntry>,
}

pub struct ChildEntry {
    node: Arc<StructuralNode>,
    multiplicity: usize,
}

pub struct NodeId {
    id: usize,
}
```

Listing 6.7: CFMTOOLS Rust core: Structural configuration representation

Structural builder and interning. To obtain unique node identifiers, structural configurations and nodes are constructed using the builder pattern. In particular,

the `StructuralBuilder`, shown in Listing 6.8, is responsible for constructing structural configurations and employs the global interning table approach described in Section 5.2.6.

Concretely, the builder maintains an interning table stored in the field `table`. This table maps `NodeKeys` (Listing 6.9) to nodes. Whenever a new node is created, its children are first normalized before constructing the interning key. This normalization step removes zero multiplicities, sorts the child identifiers, and merges identical identifiers by accumulating their multiplicities. As a result, the representation of the multiset of child configurations becomes canonical and independent of the construction order.

The resulting key is then used to check whether the node already exists in the interning table. If the node exists, it is reused. Otherwise, a new node is allocated and inserted into the table. The core procedure implementing this logic is shown in Listing 6.10.

In addition to constructing nodes, the builder maintains aggregate information for each node. In particular, it stores the size of the subtree and a feature-indexed vector containing the number of occurrences of each feature in the subtree. These values are computed during the interning step by combining the aggregate values of the child nodes. This information is later used to initialize the fields `feature_counts` and `size` in the `StructuralConfiguration` (Listing 6.7).

```rust
pub struct StructuralBuilder {
    next_id: usize,
    num_features: usize,
    table: HashMap<Arc<NodeKey>, Arc<StructuralNode>>,
    aggregate_values: HashMap<NodeId, AggregateValues>,
    insert_log: Vec<(Arc<NodeKey>, NodeId)>,
}

impl StructuralBuilder {

    pub fn begin_node(&mut self, feature: &Feature) -> NodeBuilder {
    ↪ ... }

    pub fn finish_node(&mut self, node: NodeBuilder) -> Arc<
    ↪ StructuralNode> { ... }

    pub fn finish(self, root: Arc<StructuralNode>) ->
    ↪ StructuralConfiguration { ... }

    pub fn checkpoint(&self) -> BuilderCheckpoint { ... }

    pub fn rollback(&mut self, cp: &BuilderCheckpoint) { ... }

}
```

Listing 6.8: CFMTOOLS Rust core: Structural configuration builder

```rust
#[derive(Clone, Hash, PartialEq, Eq)]
struct NodeKey {
    feature: Feature,
    children: Vec<NodeKeyChildEntry>,
}

#[derive(Clone, Hash, PartialEq, Eq)]
struct NodeKeyChildEntry {
    node_id: NodeId,
    multiplicity: usize,
}
```

Listing 6.9: CFMTOOLS Rust core: Key used for node interning

Uniqueness of node identifiers. An important property of the node identifiers is that their uniqueness guarantee holds only for nodes constructed using the same `StructuralBuilder`. In particular, two nodes have the same identifier if and only if their subtrees are identical; however, this property is guaranteed only within the scope of a single builder instance.

The reason is that node identifiers are assigned sequentially as nodes are interned. Consequently, the numeric value of a node identifier depends on the order in which nodes are created. For example, if a leaf node with feature f is interned first in an empty builder, it receives identifier 0. However, if instead a leaf node with feature $g \neq f$ were interned first, that node would receive identifier 0.

To preserve the identifier uniqueness invariant during sampling, the same builder instance is passed recursively throughout the sampling procedures, such as in UniSample-BT. This ensures that all nodes are interned within the same global table.

For this reason, the builder additionally provides a rollback mechanism. This mechanism allows the builder state to be restored after speculative node construction, which is required to support the rejection loops used by UniSample-BT.

```rust
fn intern_node(
    &mut self,
    feature: Feature,
    mut children: Vec<(Arc<StructuralNode>, usize)>,
) -> Arc<StructuralNode> {
    // normalize
    children.retain(|(_, multiplicity)| *multiplicity > 0);
    children.sort_unstable_by_key(|(g, _)| g.id.to_usize());

    // merge identical node identifiers
    let mut write = 0;
    for read in 0..children.len() {
        if write == 0 || children[read].0.id != children[write - 1].0.
    ↪ id {
            children[write] = children[read].clone();
            write += 1;
        } else {
            children[write - 1].1 += children[read].1;
        }
    }
    children.truncate(write);

    // build interning key
    let key = NodeKey { ... };

    if let Some(existing) = self.table.get(&key) {
        return existing.clone();
    }

    let id = self.alloc_id();
    let node = Arc::new(StructuralNode { id, feature, ... });

    self.table.insert(key, node.clone());
    node
}
```

Listing 6.10: CFMTOOLS Rust core: Interning of nodes

6.2.2 ComputeDP

Listing 6.11 shows the implementation of ComputeDP (Algorithm 4.1). The implementation closely follows the theoretical formulation. However, it incorporates several practical optimizations discussed in Section 5.2.1 to reduce the runtime of the naive algorithm.

In particular, the implementation applies the following optimizations:

- *first-child initialization*, which directly constructs the first DP layer ($i = 1$) instead of computing it via the general recursion,
- *bounded iteration ranges*,
- *batch computation of multiset coefficients* $\left(\!\binom{T_{c_i}}{r}\!\right)$ for all $r \in \lambda_I^F(c_i)$, and
- *forward state updates*, where contributions are propagated from layer i to layer $i + 1$.

```rust
// DP table
let mut table = DpTable::new(n + 1, c_max + 1, k_max + 1, Integer::from
    ↪ (0));
*table.get_mut(0, 0, 0) = Integer::from(1);
// --- first child: direct initialization ---
let child = children[0];
let total_child = &total_configuration_counts[child];
let rs = &child_multiplicities[0];
let weights = Multiset::count_multisets_batched(total_child, rs);
for (&r, weight) in rs.iter().zip(weights) {
    let c_rem = usize::from(r > 0);
    let k_rem = r;
    if k_rem > k_max || c_rem > c_max { break; }
    *table.get_mut(1, c_rem, k_rem) += weight;
}
// --- compute prefix bounds ---
let mut max_k_prefix = Vec::with_capacity(children.len() + 1);
max_k_prefix.push(0);
for mults in &child_multiplicities {
    let max_r = *mults.last().unwrap();
    let next = max_k_prefix.last().unwrap() + max_r;
    max_k_prefix.push(next);
}
// --- main DP (forward updates) ---
for (i, &child) in children.iter().enumerate().skip(1) {
    let num_child_configs = &total_configuration_counts[child];
    let rs = &child_multiplicities[i];
    let weights =
        Multiset::count_multisets_batched(num_child_configs, rs);
    let weighted_rs: Vec<(usize, Integer)> =
        rs.iter().copied().zip(weights).collect();

    for c_prev in 0..=c_max.min(i) {
        for k_prev in c_prev..=max_k_prefix[i].min(k_max) {
            let base_count = table.get(i, c_prev, k_prev);
            if base_count.is_zero() { continue; }
            for &(r, ref weight) in &weighted_rs {
                let k_rem = k_prev + r;
                let c_rem = c_prev + usize::from(r > 0);
                if k_rem > k_max || c_rem > c_max { break; }
                *table.get_mut(i + 1, c_rem, k_rem) +=
                    base_count * weight;
            }
        }
    }
}
```

Listing 6.11: CFMTOOLS Rust core: Implementation of COMPUTEDP

6.2.3 UniSample-BT

Listings 6.12 and 6.13 show the implementation of UniSample-BT (Algorithm 4.2). The implementation follows the theoretical description closely but incorporates several practical implementation choices.

First, all discrete random choices are performed using precomputed alias tables (see Section 5.2.2.1). In particular, the group cardinalities (c, k) and the child multiplicities r_i are sampled using the alias method. The sampling of (c, k) is shown in line 13 of Listing 6.13, while lines 15–22 sample the multiplicities r_i in reverse order.

Second, configurations are constructed using the `StructuralBuilder` introduced in Section 6.2.1. The builder also provides checkpoint and rollback operations, which are used to efficiently revert partial constructions when a multiset rejection occurs.

The final difference concerns the computation of the multiset acceptance probability. For a child feature c_i with multiplicity r_i, the acceptance probability of a sampled list of configurations is

$$p = \frac{\prod_j m_j!}{r_i!},$$

where m_j denotes the multiplicity of the j-th distinct configuration in the list.

Rather than computing p directly, the implementation evaluates the probability in log-space. A direct evaluation is not suitable, since in IEEE-754 double precision arithmetic [97] the factorial overflows at 171!, with 170! being the largest representable finite value. Moreover, naive iterative multiplication of fractional factorial terms (i.e. computing $1/r_i!$ as $1/r_i \cdot 1/(r_i - 1) \cdots 1$) leads to the accumulation of rounding errors [98, 99, 100].

To avoid overflow and improve numerical stability, the rejection loop is therefore implemented in log-space. Concretely, we compute

$$\log p = \sum_j \log(m_j!) - \log(r_i!),$$

as shown in Listing 6.14. Subsequently, a uniform random value $u \in [0, 1)$ is sampled, and the multiset is accepted if

$$\log(u) \leq \log p.$$

To evaluate these expressions efficiently, the implementation maintains a dynamically growing table of logarithmic factorials using the helper structure `LogFactorials`. This structure stores values of $\log(z!)$ for all $z \leq Z$, where the bound Z is extended on demand whenever a larger multiplicity is encountered during recursive sampling. Extending the table from Z to $Z' > Z$ is performed incrementally using the recurrence $\log(z!) = \log((z-1)!) + \log z$, ensuring that previously computed values are reused. Since evaluating $\log(z!)$ inherently requires all lower-order terms, this incremental extension introduces no additional asymptotic cost while avoiding repeated logarithm evaluations inside the rejection loop.

```rust
fn sample<R: Rng>(
    &self,
    sampler_cache: &BacktrackingSamplerCache,
    rng: &mut CompoundRng<R>,
) -> SampleResult<StructuralConfiguration, BacktrackingStatistics> {

    let root = self.config_space.cfm().root();

    loop {
        let mut builder = StructuralBuilder::new(self.config_space.cfm
    ↪ ().size());

        let root_group = self.sample_recursively(
            root,
            sampler_cache,
            &mut LogFactorials::new(0),
            &mut builder,
            rng,
        );

        let cfg = builder.finish(root_group);

        if self.config_space.cfm()
            .satisfies_cross_tree_constraints(cfg.feature_counts())
        {
            return SampleResult { value: cfg };
        }

    }
}
```

Listing 6.12: CFMTOOLS Rust core: Top-level implementation of UNISAMPLE-BT

```rust
fn sample_recursively<R: Rng>(
    &self,
    f: &Feature,
    cache: &BacktrackingSamplerCache,
    log_factorials: &mut LogFactorials,
    builder: &mut StructuralBuilder,
    rng: &mut CompoundRng<R>,
) -> Arc<StructuralNode> {
    if self.config_space.cfm().is_leaf(f) {
        return builder.finish_node(builder.begin_node(f));
    }
    // sample (c,k)
    let &(mut c_rem, mut k_rem) = cache.group_tables[f].as_ref().unwrap
    ↪ ().sample(rng);
    // sample multiplicities r_i backwards
    let children = self.config_space.cfm().children(f);
    let mut r = vec![0; children.len()];
    for i in (0..children.len()).rev() {
        let ri = *cache.multiplicity_tables[f].table(i+1, c_rem, k_rem)
    ↪ .unwrap().sample(rng);
        r[i] = ri;
        if ri > 0 { c_rem -= 1; }
        k_rem -= ri;
    }
    // generate child configurations
    let mut node = builder.begin_node(f);
    for (i, child) in children.enumerate() {
        let ri = r[i];
        if ri == 0 { continue; }
        let accepted = loop {
            let cp = builder.checkpoint();
            let list: Vec<_> = (0..ri).map(|_| {
                self.sample_recursively(child, cache, log_factorials,
    ↪ builder, rng)
            }).collect();
            let mults = builder.count_configurations(&list);
            if Self::accept_multiset(&mults, ri, log_factorials, rng) {
                break list;
            }
            builder.rollback(&cp);
        };
        node = node.add_children(accepted);
    }
    builder.finish_node(node)
}
```

Listing 6.13: CFMTOOLS Rust core: Recursive implementation of UNISAMPLE-BT

```rust
fn accept_multiset<R: Rng>(
    multiplicities: &[usize],
    r: usize,
    log_factorials: &mut LogFactorials,
    rng: &mut R,
) -> bool {

    log_factorials.ensure(r);
    let mut log_p = -log_factorials.log_fact(r);

    for &m in multiplicities {
        log_p += log_factorials.log_fact(m);
    }

    let u: f64 = rng.random();
    u.ln() <= log_p
}
```

Listing 6.14: CFMTOOLS Rust core: Log-space computation of the multiset rejection probability

6.2.4 UniSample-UR and UnrankConfig

Listings 6.15 and 6.16 show the implementations of the algorithms UniSample-UR (Algorithm 4.3) and UnrankConfig (Algorithm 4.4).

As discussed in Section 5.2.3, the unranking procedure uses prefix-sum arrays together with binary search to efficiently determine in which block and grid a given rank lies. Concretely, this functionality is implemented by the methods `find_block` and `find_grid`, which are invoked in lines 17 and 25 of Listing 6.16, respectively.

```rust
fn sample<R: Rng>(
    &self,
    config_cache: &S::Cache,
    _sampler_cache: &Self::SamplerCache,
    rng: &mut CompoundRng<R>,
) -> SampleResult<S::Config, RankingStatistics> {
    let count = self.config_space.count(config_cache);
    assert!(!count.is_zero());

    let mut sample_rejections = 0;

    loop {
        let rank = count.random_below_ref(rng.bignum_rng_mut()).
    ↪ complete();
        let config = self.config_space.unrank(config_cache, &rank);

        let counts = config.feature_counts(&self.cfm);
        if self.cfm.satisfies_cross_tree_constraints(counts) {
            return SampleResult {
                value: config,
                statistics: RankingStatistics { sample_rejections },
            };
        }
        sample_rejections += 1;
    }
}
```

Listing 6.15: CFMTOOLS Rust core: Implementation of UNISAMPLE-UR

```rust
fn unrank_recursively(
    &self,
    cfm: &CFM,
    builder: &mut StructuralBuilder,
    f: &Feature,
    global_rank: &Integer,
) -> Arc<StructuralNode> {
    // Step 1: choose (c,k) block
    let (mut block_local_rank, mut c_rem, mut k_rem) =
        self.cum_block_sizes[f].find_block(global_rank);
    let mut node_builder = builder.begin_node(f);
    // Step 2: backward unranking of children
    for (i0, child) in cfm.children(f).enumerate().rev() {
        let i = i0 + 1;
        // split grid rank into fiber index and remainder
        let (grid_local_rank, r_i, fiber_size) = self.cum_grid_sizes[f
    ↪ ].find_grid(i, c_rem, k_rem, &block_local_rank);
        let fiber_local_rank =
            (&grid_local_rank % fiber_size.as_ref()).complete();
        let fiber_index = &grid_local_rank / fiber_size.as_ref();
        block_local_rank = fiber_index.complete();
        // unrank multiset of child configurations
        let child_ranks: Vec<Integer> = Multiset::unrank(&self.
    ↪ total_config_counts()[child], r_i, &fiber_local_rank);
        let len = child_ranks.len();
        let mut i = 0;
        // recursively build children
        while i < len {
            let child_rank = &child_ranks[i];
            let mut multiplicity = 1usize;
            let mut j = i + 1;
            while j < len && &child_ranks[j] == child_rank {
                multiplicity += 1;
                j += 1;
            }
            let child_group = self.unrank_recursively(cfm, builder,
    ↪ child, child_rank);
            node_builder =
                node_builder.add_child(child_group, multiplicity);
            i = j;
        }
        c_rem -= usize::from(r_i > 0);
        k_rem -= r_i;
    }
    builder.finish_node(node_builder)
}
```

Listing 6.16: CFMTOOLS Rust core: Implementation of UNRANKCONFIG

6.3 SAT-Based Reference Implementation

To evaluate the proposed sampling and ranking algorithms, we additionally implement a reference method based on a SAT-based CSP solver, which serves as a competitive baseline for assessing the performance of our approach. This reference enables a direct comparison between the DP-based counting method introduced in this work and a state-of-the-art, general-purpose CSP solving approach.

The reference implementation is realized using the CP-SAT solver provided by Google OR-Tools[8]. CP-SAT is a hybrid constraint programming (CP) and optimization solver built on top of a modern conflict-driven clause learning (CDCL) SAT engine. It extends SAT-based reasoning with integer variables, CP propagation techniques, and optimization capabilities.

The encoding of CFMs and their configuration semantics follows the work of Zhang [18] and Eger [21]. As already discussed in Section 3.4.3, both approaches effectively encode the *semi-structural* (equivalently, *semi-canonical*) configuration semantics, rather than the full instance-based or structural/canonical configuration semantics.

More specifically, Zhang employs the SMT solver Z3[9], which supports a rich set of background theories, including linear integer arithmetic, bit-vectors, arrays, and uninterpreted functions. In contrast, our implementation, like that of Eger, relies on the CSP solver CP-SAT, which supports Boolean and bounded integer variables together with linear constraints, but does not provide native support for the broader range of background theories available in SMT solvers. However, Eger additionally proposes a symmetry-reduction mechanism that excludes valid semi-structural configurations, which we therefore do not adopt.

Encoding a CFM in CP-SAT

Let $\mathcal{M} = (\prec_F, \lambda_I^F, \lambda_I^G, \lambda_T^G, \phi_R, \phi_X)$ be a CFM over a finite feature set F with root f_r. As in Chapters 4 and 5, we assume that all feature-instance cardinalities are finite. We encode $\mathcal{M}$ into a CP-SAT model in the following steps.

1. **Finite instance bounds.** For each feature $f \in F$, we derive a finite upper bound $B_f \in \mathbb{N}_0$ on the number of instances of f that can occur in any instance-based configuration. We set $B_{f_r} = 1$ and, for every edge $(p, c) \in \prec_F$,

$$B_c \;\coloneqq\; B_p \cdot \max \lambda_I^F(c).$$

   This bound is correct because the root feature has exactly one instance, and each instance of a parent feature $p \in F$ can have at most $\max \lambda_I^F(c)$ instances of a child feature c. Hence, across all B_p parent instances, there can be at most B_c instances of c in total.

2. **Boolean variables for feature instances.** For each $f \in F$ and each index $i \in \{0, \ldots, B_f - 1\}$, we create a Boolean variable

$$\texttt{instance_f_i} \in \{0, 1\},$$

[8] https://developers.google.com/optimization
[9] https://www.microsoft.com/en-us/research/project/z3-3/

where $\texttt{instance_f_i} = 1$ indicates that the i-th potential instance of feature f is selected (i.e., present in the configuration). For the root feature, we fix

$$\texttt{instance_f}_r\texttt{_0} = 1,$$

since it must always be present.

In contrast to the rest of the thesis, we do not assign potential instances consecutively. Instead, we assign each parent instance its own block of potential child instances; in particular, this permits gaps.

Concretely, for every edge $(p, c) \in \prec_F$ and every parent-instance index $i \in \{0, \ldots, B_p - 1\}$, let

$$\mathrm{start}(p, i, c) \;\coloneqq\; i \cdot \max \lambda_I^F(c).$$

The potential child instances of c under the i-th potential instance of p are exactly those with indices

$$\mathrm{Range}(p, i, c) \coloneqq \Big\{\mathrm{start}(p, i, c), \ldots, \mathrm{start}(p, i, c) + \max \lambda_I^F(c) - 1\Big\}.$$

3. **Structural constraints (tree semantics).** If a child instance is selected, then its parent instance must also be selected. Thus, for all parent–child pairs $(p, c) \in \prec_F$, all parent instance indices $i \in \{0, \ldots, B_p - 1\}$, and all child indices $k \in \mathrm{Range}(p, i, c)$,

$$\texttt{instance_c_k} \;\leq\; \texttt{instance_p_i}.$$

4. **Cardinality interval encoding.** Cardinality constraints of the form $x \in I$, where x is an integer variable and I is a cardinality interval, are encoded as follows.

   Conditional activation. All cardinality constraints are guarded by an activity variable $\texttt{active} \in \{0, 1\}$. If $\texttt{active} = 0$, the constrained variable is forced to zero; if $\texttt{active} = 1$, the variable must take a value in the given cardinality interval I:

$$\texttt{active} = 0 \Rightarrow x = 0, \qquad \texttt{active} = 1 \Rightarrow x \in I.$$

   Interval encoding. Let I consist of the simple cardinality intervals $I_1, \ldots, I_k$ (k may be 1). We introduce Boolean selector variables $\texttt{sel_1}, \ldots, \texttt{sel_k} \in \{0, 1\}$. Each selector enforces the bounds of its corresponding interval,

$$\texttt{sel_t} = 1 \Rightarrow x \in I_t \qquad (t = 1, \ldots, k),$$

   and we require

$$\sum_{t=1}^{k} \texttt{sel_t} = \texttt{active}.$$

   Thus, exactly one interval is selected when the variable is active, and none when it is inactive.

5. **Feature-instance cardinalities.** For each parent feature $p \in F$, each parent-instance index $i \in \{0, \ldots, B_p - 1\}$, and each child feature $c \in C_{\mathcal{M}}(p)$, we introduce an integer variable

$$\texttt{feature_instance_c_under_p_i} \in \Big\{0, \ldots, \max \lambda_I^F(c)\Big\},$$

counting the number of instances of feature c below the i-th instance of p. It is defined by

$$\texttt{feature_instance_c_under_p_i} = \sum_{k \in \mathrm{Range}(p,i,c)} \texttt{instance_c_k}.$$

The admissible values of `feature_instance_c_under_p_i` are constrained by the feature-instance cardinality $\lambda_I^F(c)$. This constraint is encoded as a cardinality-interval constraint (see Item 4), using the parent-instance variable `instance_p_i` as the activity guard:

$$\texttt{instance_p_i} = 0 \Rightarrow \texttt{feature_instance_c_under_p_i} = 0,$$

$$\texttt{instance_p_i} = 1 \Rightarrow \texttt{feature_instance_c_under_p_i} \in \lambda_I^F(c),$$

since we only need to enforce the cardinality constraint if the parent is selected.

6. **Group-instance cardinalities.** For each feature $p \in F$ and each instance index $i \in \{0, \ldots, B_p - 1\}$, we introduce an integer variable

$$\texttt{group_instance_p_i} \in \left\{0, \ldots, \sum_{c \in C_{\mathcal{M}}(p)} \max \lambda_I^F(c)\right\},$$

which counts the total number of child instances attached to the i-th instance of p. Formally,

$$\texttt{group_instance_p_i} = \sum_{c \in C_{\mathcal{M}}(p)} \texttt{feature_instance_c_under_p_i}.$$

Again, the admissible values of this variable are constrained by $\lambda_I^G(p)$, where the guard is `instance_p_i` (see Item 4). Thus,

$$\texttt{instance_p_i} = 0 \;\Rightarrow\; \texttt{group_instance_p_i} = 0,$$

$$\texttt{instance_p_i} = 1 \;\Rightarrow\; \texttt{group_instance_p_i} \in \lambda_I^G(p).$$

7. **Group-type cardinalities.** For each feature $p \in F$, each instance index $i \in \{0, \ldots, B_p - 1\}$, and each child feature $c \in C_{\mathcal{M}}(p)$, we introduce a Boolean variable

$$\texttt{is_c_present_under_p_i} \in \{0, 1\},$$

which indicates whether at least one instance of feature c is attached to the i-th instance of p.

This condition is encoded using a max-equality constraint:

$$\texttt{is_c_present_under_p_i} = \max_{k \in \mathrm{Range}(p,i,c)} \texttt{instance_c_k}.$$

To count the number of distinct child feature types present under the i-th instance of p, we introduce an integer variable

$$\texttt{group_type_p_i} = \sum_{c \in C_{\mathcal{M}}(p)} \texttt{is_c_present_under_p_i}.$$

As before, we enforce $\texttt{group_type_p_i} \in \lambda_T^G(p)$ via the cardinality interval encoding from Item 4. The guard is again the parent instance:

$$\texttt{instance_p_i} = 0 \;\Rightarrow\; \texttt{group_type_p_i} = 0,$$

$$\texttt{instance_p_i} = 1 \;\Rightarrow\; \texttt{group_type_p_i} \in \lambda_T^G(p).$$

8. **Cross-tree constraints** (ϕ_R, ϕ_X)**.** For each feature $f \in F$, we introduce an integer variable

$$\texttt{cnt_f} \in \{0, \ldots, B_f\}, \qquad \texttt{cnt_f} \;=\; \sum_{i=0}^{B_f - 1} \texttt{instance_f_i},$$

which counts the total number of selected instances of feature f.

Each require constraint $(a, \alpha, \beta, b) \in \phi_R$ is encoded as the implication

$$(\texttt{cnt_a} \in \alpha) \;\Rightarrow\; (\texttt{cnt_b} \in \beta).$$

Each exclude constraint $(a, \alpha, \beta, b) \in \phi_X$ is encoded as

$$\neg\Big((\texttt{cnt_a} \in \alpha) \;\wedge\; (\texttt{cnt_b} \in \beta)\Big),$$

or, equivalently,

$$(\texttt{cnt_a} \notin \alpha) \;\vee\; (\texttt{cnt_b} \notin \beta).$$

Membership predicates of the form $(\texttt{cnt_f} \in \alpha)$ are reified using a selector-based encoding that introduces a Boolean variable $b \in \{0, 1\}$ such that

$$b = 1 \;\Leftrightarrow\; \texttt{cnt_f} \in \alpha.$$

The concrete implementation is shown in Listing 6.18.

9. **Symmetry breaking.** With the current definition, we allow gaps within each feature-instance block. For example, we currently permit $\texttt{instance_c_t+1} = 1$ while $\texttt{instance_c_t} = 0$, even if both variables refer to potential instances under the same parent instance.

To eliminate these symmetries, we enforce dense usage of indices within each block. Concretely, for every edge $(p, c) \in \prec_F$ and every parent-instance index $i \in \{0, \ldots, B_p - 1\}$, we require

$$\forall t \in \mathrm{Range}(p, i, c) \text{ with } t + 1 \in \mathrm{Range}(p, i, c) : \\ \texttt{instance_c_t+1} \;\leq\; \texttt{instance_c_t}.$$

Thus, if the $(t{+}1)$-st instance of c under the i-th instance of p is selected, then the t-th one must also be selected.

Remark. In contrast to the rest of this work, we use 0-based indexing for feature instances in the SAT encoding; this is purely notational and has no semantic impact.

Moreover, as mentioned above, we do not assign feature-instance indices densely (i.e., we do not assign them from $\mathrm{Inst}(M)$), but instead choose them from the corresponding block $\mathrm{Range}(p, i, c)$ (see Item 2). In particular, it can happen that, for example, the 10-th instance of a feature is selected while instances $0, \ldots, 9$ are not. To obtain a valid instance-based configuration from a SAT solver solution, we therefore perform a DFS traversal of the solution, visiting smaller indices first and renaming indices using a feature-global counter (see Theorems 3.18 and 3.27).

Finally, the above encoding enforces only a *semi-canonical configuration* (equivalently, a semi-structural configuration). In particular, it does not enforce sibling index monotonicity (see Section 3.3).

The reason is that sibling index monotonicity requires sibling instances of the same feature to be ordered according to the instance order, which is itself defined recursively over the subtree structure of a configuration. Such recursive dependencies are difficult to express directly in SAT formulas. Therefore, we restrict the SAT encoding to enforcing subtree index monotonicity, which already eliminates a substantial number of symmetric solutions (in particular order-isomorphic configurations).

Although this approach still leaves many symmetries in the solution space, which would typically be undesirable [101, 102, 103], it represents a reasonable trade-off between symmetry reduction and encoding complexity.

Implementation Details

Listings 6.17 and 6.18 illustrate the concrete implementation of key building blocks of the CP-SAT reference implementation.

Listing 6.17 shows the implementation of guarded cardinality constraints. Given an integer variable x, an activity variable, and a cardinality interval, the function enforces that x is forced to zero when inactive and constrained to lie within the specified interval when active.

Listing 6.18 shows how membership predicates of the form $(x \in I)$ are reified as Boolean variables.

```python
def _enforce_cardinality_when_active(
    model: CpModel,
    x: IntVar,
    active: IntVar,
    card: CardinalityInterval,
    name: str,
) -> None:
    """
    Enforce:
      - if active == 0: x == 0
      - if active == 1: x ∈ card
    """

    # Inactive -> zero
    model.add(x == 0).only_enforce_if(active.Not())

    intervals = list(card)

    selectors: list[IntVar] = []
    for k, iv in enumerate(intervals):
        sel = model.new_bool_var(f"{name}_sel_{k}")
        selectors.append(sel)

        # If this interval is selected, x must lie inside it
        model.add(x >= iv.lower).only_enforce_if(sel)
        if iv.upper is not None:
            model.add(x <= iv.upper).only_enforce_if(sel)

    # active = 1  <=>  exactly one interval is selected
    model.add(sum(selectors) == active)
```

Listing 6.17: CFMTOOLS Python CP-SAT plugin: Guarded encoding of cardinality intervals

```python
def _in_cardinality_interval_bool(
    model: CpModel,
    x: IntVar,
    card: CardinalityInterval,
    name: str,
) -> IntVar:
    """
    Returns b ∈ {0,1} such that:
        b == 1  <=>  x ∈ card
    """
    b = model.new_bool_var(f"{name}_in")
    intervals = list(card)

    selectors: list[IntVar] = []

    for k, iv in enumerate(intervals):
        sel = model.new_bool_var(f"{name}_sel_{k}")
        selectors.append(sel)

        # sel = 1 => x ∈ I_k
        model.add(x >= iv.lower).only_enforce_if(sel)
        if iv.upper is not None:
            model.add(x <= iv.upper).only_enforce_if(sel)

        # sel = 0 => x ∉ I_k
        # i.e., x < lower OR x > upper
        if iv.upper is not None:
            below = model.new_bool_var(f"{name}_below_{k}")
            above = model.new_bool_var(f"{name}_above_{k}")

            model.add(x < iv.lower).only_enforce_if(below)
            model.add(x > iv.upper).only_enforce_if(above)

            # if sel = 0, one of below/above must hold
            model.add_bool_or([below, above]).only_enforce_if(sel.Not()
    ↪ )
        else:
            below = model.new_bool_var(f"{name}_below_{k}")
            model.add(x < iv.lower).only_enforce_if(below)
            model.add(below == 1).only_enforce_if(sel.Not())

    # Exactly one interval selected iff b = 1
    model.add(sum(selectors) == b)

    return b
```

Listing 6.18: CFMTOOLS Python CP-SAT plugin: Reification of cardinality interval predicates for cross-tree constraints

7. Empirical Evaluation

This chapter presents an empirical evaluation of the algorithms proposed in Chapter 4.

It is structured as follows. In Section 7.1, we formulate the evaluation goals and research questions. Subsequently, in Section 7.2, we describe the experimental methodology, including the benchmark design, execution protocol, dataset, and hardware and software environment. Sections 7.3 and 7.4 present and discuss the results, and Section 7.5 discusses the threats to validity.

All empirical results, the models used in the evaluation, and the scripts for running the experiments and aggregating the results are available at https://github.com/BaumKM/cfmtools-artifacts.

7.1 Evaluation Goals and Research Questions

In contrast to the theoretical analysis in Chapter 5, where we establish correctness properties (in particular uniformity and termination) as well as asymptotic time and space complexity, the empirical evaluation focuses on the practical behaviour of the proposed algorithms. In particular, we investigate their runtime characteristics and validate the correctness of the implementation. The evaluation is guided by the following goals.

G1: Performance Assessment of DP-based Enumeration and Counting. The first goal is to evaluate the practical performance of the algorithm UnrankConfig (Algorithm 4.4) as an *enumeration mechanism* for generating structural configurations of CFMs, in comparison to a competitive, state-of-the-art, general-purpose CSP solver approach. In particular, we assess enumeration performance with respect to two core performance metrics in constraint programming (CP): solution latency, defined as the time until a first valid configuration is obtained, and solution generation throughput, defined as the number of configurations generated per time unit.

G2: Empirical Validation of Uniformity. The second goal is to empirically validate that the implemented sampling algorithms behave as expected. In particular, we examine whether the generated samples are uniformly distributed over the (constrained) structural configuration space.

7.1.1 Research Questions

Based on these two primary objectives, we formulate the following research questions.

RQ1: Time to first configuration (TTFC). How does the time required to obtain the first valid structural configuration compare between the DP-based unranking approach and the CSP solver approach?

RQ2: Enumeration throughput. How does the rate of structural configuration enumeration compare between the DP-based unranking approach and the CSP solver approach?

RQ3: Influence of model type. How does the performance comparison between the unranking-based approach and the constraint-solver-based baseline change when distinguishing between FMs and CFMs? In particular, does one approach systematically outperform the other on a specific model type?

RQ4: Empirical uniformity. For models with sufficiently small configuration spaces such that all configurations of the unconstrained model can be exhaustively enumerated within realistic time, do the observed output frequencies of the uniform sampling algorithms match the expected uniform distribution?

Research Questions 1 and 2 address the performance assessment goal (G1) by comparing the two approaches with respect to solution latency (TTFC) and configuration generation throughput, respectively. Research Question 3 extends this comparison by analysing how the performance difference between the two approaches varies when distinguishing between FMs and CFMs. Finally, Research Question 4 addresses the uniformity validation goal (G2) by assessing whether the observed output distributions of the sampling algorithms conform to the expected uniform distribution for models with exhaustively enumerable configuration spaces.

7.1.2 Rationale and Scope of the Evaluation Goals

Although uniformity of the proposed sampling approaches is established formally in Section 5.1, the empirical evaluation includes uniformity experiments as an end-to-end validation of the implementation. More precisely, the goal is to detect implementation errors or unintended biases that could lead to systematic deviations from the theoretically guaranteed uniform distribution.

Such deviations may arise from subtle implementation issues (e.g., incorrect ranking and unranking logic, biased branching decisions, or mistakes in rejection handling), even when the algorithmic design itself is correct. The validation therefore complements the theoretical results by verifying that the implementation is correct and consistent with the theoretical guarantees.

While the justification of Goal 2 is straightforward, the justification of Goal 1 requires additional explanation, as it represents a deliberate departure from the theoretical analysis, which focuses on uniform random sampling. In particular, the performance evaluation in this chapter does *not* benchmark the sampling algorithms UniSample-BT and UniSample-UR themselves. Instead, we evaluate UnrankConfig and the underlying DP-based counting and unranking machinery as a *high-performance enumeration mechanism*: we use it to (i) obtain a first valid configuration as quickly as possible and (ii) enumerate configurations with maximal throughput. This shift in focus is motivated by two considerations.

First, despite substantial progress in uniform random sampling and the closely related problem of model counting, that is, determining the number of valid solutions, there is currently no state-of-the-art, easy-to-use, out-of-the-box implementation that can be directly applied to CFMs. While numerous approaches exist for SAT-based problems, including hashing-based techniques, MCMC methods, heuristic approaches, and #SAT-based sampling [33, 38, 42, 45, 104], CFMs require non-trivial encodings. In particular, such encodings must capture cardinality interval constraints, feature multiplicities, and tree-based structures, and are therefore commonly handled using SMT, ILP, or CSP solvers [11, 18, 31, 48, 49]. However, tool support for uniform random sampling in the context of SMT and general CSP solvers is considerably less mature. Although general algorithms for uniform random sampling of CSPs have been proposed in the literature (e.g. [54]), these approaches are not integrated into widely used solver frameworks such as Z3 or OR-Tools.

Second, many existing (near-)uniform random sampling algorithms fundamentally rely on repeated invocations of an underlying SAT, SMT, ILP, or CSP solver. For instance, hashing-based methods partition the solution space into random buckets by adding random XOR constraints, and once a bucket of sufficiently small size is identified, enumerate its solutions to draw a sample uniformly [33, 41, 42]. As a consequence, the overall performance of uniform random sampling algorithms is largely determined by the efficiency of the underlying solver with respect to core operations such as model counting, enumeration, and solution generation [42, 105].

Evaluating the performance of these fundamental operations therefore constitutes a meaningful and necessary first step towards assessing the practical viability of uniform random sampling approaches for CFMs. By focusing on enumeration latency and throughput, the empirical evaluation characterizes the performance behaviour of the proposed DP-based approach and establishes a sound basis for future, more direct comparisons of complete sampling algorithms.

7.2 Experimental Methodology

In the following, we describe the experimental methodology and setup used to address the goals and research questions formulated in Section 7.1. In particular, we specify the execution protocols, the dataset, and the hardware and software environment under which all experiments are conducted.

7.2.1 Methodology for Performance Assessment (G1)

To address (G1) and the related research questions Research Questions 1 to 3, we conduct systematic benchmarking experiments comparing the DP-based approach with the CSP solver baseline.

7.2.1.1 Baseline Solver

For the CSP solver baseline, we use the CP-SAT implementation provided by CFM-TOOLS, as described in Section 6.3. The rationale for this choice is that CP-SAT is a state-of-the-art general-purpose solver that provides sufficiently rich modelling capabilities to efficiently capture the (semi-structural) configuration semantics of CFMs, while avoiding additional theories that could unnecessarily increase complexity.

Moreover, a solver comparison conducted by Eger [21] indicates that CP-SAT approaches can outperform corresponding ILP and SMT encodings.

7.2.1.2 Execution Protocol

For the performance measurements, we use the following execution protocol.

For each model $m \in \mathcal{D}$ in the dataset $\mathcal{D}$ (see Section 7.2.3) and solver (or approach) $s \in \{\text{DP}, \text{CP-SAT}\}$, the solver s is executed independently under identical hardware and software conditions.

Each execution is subject to a global wall-clock time limit of

$$\mathcal{T}_{\text{global}} = 20 \text{ minutes},$$

which covers the entire subprocess invocation, including preprocessing, enumeration, and result reporting. If this limit is exceeded at any point, the execution is aborted and the model is classified as a timeout for the respective solver.

Within this global limit, execution proceeds in two conceptual phases.

1. **Preprocessing phase.** The internal representation required by the respective approach is constructed. The measured wall-clock time spent in this phase is denoted by $t_{m,s}^{\text{prep}}$.

   For the DP-based approach, this includes construction of the DP tables and auxiliary data structures required for efficient ranking (see Section 5.2). For the CP-SAT baseline, this consists of translating the input CFM into CP-SAT variables and constraints.

2. **Enumeration phase.** If preprocessing completes successfully, the solver attempts to enumerate valid configurations within a fixed enumeration time budget of

   $$\mathcal{T}_{\text{enum}} = 20 \text{ seconds}.$$

   The measured wall-clock time spent in this phase is denoted by $t_{m,s}^{\text{enum}}$.

   The enumeration phase terminates when either

   (i) all valid configurations have been generated,

   (ii) the enumeration time limit $\mathcal{T}_{\text{enum}}$ is reached, or

   (iii) the global time limit $\mathcal{T}_{\text{global}}$ is reached.

Note that the measured enumeration time $t_{m,s}^{\text{enum}}$ may exceed $\mathcal{T}_{\text{enum}}$. For the DP-based approach, this occurs because ongoing unranking steps are not interrupted when the time limit is reached and are therefore always allowed to complete before termination. For the CP-SAT solver, an internal presolve phase is always executed before the actual solving process and is not affected by the enumeration time limit, which may likewise cause the measured enumeration time to exceed $\mathcal{T}_{\text{enum}}$.

By definition, $t_{m,s}^{\text{prep}} + t_{m,s}^{\text{enum}} \leq \mathcal{T}_{\text{global}}$.

Timeout Classification

Execution outcomes are classified as follows.

- **Construction timeout.** If the global time limit $\mathcal{T}_{\text{global}}$ is reached at any point during execution (either during preprocessing or enumeration), the model is classified as a construction timeout.
- **Search timeout.** If preprocessing completes successfully and the subsequent enumeration attempt terminates within $\mathcal{T}_{\text{global}}$, but no valid configuration is produced, the model is classified as a search timeout.

Number of Runs

Each model is evaluated exactly once for each solver. Repeating the full benchmark multiple times is infeasible within the scope of this work, as a single run over the entire dataset already requires several days of wall-clock time. Moreover, both implementations operate deterministically under fixed solver parameters (in particular, a fixed random seed and single-threaded execution), such that repeated runs yield identical results in principle. Consequently, additional repetitions are not expected to provide further insight but would significantly increase computational cost.

Treatment of Extremely Large Instance Bounds

A special case arises for models with extremely large finite instance bounds. In the CP-SAT encoding (see Section 6.3), at least one Boolean variable is introduced for each *potential* feature instance. Hence, the number of variables grows linearly in the instance bounds B_f and, in particular with respect to the total instance bound

$$B_{\text{tot}} \coloneqq \sum_{f \in F} B_f.$$

For large values of B_{tot}, the number of generated variables and constraints becomes so large that model construction alone may exceed the global time limit

$$\mathcal{T}_{\text{global}} = 20 \text{ minutes},$$

or exceed practical memory limits. In such cases, execution typically terminates during preprocessing and is therefore classified as a construction timeout.

Table 7.1: Summary statistics of the total instance bound $B_{\text{tot}} = \sum_{f \in F} B_f$ for the 999 models in the folder `Generator_examples`

Statistic	Total Instance Bound B_{tot}
Minimum	988,681
Median	31,869,019
Maximum	159,338,790

Within the dataset (see Section 7.2.3), 999 models in the folder `Generator_examples` exhibit extremely large instance bounds (see Tables 7.1 and 7.2). While additional models may also have large bounds, this group constitutes the majority of such cases and dominates the associated computational cost.

Executing the CP-SAT baseline on all 999 models with $\mathcal{T}_{\text{global}} = 20$ minutes would result in a total runtime of approximately two weeks, assuming that each model runs into a construction timeout. Preliminary experiments confirm that this is indeed the case: model construction consistently fails for these instances due to excessive encoding size, resulting in construction timeouts.

To address this, we evaluate the CP-SAT baseline on a representative subset of ten models from this group, namely the ten models with the smallest total instance bounds. Since the encoding size grows monotonically with B_{tot}, these models represent the most favourable cases.

If all ten models result in construction timeouts, the remaining models in the group are classified as construction timeouts without being executed. If at least one model can be constructed successfully, we evaluate an additional ten models with larger instance bounds. This process is repeated until either all models are processed or a set of ten consecutive models results in construction timeouts.

This special treatment is applied exclusively to the CP-SAT baseline and only to the 999 models in `Generator_examples`.

Table 7.2: The ten models with the smallest total instance bounds B_{tot} among the 999 models in the folder `Generator_examples`

Total Instance Bound B_{tot}	Model File
988,681	`generated_model_230.json`
1,099,523	`generated_model_365.json`
1,180,987	`generated_model_422.json`
1,397,873	`generated_model_846.json`
1,407,109	`generated_model_329.json`
1,444,381	`generated_model_906.json`
1,453,561	`generated_model_520.json`
1,621,558	`generated_model_501.json`
1,723,793	`generated_model_519.json`
1,948,653	`generated_model_588.json`

Configuration Semantics

As discussed in Section 6.3, the DP-based approach and the CP-SAT baseline generate different types of configurations.

The DP-based approach enumerates *structural configurations*, whereas the CP-SAT baseline enumerates *semi-structural configurations*, as structural semantics cannot be enforced efficiently in the CP-SAT encoding.

For the purpose of the performance evaluation, both types are treated equivalently. In particular, we only require that generated configurations are valid with respect to the underlying CFM; no distinction between structural and semi-structural configurations is made in the reported performance metrics.

7.2.1.3 Time to First Configuration (TTFC)

To answer Research Question 1, we measure the *time to first configuration* (TTFC), which captures solution latency, i.e., the wall-clock time until the first valid configuration is returned.

For each model m and solver $s \in \{\text{DP}, \text{CP-SAT}\}$, we distinguish two variants of TTFC to separate preprocessing effects from pure search behaviour.

Non-cached TTFC (End-to-End Latency). The non-cached variant measures the latency perceived by a user invoking the solver on a raw model. It includes both preprocessing and enumeration time:

$$T_{m,s}^{\text{first,e2e}} := t_{m,s}^{\text{prep}} + t_{m,s}^{\text{first_enum}},$$

where $t_{m,s}^{\text{prep}}$ denotes the preprocessing time (see Section 7.2.1.2) and $t_{m,s}^{\text{first_enum}}$ the time from the start of enumeration until the first valid configuration is produced.

Cached TTFC (Search-Only Latency). The cached variant isolates the pure search component and excludes preprocessing time:

$$T_{m,s}^{\text{first,cached}} := t_{m,s}^{\text{first_enum}}.$$

This metric reflects the latency once the internal representation (e.g., DP tables or solver model) has already been constructed and can be reused.

Similarly to $t_{m,s}^{\text{enum}}$ (Section 7.2.1.2), $t_{m,s}^{\text{first_enum}}$ may exceed $\mathcal{T}_{\text{enum}}$, as an ongoing enumeration step is not immediately interrupted when the time limit is reached.

Timeout Handling

For each model m and solver $s \in \{\text{DP}, \text{CP-SAT}\}$, the model is classified as either

(i.) solved,

(ii.) construction timeout, or

(iii.) search timeout,

as defined in Section 7.2.1.2. For timeout cases, the TTFC values are defined as follows.

- **Construction timeout.** If the global time limit $\mathcal{T}_{\text{global}}$ is reached, no valid configuration is produced and $t_{m,s}^{\text{first_enum}}$ is not observed. For consistency of the evaluation metrics, we define

$$T_{m,s}^{\text{first,e2e}} := \mathcal{T}_{\text{global}},$$
$$T_{m,s}^{\text{first,cached}} := \mathcal{T}_{\text{enum}}.$$

  The cached TTFC is defined in terms of the enumeration time limit because, in the current experimental setup, $t_{m,s}^{\text{first_enum}}$ cannot be measured when the global time limit is reached. Consequently, $T_{m,s}^{\text{first,cached}} := \mathcal{T}_{\text{enum}}$ serves as a fixed surrogate penalty.

- **Search timeout.** If preprocessing completes successfully, but no configuration is produced before the enumeration time limit is reached, we define

$$T_{m,s}^{\text{first,cached}} := t_{m,s}^{\text{enum}}, \qquad T_{m,s}^{\text{first,e2e}} := t_{m,s}^{\text{prep}} + t_{m,s}^{\text{enum}}.$$

7.2.1.4 Enumeration Throughput

To address Research Question 2, we measure the *enumeration throughput*, i.e., the rate at which valid configurations are generated.

For each model m in the dataset and solver $s \in \{\text{DP}, \text{CP-SAT}\}$, let $n_{m,s}$ denote the number of valid configurations generated during the enumeration phase.

Analogously to the TTFC, we distinguish two variants of throughput to isolate the effects of preprocessing, as preprocessing requirements differ between the two approaches.

Non-cached Throughput (End-to-End Throughput). The non-cached variant reflects the effective generation rate perceived by a user who invokes the solver on a raw model without prior preprocessing. It includes preprocessing time in the denominator:

$$\text{Thr}_{m,s}^{\text{e2e}} := \frac{n_{m,s}}{t_{m,s}^{\text{prep}} + t_{m,s}^{\text{enum}}}.$$

Cached Throughput (Steady-State Throughput). The cached variant isolates the sustained generation capacity of the solver and excludes preprocessing time:

$$\text{Thr}_{m,s}^{\text{cached}} := \frac{n_{m,s}}{t_{m,s}^{\text{enum}}}.$$

This metric reflects the steady-state throughput once the internal representation (e.g., DP tables or solver model) has been constructed and can be reused.

Timeout Handling. For throughput, no special timeout handling is required. If a timeout occurs, the number of enumerated configurations is $n_{m,s} = 0$, and therefore the throughput is 0, regardless of the time value in the denominator.

7.2.1.5 Aggregation and Statistical Evaluation

The TTFC and enumeration throughput variants, defined per model and solver, are aggregated as follows for each research question.

RQ1: Time to First Configuration

This analysis is based on:

- Non-cached TTFC $T_{m,s}^{\text{first,e2e}}$,
- Cached TTFC $T_{m,s}^{\text{first,cached}}$.

For each variant and solver $s \in \{\text{DP}, \text{CP-SAT}\}$, the following aggregated statistics are reported:

(1) Success Rate. The success rate measures the fraction of models for which solver s produces at least one valid configuration within the imposed time limits (i.e. without timing out). Formally,

$$\text{Succ}(s) \coloneqq \frac{|\{m \in \mathcal{D} \mid n_{m,s} > 0\}|}{|\mathcal{D}|}.$$

(2) PAR-1 (Timeout-Aware Latency). To obtain a single latency measure over the full dataset that incorporates timeouts, we use the penalized average runtime (PAR-k) score, i.e., the average runtime where timeouts are assigned a penalty value [106, 107, 108]. Concretely, we use PAR-1:

$$\text{PAR-1}(s) \coloneqq \frac{1}{|\mathcal{D}|} \sum_{m \in \mathcal{D}} T_{m,s}.$$

Timeouts are already incorporated in the definition of $T_{m,s}$ (see Section 7.2.1.3). In particular:

- For construction timeouts,

$$T_{m,s}^{\text{first,e2e}} = \mathcal{T}_{\text{global}}, \quad T_{m,s}^{\text{first,cached}} = \mathcal{T}_{\text{enum}}.$$

- For search timeouts,

$$T_{m,s}^{\text{first,e2e}} = t_{m,s}^{\text{prep}} + t_{m,s}^{\text{enum}}, \quad T_{m,s}^{\text{first,cached}} = t_{m,s}^{\text{enum}}.$$

(3) Cactus Plots. For each TTFC variant and solver s, we construct cactus plots [109]. Formally, let $T_{(1),s} \leq T_{(2),s} \leq \cdots \leq T_{(|\mathcal{D}|),s}$ be the non-decreasing ordering of the values $\{T_{m,s} \mid m \in \mathcal{D}\}$, including timeout-assigned values. The cactus curve for solver s is the step function

$$C_s(t) \coloneqq \left|\{\, i \in \{1, \ldots, |\mathcal{D}|\} \mid T_{(i),s} \leq t \,\}\right|.$$

Thus, for a given time budget t, $C_s(t)$ equals the number of models solved within time t under the adopted definition of $T_{m,s}$ (either $T_{m,s}^{\text{first,e2e}}$ or $T_{m,s}^{\text{first,cached}}$).

Timeout models are included in the cactus plots according to their definition in Section 7.2.1.3. To distinguish them from solved models, they are explicitly marked in the plots.

(4) Performance Profiles (Relative Latency). We additionally construct performance profiles following Dolan and Moré [110], using TTFC as the cost metric. For each model m and solver s, we define the performance ratio

$$r_{m,s} \coloneqq \frac{T_{m,s}}{\min_{s'} T_{m,s'}}.$$

The performance ratio $r_{m,s}$ describes, for each model $m \in \mathcal{D}$ and solver s, how much slower the solver is compared to the best-performing solver on that model. By definition, $r_{m,s} \geq 1$ for all m and s, and $r_{m,s} = 1$ holds for the best-performing solver(s) on model m. With this definition, the performance profile of solver s is the cumulative distribution function of these ratios:

$$\rho_s(\tau) \coloneqq \frac{1}{|\mathcal{D}|} \left|\{\, m \in \mathcal{D} \mid r_{m,s} \leq \tau \,\}\right|, \qquad \tau \geq 1.$$

Hence, for a given factor τ, $\rho_s(\tau)$ denotes the fraction of models on which solver s is within a factor τ of the best solver. For example, for $\tau = 5$, $\rho_s(5)$ equals the fraction of models on which solver s is at most five times slower than the best solver.

Interpretation and purpose of (1)–(4). The selected metrics are designed to provide complementary and non-redundant perspectives on solver performance with respect to TTFC.

The success rate assesses robustness, i.e., the ability of a solver to produce at least one valid configuration within the imposed time limits across the dataset.

Cactus plots visualize the complete runtime distribution for each solver. By showing how many models are solved within increasing time budgets, they provide a global view of solver scalability and allow differences in overall solving capacity to be observed directly.

The PAR-1 score complements these measures by yielding a single timeout-aware latency metric over the full dataset, thereby incorporating unsuccessful runs into the evaluation. Importantly, the PAR-1 score also admits a direct interpretation in terms of the *total runtime required to process the entire dataset*. Specifically, since PAR-1(s) is defined as

$$\text{PAR-1}(s) = \frac{1}{|\mathcal{D}|} \sum_{m \in \mathcal{D}} T_{m,s},$$

we can multiply it by the dataset size, to obtain

$$\sum_{m \in \mathcal{D}} T_{m,s} = |\mathcal{D}| \cdot \text{PAR-1}(s).$$

That is the total wall-clock time required to run solver s sequentially on all models under the given timeout policy. In this sense, PAR-1 provides a realistic estimate of the *end-to-end runtime experienced by a user* who applies the solver to the full benchmark set, where each run is terminated as soon as the first valid configuration is found (or a timeout occurs).

Finally, performance profiles enable relative, per-model comparisons between solvers. Rather than comparing absolute runtimes, they quantify how close each solver is to the best-performing solver on each model [110]. In particular, they allow us to assess, for any given fraction of the dataset, by which factor a solver would need to be sped up to match the performance of the best-performing solver on those models, where performance is measured with respect to TTFC.

RQ2: Enumeration Throughput

This analysis is based on:

- Non-cached throughput $\text{Thr}_{m,s}^{\text{e2e}}$,
- Cached throughput $\text{Thr}_{m,s}^{\text{cached}}$.

For each variant and solver $s \in \{\text{DP}, \text{CP-SAT}\}$, the following aggregated statistics are reported.

(1) Cactus Plots. In standard SAT benchmarking practice, cactus plots are defined over a cost metric to be minimized, typically runtime [109, 111, 112]. Since throughput is a maximization objective, we convert it into a minimization metric by considering inverse throughput:

$$C_{m,s}^{\text{thr}} := \begin{cases} \frac{1}{\text{Thr}_{m,s}} & \text{if } \text{Thr}_{m,s} > 0, \\ \infty & \text{if } \text{Thr}_{m,s} = 0. \end{cases}$$

The value $C_{m,s}^{\text{thr}}$ represents the average time per generated configuration; consequently, smaller values correspond to higher throughput. In particular, if a solver produces no valid configuration, its inverse throughput is ∞.

(2) Performance Profiles (Relative Throughput). For the performance profiles, we also use inverse throughput. For each model m and solver s, we define the performance ratio

$$r_{m,s}^{\text{thr}} := \frac{C_{m,s}^{\text{thr}}}{\min_{s'} C_{m,s'}^{\text{thr}}}.$$

Analogously to the latency setting, the corresponding cumulative distribution function yields, for each factor $\tau \geq 1$, the fraction of models on which solver s achieves throughput within a factor τ of the best solver.

If both solvers have $\text{Thr}_{m,s} = 0$ on a model m, then all inverse-throughput values are infinite and the performance ratio is undefined. Such models are therefore excluded from the throughput performance profiles.

Interpretation and purpose of (1)-(2). Similarly to the TTFC aggregation, cactus plots provide a global view of enumeration throughput. They visualize scalability by showing, for increasing cost thresholds (i.e., time per configuration), how many models achieve a given level of efficiency.

Performance profiles complement this view by enabling relative, per-model comparisons between solvers. Rather than comparing absolute throughput values, they quantify how close each solver is to the best-performing solver on each model. In particular, they allow us to assess, for any given fraction of the dataset, by which factor a solver would need to be sped up to match the throughput of the best-performing solver on those models.

We do not report a separate success rate for throughput (i.e., the number of models with non-zero throughput), as this information is already captured by the TTFC success rate. Throughput and TTFC are measured within the same run; thus, whenever a solver generates at least one valid configuration, this contributes both to the TTFC success rate and to a strictly positive throughput value.

Failure Mode Comparison via Contingency Analysis

Beyond the aggregated performance metrics for Research Questions 1 and 2, we analyse the failure behaviour of both solvers using a contingency table [113, 114, 115]. For each model, we record the outcome (solved, construction timeout, or search timeout) for both solvers and count the frequency of each outcome combination.

This analysis is reused in the structural aggregation to address Research Question 3. In particular, it reveals whether failures occur on the same models for both solvers or on distinct subsets.

RQ3: Influence of Model Type

While Research Questions 1 and 2 compare solver performance over the full dataset, Research Question 3 investigates how the relative behaviour of the two solvers differs between classical FMs and proper CFMs.

To this end, we partition the dataset by model type into two disjoint subsets.

Evaluation within Each Partition. Within each partition and for each solver, we recompute the same aggregated metrics defined for RQ1 and RQ2. Concretely, we report:

- success rate,
- PAR-1 (timeout-aware mean TTFC),
- cactus plots and performance profiles for TTFC,
- cactus plots and performance profiles for throughput, and
- contingency tables of solver failure modes.

Interpretation. This stratified analysis [116], i.e., partitioning the dataset into FMs and CFMs, allows us to assess whether solver dominance remains consistent across both model types or differs between them.

7.2.2 Methodology for Uniformity Validation (G2)

In contrast to the extensive performance evaluation presented for (G1), the uniformity validation serves as an end-to-end sanity check of the implemented sampling algorithms. Its purpose is not to establish new theoretical guarantees, which are already provided in Section 5.1, but to empirically verify that the implementation behaves as expected.

7.2.2.1 Execution Protocol

The following describes the experimental protocol used for the uniformity validation.

Model Selection

Uniformity validation requires knowledge of the complete configuration space in order to evaluate the empirical distribution of generated samples. Therefore, we manually select 8 models whose structural configuration space can be enumerated completely within reasonable time and memory limits. In addition to this feasibility constraint, we ensure that the selected models cover different structural characteristics of the dataset. In particular, the selection satisfies the following diversity criteria:

- at least one FM,
- at least one proper CFM,
- models without cross-tree constraints, and
- models with one or multiple cross-tree constraints.

Including models with cross-tree constraints is essential because both sampling procedures rely on rejection sampling in their presence. Consequently, these models allow us to verify that the rejection mechanism behaves as expected.

Table 7.3 lists the models used for the uniformity validation.

Table 7.3: Models used for the uniformity validation

ID	Category	Model File	CTCs
M1	FM	`fm/splot/aircraft_fm.json`	✗
M2	FM	`fm/collection/smart_watch.json`	✓
M3	FM	`fm/splot/car_fm.json`	✓
M4	CFM	`cfm/chain_models/chain_3_3.json`	✗
M5	CFM	`cfm/repo/model_20.json`	✗
M6	CFM	`cfm/repo/model_19.json`	✓
M7	CFM	`cfm/repo/pizzas.json`	✓
M8	CFM	`cfm/repo/running_example.json`	✓

Sampling Setup

For each selected model m, we evaluate both sampling algorithms

$$s \in \{\textsc{UniSample-BT}, \textsc{UniSample-UR}\}.$$

Similarly to the performance evaluation protocol (Section 7.2.1.2), all experiments are executed single-threaded under identical hardware and software conditions.

Number of samples per run. Let

$$k \coloneqq \left|[\![m]\!]_{\text{Struct}}\right|$$

denote the number of structural configurations of model m. For each run, we generate

$$N_m \coloneqq 20 \cdot k$$

independent samples.

The factor of 20 is chosen to satisfy standard recommendations for the applicability of Pearson's χ^2 goodness-of-fit test, which suggest that expected counts per category (outcome) should be sufficiently large (commonly at least five) to ensure that the chi-squared approximation is reliable [59, 114, 117]. Since under the uniform distribution the expected count is

$$E = \frac{N_m}{\left|[\![m]\!]_{\text{Struct}}\right|} = 20,$$

the expected frequency of each configuration comfortably exceeds the recommended minimum.

Number of runs. To obtain statistically robust results, the entire sampling procedure is repeated multiple times. For each pair (m, s) of model and sampler we perform

$$R = 10{,}000$$

independent runs. In every run, the sampler generates N_m samples using a fresh random seed.

7.2.2.2 Collected Metrics

For every run, we record the following quantities.

(1) Observed frequency distribution. Let $k = |[\![m]\!]_{\text{Struct}}|$. For each configuration $i \in \{1, \ldots, k\}$, we record the observed frequency O_i among the N_m generated samples. This yields the empirical distribution

$$\hat{p}_i = \frac{O_i}{N_m}.$$

(2) Pearson χ^2 statistic and p-value. Using the observed frequencies we compute Pearson's chi-squared statistic [114]

$$\chi^2 = \sum_{i=1}^{k} \frac{(O_i - E)^2}{E}, \qquad E = \frac{N_m}{k}.$$

This statistic measures the deviation of the empirical distribution from the uniform distribution. The corresponding p-value quantifies how likely such a deviation would be if the samples were drawn independently and uniformly. In particular, it is defined as the probability of observing a value at least as extreme, i.e., a χ^2 value greater than or equal to the observed one, under the null hypothesis [114, 118, 119, 120].

(3) Sampling runtime. For completeness, we also measure the total wall-clock time required to generate the N_m samples. This time measures only the sampling procedure itself and excludes preprocessing, ranking, or statistical evaluation steps.

7.2.2.3 Statistical Evaluation

For the statistical evaluation, we apply Pearson's χ^2 goodness-of-fit test [114] to each run to test the null hypothesis that the generated samples are uniformly distributed over the structural configuration space.

Rather than reporting the results of individual runs independently, we aggregate the resulting p-values and perform a second-level test to evaluate them jointly. Such a second-level test examines whether the collection of p-values follows the expected uniform distribution under the global null hypothesis, thereby assessing the overall consistency of the sampler across multiple runs. This reduces the variability of test outcomes and yields a more robust overall decision [121, 122, 123]. In particular, we follow an approach similar to that described by Rukhin et al. [124]

Per-Model Analysis

Let $p_{m,s,r}$ denote the p-value obtained for model m, sampler s, and run r. Under the null hypothesis, these p-values are approximately uniformly distributed on $[0, 1]$ [114, 120], i.e.,

$$p_{m,s,r} \sim \text{Uniform}(0, 1).$$

We fix a significance level

$$\alpha = 0.05.$$

For each run, we record whether the null hypothesis is rejected:

$$I_{m,s,r} = \begin{cases} 1 & \text{if } p_{m,s,r} < \alpha, \\ 0 & \text{otherwise.} \end{cases}$$

Distribution of rejections. Under the null hypothesis, the rejection probability equals α (exactly for an ideal test and approximately in practice). Hence, the rejection indicator satisfies

$$I_{m,s,r} \sim \text{Bernoulli}(\alpha).$$

Because the $R = 10{,}000$ runs are executed independently using fresh random seeds, the total number of rejections

$$K_{m,s} \coloneqq \sum_{r=1}^{R} I_{m,s,r}$$

follows a binomial distribution

$$K_{m,s} \sim \text{Binomial}(R, \alpha)$$

under the null hypothesis [59, 115].

Empirical rejection rate. For each model–sampler pair (m, s) we summarize the outcome by the empirical rejection rate

$$\hat{\rho}_{m,s} \coloneqq \frac{K_{m,s}}{R}.$$

If the sampler is implemented correctly, the rejection rate should be close to α. In contrast, if the sampler is biased, the χ^2 test will reject more frequently and the observed rejection rate will typically exceed α.

Confidence interval for the rejection probability. Using the empirical rejection rate, we compute a two-sided 95% Clopper–Pearson confidence interval [125, 126] for the true rejection probability $\rho_{m,s}$. Specifically, we compute bounds

$$[\underline{\rho}_{m,s}, \overline{\rho}_{m,s}]$$

that form a $(1 - \alpha)$-confidence interval for $\rho_{m,s}$. This means that the interval is constructed such that, with probability at least $1 - \alpha$, it contains the true rejection probability. Formally,

$$\Pr\Big(\rho_{m,s} \in [\underline{\rho}_{m,s}, \overline{\rho}_{m,s}]\Big) \geq 1 - \alpha.$$

We then evaluate whether the theoretical rejection probability α lies within this interval. If

$$\alpha \notin [\underline{\rho}_{m,s}, \overline{\rho}_{m,s}],$$

we conclude that sampler s is statistically inconsistent with uniform sampling on model m. Otherwise, the results are consistent with uniform sampling.

Resulting hypothesis test. Overall, this procedure defines a hypothesis test for

$$H_0 : \text{sampler } s \text{ samples uniformly from } [\![m]\!]_{\text{Struct}},$$
$$H_1 : \text{sampler } s \text{ does not sample uniformly from } [\![m]\!]_{\text{Struct}}.$$

Aggregation Across Models

In addition to the per-model analysis, we also aggregate results across all benchmark models. Let $\mathcal{F}$ denote the set of evaluated models and define

$$K_s^{\text{glob}} = \sum_{m \in \mathcal{F}} \sum_{r=1}^{R} I_{m,s,r}$$

as the total number of rejections observed for sampler s. The total number of runs equals

$$N_s^{\text{glob}} = |\mathcal{F}| \cdot R.$$

Under the null hypothesis we have

$$K_s^{\text{glob}} \sim \text{Binomial}(N_s^{\text{glob}}, \alpha).$$

Again, we summarize the outcomes by the global rejection rate

$$\hat{\rho}_s^{\text{glob}} = \frac{K_s^{\text{glob}}}{N_s^{\text{glob}}}$$

together with a 95% Clopper–Pearson confidence interval for the true rejection probability.

If α lies outside this interval, the sampler is considered statistically inconsistent with uniform sampling across the benchmark set. Otherwise, the results are consistent with uniform sampling.

This yields a global hypothesis test

H_0 : sampler s samples uniformly across the benchmark set $\mathcal{F}$,
H_1 : sampler s does not sample uniformly across the benchmark set $\mathcal{F}$.

7.2.3 Dataset

The dataset used for the performance evaluation is a heterogeneous collection comprising a total of 2499 CFMs. Of these, 1300 models are classical FMs, while the remaining models are proper CFMs. However, this distinction between FMs and proper CFMs is only relevant for Research Question 3, as internally all FMs are converted to CFMs.

FMs

The FMs included in the dataset originate from established public repositories. The majority is taken from the Software Product Lines Online Tools (S.P.L.O.T.) dataset [127][1], accessed via the UVL Hub[2], which provides a large collection of models in the Universal Variability Language (UVL) format[3].

[1] http://www.splot-research.org/
[2] https://www.uvlhub.io/
[3] https://universal-variability-language.github.io/

In addition, nine hand-selected FMs from the UVL Hub are included. Among these are four real-world automotive models originating from the automotive industry and originally published by Knüppel et al. [26]. To ensure compatibility with the CFMTOOLS framework, these models were slightly adapted by removing a small number of cross-tree constraints (see Section 7.2.3.2).

Compared to the relatively small models in the S.P.L.O.T. dataset (on average 45.46 features and 3.55 cross-tree constraints), the automotive models are substantially larger and structurally more complex, with an average of 17,671.50 features and 1,225.75 cross-tree constraints.

CFMs

The proper CFMs in the dataset stem from two primary sources. First, we include a collection of models used internally at the TVA institute at KIT.[4] Second, we incorporate models from an online repository associated with an unpublished study.[5]

In addition, the dataset includes a family of synthetic *chain* CFMs, illustrated in Figure 7.1. Each chain model consists of a linear hierarchy of features, where the parameter h controls the height of the chain and k determines the maximum feature-instance cardinality. Group cardinalities are permissive and do not restrict the number of valid configurations, with group-type cardinalities set to $\langle 0, 1 \rangle$ and group-instance cardinalities set to $\langle 0, k \rangle$.

The model family, referred to here as the *chain* family, has also been used by Güthing et al. [11] and Zhang [18]; we adopt it here without modification.

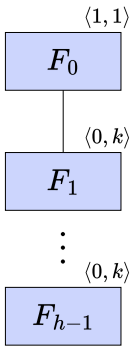


Figure 7.1: Chain CFM with parameters h and k

7.2.3.1 Preprocessing

Independently of their origin and representation, all models are converted into the JSON-based input format supported by CFMTOOLS. Furthermore, all models are bounded using the Big-M method, and trivial dead cardinalities are removed (see Section 2.2.3).

7.2.3.2 Selection Criteria

The dataset construction is guided by a combination of hard and soft criteria. Hard criteria arise from technical constraints imposed by CFMTOOLS (see Chapter 6) and the adopted CFM semantics, while soft criteria aim to ensure structural diversity.

[4] https://bwsyncandshare.kit.edu/s/ogN8Ps4ZF4DLijc
[5] https://anonymous.4open.science/r/solver-rep-subject-systems-C412/

Hard Selection Criteria

All models included in the dataset must be publicly available and provided in a format supported by CFMTOOLS. Specifically, models must either be available in the UVL format or in the JSON-based format commonly used by the TVA institute at KIT, as these are currently the only two input formats supported by CFMTOOLS.

Furthermore, UVL models must be compatible with the CFM semantics considered in this work. In particular, cross-tree constraints must be expressible using require and exclude constraints over cardinality intervals. Models containing unsupported constraint types, such as arithmetic constraints, attribute-based constraints, or general Boolean expressions beyond implication, equivalence, and mutual exclusion, cannot be processed directly. If such constraints are present but limited, they are removed or simplified, as done for the automotive models.

Finally, all models must admit at least one valid structural configuration. Models with an empty configuration space are excluded.

Table 7.4 summarizes the supported UVL constraint types and their encoding in CFM semantics. In all cases, variables may appear negated. If a variable occurs negated, the corresponding interval $\langle 1, 1\rangle$ is replaced by $\langle 0, 0\rangle$ (and vice versa for exclude constraints). For example, the constraint $\neg(\neg A \wedge \neg B)$ is encoded as $A \in \langle 0, 0\rangle \Rightarrow B \in \langle 1, 1\rangle$.

Table 7.4: Supported UVL cross-tree constraints and their encoding in CFM semantics

UVL Constraint	Encoding in CFM
$A \Rightarrow B$	$A \in \langle 1, 1\rangle \;\Rightarrow\; B \in \langle 1, 1\rangle$
$A \Leftrightarrow B$	$A \in \langle 1, 1\rangle \;\Rightarrow\; B \in \langle 1, 1\rangle$ and $B \in \langle 1, 1\rangle \;\Rightarrow\; A \in \langle 1, 1\rangle$
$\neg(A \wedge B)$	$A \in \langle 1, 1\rangle \;\Rightarrow\; B \in \langle 0, 0\rangle$

Soft Selection Criteria

Beyond the hard constraints described above, the dataset is constructed to cover a broad range of structural characteristics, including model size and the number of cross-tree constraints. In particular, the automotive feature models are included to complement the S.P.L.O.T. dataset, as they differ substantially in both size and structural complexity.

While a wide variety of FMs, including both synthetic and real-world models, is publicly available, the availability of real-world CFMs is limited [18]. Consequently, the dataset contains only synthetic CFMs. To maximize coverage, all publicly available CFMs from the previously mentioned repositories are included.

7.2.3.3 Empty Configuration Spaces

One important hard selection criterion is the non-emptiness of the structural configuration space. However, this criterion is difficult to enforce *a priori*, since the models originate from heterogeneous sources that typically do not provide information about their construction process or validity properties.

To address this, we apply a pragmatic filtering strategy during evaluation. For each model, both solvers (DP and CP-SAT) are executed as described in Section 7.2.1.2. If either solver determines that the configuration space is empty (i.e., proves unsatisfiability), the model is excluded from all reported performance metrics. Note that this procedure does not guarantee the absence of additional models with empty configuration spaces. However, any such remaining cases do not bias the comparison, as both solvers are evaluated under identical conditions and neither solver detects these inconsistencies.

In total, 10 models are excluded, resulting in a final dataset of 2489 models. Table 7.5 lists the excluded models.

Table 7.5: Evaluation dataset: Models excluded due to empty configuration spaces

Model File
`cfm/bw_sync/Test_Models/model_4.json`
`cfm/repo/model_4.json`
`fm/splot/model_20110207_1623721536.json`
`fm/splot/model_20151123_702303189.json`
`fm/splot/model_20160226_1944186213.json`
`fm/splot/model_20160519_1846923063.json`
`fm/splot/model_20161118_228131771.json`
`fm/splot/model_20170514_1578942718.json`
`fm/splot/model_20190411_1457420991.json`
`fm/splot/model_20221031_379205114.json`

7.2.3.4 Overview

Table 7.6 together with Figures 7.2 to 7.4 summarize key characteristics of the final dataset used for the evaluation, excluding models with an empty configuration space.

The figures present the cumulative distribution functions (CDFs) for FMs, proper CFMs, and the complete dataset, whereas the table reports the minimum, median, mean, and maximum values, which are difficult to extract from the plots, especially for heavy-tailed metrics shown on a logarithmic scale.

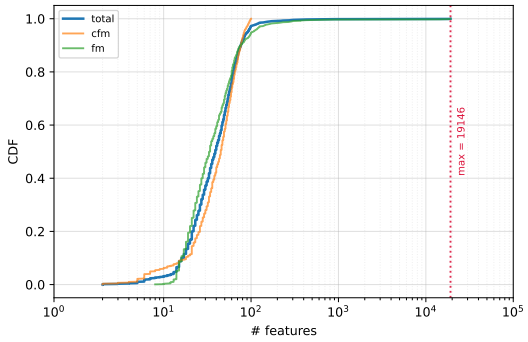
Figure 7.2 illustrates structural properties of the feature trees in the dataset, including the number of features, tree height, average internal branching factor (i.e. the average number of children per internal feature), and maximum branching factor.

Figure 7.3 shows the distribution of cross-tree constraints. In addition to the absolute number of constraints, the cross-tree constraint density is reported, where the number of require and exclude constraints is normalized by the number of theoretically possible cross-tree relations.

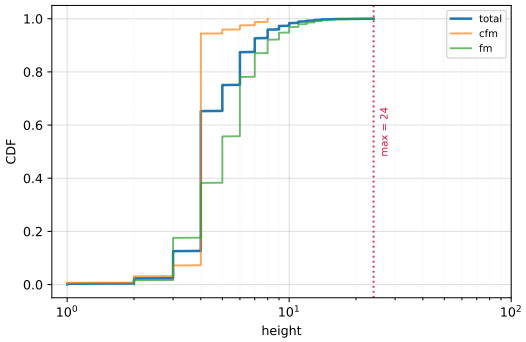
For a model with n features, at most

$$n(n-1)$$

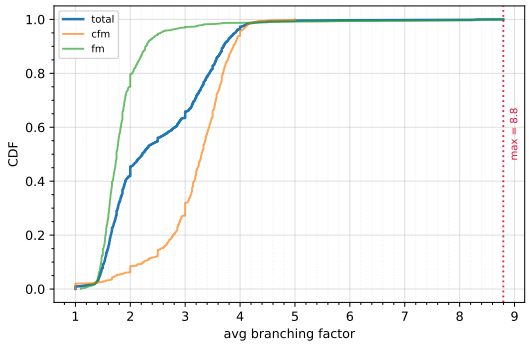
require constraints are possible, since require constraints can be interpreted as directed edges in a graph over n vertices, where self-loops as well as duplicated con-



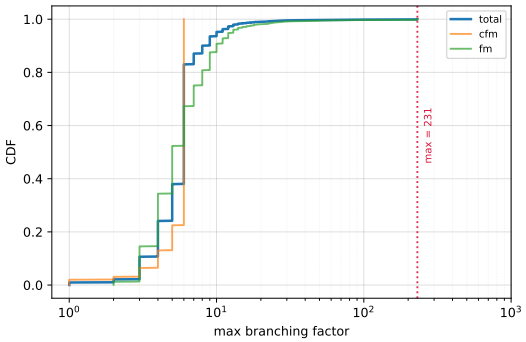
(a) Number of features (log-scale).



(b) Tree height (log-scale).



(c) Average branching factor (linear scale).



(d) Maximum branching factor (log-scale).

Figure 7.2: Evaluation dataset: Distribution of feature tree properties across CFMs, FMs, and the complete dataset

straints (i.e. multiple constraints for the same feature pair) are excluded. Likewise, at most

$$\frac{n(n-1)}{2}$$

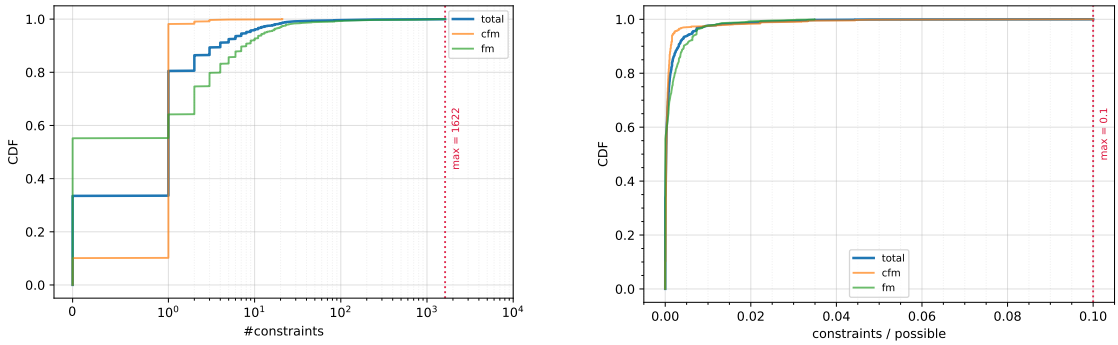
undirected exclude constraints can exist, as exclude constraints correspond to undirected edges between distinct features.

The density is therefore defined as

$$\frac{c}{n(n-1) + \frac{n(n-1)}{2}},$$

where c denotes the number of cross-tree constraints. This normalization enables meaningful comparisons across models of different sizes.

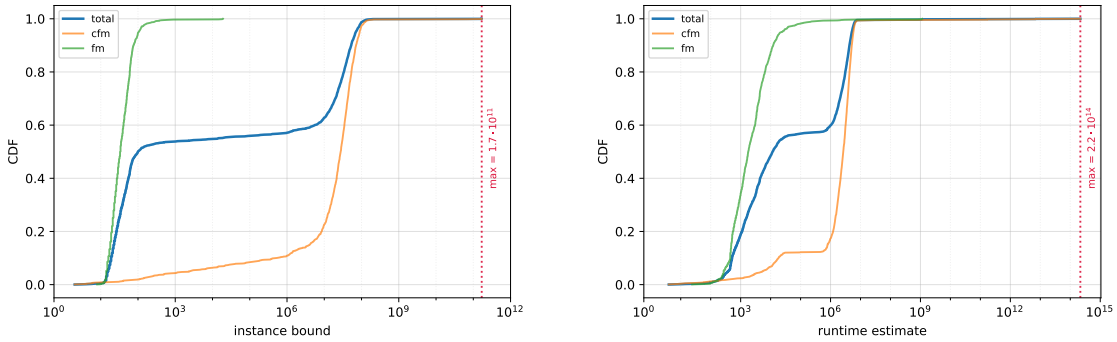
Note that this density definition is simplified, and the previous statement that we normalize by the number of theoretically possible cross-tree relations is not strictly correct. In particular, CFMs allow multiple require or exclude constraints between the same pair of features. Such multi-edges are not merely a modelling artifact, but can be necessary to express constraints such as "if there are 5 `Bathrooms` in the house, we need 10 `Solar Panels`, and if there are 10 `Bathrooms`, we need 20" (see Section 2.2.1).



(a) Number of constraints (log-scale). (b) Constraint density (linear scale).

Figure 7.3: Evaluation dataset: Distribution of cross-tree constraint characteristics across CFMs, FMs, and the complete dataset

Finally, Figure 7.4 shows two derived runtime estimators. The total instance bound, introduced in Section 6.3 and Section 7.2.1.2, serves as a predictor for the encoding size and, consequently, as an indicator of the practical runtime of the CP-SAT baseline. The DP runtime estimator corresponds to the asymptotic runtime bound of ComputeDP derived in Section 5.2.1.



(a) Total instance bound (log-scale).

(b) Estimated DP runtime (log-scale).

Figure 7.4: Evaluation dataset: Distribution of total instance bounds and estimated DP runtime across CFMs, FMs, and the complete dataset

metric	min	median	mean	max
Number of features	2	39	73.74	19,146
Tree height	1	4	4.76	24
Avg. branching factor	1	2.22	2.50	8.80
Max branching factor	1	6	6.39	231
Cross-tree constraints	0	1	4.21	1,622
Constraint density	0	0	0	0.10
Total instance bound	2	100	$1.78 \cdot 10^8$	$1.68 \cdot 10^{11}$
DP runtime estimator	4	11,664	$9.30 \cdot 10^{10}$	$2.17 \cdot 10^{14}$

Table 7.6: Evaluation dataset: Minimum, median, mean, and maximum values of feature tree properties, cross-tree constraints, and runtime estimation metrics

7.2.4 Hardware and Software Environment

All experiments are conducted on a personal computer running Windows 11, equipped with an AMD Ryzen 9 3900X processor (12 cores) and 32 GB of RAM. The experiments are executed inside the Windows Subsystem for Linux (WSL) 2, running Ubuntu 24.04.4 LTS (Noble Numbat).

WSL is configured with a memory limit of 24 GB and a swap limit of 500 GB. The large swap limit ensures that a solver does not run out of memory during execution. No explicit CPU affinity restrictions are applied.

CFMTOOLS (see Chapter 6) is executed using CPython 3.14.3. The CSP-solver baseline (CP-SAT) relies on OR-Tools 9.15.6755. The native backend is compiled with `rustc` 1.93.0 in release mode using the compiler flag `-C target-cpu=native`, which enables target-specific instruction set optimizations for the host CPU.

The Rust release profile uses optimization level 3, link-time optimization (LTO) ("fat"), a single code generation unit, panic abort semantics, symbol stripping, and disabled debug information. These settings aim to maximize runtime performance in accordance with the official Rust documentation.[6]

Listing 7.1 shows the complete Rust release profile used for compiling the native backend, and Listing 7.2 shows the corresponding build command.

```
[profile.release]
opt-level = 3
lto = "fat"
codegen-units = 1
panic = "abort"
strip = true
debug = false
```

Listing 7.1: Rust release profile for compiling the native backend

```
RUSTFLAGS="-C target-cpu=native" maturin develop --release
```

Listing 7.2: Build command for compiling the native backend and installing the Python package in development mode

7.3 Results

This section presents and analyses the empirical results of the experiments described in Section 7.2, with respect to the goals and research questions formulated in Section 7.1.

7.3.1 RQ1: Time to First Configuration

Tables 7.7 to 7.9 and Figures 7.5 and 7.6 summarize the empirical results for TTFC.

[6] https://doc.rust-lang.org/cargo/reference/profiles.html

Success Rate and Robustness

Table 7.7 reveals a substantial difference in success rates between the two approaches. The DP-based approach successfully produces at least one configuration for 2458 out of 2489 models, corresponding to a success rate of 98.8%. In contrast, CP-SAT solves only 1387 models (55.7%), with the majority of failures (1001 models) caused by construction timeouts.

This behaviour is further corroborated by the contingency analysis in Table 7.8. Most models solved by CP-SAT (1368 out of 1387) are also solved by DP, whereas a large number of models (1090) are solved exclusively by DP. 19 models are solved exclusively by CP-SAT, and 12 models are solved by neither approach.

Overall, this indicates that the DP-based approach dominates CP-SAT in terms of robustness and is able to handle a significantly broader range of models. Only a small number of exceptions exist in which CP-SAT solves a model that DP cannot.

Table 7.7: RQ1: Overview of solver outcomes and success rates

Solver	Solved	Timeout		Total	Success Rate (%)
		Search	Construction		
DP	2458	26	5	2489	98.8
CP-SAT	1387	101	1001	2489	55.7

Table 7.8: RQ1: Contingency comparison of solver outcomes

DP \ CP-SAT	Solved	Search Timeout	Construction Timeout
Solved	1368	98	992
Search Timeout	19	1	6
Construction Timeout	0	2	3

Timeout-Aware Latency (PAR-1)

Table 7.9 reports the PAR-1 score for both solvers and TTFC variants, further emphasizing the dominance of DP over CP-SAT.

Concretely, for the end-to-end variant, CP-SAT exhibits a PAR-1 score of 490,629, compared to 3,316 for DP, corresponding to a nearly 148× speedup. For the cached variant, the gap decreases but remains substantial (13×).

This reduction in the relative gap for the cached variant is expected. Construction timeouts are penalized with $\mathcal{T}_{\text{global}} = 20$ minutes in the end-to-end metric, but only with $\mathcal{T}_{\text{enum}} = 20$ seconds in the cached metric. Since CP-SAT exhibits a large number of construction timeouts, this change reduces its penalty and thus narrows the relative difference.

In terms of absolute runtimes, the end-to-end PAR-1 scores correspond to a total runtime of approximately 2 hours and 18 minutes for DP, whereas for CP-SAT it amounts to roughly 14 days and 3 hours. For the cached variants, the total runtime reduces to about 33 minutes for cached DP and approximately 7 hours and 20 minutes for cached CP-SAT.

Table 7.9: RQ1: PAR-1 of TTFC (in milliseconds)

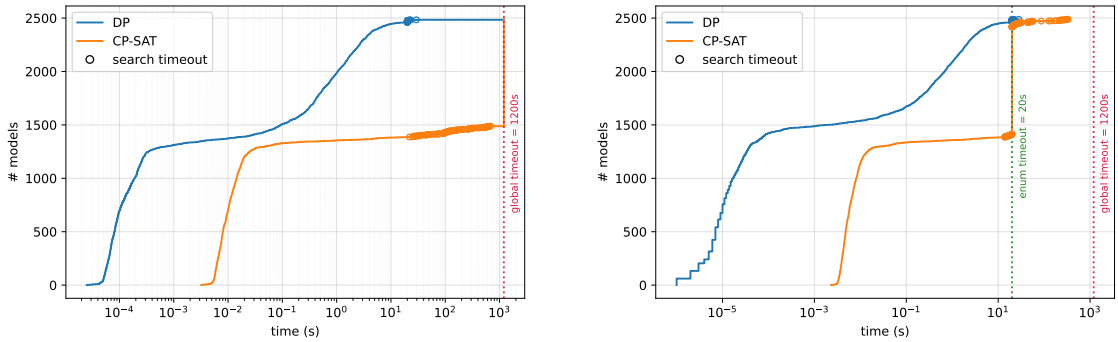
	DP		CP-SAT		Speedup	
	E2E	Cached	E2E	Cached	E2E	Cached
PAR-1	3,316.348	787.517	490,628.763	10,615.031	147.94	13.48

Cactus Plots

The cactus plots in Figure 7.5 illustrate the overall TTFC behaviour of both solvers. For both cached and non-cached variants, there is an initial steep increase up to approximately 1250 solved models, followed by a plateau.

However, the time scale differs substantially. DP solves the initial set of models within approximately $5 \cdot 10^{-5}$ to $3 \cdot 10^{-4}$ seconds, whereas CP-SAT requires roughly $5 \cdot 10^{-3}$ to $2 \cdot 10^{-2}$ seconds, corresponding to a difference of about two orders of magnitude.

After this initial phase, CP-SAT makes almost no further progress and remains largely constrained by search and construction timeouts. In contrast, DP continues to solve additional models and eventually covers almost the entire dataset within the time limit. This behaviour is consistent across both variants, with the cached curves uniformly shifted towards lower TTFC values due to the removal of preprocessing overhead.



(a) End-to-end. (b) Cached.

Figure 7.5: RQ1: Cactus plots for TTFC (in seconds and log-scale)

Performance Profiles

The performance profiles in Figure 7.6 provide a relative, per-model comparison and further confirm the dominance of DP. In the end-to-end setting, DP is the fastest solver on approximately 98% of all models, whereas CP-SAT is fastest on the remaining 2%.

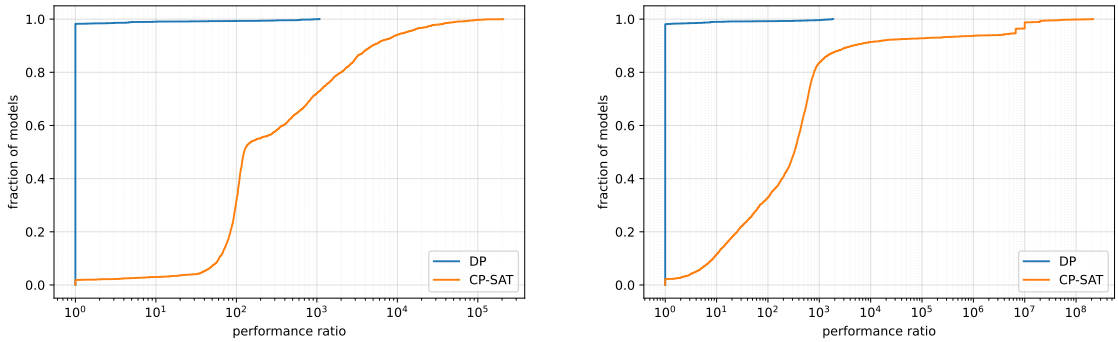
Moreover, CP-SAT requires substantial slowdowns to approach the performance of DP. In the end-to-end setting, only about 32% of models are solved within a factor of 100 of the best solver, and roughly 72% within a factor of 1000. Intuitively, even if DP were slowed down by a factor of 1000 (or CP-SAT improved accordingly), DP

would still be the fastest solver on approximately 28% of the dataset (with respect to TTFC).

Even for extremely large factors, CP-SAT does not fully close the gap. Specifically, a factor of approximately 207,183 would be required for CP-SAT to match or outperform DP on all models.

In contrast, for DP, even a speedup factor of 10 makes it the fastest solver on approximately 99% of the dataset; at 100×, on 99.3%; and at about 1081×, on all models.

In the cached setting, this difference becomes even more pronounced. Specifically, CP-SAT requires a speedup factor of approximately 326× to cover 50% of the models, compared to 125× in the end-to-end setting. Moreover, a speedup of $2.1 \cdot 10^8$ is required for CP-SAT to be at least as fast as DP on all models.



(a) End-to-end. (b) Cached.

Figure 7.6: RQ1: Performance profiles for TTFC (log-scale)

Summary

Overall, the results clearly demonstrate that the DP-based unranking approach achieves substantially lower TTFC than the CP-SAT baseline across the vast majority of models. This advantage is consistent across all evaluation perspectives.

In terms of robustness, DP solves nearly the entire dataset (98.8%), whereas CP-SAT succeeds on only 55.7% of models, with a large fraction of failures due to construction timeouts. As a result, DP is applicable to a significantly broader range of models.

The performance profiles further highlight this dominance. DP is the fastest solver on approximately 98% of all models and remains superior even under substantial hypothetical slowdowns.

This behaviour is also reflected in the total runtime for the entire dataset as captured by the PAR-1 metric. When using DP instead of CP-SAT, solving the full dataset (in the sense of finding at least one solution per model) requires only two hours, compared to two weeks for CP-SAT.

Despite this clear overall dominance, a small number of exceptions remain: CP-SAT outperforms DP on a few models and can solve some instances where DP fails. However, these cases are rare and do not affect the overall conclusion.

In summary, DP achieves significant improvements in TTFC over CP-SAT, often exceeding two orders of magnitude and in some cases even reaching three or more orders of magnitude, making it clearly superior with respect to TTFC.

7.3.2 RQ2: Enumeration Throughput

Figures 7.7 and 7.8 summarize the empirical results for enumeration throughput.

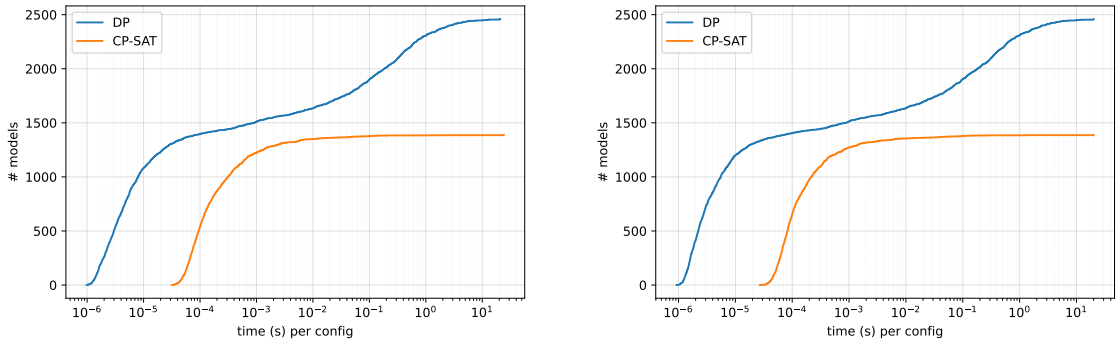
Cactus Plots

The cactus plots in Figure 7.7 illustrate the overall enumeration behaviour in terms of seconds per configuration. Similar to TTFC, both solvers exhibit an initial steep increase corresponding to models with high throughput, although the absolute scales differ by roughly one order of magnitude.

Specifically, the steep increase for DP ranges from approximately $2 \cdot 10^{-6}$ to $2 \cdot 10^{-5}$ seconds per configuration, whereas CP-SAT ranges from roughly $4 \cdot 10^{-5}$ to $4 \cdot 10^{-4}$.

After this initial phase, CP-SAT plateaus, whereas DP continues to make steady progress and eventually covers almost the entire dataset within the enumeration time limit of $\mathcal{T}_{\text{enum}} = 20$ seconds.

As in the TTFC analysis, the cached variant results in a uniform shift towards lower seconds-per-configuration values (i.e., higher throughput).



(a) End-to-end. (b) Cached.

Figure 7.7: RQ2: Cactus plots for enumeration throughput (in seconds per configuration and log-scale)

Performance Profiles

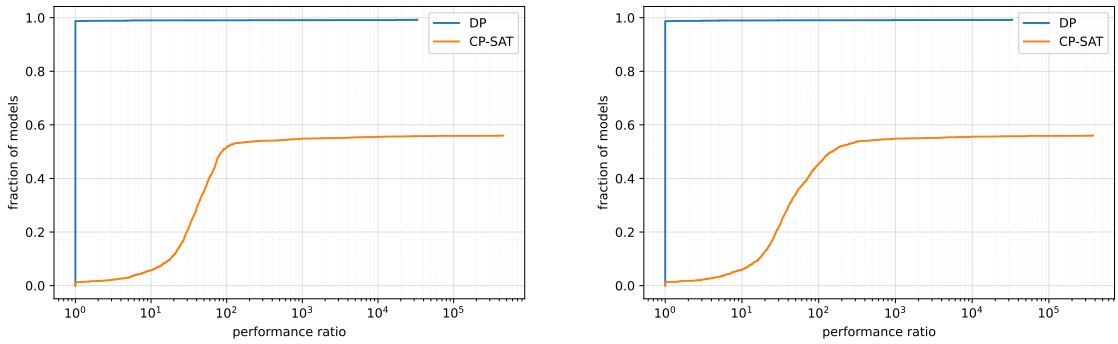
The performance profiles in Figure 7.8 provide a per-model comparison of throughput and further confirm the dominance of DP.

In the end-to-end setting, DP achieves the highest throughput on 98.7% of all models, leaving only 1.3% where CP-SAT performs better. Moreover, CP-SAT requires substantial speedups to approach the performance of DP. Even with a 100× improvement, it matches the throughput of DP on only 52% of models and plateaus at roughly 56%.

This plateau is explained by timeouts: for almost half of the dataset, CP-SAT fails to produce any solutions, resulting in zero throughput (i.e., infinite time per

configuration). In contrast, DP continues to produce configurations for almost all models.

The cached variant exhibits a similar overall pattern in terms of curve shape, but differs slightly in coverage. In particular, the CP-SAT curve is slightly flatter, indicating that larger speedup factors are required for CP-SAT to match or exceed the throughput of DP on a given fraction of models.



(a) End-to-end. (b) Cached.

Figure 7.8: RQ2: Performance profiles for enumeration throughput (log-scale)

Summary

Overall, the DP-based approach achieves substantially higher enumeration throughput than CP-SAT across the vast majority of models.

This advantage is already evident in the cactus plots. While both solvers exhibit an initial steep increase, DP operates consistently at a significantly lower time per configuration (i.e., higher throughput) and continues to make progress across almost the entire dataset. In contrast, CP-SAT plateaus and fails to enumerate solutions for almost half of the dataset.

The performance profiles further reinforce this picture. DP achieves the highest throughput on approximately 98.7% of all models and remains dominant even under substantial hypothetical speedups of CP-SAT. Due to frequent timeouts, CP-SAT is unable to match the throughput of DP on nearly half of the dataset.

As in the TTFC analysis, there are a few models for which CP-SAT achieves higher throughput. However, these cases are rare and do not affect the overall conclusion.

In summary, DP consistently delivers significantly higher enumeration throughput than CP-SAT, achieving improvements of one to two orders of magnitude on approximately half of the models. For the remaining instances, CP-SAT fails to produce any solutions within the time limit, resulting in effectively unbounded relative improvements.

7.3.3 RQ3: Influence of Model Type

As described in Section 7.2.1, to answer Research Question 3, we partition the dataset by model type and apply the same aggregation methodology as for Research Questions 1 and 2.

7.3.3.1 Time to First Configuration

Tables 7.10 and 7.11 and Figures 7.9 and 7.10 summarize the empirical results for TTFC partitioned by model type.

Success Rate and Robustness

Table 7.10 highlights a clear contrast between the two model classes. For FMs, both solvers achieve near-perfect robustness. CP-SAT solves all 1292 models (100%), while DP solves 1273 models (98.5%), with only 19 search timeouts and no construction timeouts. Thus, both approaches are broadly applicable in this setting.

In contrast, for CFMs, the behaviour changes drastically. While DP maintains a high success rate of 99.0% (1185 solved models), CP-SAT solves only 95 out of 1197 models (7.9%). The vast majority of failures (1001 models) are due to construction timeouts, indicating that the CP-SAT encoding does not scale to the more expressive CFM setting.

This difference is further reflected in the contingency analysis in Table 7.11, which shows that there are no CFMs for which DP fails while CP-SAT succeeds.

Overall, these results indicate that DP is strictly more robust than CP-SAT on CFMs. In contrast, for FMs, CP-SAT exhibits a slight robustness advantage; however, the difference is small.

Table 7.10: RQ3: Overview of solver outcomes and success rates partitioned by model type

(a) FM.

Solver	Solved	Timeout		Total	Success Rate (%)
		Search	Construction		
DP	1273	19	0	1292	98.5
CP-SAT	1292	0	0	1292	100.0

(b) CFM.

Solver	Solved	Timeout		Total	Success Rate (%)
		Search	Construction		
DP	1185	7	5	1197	99.0
CP-SAT	95	101	1001	1197	7.9

Table 7.11: RQ3: Contingency comparison of solver outcomes partitioned by model type

(a) FM

DP \ CP-SAT	Solved	Search Timeout	Construction Timeout
Solved	1273	0	0
Search Timeout	19	0	0
Construction Timeout	0	0	0

(b) CFM

DP \ CP-SAT	Solved	Search Timeout	Construction Timeout
Solved	95	98	992
Search Timeout	0	1	6
Construction Timeout	0	2	3

Timeout-Aware Latency (PAR-1)

The PAR-1 scores shown in Table 7.12 further emphasize the differences between the two model classes.

For FMs, CP-SAT achieves substantially lower PAR-1 scores than DP, outperforming it by a factor of 12.5× in the end-to-end setting and 20× in the cached setting. Nevertheless, the absolute runtimes remain modest for both solvers (6 minutes and 35 seconds for DP vs. 32.76 seconds for CP-SAT in the end-to-end setting, and 6 minutes and 30 seconds vs. 18.75 seconds in the cached setting), reflecting that FMs are generally easier to solve in our dataset.

In contrast, for CFMs, DP clearly outperforms CP-SAT. It achieves a PAR-1 improvement of approximately 155× in the end-to-end setting and 17× in the cached setting. In absolute terms, this corresponds to a total runtime of approximately 2 hours and 11 minutes (end-to-end) and 26 minutes (cached) for DP, compared to roughly 14 days and 3 hours (end-to-end) and 7 hours and 20 minutes (cached) for CP-SAT, highlighting a stark difference in scalability.

Table 7.12: RQ3: PAR-1 of TTFC (in milliseconds) partitioned by model type

(a) FM

	DP		CP-SAT		Speedup	
	E2E	Cached	E2E	Cached	E2E	Cached
PAR-1	305.883	301.762	25.352	14.511	0.08	0.05

(b) CFM

	DP		CP-SAT		Speedup	
	E2E	Cached	E2E	Cached	E2E	Cached
PAR-1	6,565.738	1,311.823	1,020,168.953	22,056.863	155.38	16.81

Cactus Plots

Compared to the TTFC analysis in Section 7.3.1 over the full dataset, where we observed an initial steep increase followed by a plateau, the cactus plots in Figure 7.9 provide a more fine-grained perspective. In particular, partitioning the data into FMs and CFMs cleanly separates these two regions.

For FMs, both solvers exhibit the steep increase observed in the overall analysis. In this regime, DP is consistently slower than CP-SAT. Towards the end of the curve, however, CP-SAT solves slightly more models within the time limit, consistent with its perfect success rate on FMs. In contrast, for CFMs, CP-SAT reaches an almost immediate plateau, whereas DP continues to make progress and solves nearly the entire dataset within the time limit.

Overall, partitioning the dataset reveals that the bimodal behaviour observed in Section 7.3.1 is not incidental but corresponds exactly to the distinction between FMs and CFMs.

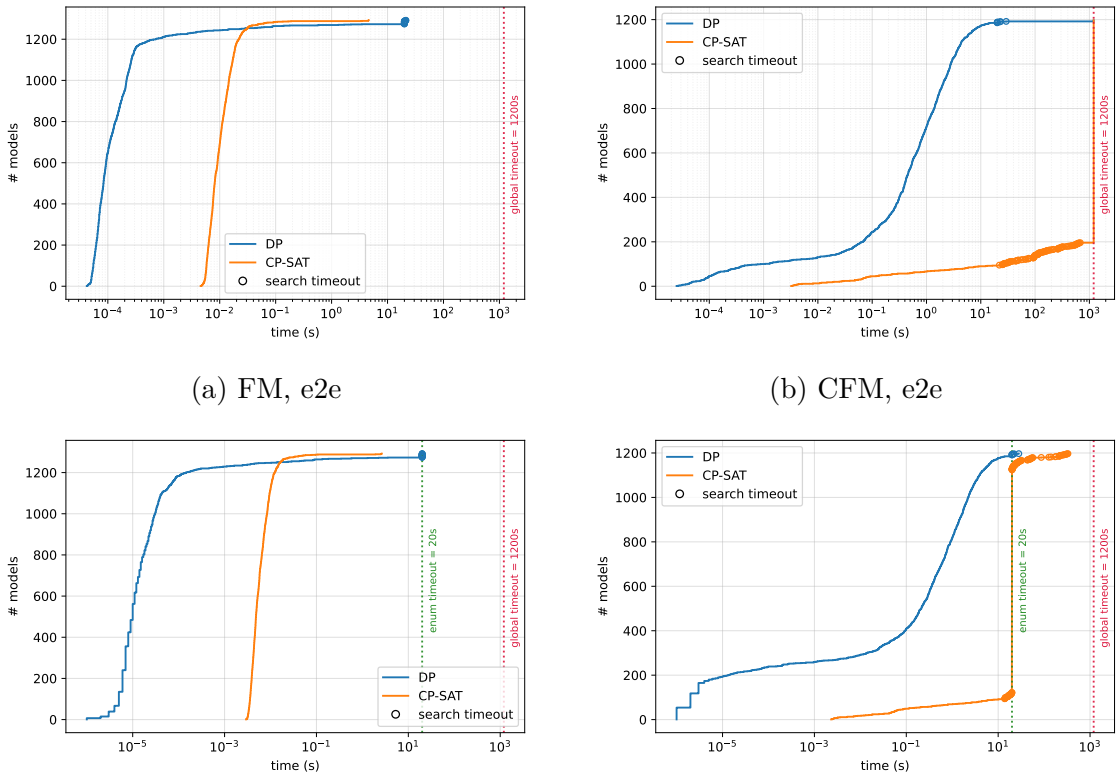
Performance Profiles

In contrast to the PAR-1 score, where we observed that CP-SAT outperforms DP on FMs by a factor of 12.5× in the end-to-end setting and 20× in the cached setting, the performance profiles in Figure 7.10 reveal a more complete picture.

For FMs, DP is the fastest solver on approximately 96.75% of the models. To match or outperform CP-SAT on all models, DP would require a speedup of about 1081×. In contrast, CP-SAT is the fastest solver on only 3.25% of the models and requires speedups of two orders of magnitude to cover more than 90% of the dataset. However, the speedup required for CP-SAT to reach 100% coverage is lower than for DP, at approximately 312×.

These observations show that the PAR-1 gap is primarily driven by a small number of difficult models on which DP exhibits comparatively high latency.

In the cached setting, the situation becomes less favourable for CP-SAT. To match DP on 90% of the FMs, CP-SAT now requires speedups approaching three orders



(a) FM, e2e

(b) CFM, e2e

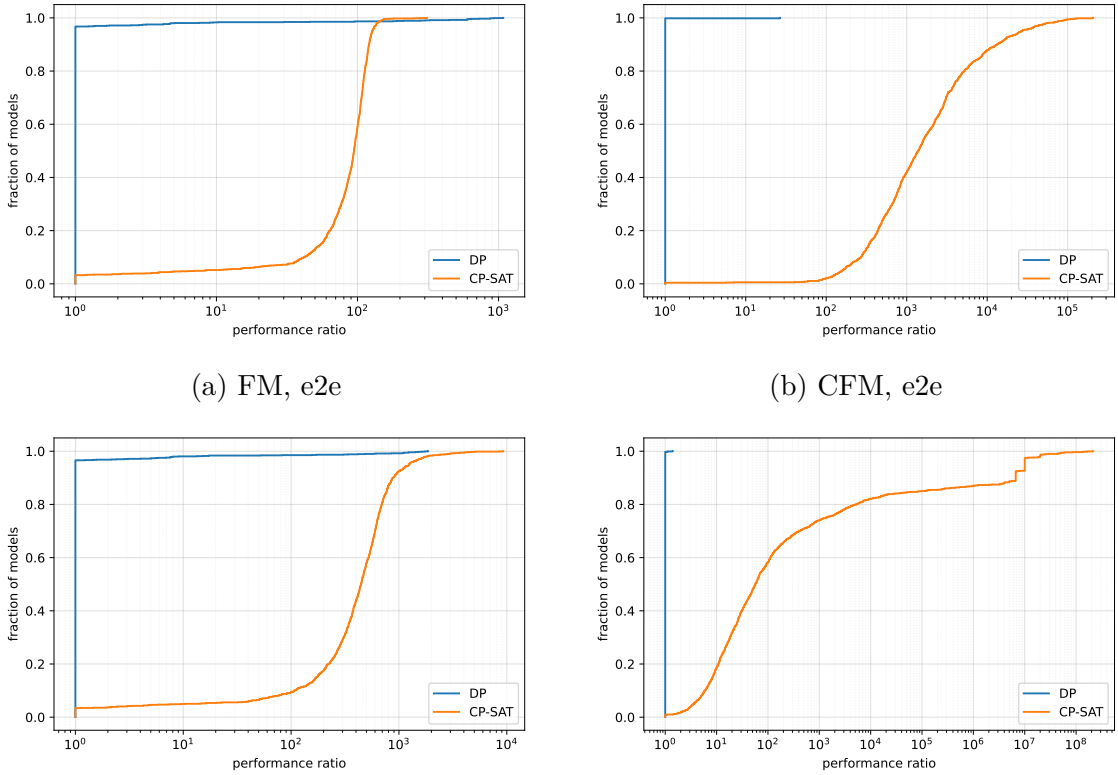
(c) FM, cached

(d) CFM, cached

Figure 7.9: RQ3: Cactus plots for TTFC partitioned by model type (in seconds and log-scale)

of magnitude (877×). Moreover, the maximum speedup required to achieve 100% coverage increases substantially and exceeds that of DP (9283× vs. 1864×).

For CFMs, the performance gap becomes even more pronounced. The DP approach requires only a modest speedup of approximately 26.8× in the end-to-end setting and 1.4× in the cached setting to match CP-SAT across all models. In contrast, CP-SAT would need speedups of approximately 207,183× in the end-to-end setting and more than $2.1 \cdot 10^8$× in the cached setting to achieve full coverage.



(a) FM, e2e

(b) CFM, e2e

(c) FM, cached

(d) CFM, cached

Figure 7.10: RQ3: Performance profiles for TTFC partitioned by model type (log scale)

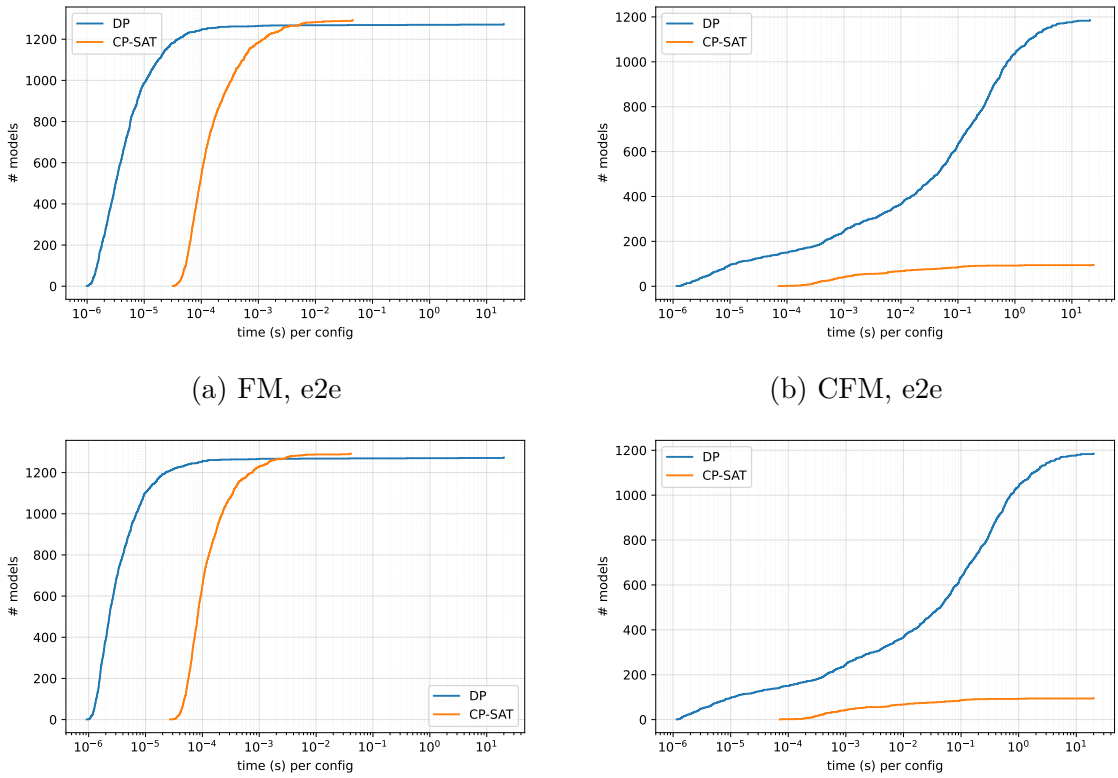
7.3.3.2 Enumeration Throughput

Figures 7.11 and 7.12 summarize the empirical results for enumeration throughput partitioned by model type.

Cactus Plots

As in the TTFC analysis, the cactus plots in Figure 7.11 separate the curves observed in the overall analysis (Sections 7.3.1 and 7.3.2) into their respective regions. In particular, we again observe a steep increase for both DP and CP-SAT on FMs, whereas CP-SAT exhibits an early plateau on CFMs, while DP continues to make progress.

Furthermore, the effect of caching is negligible for throughput, as the curves for the end-to-end and cached variants are nearly identical across both model types.



(a) FM, e2e

(b) CFM, e2e

(c) FM, cached

(d) CFM, cached

Figure 7.11: RQ3: Cactus plots for enumeration throughput partitioned by model type (in seconds per configuration and log-scale)

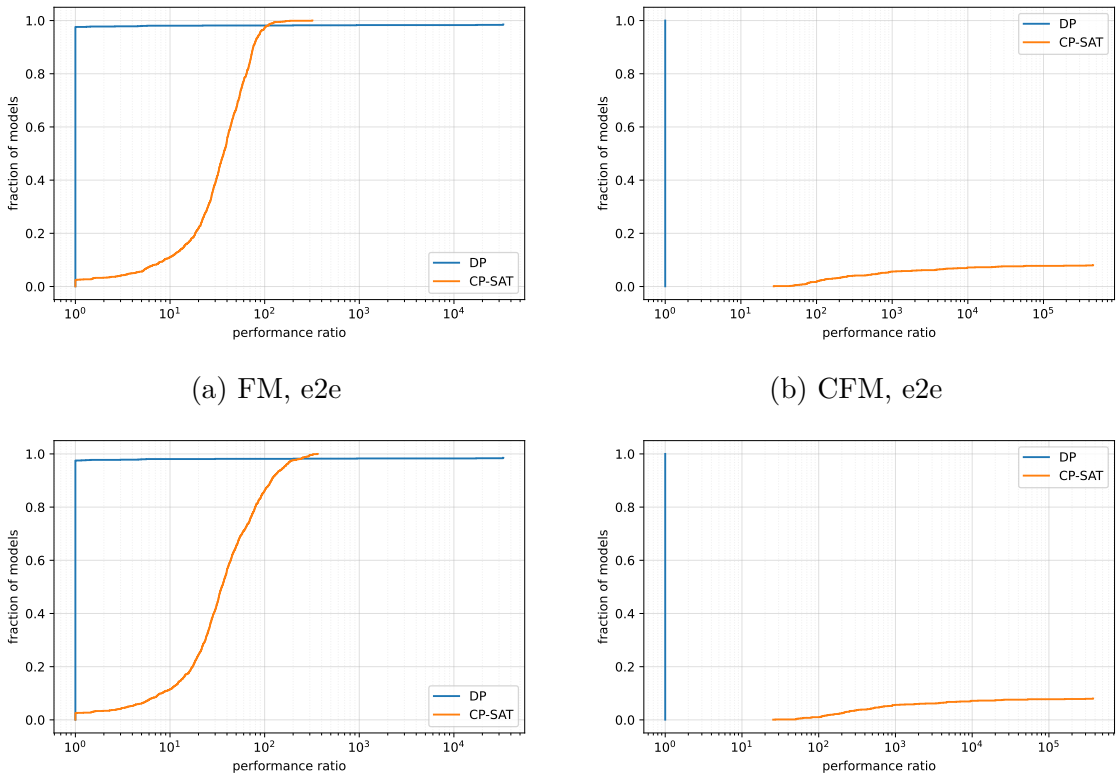
Performance Profiles

The performance profiles in Figure 7.12 exhibit the same qualitative behaviour as observed in the partitioned TTFC analysis.

For FMs, DP achieves the highest throughput on the majority of models (97.6%), whereas CP-SAT requires speedups of one to two orders of magnitude to match DP on a large fraction of the dataset.

For CFMs, the difference becomes even more pronounced than in the TTFC setting. The DP-based approach achieves the highest throughput on all CFMs, without requiring any speedup. In contrast, the speedups required for CP-SAT to match the performance of DP are extreme: even with a speedup factor of 100,000, CP-SAT reaches comparable throughput on only 7.8% of the models, both in the end-to-end and cached settings.

Notably, the performance profile of DP for CFMs (see Figures 7.12b and 7.12d) closely resembles that of an idealized "oracle" solver [128, 129] that is always optimal. Such a solver would exhibit a step function with a jump at $x = 1$, while all competing solvers remain at 0 for all $\tau \geq 1$. The DP profile approximates this behaviour, further highlighting its dominance in the CFM setting.



(a) FM, e2e

(b) CFM, e2e

(c) FM, cached

(d) CFM, cached

Figure 7.12: RQ3: Performance profiles for enumeration throughput partitioned by model type (log-scale)

7.3.3.3 Summary

Overall, the influence of model type has a decisive impact on the performance of the two approaches and fully explains the aggregate behaviour observed in Sections 7.3.1 and 7.3.2.

For FMs, both solvers perform well and achieve near-perfect robustness. CP-SAT solves all 1292 models (100%), while DP solves 98.5%. In terms of latency, CP-SAT achieves lower PAR-1 scores, outperforming DP by up to 20×. However, this advantage is driven by a small subset of models. The performance profiles show that DP is nevertheless the fastest solver on the vast majority of models (more than 96%), both in terms of TTFC and throughput.

Moreover, CP-SAT requires speedups of one to two orders of magnitude to match DP on most of the dataset (at least 90% of models). Overall, both approaches are effective on FMs, with CP-SAT exhibiting a slight advantage in robustness and aggregate latency, while DP dominates TTFC and throughput on a per-model basis for the majority of models.

In contrast, for CFMs, the performance gap becomes extreme. The DP-based approach maintains a success rate of 99.0%, whereas CP-SAT solves only 7.9% of the models. This lack of robustness directly translates into a substantial performance gap: in terms of PAR-1, DP achieves speedups of approximately 155×, reducing the total runtime from roughly 14 days and 3 hours to about 2 hours and 11 minutes.

The cactus plots further emphasize this difference. For CFMs, CP-SAT reaches an almost immediate plateau and fails to make further progress, whereas DP continues to solve nearly the entire dataset within the time limits.

This gap is also reflected in the required relative speedups. For TTFC, CP-SAT frequently requires speedups of three to four orders of magnitude to match DP, and in extreme cases up to 8 orders of magnitude.

In terms of throughput, DP is the best solver on all CFMs, while CP-SAT remains ineffective even under extreme hypothetical speedups: even with a 100,000× improvement, it matches DP on only 7.8% of the models. Beyond this point, the performance profile plateaus, as the remaining models correspond to failures, implying effectively unbounded required speedups.

Taken together, these results show that the overall superiority of DP observed in Research Questions 1 and 2 is due not only to its scalability to CFMs, but also to its dominance on the majority of FMs, where it typically outperforms CP-SAT by one to two orders of magnitude. The only setting in which CP-SAT remains competitive, and occasionally superior, is a small subset of FMs for which DP fails to find a solution.

7.3.4 RQ4: Empirical Uniformity

Tables 7.13 and 7.14 summarize the results of the uniformity validation.

Overall, the empirical rejection rates for both sampling algorithms are close to the expected significance level $\alpha = 0.05$, as predicted by the methodology in Section 7.2.2.3. For UniSample-BT, all model–sampler pairs yield rejection rates whose 95% confidence intervals contain α, indicating no statistically significant deviation from uniform sampling at the per-model level.

For UniSample-UR, the results are likewise consistent across most models. However, for Model 6, the observed rejection rate lies slightly outside the expected range, with the corresponding confidence interval not containing α (note that in the table

it appears contained due to rounding, while the exact upper bound is approximately 0.049769). According to our testing procedure, this constitutes a statistically significant deviation from uniformity for this specific model.

Despite this isolated deviation, the global aggregation across all models shows that the overall rejection rates for both samplers remain within the expected confidence bounds. Hence, at the global level, the null hypothesis of uniform sampling cannot be rejected for either algorithm.

Taken together, these results provide strong empirical evidence that both sampling procedures behave as intended and generate samples consistent with the uniform distribution in the vast majority of cases. The single deviation observed for UniSample-UR on Model 6 is minor. In particular, even a 96% confidence interval would contain α and thus would not lead to a rejection of the null hypothesis. Nevertheless, more extensive experiments, including a larger set of models and a higher number of runs, should be considered in future work to further substantiate these findings.

Table 7.13: RQ4: Uniformity validation per model. For each model–sampler pair, the empirical rejection rate of the χ^2 test (with $\alpha = 0.05$) is reported together with its 95% Clopper–Pearson confidence interval (CI). Uniformity is supported if the confidence interval contains the expected rejection probability $\alpha = 0.05$.

Sampler	Model	Rejection Rate	CI	Uniform	Avg. Sampling Time (µs)
UniSample-UR	Model 1	0.051	[0.047, 0.056]	✓	10.26
	Model 2	0.050	[0.046, 0.054]	✓	25.33
	Model 3	0.048	[0.044, 0.053]	✓	15.78
	Model 4	0.049	[0.045, 0.054]	✓	2.84
	Model 5	0.050	[0.046, 0.055]	✓	4.13
	Model 6	0.045	[0.041, 0.050]	✗	26.23
	Model 7	0.050	[0.045, 0.054]	✓	8.31
	Model 8	0.048	[0.044, 0.052]	✓	8.39
UniSample-BT	Model 1	0.050	[0.046, 0.055]	✓	5.29
	Model 2	0.050	[0.045, 0.054]	✓	15.82
	Model 3	0.049	[0.045, 0.054]	✓	9.95
	Model 4	0.053	[0.048, 0.057]	✓	3.81
	Model 5	0.051	[0.047, 0.055]	✓	1.92
	Model 6	0.049	[0.045, 0.054]	✓	16.93
	Model 7	0.049	[0.045, 0.053]	✓	6.71
	Model 8	0.047	[0.043, 0.052]	✓	6.82

Average Sampling Time

An additional observation from the uniformity experiments concerns the average sampling time. In most cases, UniSample-BT is faster than UniSample-UR, with the notable exception of Model 4. This behaviour is consistent with the theoretical analysis in Section 5.2.5, where the performance trade-off between UniSample-BT

Table 7.14: RQ4: Global uniformity validation aggregated over all models. The table reports the overall rejection rate and corresponding 95% Clopper–Pearson confidence interval (CI) for each sampler. Uniformity is supported if the confidence interval contains $\alpha = 0.05$.

Sampler	Rejection Rate	CI	Uniform
UniSample-UR	0.049	[0.048, 0.051]	✓
UniSample-BT	0.050	[0.048, 0.051]	✓

and UniSample-UR was shown to depend strongly on the structural properties of the model.

A key factor is that all considered models exhibit only small feature-instance cardinalities for non-leaf features (typically 1 or 2), while larger cardinalities, if present, occur only at leaf nodes. In the latter case, multiset generation is trivial, since leaf features admit only a single configuration. As a consequence, the acceptance probability in UniSample-BT remains high (i.e., q_i is close to 1), and the expected number of rejection iterations remains small. In particular, the factorial blow-up of the rejection cost (see Section 5.2.2.2) does not occur in these models. This allows UniSample-BT to fully exploit its advantages discussed in Section 5.2.5.

The model Model 4 constitutes an exception, as it contains a non-leaf feature with a feature-instance cardinality $\langle 0, 3 \rangle$. In this case, the rejection overhead of UniSample-BT becomes noticeable, reducing its performance advantage over UniSample-UR.

7.4 Discussion of Findings and Failure Cases

This section summarizes the findings from Section 7.3 by giving compact answers to the research questions and by examining the failure cases in which DP fails while CP-SAT performs better.

7.4.1 Answers to Research Questions

Research Question 1 (TTFC). The DP-based approach achieves substantially lower time to first configuration (TTFC) and markedly higher robustness than the CP-SAT baseline. Over the full dataset, DP produces a valid configuration for almost all models (about 99%), whereas CP-SAT succeeds on only a little more than half (about 56%), with most failures caused by construction timeouts. This is also evident in the total runtime, where in the end-to-end setting the total runtime reduces from 2 weeks to 2 hours. In a per-model comparison, DP outperforms CP-SAT by often *one to two* and sometimes *three or more* orders of magnitude.

Research Question 2 (Enumeration throughput). The DP-based approach delivers substantially higher enumeration throughput. In a per-model comparison, DP achieves on half of the dataset *one to two orders* of magnitude of improvements in enumeration throughput, whereas on the other half theoretically *infinite* speedups, due to the inability of CP-SAT to produce any solutions.

Research Question 3 (Influence of model type). Model type decisively explains the overall performance picture. For classical FMs, both approaches are robust (near-100% success), and CP-SAT can exhibit an advantage in total runtime, driven by a small subset of harder FMs; however, even here DP is still fastest in a per-model comparison, on the large majority of individual models and differences are typically within *one to two orders of magnitude*. For proper CFMs, the situation changes severely: DP remains highly robust (about 99% success), whereas CP-SAT succeeds on only a small minority (single-digit percent), overwhelmingly failing during model construction. In total, the overall dominance of DP in Research Questions 1 and 2, is not only due to scalability to CFMs, but also because of the dominance on the majority of FMs.

Research Question 4 (Empirical uniformity). Yes. For models with sufficiently small configuration spaces to allow exhaustive enumeration, the observed output frequencies of both uniform samplers are empirically consistent with the expected uniform distribution. Per-model rejection rates are generally close to the expected, and the aggregated confidence intervals support uniformity both per model (almost everywhere) and globally. The only exception is a small deviation for UniSample-UR on one model, but it is isolated and disappears under global aggregation, indicating no systematic bias.

7.4.2 Interpretation of Failure Cases

Although DP dominates on the majority of both classical FMs and proper CFMs instances, there are rare cases in which this is not the case. In particular, the most interesting exceptions are those in which CP-SAT succeeds while DP fails (due to either a construction or an enumeration timeout), as well as cases in which both approaches fail but CP-SAT exhibits the better timeout category, i.e., an enumeration timeout instead of a construction timeout.

DP Fails - CP-SAT Succeeds

The only models where DP fails while CP-SAT succeeds are classical FMs. Moreover, in all those cases, DP fails not due to construction but because it fails to find a valid configuration during the enumeration phase (see Section 7.3.3).

All of these models contain cross-tree constraints and are large enough that DP is not able to explore the whole unconstrained configuration space within the enumeration time window of $\mathcal{T}_{\text{enum}} = 20$ seconds. In particular, three out of the four automotive models (see Section 7.2.3) are among those failed models.

This behaviour is consistent and expected, since DP enumerates all configurations from the unconstrained configuration space, without incorporating cross-tree constraints, which are only checked after the full configuration has been built. As a result, DP is likely to fail for models with cross-tree constraints, where it is infeasible to exhaustively explore the whole unconstrained configuration space.

In contrast, CP-SAT, as a SAT-based solver, incorporates cross-tree constraints directly, which makes it more likely to solve such models.

DP Construction Timeout - CP-SAT Enumeration Timeout

Construction timeouts for either solver occur only on proper CFMs, as shown in Section 7.3.3. Among these, there are a few instances in which DP times out during construction while CP-SAT still progresses to enumeration. In particular, this occurs for `cfm\repo\model_22` and `cfm\repo\model_23`. The corresponding models are shown in Figure 7.13.

Although both models have comparatively low total instance bounds (374,476), they exhibit extremely large DP runtime estimators (168,280,077,600). This is due to the relatively large group-instance cardinality in the root-group and the dense and large feature-instance cardinalities at the leaves.

Concretely, a naive implementation of ComputeDP would, for each of the five children, iterate over every pair $(c, k) \in \langle 0, 5\rangle \times \langle 0, 74895\rangle$ and every possible feature-instance cardinality $r \in \langle 0, 74895\rangle$. Even with the optimization discussed in Section 5.2.1.4, this amounts to an extreme number of iterations.

The remaining DP construction timeouts follow the same pattern, often in an even more extreme form (i.e. even larger feature-instance and group cardinalities). However, in those cases the feature-instance bounds are so large that CP-SAT also fails, such that the effect is not visible in the comparative analysis.

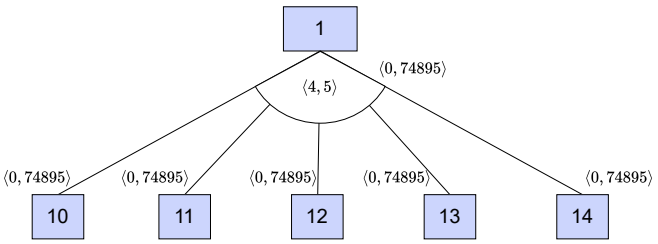


Figure 7.13: CFMs `cfm\repo\model_22` and `cfm\repo\model_23` from the evaluation dataset, where DP has a construction timeout and CP-SAT an enumeration timeout. Both models differ only in their cross-tree constraints, which are not shown.

7.5 Threats to Validity

The results presented in the previous sections are subject to several limitations that may affect their interpretation and the conclusions drawn from them. In the following, we discuss the most relevant threats to validity, distinguishing between internal, construct, conclusion, and external validity.

7.5.1 Internal Validity

Internal validity concerns whether the observed differences can be attributed to the compared algorithms rather than to unintended properties of the experimental setup.

Uncontrolled Execution Environment

The evaluation was conducted on a shared software stack without full system isolation. In particular, other operating-system activities and background processes may have interfered with runtime measurements. Moreover, no explicit CPU affinity was enforced, such that solver runs could be affected by scheduling noise and cache effects. This introduces measurement noise, especially for very short runtimes.

The fact that the experiments were conducted inside a WSL environment is unlikely to introduce systematic bias, as any overhead should affect both solvers similarly.

Implementation Differences

The two approaches are not implemented within the same software stack, introducing a potential confounding variable when comparing their performance. The DP-based approach is implemented in Rust, whereas the CP-SAT baseline relies on the Python interface of OR-Tools, at least for model construction. As a result, part of the measured end-to-end time for CP-SAT includes Python-side overhead, while DP benefits from a compiled systems-language implementation. This distinction is particularly relevant given that the Rust code was compiled with CPU-specific optimizations enabled (see Section 7.2.4). More generally, differences in compiler optimizations, memory layout, and runtime environments may influence the results in ways that are not solely attributable to the underlying algorithms, thereby confounding algorithmic performance with implementation efficiency.

Solver-Dependent Instance Filtering

As described in Section 7.2.3, models with an empty configuration space are removed only after both solvers have been executed. More precisely, a model is excluded from the aggregation whenever at least one of the two solvers identifies it as unsatisfiable.

This creates a potential threat to internal validity because the effective evaluation dataset then depends on solver outcomes. In particular, if one solver is systematically better at detecting empty configuration spaces, then excluding instances that are easy for that solver but difficult for the other may bias the comparison.

For the experiments in this thesis, this effect is likely minor: only ten models were excluded in total, nine of which were identified as empty by both solvers, while one additional model was detected only by CP-SAT. Nevertheless, the procedure remains methodologically suboptimal.

Timeout Handling Asymmetry

A further threat arises from the handling of time limits. In both the cached and the non-cached setting, the enumeration phase is not enforced as a strict wall-clock cap at $\mathcal{T}_{\text{enum}} = 20$ seconds; instead, whenever enumeration exceeds the nominal limit, the actual elapsed time is recorded. While this is generally acceptable, it creates an asymmetry in the cached setting: construction timeouts are penalized with the fixed value $\mathcal{T}_{\text{enum}}$, whereas enumeration failures are penalized by their actual runtime, which may exceed this limit substantially. Consequently, a solver that finishes construction but later times out during enumeration can receive a

worse penalty than a solver that already fails during construction. This effect is particularly visible for CP-SAT, where TTFC may exceed the enumeration limit by several minutes, as shown in Figure 7.5b. For DP, the same issue exists in principle, but in the conducted experiments the limit is exceeded only by seconds rather than by minutes.

The cause of this behaviour is CP-SAT's internal presolve, which is not controlled by the external enumeration timeout. Although this time could in principle be treated as preprocessing, doing so is not appropriate in practice, as CP-SAT does not support storing and reloading presolved models.

Limited Interpretability of Timeout Cutoffs

A limitation of the chosen timeout policy is that, once a run is cut off, we do not know how quickly the solver would have found a solution beyond that cutoff. Consequently, all instances that hit the timeout are collapsed into the same outcome category, even though their actual remaining solving times may differ substantially. Using larger time limits, or additionally repeating the analysis with different timeout values, could provide a more differentiated view of this behaviour. This is particularly relevant for the large block of CP-SAT construction timeouts at $\mathcal{T}_{\text{global}} = 20$ minutes, where the current results only show that many models exceed the cutoff, but do not reveal whether these instances are concentrated in a narrow range just above the limit or spread out over much larger construction times.

Warm-Up and Caching Effects

Warm-up effects, repeated-run caching effects, and extensive fine-tuning of both approaches were not systematically controlled. In particular, effects such as cache population, just-in-time initialization, or data structure reuse may influence runtime measurements across runs. This introduces additional uncontrolled variation that may bias the observed differences between the approaches.

7.5.2 Construct Validity

Construct validity concerns whether the chosen evaluation metrics and experimental setup appropriately capture the concepts of interest and are suitable to answer the research goals defined in Section 7.1.

Non-Equivalent Configuration Semantics

The most important threat to construct validity is that the two approaches do not solve exactly the same configuration-generation task. The DP-based approach generates structural configurations, whereas the CP-SAT baseline generates semi-structural configurations. As a result, the corresponding solution spaces are not identical: the semi-structural configuration space can be strictly larger, such that CP-SAT may in principle have more solutions available to enumerate.

Enforcing proper structural configurations in the CP-SAT encoding would require additional variables and constraints. The exact impact on performance is difficult to predict, since SAT-based solving is subject to effects such as phase transitions, where difficulty initially increases as constraints are added, but may decrease again beyond

a certain point [130, 131, 132, 133]. However, this phenomenon is typically not observed for realistic feature models, which have been shown to yield comparatively easy SAT instances [34].

Nevertheless, enforcing structural configurations is likely to worsen performance in many cases, as representing instance order requires a substantial number of additional variables rather than only a few extra constraints.

Enumeration versus Random-Sampling Behaviour

The reported performance results for the DP-based approach cannot be directly transferred to UniSample-UR, as the underlying procedures differ fundamentally. The DP-based enumeration exploits optimizations that are not applicable to random sampling: it processes consecutive ranks and reuses large parts of previously constructed configurations, since only a small portion changes between successive ranks. In contrast, UniSample-UR must unrank independent random ranks, which prevents such reuse and therefore requires reconstructing configurations from scratch.

Consequently, the reported enumeration performance overestimates the efficiency that can be achieved in a true random-sampling setting. However, this is acceptable in the context of the present evaluation, as the goal is to compare the raw enumeration and solution-generation capabilities of the approaches rather than end-to-end random-sampling performance (see Section 7.1).

Incomplete Performance Metrics

The evaluation focuses primarily on latency- and throughput-related metrics and does not include memory consumption. This is a relevant limitation, since memory usage is often a decisive factor for tractability and practical applicability. Consequently, the operationalization of "performance" in this study is incomplete, and the reported results capture only a subset of relevant characteristics.

7.5.3 Conclusion Validity

Conclusion validity concerns whether the empirical evidence supports the drawn conclusions in a statistically sound manner.

Single-Run Performance Measurements

The runtime experiments are based on a single execution per model and solver variant. Although the algorithms themselves are largely deterministic under fixed settings, measured runtimes can still vary due to operating-system noise, scheduler behaviour, memory allocation effects, and caching. As a consequence, the study does not explicitly quantify runtime variance and does not provide confidence intervals or hypothesis tests for the performance differences. Repeating the experiments multiple times and reporting aggregate statistics would strengthen the robustness of the conclusions, especially for models with short runtimes or marginal differences.

Limitations of the Uniformity Validation

The empirical uniformity study already goes beyond a simple visual inspection by using repeated χ^2 tests and confidence intervals for rejection rates. Nevertheless, the validation remains limited in scope. In particular, it primarily checks whether rejection rates concentrate around the expected values, but it does not test whether the resulting p-values follow the expected uniform distribution under the null hypothesis. Additional tests such as a Kolmogorov–Smirnov test on the p-value distribution could provide stronger evidence [114, 121, 124].

7.5.4 External Validity

External validity concerns the extent to which the results generalize beyond the concrete dataset, implementation, and hardware platform used in the study.

Dataset Representativeness

The dataset is not fully curated and consists of models collected from different sources, which often do not document their origin, purpose, or additional metadata. As a result, duplicate and similar models may be present.

For FMs, this issue already arises from the S.P.L.O.T. collection, which is known to be uncurated and to contain duplicate or very similar models [134].

For proper CFMs, the situation is even more challenging, as duplicates are difficult to detect: some models in the original sources exist in both bounded and unbounded form. In our preprocessing, we also bound the unbounded models, but using a different big-M scheme than the original one. This can result in models that are syntactically different, but closely related. Consequently, certain structural patterns may be overrepresented, potentially limiting the generalizability of the results.

Synthetic Bias and limited Real-World Coverage

Another important limitation is the lack of real-world models. All proper CFMs in the dataset are synthetic, and real-world scalability remains uncertain. This is particularly relevant because DP fails on three out of four automotive models, which are real-world models from the industry (see Section 7.4.2).

Hardware and Platform dependence

All experiments were conducted on a single hardware platform. Therefore, the reported runtimes and relative speedups may depend on architectural properties such as cache sizes, memory bandwidth, processor microarchitecture, and system configuration.

Incomplete Optimization

Finally, neither approach was exhaustively tuned to its practical limits. The present study does not systematically explore alternative CP-SAT encodings, advanced solver parameter tuning and different DP implementation optimizations. Thus, the observed performance differences should be understood as evidence for the effectiveness of the compared implementations in their present form, rather than as an absolute statement about the maximal achievable performance of either methodological family.

7.6 Summary and Outlook

In this chapter, we empirically evaluated two main aspects of the algorithms developed in Chapter 4. First, we studied the *performance* of the DP-based counting and unranking machinery, in particular ComputeDP and UnrankConfig, when used as a high-performance enumeration mechanism. Second, we validated the end-to-end *correctness* of the implemented sampling algorithms by validating their output distributions empirically. In particular, the results of those experiments provided strong empirical evidence that the implementations are correct.

For the performance analysis, we deliberately did *not* benchmark the sampling algorithms UniSample-BT and UniSample-UR directly. Instead, we benchmarked the underlying DP-based counting and unranking infrastructure against a CP-SAT baseline implemented in cfmtools. This choice is motivated by two considerations. On the one hand, readily available tool support for uniform random sampling that directly supports CFMs and their configuration semantics is currently limited. On the other hand, evaluating the performance of ComputeDP and UnrankConfig constitutes a meaningful first step towards assessing the practical viability of the sampling algorithms themselves, since both UniSample-BT and UniSample-UR fundamentally rely on the prior computation of the DP tables.

Overall, the results in Section 7.3 show that the DP-based approach clearly outperforms the CP-SAT baseline, both in terms of throughput and latency (TTFC) on a majority of the models. The performance differences are typically one to two orders of magnitude and hold not only for proper CFMs but also for classical FMs. The only current challenge of the DP-based approach is its inability to incorporate cross-tree constraints. Consequently, for models with cross-tree constraints and an unconstrained configuration space too large to be enumerated in reasonable time, CP-SAT is currently the only viable option.

Even when the results are interpreted in light of the threats to validity discussed in Section 7.5, the overall conclusion remains unchanged: DP-based enumeration of structural configurations, and consequently uniform random sampling via UniSample-BT and UniSample-UR, is a promising new approach that operates directly on CFMs rather than relying on general-purpose solver technology. As suggested by the empirical results, this direct treatment of CFMs may enable performance improvements of multiple orders of magnitude over state-of-the-art approaches to finding, enumerating, counting, and sampling structural configurations of CFMs and FMs. This makes the approach well worth further investigation and optimization.

8. Related Work

This chapter reviews the literature related to the main contributions of this thesis.

Configuration Semantics

A central question in the semantics of CFMs concerns how configurations should be represented. In the classical FM setting, features are either present or absent, so configurations can be represented simply as sets of selected features [5, 9]. Once feature multiplicities are admitted, however, this set-based view is no longer sufficient, since one must additionally capture how often a feature occurs and how feature occurrences are arranged hierarchically [2, 5, 14, 28].

The extension of FMs with feature multiplicities was first proposed by Riebisch et al. [13]. A first formal semantics for CFMs was then given by Czarnecki et al. [2], who model configurations using a context-free grammar. Subsequent work developed several semantic views of configurations, including clone-based (index-free) semantics, instance-based semantics with explicit feature-instance indices, and flat multiset abstractions [2, 10, 11, 14, 28, 31]. For a detailed comparison of these semantics, see Section 3.4.3.

From the perspective of this thesis, the crucial point is that the clone-based, index-free view of configurations is not new; rather, it is the configuration semantics most commonly used in the CFM literature [2, 10, 11, 14, 18, 21]. What is new in this thesis is the observation that this clone-based semantics is *not* equivalent to the instance-based semantics introduced by Weckesser et al. [10], even though Weckesser et al. and subsequent work [11, 18, 21] effectively treat the two as interchangeable.

The main contribution of this thesis is therefore a semantic clarification and reconstruction of the relationship between these two views. More precisely, we show that the explicit indices of instance-based configurations introduce semantic artifacts that are absent from the intended clone-based reading. We then recover the clone-based semantics from the instance-based one by quotienting instance-based configurations by an appropriate equivalence relation. The resulting equivalence classes are called *structural configurations* in this thesis; semantically, they correspond to the clone-based, index-free configurations used in earlier work.

Building on this construction, the thesis introduces several genuinely new configuration notions. In particular, we define *semi-structural configurations* by quotienting instance-based configurations with respect to an equivalence relation that is more restrictive than the one used for structural configurations. As a result, semi-structural configurations constitute an intermediate semantics between instance-based and structural configurations: they abstract from some index-based distinctions, but retain more information than fully structural (i.e., clone-based) configurations. We also define *canonical* and *semi-canonical* configurations as distinguished instance-based representatives of structural and semi-structural configurations, respectively.

Although *semi-structural configurations* have not, to our knowledge, been identified explicitly in the literature before, this semantics has, in fact, already been used implicitly in recent solver-based work. In particular, the encodings proposed by Zhang [18] and Eger [21] do not realize clone-based, i.e., structural, configurations directly, since their representations distinguish sibling positions by means of fixed variables or slots. At the same time, they also do not realize unrestricted instance-based configurations, since they do not allow arbitrary index assignments but instead enforce subtree index monotonicity (see Definition 3.16) and thus encode *semi-structural configurations*.

Instance Order and Canonicalization

The techniques used to construct structural, semi-structural, canonical, and semi-canonical configurations build on established methods from graph theory, adapted to the setting of CFMs and instance-based configurations.

Concretely, *tree isomorphism* and *order isomorphism* (Definitions 3.7 and 3.22), used to define structural and semi-structural configurations, correspond to the standard notions of isomorphism for coloured graphs and coloured ordered trees, respectively [71, 72, 73, 78].

Furthermore, the *instance order* (Definition 3.12) is inspired by AHU-style algorithms for solving the tree isomorphism problem [75, 76, 135]. In particular, Chi et al. [72] employ the same underlying idea for the canonicalization of unordered, labelled trees, defining an order on rooted trees recursively by comparing canonically ordered sequences of child subtrees.

Uniform Random Sampling

In contrast to the extensive literature on uniform (and almost-uniform) sampling for Boolean FMs, where configurations can be sampled using established SAT-based techniques, comparatively little work has addressed uniform random sampling for CFMs (see Section 2.3).

A notable exception is the sampler of Zhang [18], which generates configurations via a randomized, recursive top-down construction along the feature tree. Concretely, the algorithm incrementally decides local group cardinalities and child multiplicities and relies on rejection sampling to enforce group constraints and cross-tree constraints. However, because these local choices are drawn uniformly from their respective cardinality intervals, the resulting distribution is not uniform in general.

Our sampling algorithm UniSample-BT (Algorithm 4.2) builds on the same recursive construction, but refines it into a provably uniform sampler. Specifically,

uniformity is achieved by weighting each local choice by the number of valid configurations compatible with it. The general idea is already suggested by Zhang [18] in the CFM setting and is widely used in uniform sampling for Boolean FMs, where the required completion counts are typically obtained via #SAT solvers [20, 36, 37].

9. Conclusion and Outlook

This thesis studied the problem of uniformly sampling configurations of CFMs. Starting from the observation that existing configuration semantics are not fully satisfactory for this purpose, we developed new configuration notions, a DP-based counting and unranking framework, and two uniform random sampling algorithms. In addition, we provided rigorous correctness proofs, a theoretical time and space complexity analysis together with a discussion of possible improvements, an open-source implementation of these ideas in the tool CFMTOOLS, and an extensive empirical evaluation. Taken together, these results provide both new theoretical foundations for reasoning about CFMs and evidence that the developed DP-based approach can serve as a practically effective alternative to state-of-the-art general-purpose solver technology, with the potential to outperform it by orders of magnitude.

9.1 Summary of Contributions

The main contributions of this thesis can be summarized as follows.

Configuration semantics of CFMs. A first central contribution of this thesis is the systematic study of configuration semantics for CFMs. We first showed that the currently used instance-based representation is problematic when configurations are to be constructed recursively from smaller sub-models. In particular, arbitrary feature-instance indices introduce index anomalies that obstruct closure under natural merge operations and inflate the configuration space unnecessarily.

To address this, we introduced and analysed the notions of *structural*, *semi-structural*, *canonical*, and *semi-canonical* configurations. The key idea was to define suitable equivalence relations on instance-based configurations, which allow us to form equivalence classes and thereby identify instance-based configurations that are structurally the same but differ only in their feature-instance indices. Canonical and semi-canonical configurations were then defined as special instance-based configurations that form systems of representatives for the structural and semi-structural configuration spaces, respectively.

During the development of these new configuration semantics, we transferred and adapted several notions from graph theory to the domain of CFMs, including isomorphism notions for coloured graphs and coloured ordered trees, as well as an AHU-inspired instance order. Moreover, we established a range of new theorems and proofs for these concepts, including a characterization of feature instances that are equivalent with respect to the instance order, existence and uniqueness results for canonical and semi-canonical configurations, and the closure of canonical and semi-canonical configurations under the MERGE algorithm.

An important clarification is that the structural configuration semantics itself is not introduced here as a fundamentally new semantics. Rather, structural configurations correspond to the clone-based, index-free view that has been the predominant configuration semantics throughout most of the CFM literature (see Section 3.4.3 and Chapter 8). What is new in this thesis is the rigorous reconstruction of this semantics from the instance-based one and the precise analysis of the relationship between these two views. In particular, one cannot simply treat instance-based configurations as if they were structural ones by informally ignoring feature-instance indices or assuming that these indices are always well-behaved.

This clarification is relevant not only for mathematical precision, but also at a more practical level. In particular, we showed that conflating instance-based and structural semantics has already led in prior work to technical errors, including solver encodings that actually capture only semi-structural rather than structural configurations, and sampling procedures that are not uniform for the intended structural configuration space (see Sections 3.4.3 and 3.4.4). At the same time, the equivalence-based construction developed in this thesis is not merely corrective, but also constructive: it provides a general tool for defining new configuration semantics by quotienting out exactly those distinctions that are irrelevant for a given application. The notion of *semi-structural configurations* introduced in this thesis is one concrete example of this idea and suggests the broader possibility of tailoring configuration semantics systematically to application-specific needs.

DP-based counting and unranking. A second major contribution is the development of a DP-based framework for counting canonical configurations of CFMs without cross-tree constraints. To this end, we introduced suitable DP variables that capture the number of canonical configurations of a CFM without cross-tree constraints satisfying certain properties, for example prescribed realised root-group cardinalities and the restriction that child instances may only be chosen from the first i children. Based on these variables, we derived a recursion formula (Theorem 4.5) that allows the corresponding values to be computed in a bottom-up manner.

Building on this counting framework, we then developed two uniform random sampling algorithms for canonical, and therefore structural, configurations. The first algorithm, UNISAMPLE-BT, constructs a configuration recursively by making DP-guided weighted random choices and uses rejection sampling to generate child multisets uniformly. The second algorithm, UNISAMPLE-UR, is based on a ranking and unranking scheme: after assigning each canonical configuration a unique rank, a configuration can be sampled uniformly by drawing a rank uniformly at random and reconstructing the corresponding configuration via unranking. We showed that both approaches can be extended generically to CFMs with cross-tree constraints by rejection sampling over the unconstrained structural configuration space.

Time and space complexity analysis. A further contribution is the theoretical evaluation of these algorithms. We proved correctness of UniSample-BT, UniSample-UR, and CTC-UniSample, in particular almost-sure termination and uniformity. We also analysed the time and space complexity of the DP-table construction, the time complexity of both sampling algorithms, and the cost of the rejection-sampling extension for cross-tree constraints via CTC-UniSample. This not only strengthened the theoretical understanding of the algorithms and their trade-offs, but also helped identify several concrete optimization opportunities. In particular, we showed that the preprocessing overhead of the two approaches is asymptotically the same.

Tool support. The thesis also contributes an implementation of the developed ideas in the open-source tool cfmtools. This is particularly relevant because, to the best of our knowledge, cfmtools is the first tool that operates directly on structural configurations, rather than only on semi-structural configurations (see Section 6.3).

Empirical evaluation. Finally, we provided an empirical evaluation of the developed techniques. Since direct comparisons with mature off-the-shelf uniform samplers for CFMs are currently not available, we evaluated the DP-based counting and unranking infrastructure as a high-performance enumeration mechanism and compared it against a CP-SAT baseline. The results indicate that the DP-based approach clearly outperforms the solver-based baseline on the majority of instances, often by one to two orders of magnitude and, for proper CFMs, sometimes by much more. In particular, the DP-based approach remains highly robust on proper CFMs, whereas CP-SAT fails already during model construction. In addition, an empirical uniformity validation provided strong evidence that the implemented sampling algorithms behave as intended and produce samples consistent with the uniform distribution.

9.2 Future Work

The results of this thesis open several directions for future work. These directions are not independent, but may strongly influence and reinforce each other. In particular, the usefulness of extending the DP-based framework and of systematically tailoring configuration semantics would be amplified by improvements in the handling of cross-tree constraints and, more generally, by further speedups of DP-based counting and unranking.

The following directions are presented in decreasing order of their presumed relevance.

Extension of the DP-based framework. One of the most important directions concerns the extension of the DP-based framework beyond uniform random sampling. As already indicated in this thesis, the same machinery can be adapted to compute further statistics of the configuration space, such as the expected configuration size. More generally, it may provide the basis for replacing the use of SAT and #SAT solvers for analysis tasks in the Boolean setting and the use of SMT, ILP, or CSP solvers in the non-Boolean case. Examples include the detection of dead

features, dead cardinalities, void models, combinatorial sampling (t-wise sampling), and, more generally, many of the applications identified by Sundermann et al. [20] and Eger [21].

For CFMs without cross-tree constraints, several such analyses can, in fact, be answered directly from the computed DP tables themselves. This includes, for example, the detection of dead features, dead cardinalities, and void models. More systematically investigating which analysis tasks admit such direct DP-based solutions, how this picture changes in the presence of cross-tree constraints, and how the resulting approaches perform in practice, is an interesting direction for future work.

Tailoring configuration semantics. An equally important direction concerns the systematic tailoring of configuration semantics. As discussed in Section 3.4.5, the structural and semi-structural notions introduced in this thesis suggest a more general methodology in which one chooses explicitly which distinctions should be preserved and which symmetries should be factored out. This could lead to intermediate semantics tailored to the intended application, while keeping the CFM syntax unchanged and defining validity, as before, via instance-based configurations.

Incorporating cross-tree constraints. Whereas the first two directions open broader new research areas, the following direction is more specific and concrete. In particular, it concerns extending the framework beyond the exact rejection-sampling approach used in this thesis. Here, cross-tree constraints were handled generically by filtering configurations generated for the relaxed model without cross-tree constraints. While this yields a simple and correct extension, it can become inefficient when the acceptance probability is small. Developing methods that incorporate cross-tree constraints more directly into the DP framework would therefore be a relevant next step. Two concrete ideas are the following.

First, one could enrich the existing samplers with lightweight short-circuit evaluations. For example, during UniSample-BT or UnrankConfig, the algorithm could abort as soon as it detects that cross-tree constraints are already violated and cannot be repaired.

Second, from the ranking-based perspective, cross-tree constraints partition the unconstrained rank set into valid and invalid regions. It would therefore be interesting to study this partition more systematically. In particular, one may ask whether valid and invalid ranks typically occur in short alternating blocks or whether large contiguous invalid regions arise. If the blocks are usually small, this would already be favourable for enumeration, since after entering an invalid region one would quickly encounter valid configurations again. If, however, large invalid blocks occur frequently, one could investigate how to estimate or even compute their sizes in order to skip them efficiently.

Hybrid methods and transfers from SAT-based sampling. More generally, it may be fruitful to investigate whether techniques known from exact or approximate uniform sampling for SAT can be transferred to the DP-based setting. This can range from hashing-based techniques, such as those used for the `UniGen` sampler family [33, 41, 42], to MCMC methods [45, 46]. More broadly, one could also

try to combine general-purpose solvers with the DP-based approach, where the former would typically handle the cross-tree constraints and the latter the hierarchical structure.

Approximate DP tables and approximate sampling. It would also be interesting to investigate approximate variants of the DP framework itself. Instead of exact DP tables, one could consider approximate configuration counters and study whether they can still support useful approximate unranking or approximately uniform random sampling, together with meaningful guarantees. Such an approach may be particularly attractive in the presence of cross-tree constraints, where exact counting quickly becomes difficult anyway. In this setting, combining approximate DP tables with heuristic guidance may lead to practically effective methods even when exact uniformity is out of reach.

Algorithm engineering and more realistic cost models. The runtime analysis in this thesis deliberately abstracts from the cost of arbitrary-precision arithmetic, even though configuration counts can become extremely large in practice. A more realistic complexity analysis that accounts explicitly for big-integer arithmetic, binomial-coefficient computation, and memory usage would therefore be valuable.

On the implementation side, further optimizations are conceivable as well. Concretely, several optimizations discussed in Chapter 5, such as multiset hashing and sparse feature-count aggregation, are not yet implemented in CFMTOOLS. Beyond that, sparse DP-table representations, as well as a more careful choice of iteration order, may yield significant runtime improvements when adapted to the structure of the concrete CFM. In particular, throughout this thesis we have assumed an arbitrary order on the children, that is, an arbitrary choice of which child is treated as the first one. Choosing this order more deliberately, as well as adapting the traversal order of the blocks (c, k), could further improve performance in practice.

Structural configurations in solver encodings. Another interesting direction concerns whether full structural, or equivalently canonical, configurations can be enforced directly in SAT, SMT, or CSP encodings. As discussed in Chapters 6 and 7, existing solver-based encodings typically capture only semi-structural configurations. To move from semi-structural to structural semantics, one would additionally have to encode sibling index monotonicity and therefore the instance order. It is currently unclear whether this can be achieved in a practically efficient way, but investigating how such encodings can be formulated and what performance impact they have would be an interesting topic for future work.

Broader empirical validation and benchmarking. Finally, a broader empirical evaluation would also be valuable. One important limitation is the still very limited availability of real-world CFMs, in particular industrial ones. Moreover, several of the threats to validity identified in Chapter 7 should be addressed more systematically in future experiments. In particular, it would be valuable to compare UNISAMPLE-BT and UNISAMPLE-UR directly against existing sampling approaches from the SAT and general CSP literature, even though the currently available tool support remains limited.

Bibliography

[1] Krzysztof Czarnecki et al. “Staged Configuration Using Feature Models”. In: *Software Product Lines*. Ed. by Robert L. Nord. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, pp. 266–283. ISBN: 978-3-540-28630-1. DOI: 10.1007/978-3-540-28630-1_17.

[2] Krzysztof Czarnecki et al. “Formalizing cardinality-based feature models and their specialization”. In: *Software Process: Improvement and Practice* 10.1 (2005), pp. 7–29. DOI: 10.1002/spip.213.

[3] Rafael Capilla et al., eds. *Systems and Software Variability Management. Concepts, Tools and Experiences*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013. ISBN: 978-3-642-36582-9. DOI: 10.1007/978-3-642-36583-6.

[4] Periklis Sochos et al. “Feature-Oriented Development of Software Product Lines: Mapping Feature Models to the Architecture”. In: *Object-Oriented and Internet-Based Technologies*. Ed. by Mathias Weske and Peter Liggesmeyer. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, pp. 138–152. ISBN: 978-3-540-30196-7. DOI: 10.1007/978-3-540-30196-7_11.

[5] Pierre-Yves Schobbens et al. “Feature Diagrams: A Survey and a Formal Semantics”. In: *14th IEEE International Requirements Engineering Conference (RE'06)*. 2006, pp. 139–148. DOI: 10.1109/RE.2006.23.

[6] Sven Apel et al. *Feature-Oriented Software Product Lines. Concepts and Implementation*. Springer Berlin Heidelberg, 2013. ISBN: 978-3-642-37520-0. DOI: 10.1007/978-3-642-37521-7.

[7] Ian Sommerville. *Software Engineering*. 9th ed. Boston: Addison-Wesley, 2011. ISBN: 978-0-13-703515-1.

[8] Kyo Kang et al. *Feature-Oriented Domain Analysis (FODA) Feasibility Study*. Tech. rep. CMU/SEI-90-TR-021. Carnegie Mellon University, Software Engineering Institute, Nov. 1990. URL: https://www.sei.cmu.edu/library/feature-oriented-domain-analysis-foda-feasibility-study/.

[9] David Benavides et al. “Automated analysis of feature models 20 years later: A literature review”. In: *Information Systems* 35.6 (2010), pp. 615–636. ISSN: 0306-4379. DOI: 10.1016/j.is.2010.01.001.

[10] Markus Weckesser et al. “Mind the Gap! Automated Anomaly Detection for Potentially Unbounded Cardinality-Based Feature Models”. In: *Fundamental Approaches to Software Engineering*. Ed. by Perdita Stevens and Andrzej Wąsowski. Berlin, Heidelberg: Springer Berlin Heidelberg, 2016, pp. 158–175. ISBN: 978-3-662-49665-7. DOI: 10.1007/978-3-662-49665-7_10.

[11] Lukas Güthing et al. “Sampling Cardinality-Based Feature Models”. In: *Proceedings of the 18th International Working Conference on Variability Modelling of Software-Intensive Systems*. VaMoS ’24. Bern, Switzerland: Association for Computing Machinery, 2024, pp. 46–55. ISBN: 979-8-4007-0877-0. DOI: 10.1145/3634713.3634719.

[12] Robert Müller et al. “Mapping Cardinality-based Feature Models to Weighted Automata over Featured Multiset Semirings”. In: *Proceedings of the 28th ACM International Systems and Software Product Line Conference*. SPLC ’24. Dommeldange, Luxembourg: Association for Computing Machinery, 2024, pp. 1–11. ISBN: 9798400705939. DOI: 10.1145/3646548.3676539.

[13] Matthias Riebisch et al. “Extending Feature Diagrams with UML Multiplicities”. In: *Proceedings of the Integrated Design and Process Technology Conference (IDPT 2002)*. United States of America: Society for Design and Process Science, 2002. URL: https://api.semanticscholar.org/CorpusID:16322983.

[14] Raphael Michel et al. “A formal semantics for feature cardinalities in feature diagrams”. In: *Proceedings of the 5th International Workshop on Variability Modeling of Software-Intensive Systems*. VaMoS ’11. Namur, Belgium: Association for Computing Machinery, 2011, pp. 82–89. ISBN: 9781450305709. DOI: 10.1145/1944892.1944902.

[15] Nadia Gamez and Lidia Fuentes. “Software Product Line Evolution with Cardinality-Based Feature Models”. In: *Top Productivity through Software Reuse*. Ed. by Klaus Schmid. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 102–118. ISBN: 978-3-642-21347-2. DOI: 10.1007/978-3-642-21347-2_9.

[16] Mahsa Varshosaz et al. “A classification of product sampling for software product lines”. In: *Proceedings of the 22nd International Systems and Software Product Line Conference - Volume 1*. SPLC ’18. Gothenburg, Sweden: Association for Computing Machinery, 2018, pp. 1–13. ISBN: 978-1-4503-6464-5. DOI: 10.1145/3233027.3233035.

[17] Sebastian Krieter et al. *T-Wise Presence Condition Coverage and Sampling for Configurable Systems*. 2022. arXiv: 2205.15180 [cs.SE]. URL: https://arxiv.org/abs/2205.15180.

[18] Daniel-Delong Zhang. “Lifting Classical Sampling Criteria for Cardinality-Based Feature Models”. Unpublished Master’s Thesis. Master’s thesis (unpublished). Karlsruhe, Germany: Karlsruhe Institute of Technology (KIT), Department of Informatics, Institute of Information Security and Dependability (KASTEL), Apr. 2025.

[19] Matthias Kowal et al. “Explaining Anomalies in Feature Models”. In: *Proceedings of the 2016 ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences*. GPCE 2016. Amsterdam, Netherlands: Association for Computing Machinery, 2016, pp. 132–143. ISBN: 9781450344463. DOI: 10.1145/2993236.2993248.

[20] Chico Sundermann et al. “Applications of #SAT Solvers on Feature Models”. In: *Proceedings of the 15th International Working Conference on Variability Modelling of Software-Intensive Systems*. VaMoS ’21. Krems, Austria: Association for Computing Machinery, 2021. ISBN: 9781450388245. DOI: 10.1145/3442391.3442404.

[21] Fabian Eger. “Solver Representations for Cardinality-based Feature Models”. Unpublished Master’s Thesis. Master’s thesis (unpublished). Karlsruhe, Germany: Karlsruhe Institute of Technology (KIT), Department of Informatics, Institute of Information Security, Dependability, Test, Validation, and Analysis (TVA), June 2025.

[22] Krzysztof Czarnecki and Ulrich W. Eisenecker. *Generative Programming: Methods, Tools, and Applications*. New York, NY: ACM Press/Addison-Wesley, 2000. ISBN: 978-0-201-30977-5.

[23] Jan Bosch. *Design and Use of Software Architectures: Adopting and Evolving a Product-Line Approach*. New York, NY: ACM Press/Addison-Wesley, 2000. ISBN: 978-0-201-67494-1.

[24] D. Batory et al. “Scaling step-wise refinement”. In: *IEEE Transactions on Software Engineering* 30.6 (2004), pp. 355–371. DOI: 10.1109/TSE.2004.23.

[25] Sven Apel et al. “An algebraic foundation for automatic feature-based program synthesis”. In: *Science of Computer Programming* 75.11 (2010). Special Section on the Programming Languages Track at the 23rd ACM Symposium on Applied Computing, pp. 1022–1047. ISSN: 0167-6423. DOI: 10.1016/j.scico.2010.02.001.

[26] Alexander Knüppel et al. “Is there a mismatch between real-world feature models and product-line research?” In: *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*. ESEC/FSE 2017. Paderborn, Germany: Association for Computing Machinery, 2017, pp. 291–302. ISBN: 9781450351058. DOI: 10.1145/3106237.3106252.

[27] Don Batory. “Feature Models, Grammars, and Propositional Formulas”. In: *Software Product Lines*. Ed. by Henk Obbink and Klaus Pohl. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 7–20. ISBN: 978-3-540-32064-7. DOI: 10.1007/11554844_3.

[28] Aliakbar Safilian and Tom Maibaum. “Hierarchical Multiset Theories of Cardinality-Based Feature Diagrams”. In: *2016 10th International Symposium on Theoretical Aspects of Software Engineering (TASE)*. 2016, pp. 136–143. DOI: 10.1109/TASE.2016.14.

[29] Matthias Riebisch. *Towards a More Precise Definition of Feature Models*. 2003. URL: https://www.inf.uni-hamburg.de/en/inst/ab/swk/research/publications/pdf/2003-06-riebisch.pdf.

[30] Aliakbar Safilian et al. “A theoretical framework for cardinality-based feature models: The semantics and computational aspects”. In: *Journal of Logical and Algebraic Methods in Programming* 97 (2018), pp. 30–54. ISSN: 2352-2208. DOI: 10.1016/j.jlamp.2018.02.002.

[31] Thomas Schnabel et al. “CardyGAn: Tool Support for Cardinality-based Feature Models”. In: *Proceedings of the 10th International Workshop on Variability Modelling of Software-Intensive Systems*. VaMoS ’16. Salvador, Brazil: Association for Computing Machinery, 2016, pp. 33–40. ISBN: 9781450340199. DOI: 10.1145/2866614.2866619.

[32] Jörg Liebig et al. “Scalable analysis of variable software”. In: *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*. ESEC/FSE 2013. Saint Petersburg, Russia: Association for Computing Machinery, 2013, pp. 81–91. ISBN: 9781450322379. DOI: 10.1145/2491411.2491437.

[33] Mathieu Acher et al. “BURST: a benchmarking platform for uniform random sampling techniques”. In: *Proceedings of the 25th ACM International Systems and Software Product Line Conference - Volume B*. SPLC ’21. Leicester, United Kingdom: Association for Computing Machinery, 2021, pp. 36–40. ISBN: 9781450384704. DOI: 10.1145/3461002.3473070.

[34] Marcilio Mendonca et al. “SAT-based analysis of feature models is easy”. In: *Proceedings of the 13th International Software Product Line Conference*. SPLC ’09. San Francisco, California, USA: Carnegie Mellon University, 2009, pp. 231–240. URL: https://dl.acm.org/doi/10.5555/1753235.1753267.

[35] Daniel-Jesus Munoz et al. “Uniform Random Sampling Product Configurations of Feature Models That Have Numerical Features”. In: *Proceedings of the 23rd International Systems and Software Product Line Conference - Volume A*. SPLC ’19. Paris, France: Association for Computing Machinery, 2019, pp. 289–301. ISBN: 9781450371384. DOI: 10.1145/3336294.3336297.

[36] Jeho Oh et al. “Scalable Uniform Sampling for Real-World Software Product Lines”. In: 2020. URL: https://api.semanticscholar.org/CorpusID:219699491.

[37] Dimitris Achlioptas et al. “Fast Sampling of Perfectly Uniform Satisfying Assignments”. In: *Theory and Applications of Satisfiability Testing – SAT 2018*. Ed. by Olaf Beyersdorff and Christoph M. Wintersteiger. Cham: Springer International Publishing, 2018, pp. 135–147. ISBN: 978-3-319-94144-8. DOI: 10.1007/978-3-319-94144-8_9.

[38] Shubham Sharma et al. “GANAK: A Scalable Probabilistic Exact Model Counter”. In: *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI-19*. International Joint Conferences on Artificial Intelligence Organization, July 2019, pp. 1169–1176. DOI: 10.24963/ijcai.2019/163.

[39] Olivier Zeyen et al. “Exploring the Computational Complexity of Uniform Random Sampling and SAT Counting with Phase Transitions”. In: *Proceedings of the 29th ACM International Systems and Software Product Line Conference - Volume A*. SPLC-A ’25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 149–160. ISBN: 9798400720246. DOI: 10.1145/3744915.3748473.

[40] Marc Thurley. “sharpSAT – Counting Models with Advanced Component Caching and Implicit BCP”. In: *Theory and Applications of Satisfiability Testing - SAT 2006*. Ed. by Armin Biere and Carla P. Gomes. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 424–429. ISBN: 978-3-540-37207-3. DOI: 10.1007/11814948_38.

[41] Supratik Chakraborty et al. “Balancing Scalability and Uniformity in SAT Witness Generator”. In: *Proceedings of the 51st Annual Design Automation Conference*. DAC ’14. San Francisco, CA, USA: Association for Computing Machinery, 2014, pp. 1–6. ISBN: 9781450327305. DOI: 10.1145/2593069.2593097.

[42] Mate Soos et al. “Tinted, Detached, and Lazy CNF-XOR Solving and Its Applications to Counting and Sampling”. In: *Computer Aided Verification: 32nd International Conference, CAV 2020, Los Angeles, CA, USA, July 21–24, 2020, Proceedings, Part I*. Los Angeles, CA, USA: Springer-Verlag, 2020, pp. 463–484. ISBN: 978-3-030-53287-1. DOI: 10.1007/978-3-030-53288-8_22.

[43] Supratik Chakraborty et al. “On Parallel Scalable Uniform SAT Witness Generation”. In: *Tools and Algorithms for the Construction and Analysis of Systems*. Ed. by Christel Baier and Cesare Tinelli. Berlin, Heidelberg: Springer Berlin Heidelberg, 2015, pp. 304–319. ISBN: 978-3-662-46681-0. DOI: 10.1007/978-3-662-46681-0_25.

[44] Mate Soos and Kuldeep S. Meel. “BIRD: Engineering an Efficient CNF-XOR SAT Solver and Its Applications to Approximate Model Counting”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 33.1 (July 2019), pp. 1592–1599. DOI: 10.1609/aaai.v33i01.33011592.

[45] Rafael Dutra et al. “Efficient sampling of SAT solutions for testing”. In: *Proceedings of the 40th International Conference on Software Engineering*. ICSE ’18. Gothenburg, Sweden: Association for Computing Machinery, 2018, pp. 549–559. ISBN: 9781450356381. DOI: 10.1145/3180155.3180248.

[46] Priyanka Golia et al. “Designing Samplers is Easy: The Boon of Testers”. In: *2021 Formal Methods in Computer Aided Design (FMCAD)*. 2021, pp. 222–230. DOI: 10.34727/2021/isbn.978-3-85448-046-4_31.

[47] Olivier Zeyen et al. “Testing Uniform Random Samplers: Methods, Datasets and Protocols”. In: *ACM Trans. Softw. Eng. Methodol.* (Feb. 2026). Just Accepted. ISSN: 1049-331X. DOI: 10.1145/3797477.

[48] Michał Antkiewicz et al. “Clafer tools for product line engineering”. In: *Proceedings of the 17th International Software Product Line Conference Co-Located Workshops*. SPLC ’13 Workshops. Tokyo, Japan: Association for Computing Machinery, 2013, pp. 130–135. ISBN: 9781450323253. DOI: 10.1145/2499777.2499779.

[49] Markus Weckesser et al. “Mathematical Programming for Anomaly Analysis of Clafer Models”. In: *Proceedings of the 21th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems*. MODELS ’18. Copenhagen, Denmark: Association for Computing Machinery, 2018, pp. 34–44. ISBN: 9781450349499. DOI: 10.1145/3239372.3239398.

[50] Andrei A. Bulatov. “The Complexity of the Counting Constraint Satisfaction Problem”. In: *Automata, Languages and Programming*. Ed. by Luca Aceto et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 646–661. ISBN: 978-3-540-70575-8. DOI: 10.1145/2528400.

[51] Mark Jerrum. “Counting Constraint Satisfaction Problems”. In: *The Constraint Satisfaction Problem: Complexity and Approximability*. Ed. by Andrei Krokhin and Stanislav Zivny. Vol. 7. Dagstuhl Follow-Ups. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2017, pp. 205–231. ISBN: 978-3-95977-003-3. DOI: 10.4230/DFU.Vol7.15301.205.

[52] Rafael Tupynambá Dutra. “Efficient Sampling of SAT and SMT Solutions for Testing and Verification”. PhD thesis. University of California, Berkeley, 2019. URL: https://escholarship.org/uc/item/2g21k1x9.

[53] Arijit Shaw and Kuldeep S. Meel. “Approximate SMT Counting Beyond Discrete Domains”. In: *Proceedings of the 62nd Annual ACM/IEEE Design Automation Conference*. DAC ’25. San Francisco, California, United States: IEEE Press, 2025. ISBN: 9798331503048. DOI: 10.1109/DAC63849.2025.11133351.

[54] Ghiles Ziat and Martin Pépin. “An Efficient and Uniform CSP Solution Generator Generator”. In: *31st International Conference on Principles and Practice of Constraint Programming (CP 2025)*. Ed. by Maria Garcia de la Banda. Vol. 340. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 40:1–40:18. ISBN: 978-3-95977-380-5. DOI: 10.4230/LIPIcs.CP.2025.40.

[55] Luca Martino et al. *Independent Random Sampling Methods*. 1st ed. Statistics and Computing. Springer Cham, 2018. ISBN: 978-3-319-72634-2. DOI: 10.1007/978-3-319-72634-2.

[56] David R. Mazur. *Combinatorics: A Guided Tour*. The Mathematical Association of America, 2010. ISBN: 978-0-88385-762-5.

[57] Richard P. Stanley. *Enumerative Combinatorics*. 2nd ed. Vol. 1. Cambridge Studies in Advanced Mathematics. Cambridge: Cambridge University Press, 2012. ISBN: 978-1-139-05852-0.

[58] Pavle Mladenović. *Combinatorics. A Problem-Based Approach*. 1st ed. Problem Books in Mathematics. Springer Cham, 2019. ISBN: 978-3-030-00831-4. DOI: 10.1007/978-3-030-00831-4.

[59] Norbert Henze. *Stochastik: Eine Einführung mit Grundzügen der Maßtheorie*. 2nd ed. Springer Spektrum Berlin, Heidelberg, 2024. ISBN: 978-3-662-68649-2. DOI: 10.1007/978-3-662-68649-2.

[60] Donald L. Kreher and Douglas R. Stinson. *Combinatorial Algorithms. Generation, Enumeration, and Search*. The CRC Press Series on Discrete Mathematics and Its Applications. Boca Raton, FL: CRC Press, 1999. ISBN: 0-8493-3988-X.

[61] Donald E. Knuth. *The Art of Computer Programming. Combinatorial Algorithms, Part 1*. Vol. 4A. Upper Saddle River, NJ: Addison-Wesley, 2011. ISBN: 978-0-201-03804-0.

[62] Xavier Molinero and Josep Vives. “Unranking Algorithms for Combinatorial Structures”. In: *International Journal of Applied Mathematics and Informatics* 9 (2015), pp. 110–115. ISSN: 2074-1278. URL: https://hdl.handle.net/2117/96502.

[63] Stephen Melczer. *An Invitation to Analytic Combinatorics. From One to Several Variables*. 1st ed. Texts & Monographs in Symbolic Computation. Springer Cham, 2020. ISBN: 978-3-030-67080-1. DOI: 10.1007/978-3-030-67080-1.

[64] Boris Ryabko. *Fast enumeration of combinatorial objects*. 2006. arXiv: cs/0601069 [cs.CC]. URL: https://arxiv.org/abs/cs/0601069.

[65] Daniel Gabric. "Ranking and unranking bordered and unbordered words". In: *Information Processing Letters* 184 (2024), p. 106452. ISSN: 0020-0190. DOI: 10.1016/j.ipl.2023.106452.

[66] Wendy Myrvold and Frank Ruskey. "Ranking and unranking permutations in linear time". In: *Information Processing Letters* 79.6 (2001), pp. 281–284. ISSN: 0020-0190. DOI: 10.1016/S0020-0190(01)00141-7.

[67] Yuxuan Ma et al. *Theoretical and Empirical Analysis of Lehmer Codes to Search Permutation Spaces with Evolutionary Algorithms*. 2025. arXiv: 2511.19089 [cs.NE]. URL: https://arxiv.org/abs/2511.19089.

[68] Frank Ruskey. *Combinatorial Generation*. Working version (1j–CSC 425/520). Accessed: 30 December 2025. Oct. 2003. URL: http://page.math.tu-berlin.de/~felsner/SemWS17-18/Ruskey-Comb-Gen.pdf.

[69] Vladimir Kruchinin et al. "Unranking Small Combinations of a Large Set in Co-Lexicographic order". In: *Algorithms* 15.2 (2022), p. 36. ISSN: 1999-4893. DOI: 10.3390/a15020036.

[70] Christian Karpfinger and Kurt Meyberg. *Algebra. Gruppen – Ringe – Körper*. 5th ed. Springer Spektrum Berlin Heidelberg, 2021. ISBN: 978-3-662-61952-0. DOI: 10.1007/978-3-662-61952-0.

[71] Johannes Köbler and Jacobo Torán. "The Complexity of Graph Isomorphism for Colored Graphs with Color Classes of Size 2 and 3". In: *STACS 2002*. Ed. by Helmut Alt and Afonso Ferreira. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 121–132. ISBN: 978-3-540-45841-8. DOI: 10.1007/3-540-45841-7_9.

[72] Y. Chi et al. "Indexing and mining free trees". In: *Third IEEE International Conference on Data Mining*. 2003, pp. 509–512. DOI: 10.1109/ICDM.2003.1250964.

[73] Shu-Ming Hsieh et al. "Efficient Method to Perform Isomorphism Testing of Labeled Graphs". In: *Computational Science and Its Applications – ICCSA 2006*. Ed. by Marina L. Gavrilova et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 422–431. ISBN: 978-3-540-34080-5. DOI: 10.1007/11751649_46.

[74] George Grätzer. *Lattice Theory: Foundation*. 1st ed. Birkhäuser Basel, 2011. ISBN: 978-3-0348-0017-4. DOI: 10.1007/978-3-0348-0018-1.

[75] Alfred V. Aho et al. *The Design and Analysis of Computer Algorithms*. Reading, Mass.: Addison-Wesley, 1974.

[76] Florian Ingels. *Revisiting Tree Isomorphism: An Algorithmic Bric-à-Brac*. 2024. arXiv: 2309.14441 [cs.DS]. URL: https://arxiv.org/abs/2309.14441.

[77] Egbert Harzheim. *Ordered Sets*. 1st ed. Vol. 7. Advances in Mathematics. New York, NY: Springer New York, 2005. ISBN: 978-0-387-24219-4. DOI: 10.1007/b104891.

[78] Gabriel Valiente. *Algorithms on Trees and Graphs. With Python Code*. 2nd ed. Texts in Computer Science. Springer Cham, 2021. ISBN: 978-3-030-81885-2. DOI: 10.1007/978-3-030-81885-2.

[79] Thomas Thum et al. "Abstract Features in Feature Modeling". In: *2011 15th International Software Product Line Conference*. 2011, pp. 191–200. DOI: 10.1109/SPLC.2011.53.

[80] Kurt Mehlhorn and Peter Sanders. *Algorithms and Data Structures. The Basic Toolbox*. 1st ed. Springer Berlin Heidelberg, 2008. ISBN: 978-3-540-77978-0. DOI: 10.1007/978-3-540-77978-0.

[81] Jeff Erickson. *Algorithms*. 1st ed. Available under Creative Commons Attribution 4.0 International License. Self-published, 2019. ISBN: 978-1-792-64483-2. URL: https://jeffe.cs.illinois.edu/teaching/algorithms/.

[82] Thomas H. Cormen et al. *Introduction to Algorithms*. 4th ed. Cambridge, MA: The MIT Press, 2022. ISBN: 978-0-262-36750-9.

[83] Jacopo Mauro. "Anomaly detection in Context-aware Feature Models". In: *Proceedings of the 15th International Working Conference on Variability Modelling of Software-Intensive Systems*. VaMoS '21. Krems, Austria: Association for Computing Machinery, 2021. ISBN: 9781450388245. DOI: 10.1145/3442391.3442405.

[84] Patrick Billingsley. *Probability and Measure*. 3rd ed. Wiley Series in Probability and Mathematical Statistics. New York: John Wiley & Sons, 1995. ISBN: 0-471-00710-2.

[85] John E. Savage. *Models of Computation. Exploring the Power of Computing*. Electronic edition, Creative Commons Attribution–NonCommercial–NoDerivatives 3.0. Originally published by Addison-Wesley in 1998. All rights reverted to the author. 2008. URL: https://cs.brown.edu/people/jsavage/book/.

[86] Richard A. Kronmal and Arthur V. Peterson. "On the Alias Method for Generating Random Variables From a Discrete Distribution". In: *The American Statistician* 33.4 (1979), pp. 214–218. DOI: 10.2307/2683739.

[87] M. D. Vose. "A linear algorithm for generating random numbers with a given distribution". In: *IEEE Transactions on Software Engineering* 17.9 (1991), pp. 972–975. DOI: 10.1109/32.92917.

[88] Chris Wyman. "The Alias Method for Sampling Discrete Distributions". In: *Ray Tracing Gems II: Next Generation Real-Time Rendering with DXR, Vulkan, and OptiX*. Ed. by Adam Marrs et al. Berkeley, CA: Apress, 2021, pp. 339–343. ISBN: 978-1-4842-7185-8. DOI: 10.1007/978-1-4842-7185-8_21.

[89] Joseph K. Blitzstein and Jessica Hwang. *Introduction to Probability*. 2nd ed. Texts in Statistical Science. Boca Raton, FL: CRC Press, 2019. ISBN: 978-1-1383-6991-7.

[90] Achim Klenke. *Wahrscheinlichkeitstheorie*. 4th ed. Masterclass. Springer Spektrum Berlin, Heidelberg, 2020. ISBN: 978-3-662-62089-2. DOI: 10.1007/978-3-662-62089-2.

[91] Ludger Rüschendorf. *Wahrscheinlichkeitstheorie*. 1st ed. Masterclass. Springer Spektrum Berlin, Heidelberg, 2016. ISBN: 978-3-662-48937-6. DOI: 10.1007/978-3-662-48937-6.

[92] Peyman Afshani et al. "Fragile complexity of comparison-based algorithms". In: *Proceedings of the 27th Annual European Symposium on Algorithms*. Vol. 144. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, 2:1–2:19. DOI: 10.4230/LIPIcs.ESA.2019.2.

[93] Dwaine Clarke et al. "Incremental Multiset Hash Functions and Their Application to Memory Integrity Checking". In: *Advances in Cryptology – ASIACRYPT 2003*. Ed. by Chi-Sung Laih. Berlin, Heidelberg: Springer Berlin Heidelberg, 2003, pp. 188–207. ISBN: 978-3-540-40061-5. DOI: 10.1007/978-3-540-40061-5_12.

[94] Jeremy Maitin-Shepard et al. "Elliptic Curve Multiset Hash". In: *The Computer Journal* 60.4 (2016), pp. 476–490. ISSN: 0010-4620. DOI: 10.1093/comjnl/bxw053.

[95] Peter J. Downey et al. "Variations on the Common Subexpression Problem". In: *J. ACM* 27.4 (Oct. 1980), pp. 758–771. ISSN: 0004-5411. DOI: 10.1145/322217.322228.

[96] Max Bannach et al. "Faster Graph Algorithms Through DAG Compression". In: *41st International Symposium on Theoretical Aspects of Computer Science (STACS 2024)*. Ed. by Olaf Beyersdorff et al. Vol. 289. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, 8:1–8:18. ISBN: 978-3-95977-311-9. DOI: 10.4230/LIPIcs.STACS.2024.8.

[97] IEEE. "IEEE standard for floating-point arithmetic". In: *IEEE Std 754-2008* (2008), pp. 1–70. DOI: 10.1109/IEEESTD.2008.4610935.

[98] David Goldberg. "What every computer scientist should know about floating-point arithmetic". In: *ACM Computing Surveys* 23.1 (1991), pp. 5–48. ISSN: 0360-0300. DOI: 10.1145/103162.103163.

[99] William H. Press et al. *Numerical Recipes. The Art of Scientific Computing*. 3rd ed. Cambridge: Cambridge University Press, 2007. ISBN: 978-0-521-88068-8.

[100] Peter Deuflhard and Andreas Hohmann. *Numerische Mathematik 1. Eine algorithmisch orientierte Einführung*. 5th ed. Berlin, Boston: De Gruyter, 2019. ISBN: 978-3-11-061432-9. DOI: 10.1515/9783110614329.

[101] Toby Walsh. "General symmetry breaking constraints". In: *Principles and Practice of Constraint Programming*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 650–664. ISBN: 978-3-540-46268-2. DOI: 10.1007/11889205_46.

[102] Jean-François Puget. “Dynamic lex constraints”. In: *Principles and Practice of Constraint Programming*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 453–467. ISBN: 978-3-540-46268-2. DOI: 10.1007/11889205_33.

[103] Roland Martin. “Breaking weak symmetries in constraint programming”. PhD thesis. Darmstadt, Germany: Technische Universität Darmstadt, 2008. DOI: 10.26083/tuprints-00000863.

[104] Mate Soos and Kuldeep S. Meel. “Engineering an Efficient Probabilistic Exact Model Counter”. In: *Computer Aided Verification: 37th International Conference, CAV 2025, Zagreb, Croatia, July 23-25, 2025, Proceedings, Part III*. Zagreb, Croatia: Springer-Verlag, 2025, pp. 72–91. ISBN: 978-3-031-98681-9. DOI: 10.1007/978-3-031-98682-6_5.

[105] Supratik Chakraborty et al. “Approximate model counting”. In: *Handbook of Satisfiability*. Ed. by Armin Biere et al. Vol. 336. Frontiers in Artificial Intelligence and Applications. IOS Press, 2021, pp. 1015–1045. ISBN: 978-1-64368-161-0. DOI: 10.3233/FAIA201010.

[106] Frank Hutter et al. “ParamILS: an automatic algorithm configuration framework”. In: *J. Artif. Int. Res.* 36.1 (Sept. 2009), pp. 267–306. ISSN: 1076-9757. URL: https://dl.acm.org/doi/abs/10.5555/1734953.1734959.

[107] Carlos Ansótegui et al. “MaxSAT by improved instance-specific algorithm configuration”. In: *Artificial Intelligence* 235 (2016), pp. 26–39. ISSN: 0004-3702. DOI: 10.1016/j.artint.2015.12.006.

[108] Marius Lindauer et al. “The algorithm selection competitions 2015 and 2017”. In: *Artificial Intelligence* 272 (2019), pp. 86–100. ISSN: 0004-3702. DOI: 10.1016/j.artint.2018.10.004.

[109] Martin Brain et al. “Benchmarking Solvers, SAT-style”. In: *Proceedings of the 2nd International Workshop on Satisfiability Checking and Symbolic Computation co-located with the 42nd International Symposium on Symbolic and Algebraic Computation (ISSAC 2017), Kaiserslautern, Germany, July 29, 2017*. Ed. by Matthew England and Vijay Ganesh. Vol. 1974. CEUR Workshop Proceedings. CEUR-WS.org, 2017. URL: https://ceur-ws.org/Vol-1974/RP3.pdf.

[110] Elizabeth D. Dolan and Jorge J. Moré. “Benchmarking optimization software with performance profiles”. In: *Mathematical Programming* 91.2 (2002), pp. 201–213. ISSN: 1436-4646. DOI: 10.1007/s101070100263.

[111] Mladen Nikolić. “Statistical methodology for comparison of SAT solvers”. In: *Proceedings of the 13th International Conference on Theory and Applications of Satisfiability Testing*. SAT’10. Edinburgh, UK: Springer-Verlag, 2010, pp. 209–222. ISBN: 3642141854. DOI: 10.1007/978-3-642-14186-7_18.

[112] Tobias Fuchs et al. “Active Learning for SAT Solver Benchmarking”. In: *Journal of Automated Reasoning* 69.3 (2025), p. 16. ISSN: 1573-0670. DOI: 10.1007/s10817-025-09729-6.

[113] William Mendenhall et al. *Introduction to Probability and Statistics*. 13th ed. Belmont, CA: Brooks/Cole, Cengage Learning, 2009. ISBN: 9780495389538.

[114] E. L. Lehmann and Joseph P. Romano. *Testing Statistical Hypotheses*. 4th ed. Springer Texts in Statistics. Springer Cham, 2022. ISBN: 978-3-030-70578-7. DOI: 10.1007/978-3-030-70578-7.

[115] Norbert Henze. *Stochastik für Einsteiger. Eine Einführung in die faszinierende Welt des Zufalls*. 14th ed. Springer Spektrum Berlin, Heidelberg, 2024. ISBN: 978-3-662-67728-5. DOI: 10.1007/978-3-662-67729-2.

[116] Kenneth J. Rothman. *Modern Epidemiology*. Boston: Little, Brown and Company, 1986. ISBN: 0-316-75776-4.

[117] Dennis D. Wackerly et al. *Mathematical Statistics with Applications*. 7th ed. Belmont, CA: Thomson Higher Education, 2008. ISBN: 9780495385080.

[118] Jonathan Gillard. *A First Course in Statistical Inference*. 1st ed. Springer Undergraduate Mathematics Series. Cham: Springer, 2020. ISBN: 978-3-030-39560-5. DOI: 10.1007/978-3-030-39561-2.

[119] Rose R. Chamberlain et al. "P-Value Chronicles, The: Unmasking Fourteen Widespread Misconceptions". In: *International Encyclopedia of Statistical Science*. Ed. by Miodrag Lovric. Berlin, Heidelberg: Springer Berlin Heidelberg, 2025, pp. 2006–2024. ISBN: 978-3-662-69359-9. DOI: 10.1007/978-3-662-69359-9_705.

[120] Raymond Hubbard. "P-Values". In: *International Encyclopedia of Statistical Science*. Ed. by Miodrag Lovric. Berlin, Heidelberg: Springer Berlin Heidelberg, 2025, pp. 2024–2026. ISBN: 978-3-662-69359-9. DOI: 10.1007/978-3-662-69359-9_458.

[121] Pierre L'Ecuyer and Richard Simard. "TestU01: A C library for empirical testing of random number generators". In: *ACM Trans. Math. Softw.* 33.4 (Aug. 2007), p. 22. ISSN: 0098-3500. DOI: 10.1145/1268776.1268777.

[122] N. A. Heard and P. Rubin-Delanchy. "Choosing Between Methods of Combining p-Values". In: *Biometrika* 105.1 (2018), pp. 239–246. ISSN: 1464-3510. DOI: 10.1093/biomet/asx076.

[123] Dinis Pestana et al. "P-Values, Combining of". In: *International Encyclopedia of Statistical Science*. Ed. by Miodrag Lovric. Berlin, Heidelberg: Springer Berlin Heidelberg, 2025, pp. 2026–2033. ISBN: 978-3-662-69359-9. DOI: 10.1007/978-3-662-69359-9_459.

[124] Andrew Rukhin et al. *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications*. NIST Special Publication 800-22 Revision 1a. Gaithersburg, MD, USA: National Institute of Standards and Technology (NIST), Apr. 2010. URL: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=906762.

[125] C. J. Clopper and E. S. Pearson. "The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial". In: *Biometrika* 26.4 (1934), pp. 404–413. ISSN: 0006-3444. DOI: 10.1093/biomet/26.4.404.

[126] Robert G. Newcombe. "Two-sided confidence intervals for the single proportion: comparison of seven methods". In: *Statistics in Medicine* 17.8 (1998), pp. 857–872. DOI: 10.1002/(SICI)1097-0258(19980430)17:8<857::AID-SIM777>3.0.CO;2-E.

[127] Marcilio Mendonca et al. “S.P.L.O.T.: software product lines online tools”. In: *Proceedings of the 24th ACM SIGPLAN Conference Companion on Object Oriented Programming Systems Languages and Applications*. OOPSLA ’09. Orlando, Florida, USA: Association for Computing Machinery, 2009, pp. 761–762. ISBN: 9781605587684. DOI: 10.1145/1639950.1640002.

[128] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge: Cambridge University Press, 2009. ISBN: 978-0-521-42426-4.

[129] Michael Sipser. *Introduction to the Theory of Computation*. 3rd ed. Boston, MA: Cengage Learning, 2013. ISBN: 978-1-133-18779-0.

[130] Ian P. Gent and Toby Walsh. “The SAT Phase Transition”. In: *Proceedings of the 11th European Conference on Artificial Intelligence*. ECAI ’94. Amsterdam, The Netherlands: John Wiley & Sons, Inc., 1994, pp. 105–109.

[131] Ke Xu and Wei Li. “The SAT Phase Transition”. In: *Science in China Series E: Technological Sciences* 42.5 (1999), pp. 494–501. ISSN: 1862-281X. DOI: 10.1007/BF02917402.

[132] Dimitris Achlioptas and Cristopher Moore. “Random k-SAT: Two Moments Suffice to Cross a Sharp Threshold”. In: *SIAM Journal on Computing* 36.3 (2006), pp. 740–762. DOI: 10.1137/S0097539703434231.

[133] Hendrik Schawe et al. “Phase Transitions of the Typical Algorithmic Complexity of the Random Satisfiability Problem Studied with Linear Programming”. In: *PLOS ONE* 14.4 (2019), pp. 1–16. DOI: 10.1371/journal.pone.0215309.

[134] Önder Babur et al. “Model analytics for feature models: Case studies for S.P.L.O.T. Repository”. English. In: *Proceedings of MODELS 2018 Workshops: ModComp, MRT, OCL, FlexMDE, EXE, COMMitMDE, MDETools, GEMOC, MORSE, MDE4IoT, MDEbug, MoDeVVa, ME, MULTI, HuFaMo, AMMoRe, PAINS*. Ed. by R. Hebig and T. Berger. CEUR Workshop Proceedings. CEUR Workshop Proceedings, 2018, pp. 787–792. URL: http://ceur-ws.org/Vol-2245/ammore_paper_2.pdf.

[135] Anna Lindeberg. *Isomorphism Testing of Rooted Trees in Linear Time*. 2024. arXiv: 2401.07636 [cs.CC]. URL: https://arxiv.org/abs/2401.07636.

A. Use of AI Tools

In addition to conventional internet research, generative AI, primarily ChatGPT, was used as an auxiliary tool for rephrasing individual sentences, supporting code development and review, and sanity-checking methodology, grammar, spelling, and punctuation. All such outputs were critically reviewed and, where necessary, revised before inclusion.